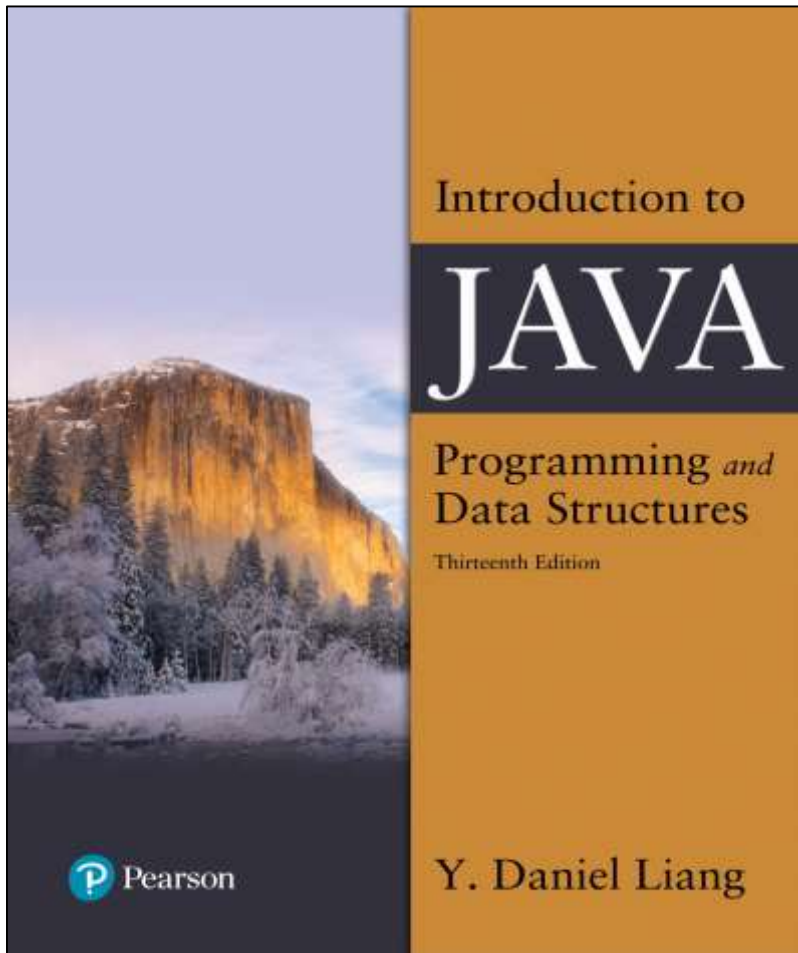


■çindekiler

Copy of 09slide_accessible.pdf	2
Copy of 10slide_accessible.pdf	68
Copy of 11slide_accessible.pdf	127
Copy of 12slide_accessible.pdf	189
Copy of 13slide_accessible.pdf	257
Copy of 14slide_accessible.pdf	306
Copy of 15slide_accessible.pdf	343
Copy of 16slide_accessible.pdf	385
Copy of 19slide_accessible.pdf	420
Copy of 24slide_accessible.pdf	447

Introduction to Java Programming and Data Structures

Thirteenth Edition

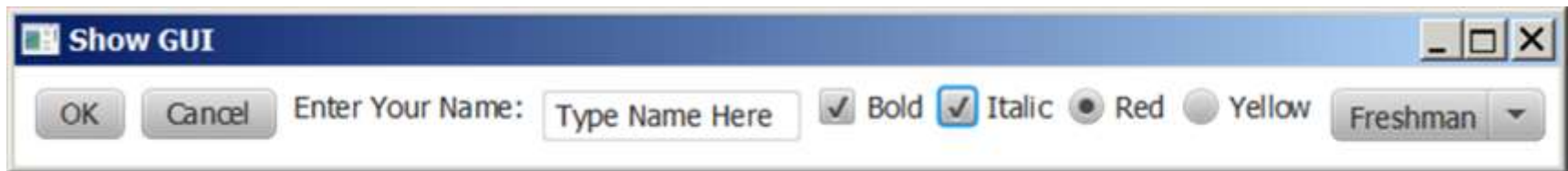


Chapter 9

Objects and Classes

Motivations

After learning the preceding chapters, you are capable of solving many programming problems using selections, loops, methods, and arrays. However, these Java features are not sufficient for developing graphical user interfaces and large scale software systems. Suppose you want to develop a graphical user interface as shown below. How do you program it?



Objectives (1 of 3)

- 9.1** To describe objects and classes, and use classes to model objects (§9.2).
- 9.2** To use UML graphical notation to describe classes and objects (§9.2).
- 9.3** To demonstrate how to define classes and create objects (§9.3).
- 9.4** To create objects using constructors (§9.4).
- 9.5** To define a reference variable using a reference type and access objects via object reference variables (§9.5).
- 9.6** To access an object's data and methods using the object member access operator (.) (§9.5.1).
- 9.7** To define data fields of reference types and assign default values for an object's data fields (§9.5.2).

Objectives (2 of 3)

9.8 To distinguish between object reference variables and primitive data type variables (§9.5.3).

9.9 To use the Java library classes **Date**, **Random**, and **Point2D** (§9.6).

9.10 To distinguish between instance and static variables and methods (§9.7).

9.11 To define private data fields with appropriate **get** and **set** methods (§9.8).

9.12 To encapsulate data fields to make classes easy to maintain (§9.9).

9.13 To develop methods with object arguments and differentiate between primitive-type arguments and object-type arguments (§9.10).

9.14 To store and process objects in arrays (§9.11).

Objectives (3 of 3)

9.15 To create immutable objects from immutable classes to protect the contents of objects (§9.12).

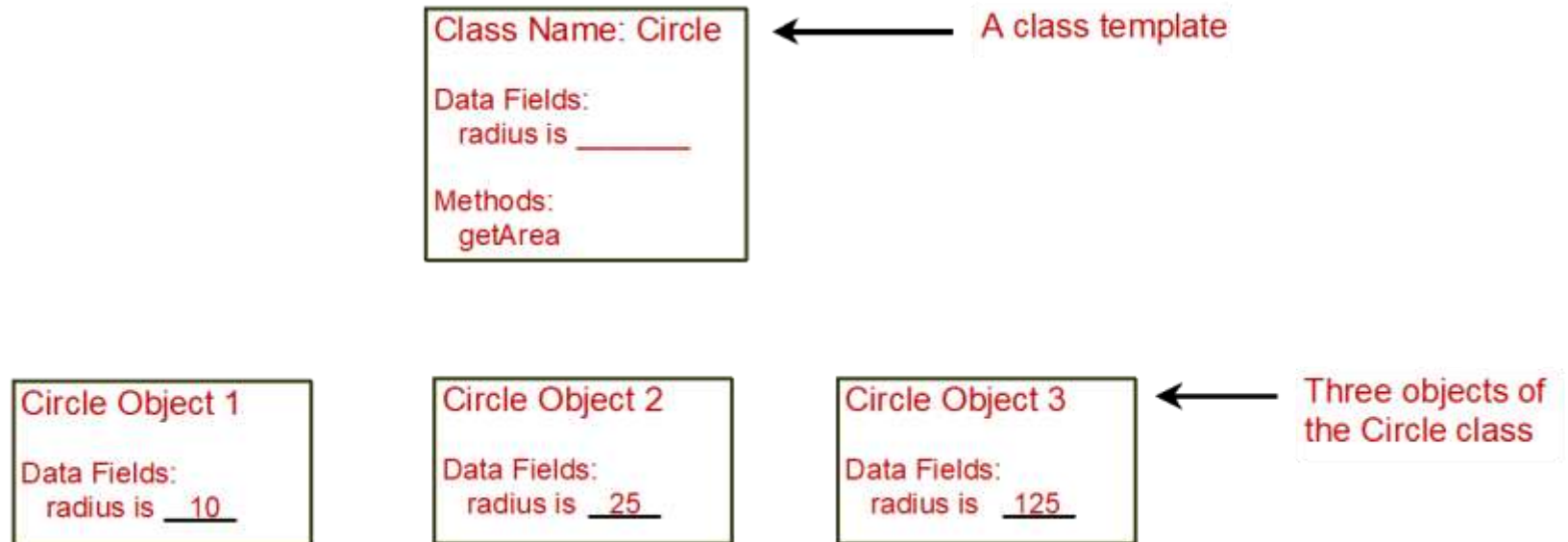
9.16 To determine the scope of variables in the context of a class (§9.13).

9.17 To use the keyword **this** to refer to the calling object itself (§9.14).

OO Programming Concepts

Object-oriented programming (OOP) involves programming using objects. An **object** represents an entity in the real world that can be distinctly identified. For example, a student, a desk, a circle, a button, and even a loan can all be viewed as objects. An object has a unique identity, state, and behaviors. The **state** of an object consists of a set of **data fields** (also known as **properties**) with their current values. The **behavior** of an object is defined by a set of methods.

Objects



An object has both a state and behavior. The state defines the object, and the behavior defines what the object does.

Classes (1 of 2)

Classes are constructs that define objects of the same type. A Java class uses variables to define data fields and methods to define behaviors. Additionally, a class provides a special type of methods, known as constructors, which are invoked to construct objects from the class.

Classes (2 of 2)

```
class Circle {  
    /** The radius of this circle */  
    double radius = 1.0;  
  
    /** Construct a circle object */  
    Circle() {  
    }  
  
    /** Construct a circle object */  
    Circle(double newRadius) {  
        radius = newRadius;  
    }  
  
    /** Return the area of this circle */  
    double getArea() {  
        return radius * radius * 3.14159;  
    }  
}
```

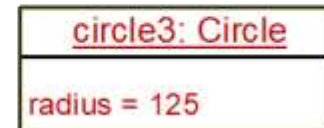
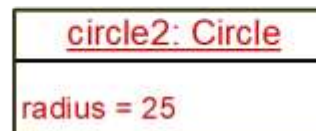
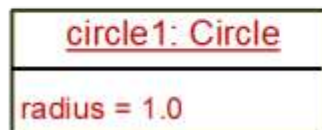
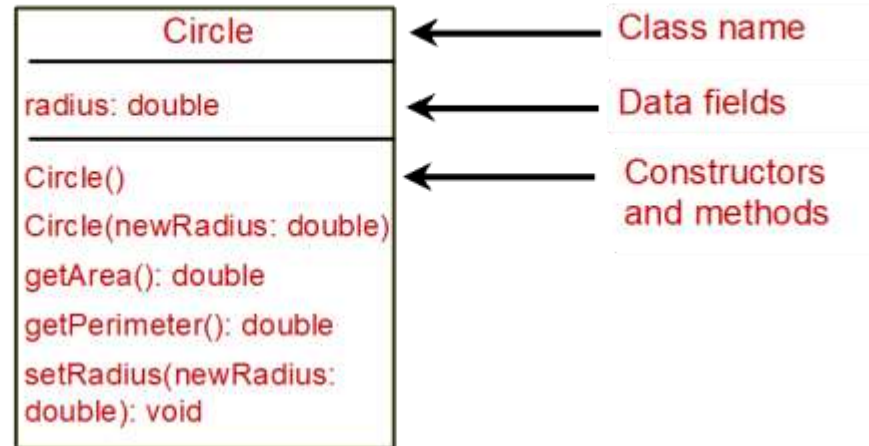
← Data field

← Constructors

← Method

UML Class Diagram

UML Class Diagram



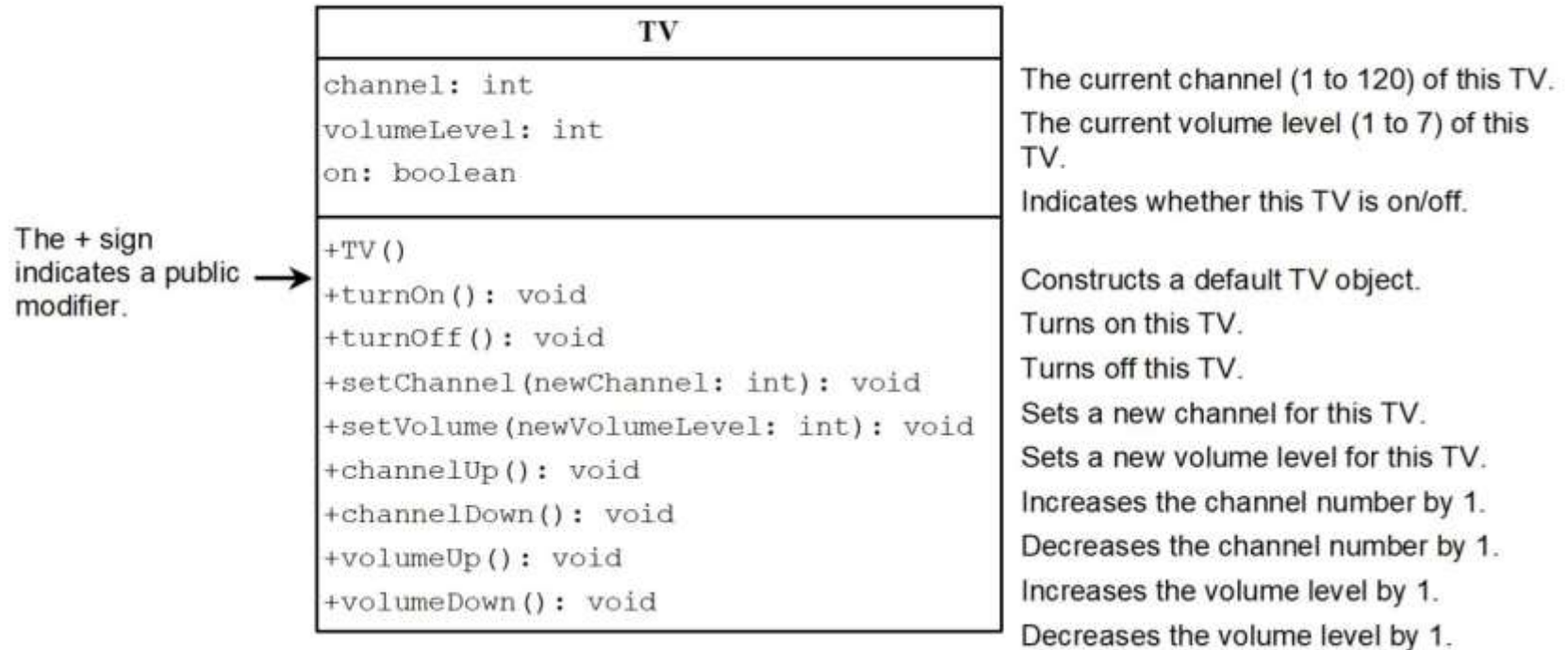
UML notation
for objects

Example: Defining Classes and Creating Objects (1 of 2)

Objective: Demonstrate creating objects, accessing data, and using methods.

TestSimpleCircle

Example: Defining Classes and Creating Objects (2 of 2)



TV

TestTV

Constructors (1 of 2)

```
Circle() {  
  
}  
  
Circle(double newRadius) {  
    radius = newRadius;  
}
```

Constructors are a special kind of methods that are invoked to construct objects.

Constructors (2 of 2)

A constructor with no parameters is referred to as a **no-arg constructor**.

- Constructors must have the same name as the class itself.
- Constructors do not have a return type—not even void.
- Constructors are invoked using the new operator when an object is created. Constructors play the role of initializing objects.

Creating Objects Using Constructors

```
new ClassName() ;
```

Example:

```
new Circle() ;
```

```
new Circle(5.0) ;
```

Default Constructor

A class may be defined without constructors. In this case, a no-arg constructor with an empty body is implicitly defined in the class. This constructor, called a **default constructor**, is provided automatically **only if no constructors are explicitly defined in the class**.

Declaring Object Reference Variables

To reference an object, assign the object to a reference variable.

To declare a reference variable, use the syntax:

```
ClassName objectRefVar;
```

Example:

```
Circle myCircle;
```

Declaring/Creating Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```

Example:

Assign object reference Create an object

```
Circle myCircle = new Circle();
```

Accessing Object's Members

- Referencing the object's data:

`objectRefVar.data`

e.g., `myCircle.radius`

- Invoking the object's method:

`objectRefVar.methodName (arguments)`

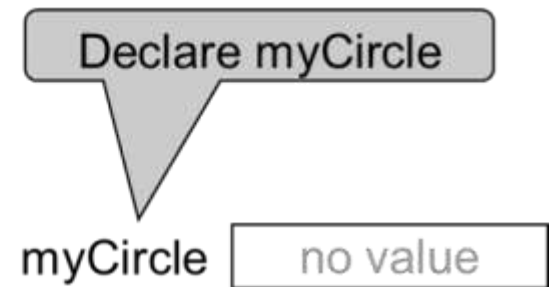
e.g., `myCircle.getArea ()`

Trace Code (1 of 7)

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```



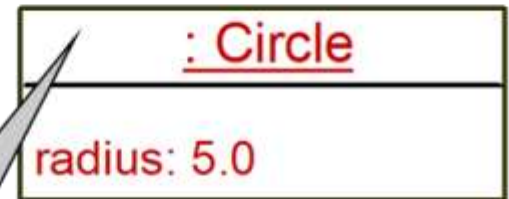
Trace Code (2 of 7)

Circle myCircle = new Circle(5.0);

myCircle no value

Circle yourCircle = new Circle();

yourCircle.radius = 100;



Create a circle

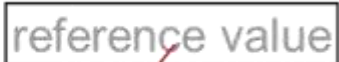
Trace Code (3 of 7)

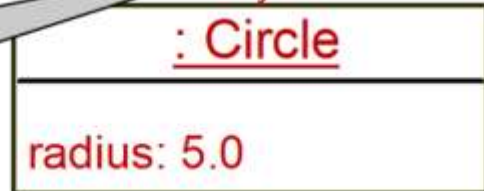
Circle myCircle  **new Circle(5.0);**

Circle yourCircle = new Circle();

yourCircle.radius = 100;

Assign object
reference to myCircle

myCircle 



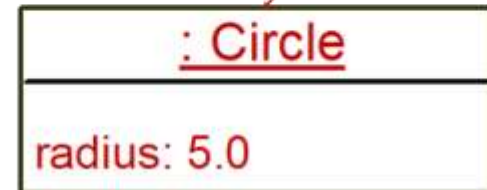
Trace Code (4 of 7)

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

myCircle reference value



yourCircle no value

Declare yourCircle

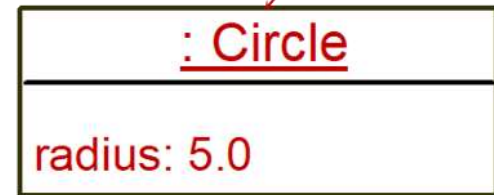
Trace Code (5 of 7)

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

myCircle reference value



yourCircle no value

Create a new
Circle object

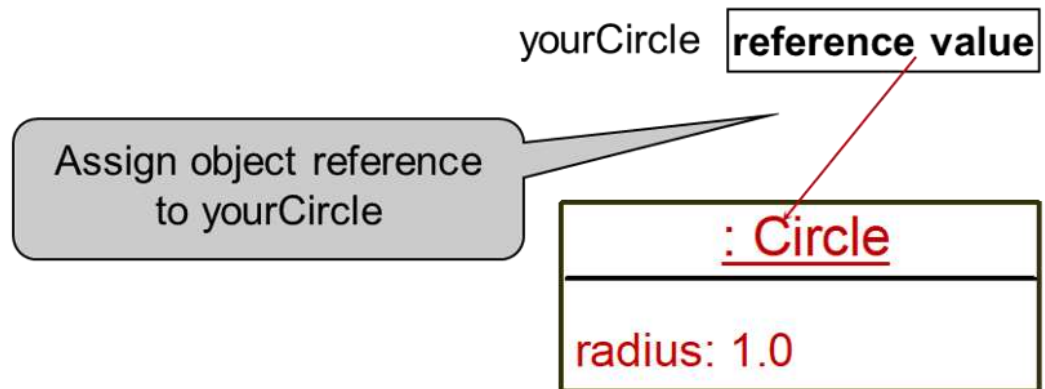
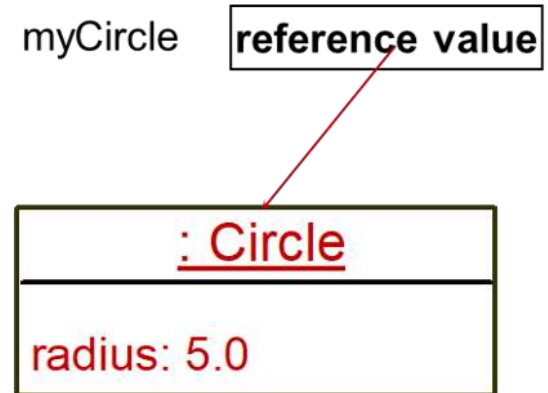


Trace Code (6 of 7)

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

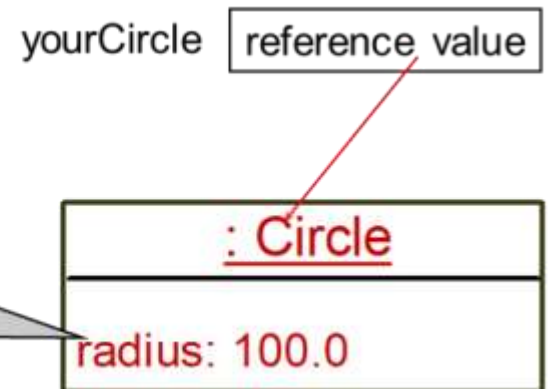
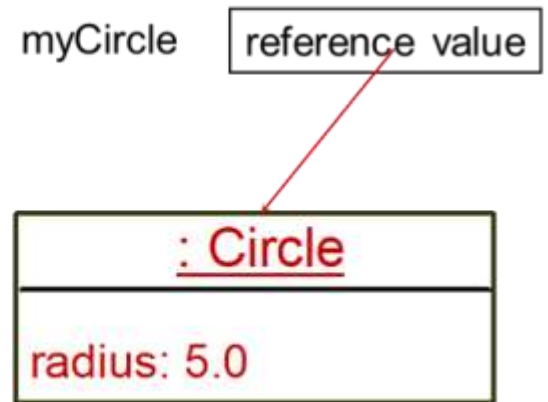


Trace Code (7 of 7)

```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```



Change radius in
yourCircle

Caution

Recall that you use

```
Math.methodName(arguments) (e.g., Math.pow(3, 2.5))
```

to invoke a method in the Math class. Can you invoke `getArea()` using `SimpleCircle.getArea()`? The answer is no. All the methods used before this chapter are static methods, which are defined using the `static` keyword. However, `getArea()` is non-static. It must be invoked from an object using

```
objectRefVar.methodName(arguments) (e.g.,  
myCircle.getArea()).
```

More explanations will be given in the section on “Static Variables, Constants, and Methods.”

Reference Data Fields

The data fields can be of reference types. For example, the following Student class contains a data field name of the String type.

```
public class Student {  
    String name; // name has default value null  
    int age; // age has default value 0  
    boolean isScienceMajor; // isScienceMajor  
    has default value false  
    char gender; // c has default value  
    '\u0000'  
}
```

The Null Value

If a data field of a reference type does not reference any object, the data field holds a special literal value, null.

Default Value for a Data Field

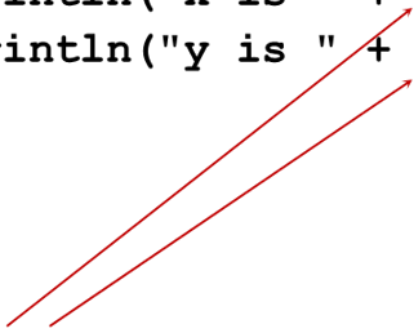
The default value of a data field is null for a reference type, 0 for a numeric type, false for a boolean type, and ' \u0000 ' for a char type. However, Java assigns no default value to a local variable inside a method.

```
public class Test {  
    public static void main(String[] args) {  
        Student student = new Student();  
        System.out.println("name? " + student.name);  
        System.out.println("age? " + student.age);  
        System.out.println("isScienceMajor? " +  
            student.isScienceMajor);  
        System.out.println("gender? " + student.gender);  
    }  
}
```

Example (1 of 3)

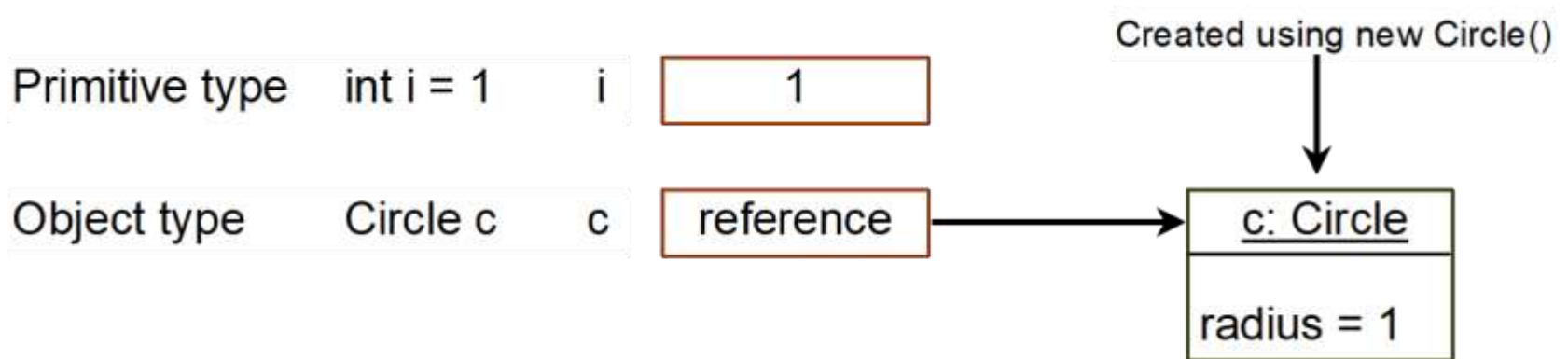
Java assigns no default value to a local variable inside a method.

```
public class Test {  
    public static void main(String[] args) {  
        int x; // x has no default value  
        String y; // y has no default value  
        System.out.println("x is " + x);  
        System.out.println("y is " + y);  
    }  
}
```



Compile error: variable not initialized

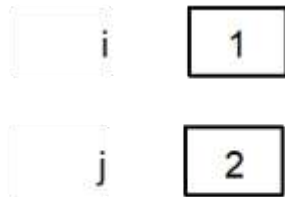
Differences Between Variables of Primitive Data Types and Object Types



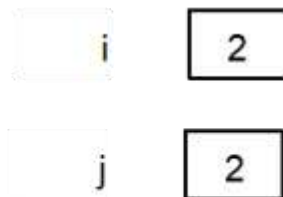
Copying Variables of Primitive Data Types and Object Types

Primitive type assignment $i = j$

Before:

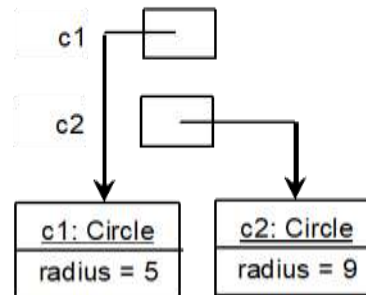


After:

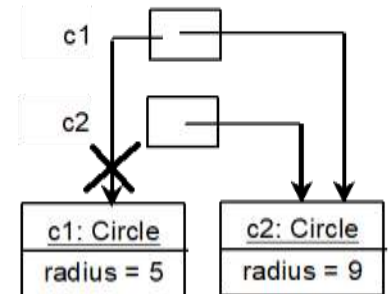


Object type assignment $c1 = c2$

Before:



After:



Garbage Collection (1 of 2)

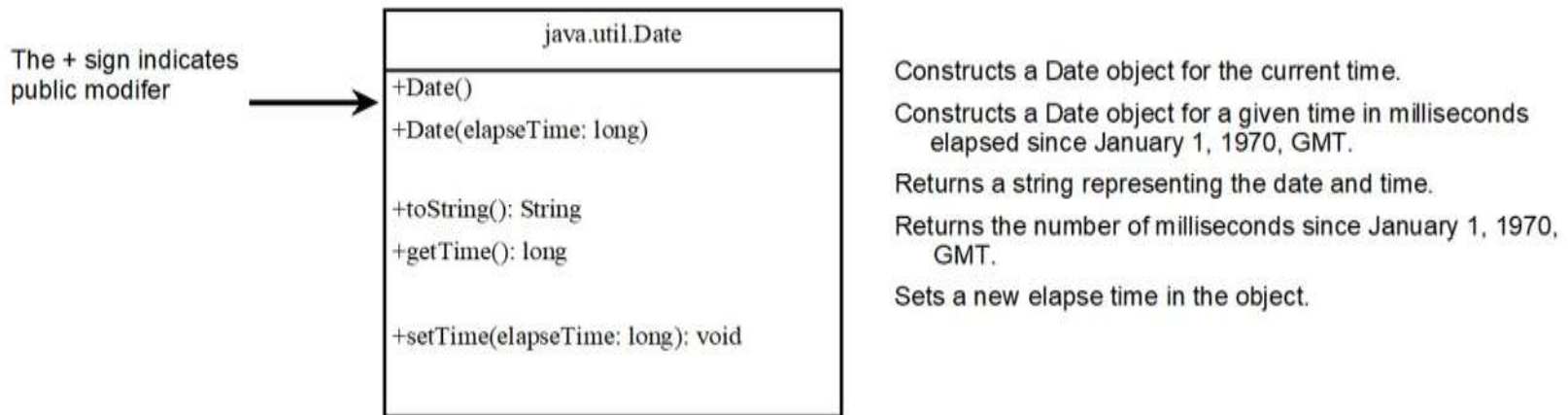
As shown in the previous figure, after the assignment statement `c1 = c2`, `c1` points to the same object referenced by `c2`. The object previously referenced by `c1` is no longer referenced. This object is known as garbage. Garbage is automatically collected by JVM.

Garbage Collection (2 of 2)

Tip: If you know that an object is no longer needed, you can explicitly assign null to a reference variable for the object. The JVM will automatically collect the space if the object is not referenced by any variable.

The Date Class

Java provides a system-independent encapsulation of date and time in the `java.util.Date` class. You can use the `Date` class to create an instance for the current date and time and use its `toString` method to return the date and time as a string.



The Date Class Example

For example, the following code

```
java.util.Date date = new java.util.Date();  
System.out.println(date.toString());
```

displays a string like **Sun Mar 09 13:50:19 EST 2003**.

The Random Class

You have used `Math.random()` to obtain a random double value between 0.0 and 1.0 (excluding 1.0). A more useful random number generator is provided in the `java.util.Random` class.

java.util.Random	
+Random()	Constructs a Random object with the current time as its seed.
+Random(seed: long)	Constructs a Random object with a specified seed.
+nextInt(): int	Returns a random int value.
+nextInt(n: int): int	Returns a random int value between 0 and n (exclusive).
+nextLong(): long	Returns a random long value.
+nextDouble(): double	Returns a random double value between 0.0 and 1.0 (exclusive).
+nextFloat(): float	Returns a random float value between 0.0F and 1.0F (exclusive).
+nextBoolean(): boolean	Returns a random boolean value.

The Random Class Example

If two **Random** objects have the same seed, they will generate identical sequences of numbers. For example, the following code creates two **Random** objects with the same seed 3.

```
Random random1 = new Random(3);  
System.out.print("From random1: ");  
for (int i = 0; i < 10; i++)  
    System.out.print(random1.nextInt(1000) + " ");  
Random random2 = new Random(3);  
System.out.print("\nFrom random2: ");  
for (int i = 0; i < 10; i++)  
    System.out.print(random2.nextInt(1000) + " ");
```

From random1: 734 660 210 581 128 202 549 564 459 961

From random2: 734 660 210 581 128 202 549 564 459 961

The Point2D Class

Java API has a convenient **Point2D** class in the **javafx.geometry** package for representing a point in a two-dimensional plane.

javafx.geometry.Point2D

```
+Point2D(x: double, y: double)
+distance(x: double, y: double): double
+distance(p: Point2D): double
+getX(): double
+getY(): double
+toString(): String
```

Constructs a **Point2D** object with the specified *x*- and *y*-coordinates.
Returns the distance between this point and the specified point (*x*, *y*).
Returns the distance between this point and the specified point *p*.
Returns the *x*-coordinate from this point.
Returns the *y*-coordinate from this point.
Returns a string representation for the point.

[TestPoint2D](#)

Instance Variables, and Methods

Instance variables belong to a specific instance.

Instance methods are invoked by an instance of the class.

Static Variables, Constants, and Methods (1 of 3)

Static variables are shared by all the instances of the class.

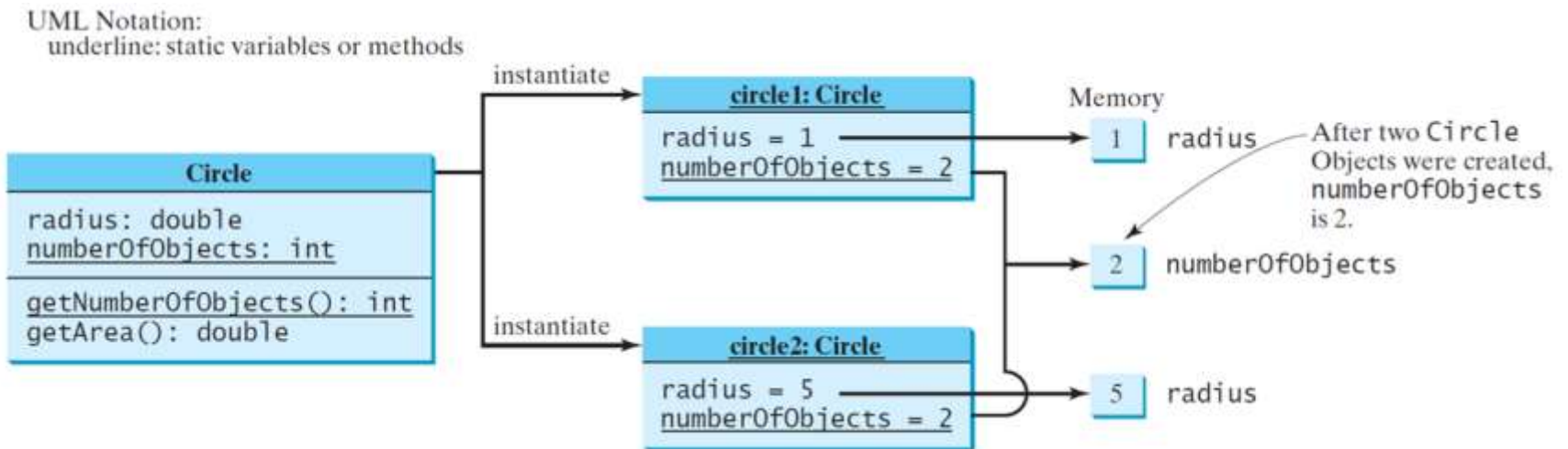
Static methods are not tied to a specific object.

Static constants are final variables shared by all the instances of the class.

Static Variables, Constants, and Methods (2 of 3)

To declare static variables, constants, and methods, use the static modifier.

Static Variables, Constants, and Methods (3 of 3)



Example of Using Instance and Class Variables and Method

Objective: Demonstrate the roles of instance and class variables and their uses. This example adds a class variable `numberOfObjects` to track the number of `Circle` objects created.

[CircleWithStaticMembers](#)

[TestCircleWithStaticMembers](#)

Visibility Modifiers and Accessor/Mutator Methods (1 of 3)

By default, the class, variable, or method can be accessed by any class in the same package.

- `Public`

The class, data, or method is visible to any class in any package.

- `Private`

The data or methods can be accessed only by the declaring class.

The get and set methods are used to read and modify private properties.

Visibility Modifiers and Accessor/Mutator Methods (2 of 3)

```
package p1;

public class C1 {
    public int x;
    int y;
    private int z;

    public void m1() {
    }
    void m2() {
    }
    private void m3() {
    }
}
```

```
package p1;

public class C2 {
    void aMethod() {
        C1 o = new C1();
        can access o.x;
        can access o.y;
        cannot access o.z;

        can invoke o.m1();
        can invoke o.m2();
        cannot invoke o.m3();
    }
}
```

```
package p2;

public class C3 {
    void aMethod() {
        C1 o = new C1();
        can access o.x;
        cannot access o.y;
        cannot access o.z;

        can invoke o.m1();
        cannot invoke o.m2();
        cannot invoke o.m3();
    }
}
```

The private modifier restricts access to within a class, the default modifier restricts access to within a package, and the public modifier enables unrestricted access.

Visibility Modifiers and Accessor/Mutator Methods (3 of 3)

```
package p1;  
  
class C1 {  
    ...  
}
```

```
package p1;  
  
public class C2 {  
    can access C1  
}
```

```
package p2;  
  
public class C3 {  
    cannot access C1;  
    can access C2;  
}
```

The default modifier on a class restricts access to within a package, and the public modifier enables unrestricted access.

Note

An object cannot access its private members, as shown in (b). It is **Ok**, however, if the object is declared in its own class, as shown in (a).

```
public class C {  
    private boolean x;  
  
    public static void main(String[] args) {  
        C c = new C();  
        System.out.println(c.x);  
        System.out.println(c.convert());  
    }  
  
    private int convert() {  
        return x ? 1 : -1;  
    }  
}
```

(a) This is okay because object `c` is used inside the class `C`.

```
public class Test {  
    public static void main(String[] args) {  
        C c = new C();  
        System.out.println(c.x);  
        System.out.println(c.convert());  
    }  
}
```

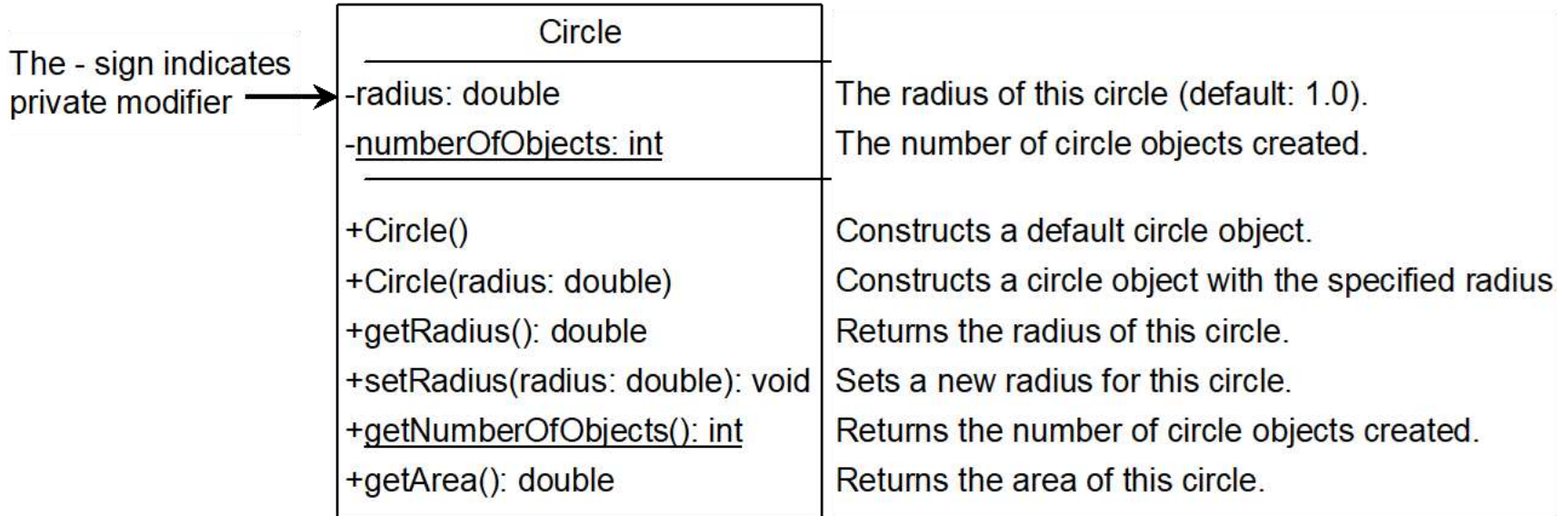
(b) This is wrong because `x` and `convert` are private in class `C`.

Why Data Fields Should Be Private?

To protect data.

To make code easy to maintain.

Example of Data Field Encapsulation



CircleWithPrivateDataFields

TestCircleWithPrivateDataFields

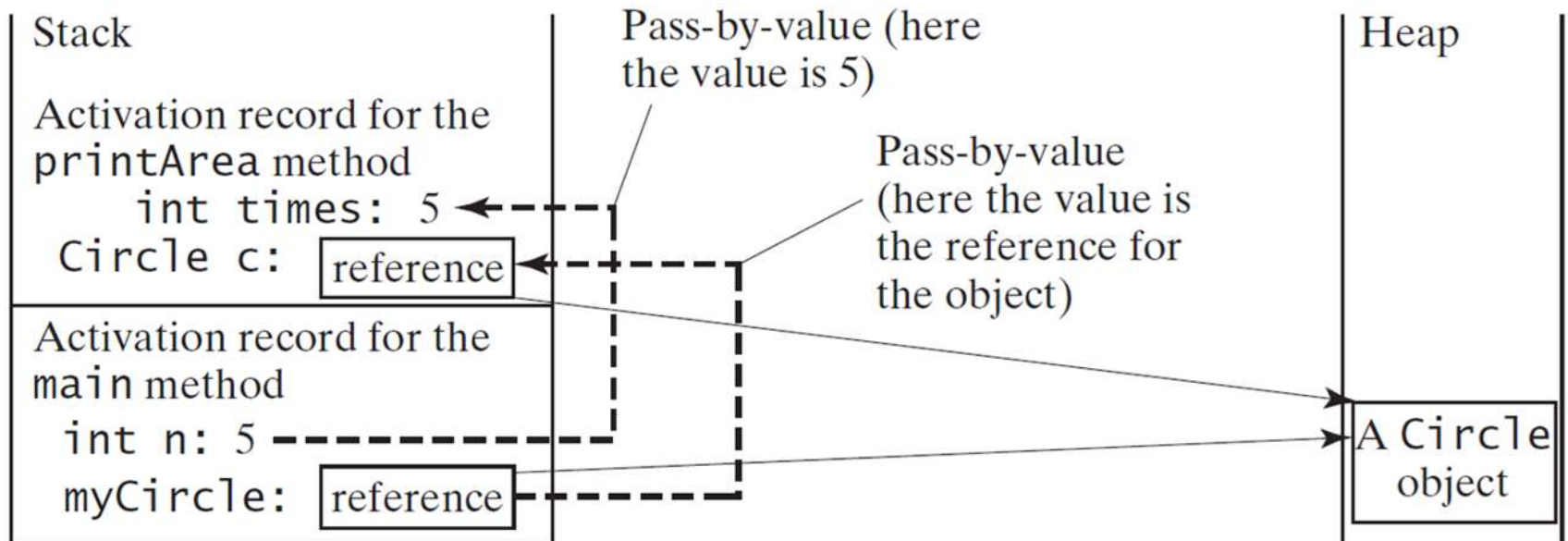
Modifier	Class	Package	Subclasses	World
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
no modifier	✓	✓	✗	✗
private	✓	✗	✗	✗

Passing Objects to Methods (1 of 2)

- Passing by value for primitive type value (the value is passed to the parameter)
- Passing by value for reference type value (the value is the reference to the object)

[TestPassObject](#)

Passing Objects to Methods (2 of 2)



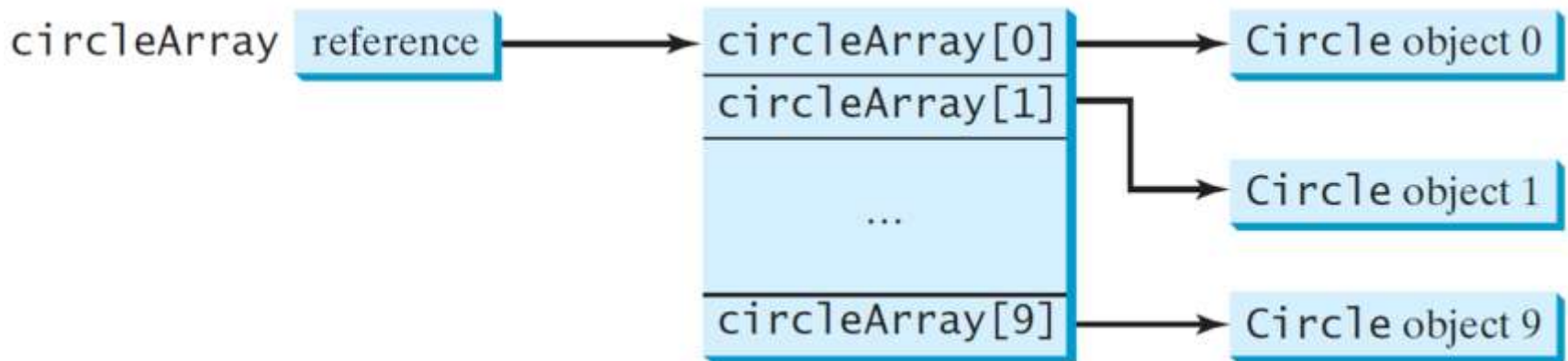
Array of Objects (1 of 3)

```
Circle[] circleArray = new Circle[10];
```

An array of objects is actually an **array of reference variables**. So invoking `circleArray[1].getArea()` involves two levels of referencing as shown in the next figure. `circleArray` references to the entire array. `circleArray[1]` references to a `Circle` object.

Array of Objects (2 of 3)

```
Circle[] circleArray = new Circle[10];
```



Array of Objects (3 of 3)

Summarizing the areas of the circles

TotalArea

Immutable Objects and Classes

If the contents of an object cannot be changed once the object is created, the object is called an **immutable object** and its class is called an **immutable class**. If you delete the set method in the Circle class in Listing 8.10, the class would be immutable because radius is private and cannot be changed without a set method.

A class with all private data fields and without mutators is not necessarily immutable. For example, the following class Student has all private data fields and no mutators, but it is mutable.

Example (2 of 3)

```
public class Student {
    private int id;
    private BirthDate birthDate;
    public Student(int ssn,
        int year, int month, int
        day) {
        id = ssn;
        birthDate = new
        BirthDate(year, month, day);
    }
    public int getId() {
        return id;
    }
    public BirthDate
    getBirthDate() {
        return birthDate;
    }
}
```

```
public class BirthDate {
    private int year;
    private int month;
    private int day;

    public BirthDate(int newYear,
        int newMonth, int newDay)
    {
        year = newYear;
        month = newMonth;
        day = newDay;
    }

    public void setYear(int
    newYear) {
        year = newYear;
    }
}
```


Example (3 of 3)

```
public class Test {  
    public static void main(String[] args) {  
        Student student = new Student(111223333,  
            1970, 5, 3);  
        BirthDate date = student.getBirthDate();  
        date.setYear(2010); // Now the student  
            birth year is changed!  
    }  
}
```

What Class Is Immutable?

For a class to be immutable, it must mark all data fields private and provide no mutator methods and no accessor methods that would return a reference to a mutable data field object.

Scope of Variables

- The scope of instance and static variables is the entire class. They can be declared anywhere inside a class.
- The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable. A local variable must be initialized explicitly before it can be used.

The `this` Keyword

- The `this` keyword is the name of a reference that refers to an object itself. One common use of the `this` keyword is reference a class's **hidden data fields**.
- Another common use of the `this` keyword to enable a constructor to invoke another constructor of the same class.

Reference the Hidden Data Fields

```
public class F {  
    private int i = 5;  
    private static double k = 0;  
  
    void setI(int i) {  
        this.i = i;  
    }  
  
    static void setK(double k) {  
        F.k = k;  
    }  
}
```

Suppose that f1 and f2 are two objects of F.
F f1 = new F(); F f2 = new F();

Invoking f1.setI(10) is to execute
 this.i = 10, where **this** refers f1

Invoking f2.setI(45) is to execute
 this.i = 45, where **this** refers f2

Calling Overloaded Constructor

```
public class Circle {  
    private double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    public Circle() {  
        this(1.0);  
    }  
  
    public double getArea() {  
        return this.radius * this.radius * Math.PI;  
    }  
}
```

 this must be explicitly used to reference the data field radius of the object being constructed

 this is used to invoke another constructor

 Every instance variable belongs to an instance represented by this, which is normally omitted

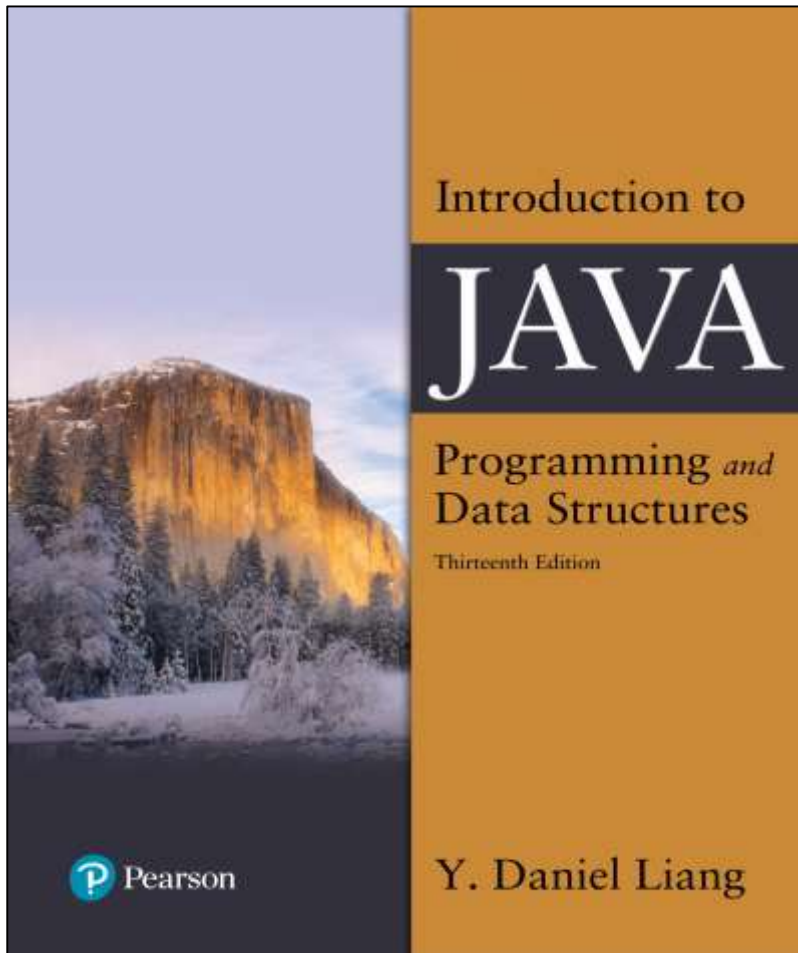
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 10

Thinking in Objects

Motivations

You see the advantages of object-oriented programming from the preceding chapter. This chapter will demonstrate how to solve problems using the object-oriented paradigm.

Objectives (1 of 2)

10.1 To apply class abstraction to develop software (§10.2).

10.2 To explore the differences between the procedural paradigm and object-oriented paradigm (§10.3).

10.3 To discover the relationships between classes (§10.4).

10.4 To design programs using the object-oriented paradigm (§§10.5–10.6).

10.5 To create objects for primitive values using the wrapper classes (**Byte**, **Short**, **Integer**, **Long**, **Float**, **Double**, **Character**, and **Boolean**) (§10.7).

10.6 To simplify programming using automatic conversion between primitive types and wrapper class types (§10.8).

10.7 To use the **BigInteger** and **BigDecimal** classes for computing very large numbers with arbitrary precisions (§10.9).

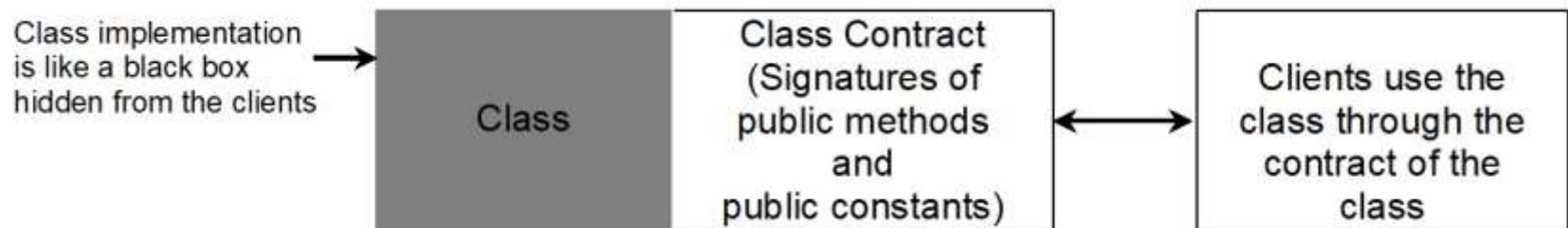
Objectives (2 of 2)

10.8 To use the **String** class to process immutable strings (§10.10).

10.9 To use the **StringBuilder** and **StringBuffer** classes to process mutable strings (§10.11).

Class Abstraction and Encapsulation

Class abstraction means to separate class implementation from the use of the class. The creator of the class provides a description of the class and let the user know how the class can be used. The user of the class does not need to know how the class is implemented. The detail of implementation is encapsulated and hidden from the user.



Designing the Loan Class

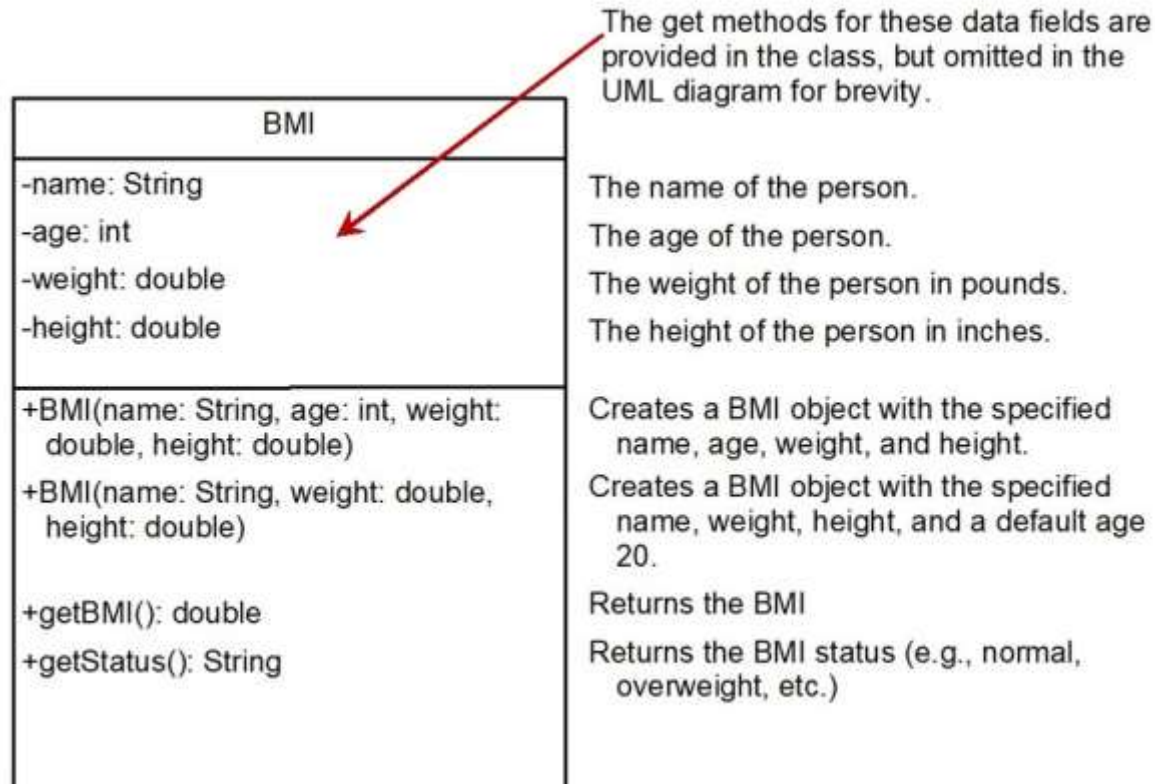
Loan	
-annualInterestRate: double	The annual interest rate of the loan (default: 2.5).
-numberOfYears: int	The number of years for the loan (default: 1)
-loanAmount: double	The loan amount (default: 1000).
-loanDate: Date	The date this loan was created.
+Loan()	Constructs a default Loan object.
+Loan(annualInterestRate: double, numberOfYears: int, loanAmount: double)	Constructs a loan with specified interest rate, years, and loan amount.
+getAnnualInterestRate(): double	Returns the annual interest rate of this loan.
+getNumberOfYears(): int	Returns the number of the years of this loan.
+getLoanAmount(): double	Returns the amount of this loan.
+getLoanDate(): Date	Returns the date of the creation of this loan.
+setAnnualInterestRate(annualInterestRate: double): void	Sets a new annual interest rate to this loan.
+setNumberOfYears(numberOfYears: int): void	Sets a new number of years to this loan.
+setLoanAmount(loanAmount: double): void	Sets a new amount to this loan.
+getMonthlyPayment(): double	Returns the monthly payment of this loan.
+getTotalPayment(): double	Returns the total payment of this loan.

[Loan](#) [TestLoanClass](#)

Object-Oriented Thinking

Chapters 1-8 introduced fundamental programming techniques for problem solving using loops, methods, and arrays. The studies of these techniques lay a solid foundation for object-oriented programming. Classes provide more flexibility and modularity for building reusable software. This section improves the solution for a problem introduced in Chapter 3 using the object-oriented approach. From the improvements, you will gain the insight on the differences between the procedural programming and object-oriented programming and see the benefits of developing reusable code using objects and classes.

The BMI Class



[BMI](#)

[UseBMIClass](#)

Class Relationships

Association

Aggregation

Composition

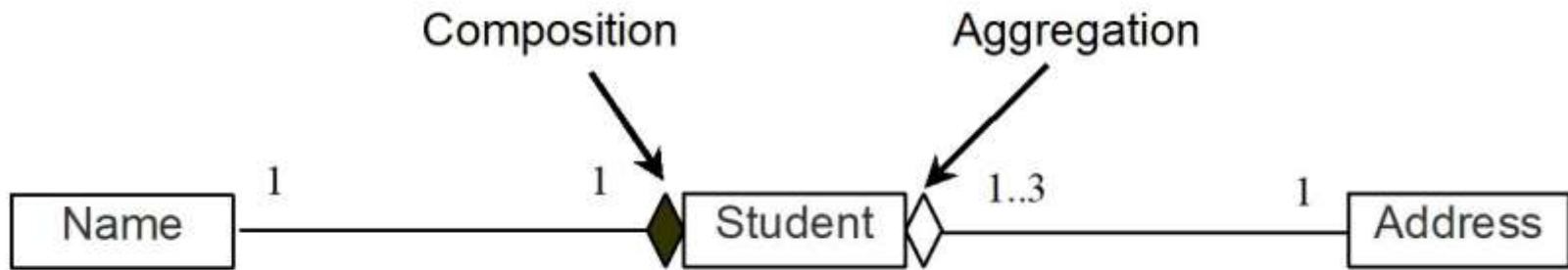
Inheritance (Chapter 13)

Association: is a general binary relationship that describes an activity between two classes.



Object Composition

Composition is actually a special case of the aggregation relationship. Aggregation models **has-a** relationships and represents an ownership relationship between two objects. The owner object is called an **aggregating object** and its class an **aggregating class**. The subject object is called an **aggregated object** and its class an **aggregated class**.



Class Representation

An aggregation relationship is usually represented as a data field in the aggregating class. For example, the relationship in Figure 10.6 can be represented as follows:

```
public class Name {  
    ...  
}
```

Aggregated class

```
public class Student {  
    private Name name;  
    private Address address;  
  
    ...  
}
```

Aggregating class

```
public class Address {  
    ...  
}
```

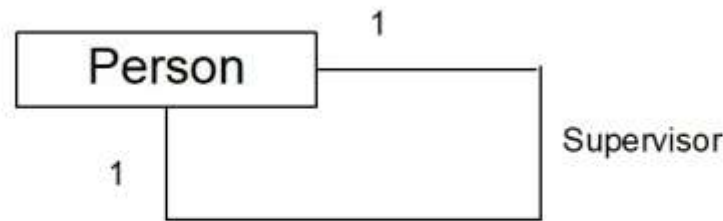
Aggregated class

Aggregation or Composition

Since aggregation and composition relationships are represented using classes in similar ways, many texts don't differentiate them and call both compositions.

Aggregation Between Same Class (1 of 2)

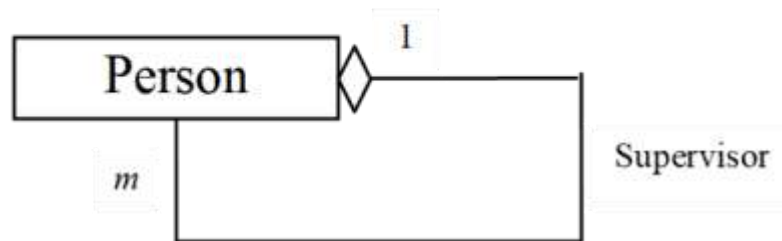
Aggregation may exist between objects of the same class. For example, a person may have a supervisor.



```
public class Person {  
    // The type for the data is the class itself  
    private Person supervisor;  
    ...  
}
```

Aggregation Between Same Class (2 of 2)

What happens if a person has several supervisors?



```
public class Person {  
    ...  
    private Person[] supervisors;  
}
```

Example: The Course Class

Course	
-courseName: String	The name of the course.
-students: String[]	An array to store the students for the course.
-numberOfStudents: int	The number of students (default: 0).
+Course(courseName: String)	Creates a course with the specified name.
+getCourseName(): String	Returns the course name.
+addStudent(student: String): void	Adds a new student to the course.
+dropStudent(student: String): void	Drops a student from the course.
+getStudents(): String[]	Returns the students in the course.
+getNumberOfStudents(): int	Returns the number of students in the course.

[Course](#)

[TestCourse](#)

Example: The StackOfIntegers Class

StackOfIntegers
-elements: int[]
-size: int
+StackOfIntegers()
+StackOfIntegers(capacity: int)
+empty(): boolean
+peek(): int
+push(value: int): int
+pop(): int
+getSize(): int

An array to store integers in the stack.

The number of integers in the stack.

Constructs an empty stack with a default capacity of 16.

Constructs an empty stack with a specified capacity.

Returns true if the stack is empty.

Returns the integer at the top of the stack without removing it from the stack.

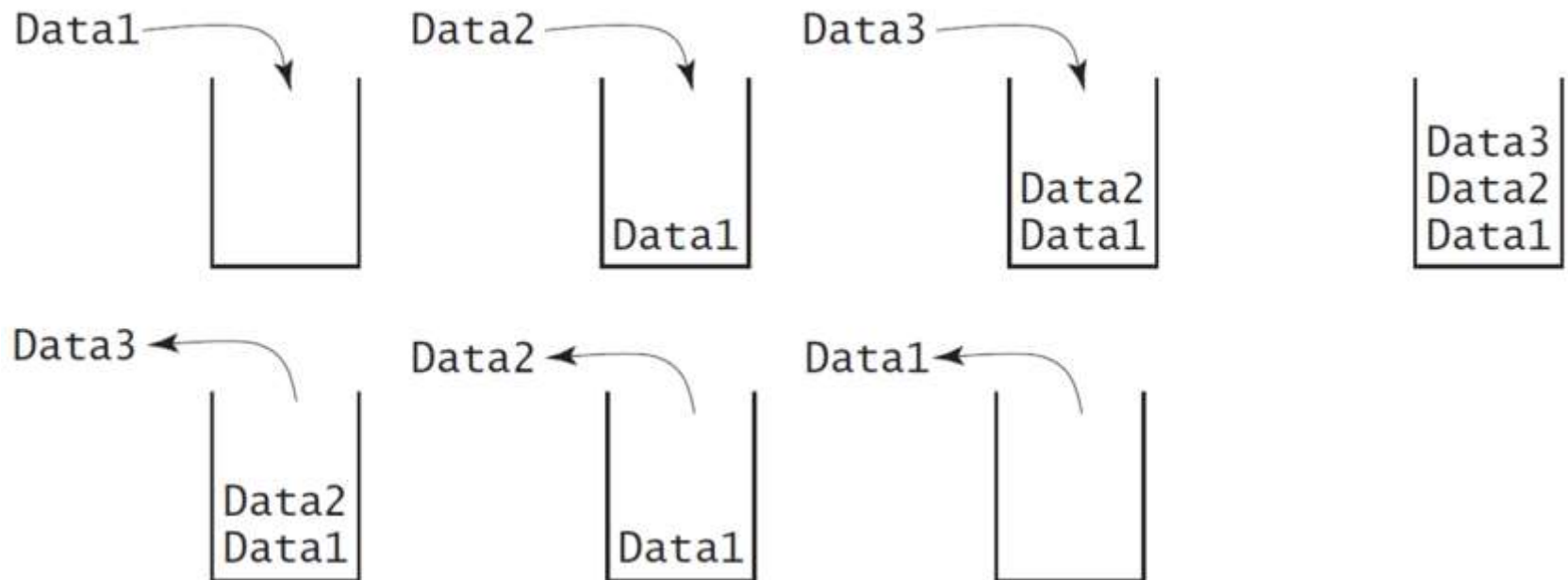
Stores an integer into the top of the stack.

Removes the integer at the top of the stack and returns it.

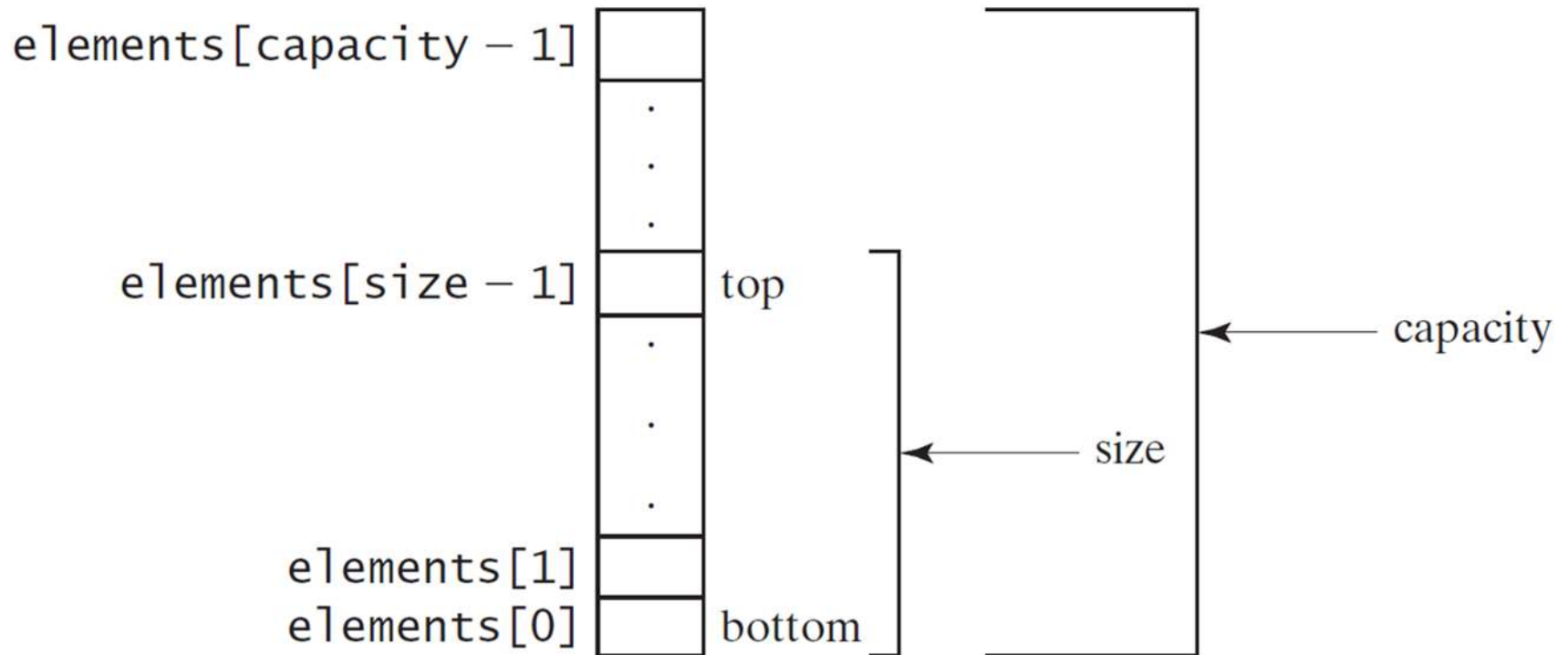
Returns the number of elements in the stack.

[TestStackOfIntegers](#)

Designing the Stackofintegers Class



Implementing Stackofintegers Class



[StackOfIntegers](#)

Wrapper Classes

- Boolean
- Character
- Short
- Byte
- Integer
- Long
- Float
- Double

Note: (1) The wrapper classes do not have no-arg constructors. (2) The instances of all wrapper classes are immutable, i.e., their internal values cannot be changed once the objects are created.

The Integer and Double Classes

java.lang.Integer
-value: int +MAX_VALUE: int +MIN_VALUE: int
+Integer(value: int) +Integer(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longVlaue(): long +floatValue(): float +doubleValue():double +compareTo(o: Integer): int +toString(): String +valueOf(s: String): Integer +valueOf(s: String, radix: int): Integer +parseInt(s: String): int +parseInt(s: String, radix: int): int

java.lang.Double
-value: double +MAX_VALUE: double +MIN_VALUE: double
+Double(value: double) +Double(s: String) +byteValue(): byte +shortValue(): short +intValue(): int +longVlaue(): long +floatValue(): float +doubleValue():double +compareTo(o: Double): int +toString(): String +valueOf(s: String): Double +valueOf(s: String, radix: int): Double +parseDouble(s: String): double +parseDouble(s: String, radix: int): double

The Integer Class and the Double Class

- Constructors
- Class Constants `MAX_VALUE`, `MIN_VALUE`
- Conversion Methods

Numeric Wrapper Class Constructors

You can construct a wrapper object either from a primitive data type value or from a string representing the numeric value. The constructors for `Integer` and `Double` **are**:

```
public Integer(int value)
public Integer(String s)
public Double(double value)
public Double(String s)
```

Numeric Wrapper Class Constants

Each numerical wrapper class has the constants **MAX_VALUE** and **MIN_VALUE**. **MAX_VALUE** represents the maximum value of the corresponding primitive data type. For **Byte**, **Short**, **Integer**, and **Long**, **MIN_VALUE** represents the minimum **byte**, **short**, **int**, and **long** values. For **Float** and **Double**, **MIN_VALUE** represents the minimum **positive float** and **double** values. The following statements display the maximum integer (2,147,483,647), the minimum positive float (1.4E-45), and the maximum double floating-point number (1.79769313486231570e+308d).

Conversion Methods

Each numeric wrapper class implements the abstract methods **doubleValue**, **floatValue**, **intValue**, **longValue**, and **shortValue**, which are defined in the **Number** class. These methods “convert” objects into primitive type values.

The Static `valueOf` Methods

The numeric wrapper classes have a useful class method, `valueOf(String s)`. This method creates a new object initialized to the value represented by the specified string. For example:

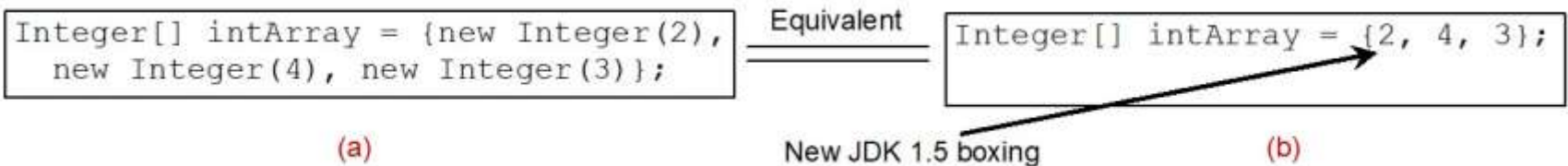
```
Double doubleObject = Double.valueOf("12.4");  
Integer integerObject = Integer.valueOf("12");
```


The Methods for Parsing Strings Into Numbers

You have used the `parseInt` method in the `Integer` class to parse a numeric string into an `int` value and the `parseDouble` method in the `Double` class to parse a numeric string into a `double` value. Each numeric wrapper class has two overloaded parsing methods to parse a numeric string into an appropriate numeric value.

Automatic Conversion Between Primitive Types and Wrapper Class Types

JDK 1.5 allows primitive type and wrapper classes to be converted automatically. For example, the following statement in (a) can be simplified as in (b):



Integer[] intArray = {1, 2, 3};
System.out.println(intArray[0] + intArray[1] + intArray[2]);

Unboxing

BigInteger and BigDecimal (1 of 2)

If you need to compute with very large integers or high precision floating-point values, you can use the **BigInteger** and **BigDecimal** classes in the **java.math** package. Both are **immutable**. Both extend the **Number** class and implement the **Comparable** interface.

BigInteger and BigDecimal (2 of 2)

```
BigInteger a = new BigInteger("9223372036854775807");
```

```
BigInteger b = new BigInteger("2");
```

```
BigInteger c = a.multiply(b); // 9223372036854775807 * 2
```

```
System.out.println(c);
```

```
BigDecimal a = new BigDecimal(1.0);
```

```
BigDecimal b = new BigDecimal(3);
```

```
BigDecimal c = a.divide(b, 20, BigDecimal.ROUND_UP);
```

```
System.out.println(c);
```

[LargeFactorial](#)

The String Class

- Constructing a String:

```
String message = "Welcome to Java";
```

```
String message = new String("Welcome to Java");
```

```
String s = new String();
```

- Obtaining String length and Retrieving Individual Characters in a string
- String Concatenation (concat)
- Substrings (substring(index), substring(start, end))
- Comparisons (equals, compareTo)
- String Conversions
- Finding a Character or a Substring in a String
- Conversions between Strings and Arrays
- Converting Characters and Numeric Values to Strings

Constructing Strings

```
String newString = new String(stringLiteral);  
String message = new String("Welcome to Java");
```

Since strings are used frequently, Java provides a shorthand initializer for creating a string:

```
String message = "Welcome to Java";
```

Strings Are Immutable

A String object is immutable; its contents cannot be changed. Does the following code change the contents of the string?

```
String s = "Java";
```

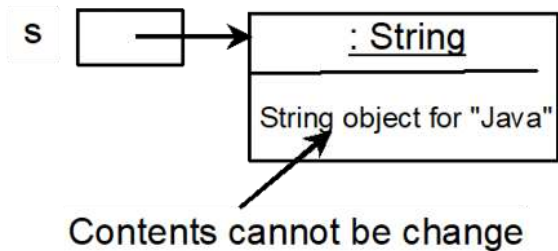
```
s = "HTML";
```

Trace Code (1 of 5)

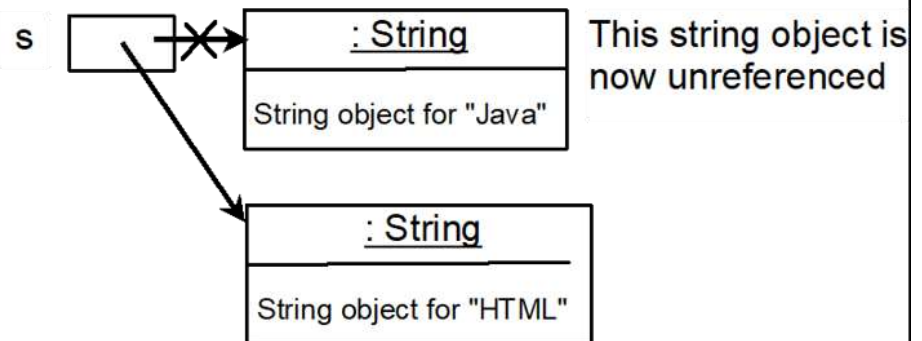
```
String s = "Java";
```

```
s = "HTML";
```

After executing `String s = "Java";`



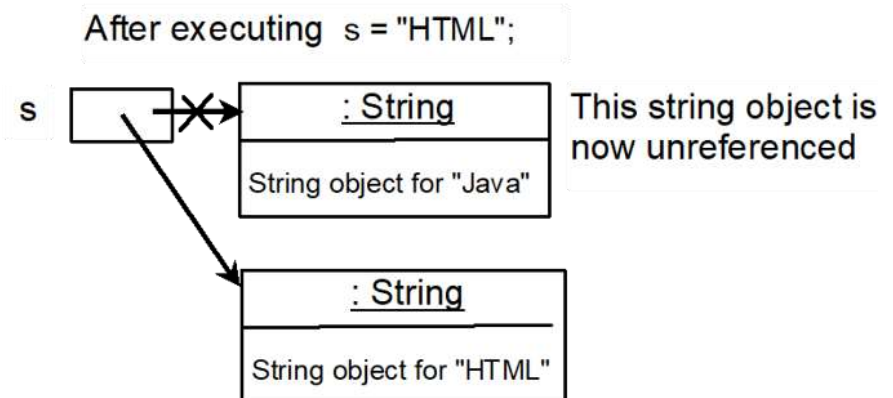
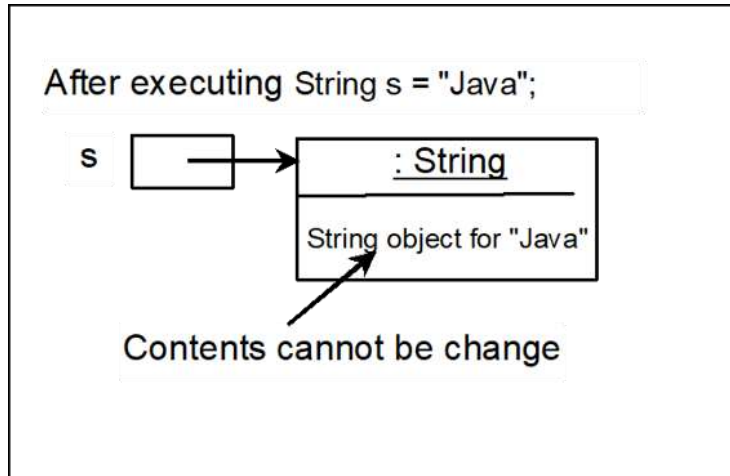
After executing `s = "HTML";`



Trace Code (2 of 5)

```
String s = "Java";
```

```
s = "HTML";
```

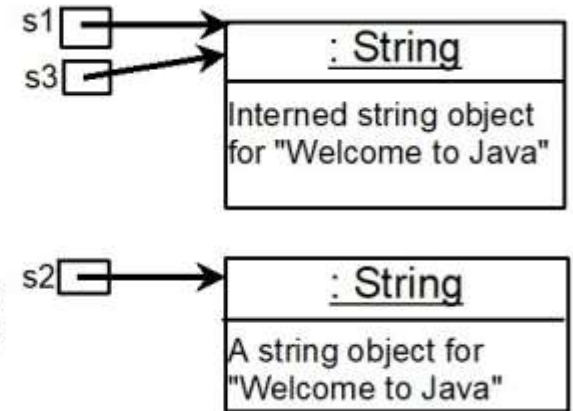


Interned Strings

Since strings are immutable and are frequently used, to improve efficiency and save memory, the JVM uses a unique instance for string literals with the same character sequence. Such an instance is called **interned**. For example, the following statements:

Examples (1 of 4)

```
String s1 = "Welcome to Java";  
String s2 = new String("Welcome to Java");  
String s3 = "Welcome to Java";  
System.out.println("s1 == s2 is " + (s1 == s2));  
System.out.println("s1 == s3 is " + (s1 == s3));
```



display

s1 == s2 is false

s1 == s3 is true

A new object is created if you use the new operator.

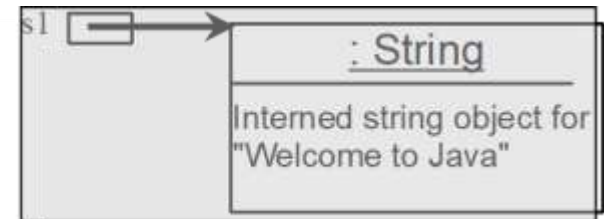
If you use the string initializer, no new object is created if the interned object is already created.

Trace Code (3 of 5)

```
String s1 = "Welcome to Java";
```

```
String s2 = new String("Welcome to Java");
```

```
String s3 = "Welcome to Java";
```

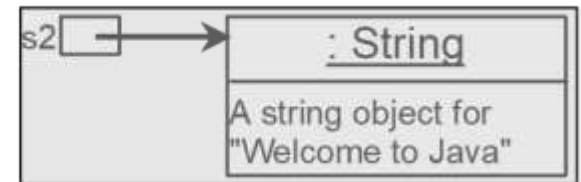
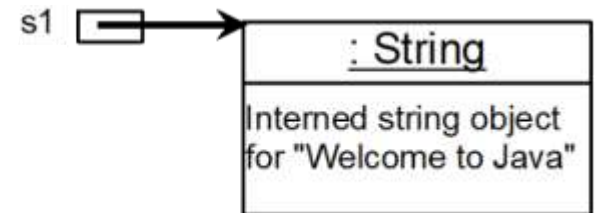


Trace Code (4 of 5)

```
String s1 = "Welcome to Java";
```

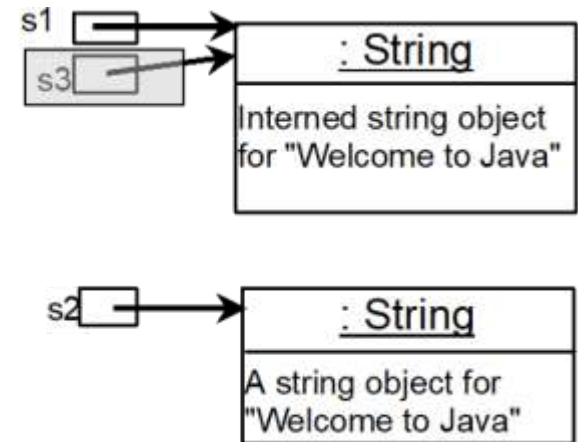
```
String s2 = new String("Welcome to Java");
```

```
String s3 = "Welcome to Java";
```



Trace Code (5 of 5)

```
String s1 = "Welcome to Java";  
String s2 = new String("Welcome to Java");  
String s3 = "Welcome to Java";
```



Replacing and Splitting Strings (1 of 2)

java.lang.String	
+replace(oldChar: char, newChar: char): String	Returns a new string that replaces all matching character in this string with the new character.
+replaceFirst(oldString: String, newString: String): String	Returns a new string that replaces the first matching substring in this string with the new substring.
+replaceAll(oldString: String, newString: String): String	Returns a new string that replace all matching substrings in this string with the new substring.
+split(delimiter: String): String[]	Returns an array of strings consisting of the substrings split by the delimiter.

Examples (2 of 4)

`"Welcome".replace('e', 'A')` returns a new string, `WAlcomA`.

`"Welcome".replaceFirst("e", "AB")` returns a new string, `WABlcome`.

`"Welcome".replace("e", "AB")` returns a new string, `WABlcomAB`.

`"Welcome".replace("el", "AB")` returns a new string, `WABcome`.

Splitting a String

```
String[] tokens =  
"Java#HTML#Perl".split("#", 0);  
  
for (int i = 0; i < tokens.length; i++)  
    System.out.print(tokens[i] + " ");
```

displays

Java HTML Perl

Matching, Replacing and Splitting by Patterns (1 of 3)

You can match, replace, or split a string by specifying a pattern. This is an extremely useful and powerful feature, commonly known as **regular expression**. Regular expression is complex to beginning students. For this reason, two simple patterns are used in this section. Please refer to Supplement III.F, “Regular Expressions,” for further studies.

```
"Java".matches("Java");
```

```
"Java".equals("Java");
```

```
"Java is fun".matches("Java.*");
```

```
"Java is cool".matches("Java.*");
```

Matching, Replacing and Splitting by Patterns (2 of 3)

The `replaceAll`, `replaceFirst`, and `split` methods can be used with a regular expression. For example, the following statement returns a new string that replaces `$`, `+`, or `#` in `"a+b$#c"` by the string `NNN`.

```
String s = "a+b$#c".replaceAll("[$+#]", "NNN");  
System.out.println(s);
```

Here the regular expression `[$+#]` specifies a pattern that matches `$`, `+`, or `#`. So, the output is `aNNNbNNNNNNc`.

Matching, Replacing and Splitting by Patterns (3 of 3)

The following statement splits the string into an array of strings delimited by some punctuation marks.

```
String[] tokens = "Java,C?C#,C++".split("[.,:;?]");  
for (int i = 0; i < tokens.length; i++)  
    System.out.println(tokens[i]);
```

Convert Character and Numbers to Strings

The `String` class provides several static `valueOf` methods for converting a character, an array of characters, and numeric values to strings. These methods have the same name `valueOf` with different argument types `char`, `char[]`, `double`, `long`, `int`, and `float`. For example, to convert a double value to a string, use `String.valueOf(5.44)`. The return value is string consists of characters `'5'`, `'.'`, `'4'`, and `'4'`.

StringBuilder and StringBuffer

The `StringBuilder/StringBuffer` class is an alternative to the `String` class. In general, a `StringBuilder/StringBuffer` can be used wherever a string is used. `StringBuilder/StringBuffer` is more flexible than `String`. You can add, insert, or append new contents into a string buffer, whereas the value of a `String` object is fixed once the string is created.

StringBuilder Constructors

java.lang.StringBuilder	
+StringBuilder()	Constructs an empty string builder with capacity 16.
+StringBuilder(capacity: int)	Constructs a string builder with the specified capacity.
+StringBuilder(s: String)	Constructs a string builder with the specified string.

Modifying Strings in the Builder

java.lang.StringBuilder	
+append(data: char[]): StringBuilder	Appends a char array into this string builder.
+append(data: char[], offset: int, len: int): StringBuilder	Appends a subarray in data into this string builder.
+append(v: <i>aPrimitiveType</i>): StringBuilder	Appends a primitive type value as a string to this builder.
+append(s: String): StringBuilder	Appends a string to this string builder.
+delete(startIndex: int, endIndex: int): StringBuilder	Deletes characters from startIndex to endIndex.
+deleteCharAt(index: int): StringBuilder	Deletes a character at the specified index.
+insert(index: int, data: char[], offset: int, len: int): StringBuilder	Inserts a subarray of the data in the array to the builder at the specified index.
+insert(offset: int, data: char[]): StringBuilder	Inserts data into this builder at the position offset.
+insert(offset: int, b: <i>aPrimitiveType</i>): StringBuilder	Inserts a value converted to a string into this builder.
+insert(offset: int, s: String): StringBuilder	Inserts a string into this builder at the position offset.
+replace(startIndex: int, endIndex: int, s: String): StringBuilder	Replaces the characters in this builder from startIndex to endIndex with the specified string.
+reverse(): StringBuilder	Reverses the characters in the builder.
+setCharAt(index: int, ch: char): void	Sets a new character at the specified index in this builder.

Examples (3 of 4)

```
StringBuilder.append("Java");  
StringBuilder.insert(11, "HTML and ");  
StringBuilder.delete(8, 11) changes the builder to Welcome  
Java.  
StringBuilder.deleteCharAt(8) changes the builder to  
Welcome o Java.  
StringBuilder.reverse() changes the builder to avaJ ot  
emocleW.  
StringBuilder.replace(11, 15, "HTML")  
    changes the builder to Welcome to HTML.  
StringBuilder.setCharAt(0, 'w') sets the builder to welcome  
to Java.
```

The toString, capacity, length, setLength, and charAt Methods

java.lang.StringBuilder	
+toString(): String	Returns a string object from the string builder.
+capacity(): int	Returns the capacity of this string builder.
+charAt(index: int): char	Returns the character at the specified index.
+length(): int	Returns the number of characters in this builder.
+setLength(newLength: int): void	Sets a new length in this builder.
+substring(startIndex: int): String	Returns a substring starting at startIndex.
+substring(startIndex: int, endIndex: int): String	Returns a substring from startIndex to endIndex-1.
+trimToSize(): void	Reduces the storage size used for the string builder.

Problem: Checking Palindromes Ignoring Non-alphanumeric Characters

This example gives a program that counts the number of occurrence of each letter in a string. Assume the letters are not case-sensitive.

[PalindromelgnoreNonAlphanumeric](#)

Regular Expressions

A **regular expression** (abbreviated **regex**) is a string that describes a pattern for matching a set of strings. Regular expression is a powerful tool for string manipulations. You can use regular expressions for matching, replacing, and splitting strings.

Matching Strings

```
"Java".matches("Java");
```

```
"Java".equals("Java");
```

```
"Java is fun".matches("Java.*")
```

```
"Java is cool".matches("Java.*")
```

```
"Java is powerful".matches("Java.*")
```

Regular Expression Syntax (1 of 2)

Regular Expression	Matches	Example
x	a specified character x	Java matches Java
.	any single character	Java matches J..a
(ab cd)	ab or cd	ten matches t(en im)
[abc]	a, b, or c	Java matches Ja[uvw]a
[^abc]	any character except a, b, or c	Java matches Ja[^ars]a
[a-z]	a through z	Java matches [A-M]av[a-d]
[^a-z]	any character except a through z	Java matches Jav[^b-d]
[a-e[m-p]]	a through e or m through p	Java matches [A-G[I-M]]av[a-d]
[a-e&& [c-p]]	intersection of a-e with c-p	Java matches [A-P&& [I-M]]av[a-d]
\d	a digit, same as [0-9]	Java2 matches "Java[\\d]"
\D	a non-digit	\$Java matches "[\\D][\\D]ava"
\w	a word character	Java1 matches "[\\w]ava[\\w]"
\W	a non-word character	\$Java matches "[\\W][\\w]ava"

Regular Expression Syntax (2 of 2)

Regular Expression	Matches	Example
<code>\s</code>	a whitespace character	"Java 2" matches " <code>Java\s2</code> "
<code>\S</code>	a non-whitespace char	Java matches " <code>[\\S]ava</code> "
<code>p*</code>	zero or more occurrences of pattern <i>p</i>	aaaabb matches " <code>a*b</code> " ababab matches " <code>(ab)*</code> "
<code>p+</code>	one or more occurrences of pattern <i>p</i>	a matches " <code>a+b*</code> " able matches " <code>(ab)+.*</code> "
<code>p?</code>	zero or one occurrence of pattern <i>p</i>	Java matches " <code>J?Java</code> " Java matches " <code>J?ava</code> "
<code>p{n}</code>	exactly <i>n</i> occurrences of pattern <i>p</i>	Java matches " <code>Ja{1}.*</code> " Java does not match " <code>.{2}</code> "
<code>p{n,}</code>	at least <i>n</i> occurrences of pattern <i>p</i>	aaaa matches " <code>a{1,}</code> " a does not match " <code>a{2,}</code> "
<code>p{n,m}</code>	between <i>n</i> and <i>m</i> occurrences (inclusive)	aaaa matches " <code>a{1,9}</code> " abb does not match " <code>a{2,9}bb</code> "

Replacing and Splitting Strings (2 of 2)

java.lang.String

+matches(regex: String): boolean

+replaceAll(regex: String,
replacement: String): String

+replaceFirst(regex: String,
replacement: String): String

+split(regex: String): String[]

Returns true if this string matches the pattern.

Returns a new string that replaces all matching substrings with the replacement.

Returns a new string that replaces the first matching substring with the replacement.

Returns an array of strings consisting of the substrings split by the matches.

Examples (4 of 4)

```
String s = "Java Java Java".replaceAll("v\\w", "wi") ;
```

```
String s = "Java Java Java".replaceFirst("v\\w", "wi") ;
```

```
String[] s = "Java1HTML2Perl".split("\\d");
```

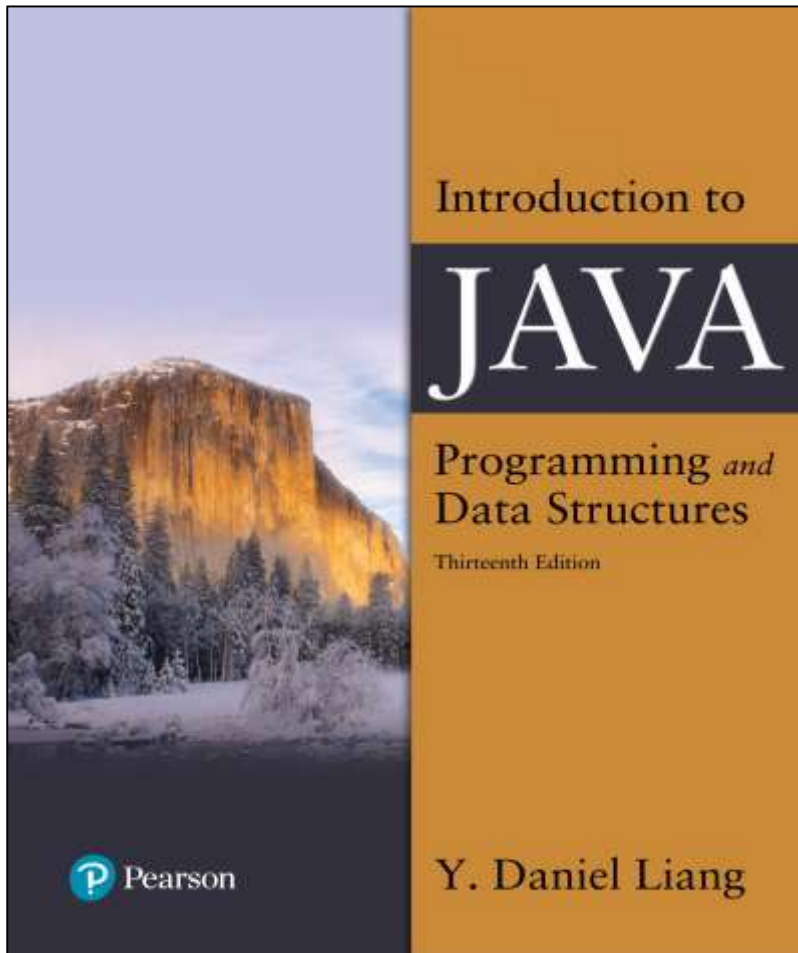
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 11

Inheritance and
Polymorphism

Motivations

Suppose you will define classes to model circles, rectangles, and triangles. These classes have many common features. What is the best way to design these classes so to avoid redundancy? The answer is to use inheritance.

Objectives (1 of 2)

11.1 To define a subclass from a superclass through inheritance (§11.2).

11.2 To invoke the superclass's constructors and methods using the **super** keyword (§11.3).

11.3 To override instance methods in the subclass (§11.4).

11.4 To distinguish differences between overriding and overloading (§11.5).

11.5 To explore the **toString()** method in the **Object** class (§11.6).

11.6 To discover polymorphism and dynamic binding (§§11.7–11.8).

11.7 To describe casting and explain why explicit downcasting is necessary (§11.9).

Objectives (2 of 2)

11.8 To explore the **equals** method in the **Object** class (§11.10).

11.9 To store, retrieve, and manipulate objects in an **ArrayList** (§11.11).

11.10 To construct an array list from an array, to sort and shuffle a list, and to obtain max and min element from a list (§11.12).

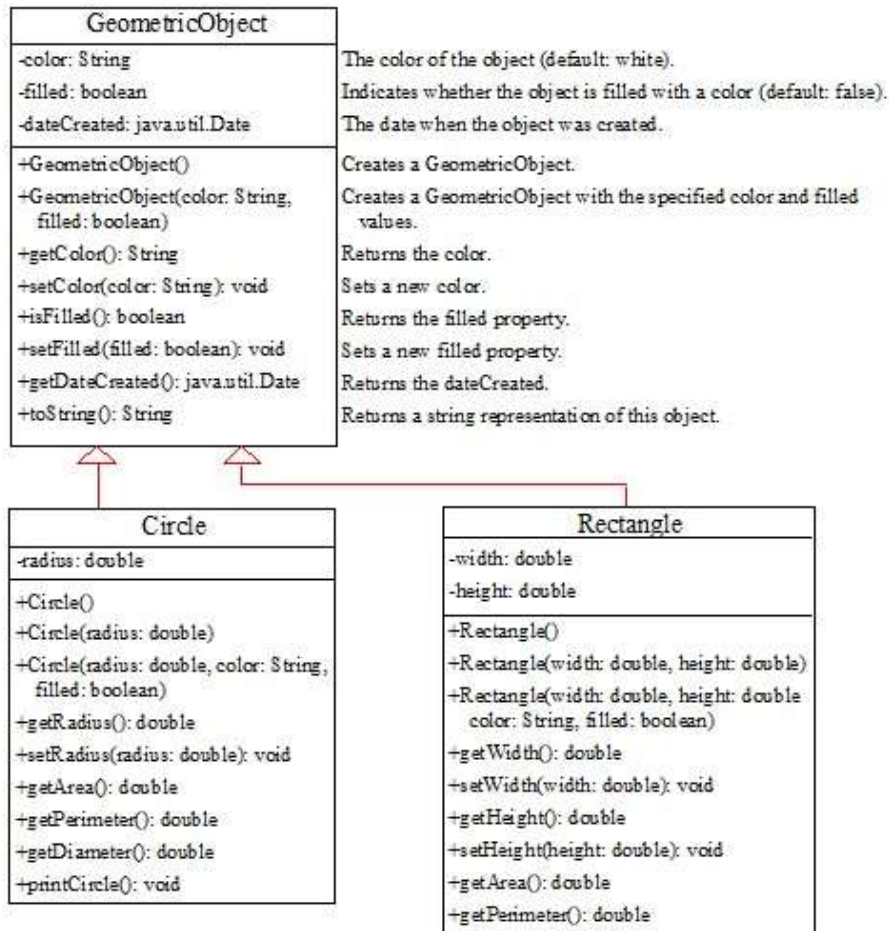
11.11 To implement a **Stack** class using **ArrayList** (§11.13).

11.12 To enable data and methods in a superclass accessible from subclasses using the **protected** visibility modifier (§11.14).

11.13 To prevent class extending and method overriding using the **final** modifier (§11.14).

11.14 To automatically generate boilerplate code using Lombok (§11.16).

Superclasses and Subclasses



[GeometricObject](#)

[Circle](#)

[Rectangle](#)

[TestCircleRectangle](#)

Are Superclass's Constructor Inherited?

No. They are not inherited.

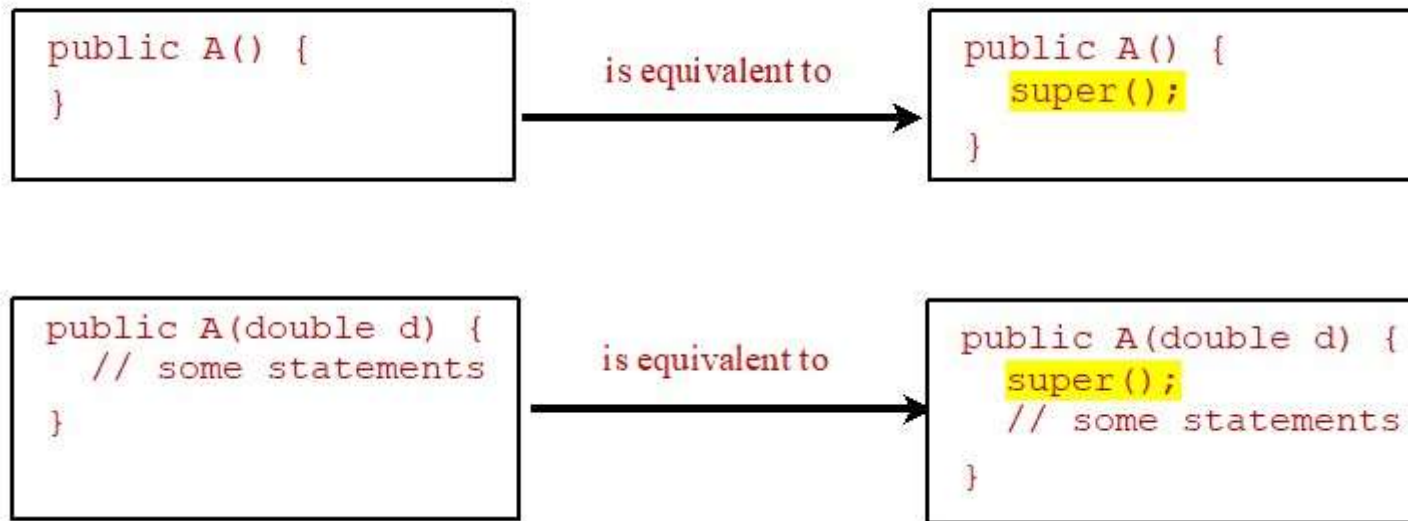
They are invoked explicitly or implicitly.

Explicitly using the super keyword.

A constructor is used to construct an instance of a class. Unlike properties and methods, a superclass's constructors are not inherited in the subclass. They can only be invoked from the subclasses' constructors, using the keyword super. **If the keyword super is not explicitly used, the superclass's no-arg constructor is automatically invoked.**

Superclass's Constructor Is Always Invoked

A constructor may invoke an overloaded constructor or its superclass's constructor. If none of them is invoked explicitly, the compiler puts **super()** as the first statement in the constructor. For example,



Using the Keyword `super`

The keyword `super` refers to the superclass of the class in which `super` appears. This keyword can be used in two ways:

- To call a superclass constructor
- To call a superclass method

Caution

You must use the keyword **super** to call the superclass constructor. Invoking a superclass constructor's name in a subclass causes a syntax error. Java requires that the statement that uses the keyword **super** appear first in the constructor.

Constructor Chaining (1 of 2)

Constructing an instance of a class invokes all the superclasses' constructors along the inheritance chain. This is known as **constructor chaining**.

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is  
        invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {
```

Constructor Chaining (2 of 2)

```
this(" (2) Invoke Employee's overloaded constructor");  
System.out.println(" (3) Employee's no-arg constructor is  
invoked");  
}  
public Employee(String s) {  
    System.out.println(s);  
}  
}  
class Person {  
    public Person() {  
        System.out.println(" (1) Person's no-arg constructor is  
invoked");  
    }  
}
```

Trace Execution (1 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

1. Start from the
main method

Trace Execution (2 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

2. Invoke Faculty
constructor

Trace Execution (3 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
}
```

```
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}
```

3. Invoke Employee's no-arg constructor

```
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
}
```

```
    public Employee(String s) {  
        System.out.println(s);  
    }  
}
```

```
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```


Trace Execution (4 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

4. Invoke Employee(String) constructor

Trace Execution (5 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

5. Invoke Person()
constructor

Trace Execution (6 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

6. Execute println

Trace Execution (7 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

7. Execute println

Trace Execution (8 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

8. Execute println

Trace Execution (9 of 9)

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

9. Execute println

Example on the Impact of a Superclass Without no-arg Constructor

Find out the errors in the program:

```
public class Apple extends Fruit {  
}  
class Fruit {  
    public Fruit(String name) {  
        System.out.println("Fruit's constructor is  
            invoked");  
    }  
}
```

Defining a Subclass

A subclass inherits from a superclass. You can also:

- Add new properties
- Add new methods
- Override the methods of the superclass

Calling Superclass Methods

You could rewrite the `printCircle()` method in the `Circle` class as follows:

```
public void printCircle() {  
    System.out.println("The circle is created " +  
        super.getDateCreated() + " and the radius is " + radius);  
}
```

Overriding Methods in the Superclass

A subclass inherits methods from a superclass. Sometimes it is necessary for the subclass to modify the implementation of a method defined in the superclass. This is referred to as **method overriding**.

```
public class Circle extends GeometricObject {  
    // Other methods are omitted  
  
    /** Override the toString method defined in  
        GeometricObject */  
    public String toString() {  
        return super.toString() + "\nradius is " + radius;  
    }  
}
```

Note (1 of 4)

An instance method can be overridden only if it is accessible. Thus a private method cannot be overridden, because it is not accessible outside its own class. If a method defined in a subclass is private in its superclass, the two methods are completely unrelated.

Note (2 of 4)

Like an instance method, a static method can be inherited. However, a static method cannot be overridden. If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden.

Overriding vs Overloading

```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overrides the method in B  
    public void p(double i) {  
        System.out.println(i);  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overloads the method in B  
    public void p(int i) {  
        System.out.println(i);  
    }  
}
```

The Object Class and Its Methods

Every class in Java is descended from the `java.lang.Object` class. If no inheritance is specified when a class is defined, the superclass of the class is `Object`.

```
public class Circle {  
    ...  
}
```

Equivalent

```
public class Circle extends Object {  
    ...  
}
```

The `toString()` Method in Object

The `toString()` method returns a string representation of the object. The default implementation returns a string consisting of a class name of which the object is an instance, the at sign (`@`), and a number representing this object.

```
Loan loan = new Loan();
```

```
System.out.println(loan.toString());
```

The code displays something like `Loan@15037e5` . This message is not very helpful or informative. Usually you should override the `toString` method so that it returns a digestible string representation of the object.

Polymorphism

Polymorphism means that a variable of a supertype can refer to a subtype object.

A class defines a type. A type defined by a subclass is called a **subtype**, and a type defined by its superclass is called a **supertype**. Therefore, you can say that **Circle** is a subtype of **GeometricObject** and **GeometricObject** is a supertype for **Circle**.

[PolymorphismDemo](#)

Polymorphism, Dynamic Binding and Generic Programming (1 of 2)

```
public class PolymorphismDemo {  
    public static void main(String[] args) {  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }  
  
    public static void m(Object x) {  
        System.out.println(x.toString());  
    }  
}  
  
class GraduateStudent extends Student {  
}  
  
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}  
  
class Person extends Object {  
    public String toString() {  
        return "Person";  
    }  
}
```

Method m takes a parameter of the Object type. You can invoke it with any object.

[DynamicBindingDemo](#)

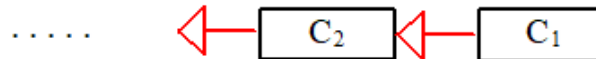
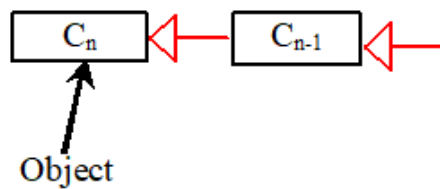
Polymorphism, Dynamic Binding and Generic Programming (2 of 2)

An object of a subtype can be used wherever its supertype value is required. This feature is known as **polymorphism**.

When the method `m(Object x)` is executed, the argument `x`'s `toString` method is invoked. `x` may be an instance of `GraduateStudent`, `Student`, `Person`, or `Object`. **Classes** `GraduateStudent`, `Student`, `Person`, and `Object` have their own implementation of the `toString` method. Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime. This capability is known as **dynamic binding**.

Dynamic Binding

Dynamic binding works as follows: Suppose an object o is an instance of classes $C_1, C_2, \dots, C_{n-1}$, and C_n , where C_1 is a subclass of C_2 , C_2 is a subclass of C_3 , ..., and C_{n-1} is a subclass of C_n . That is, C_n is the most general class, and C_1 is the most specific class. In Java, C_n is the Object class. If o invokes a method p , the JVM searches the implementation for the method p in $C_1, C_2, \dots, C_{n-1}$ and C_n , in this order, until it is found. Once an implementation is found, the search stops and the first-found implementation is invoked.



Since o is an instance of C_1 , o is also an instance of $C_2, C_3, \dots, C_{n-1}$, and C_n

Method Matching vs Binding

Matching a method signature and binding a method implementation are two issues. The compiler finds a matching method according to parameter type, number of parameters, and order of the parameters at compilation time. A method may be implemented in several subclasses. The Java Virtual Machine dynamically binds the implementation of the method at runtime.

Generic Programming (1 of 2)

```
public class
PolymorphismDemo {
    public static void
main(String[] args) {
    m(new GraduateStudent());
    m(new Student());
    m(new Person());
    m(new Object());
    }
    public static void
m(Object x) {
    System.out.println(x.toSt
ring());
    }
}
```

```
class GraduateStudent extends
Student {
}
class Student extends Person {
    public String toString() {
        return "Student";
    }
}
class Person extends Object {
    public String toString() {
        return "Person";
    }
}
```

Generic Programming (2 of 2)

Polymorphism allows methods to be used generically for a wide range of object arguments. This is known as generic programming. If a method's parameter type is a superclass (e.g., `Object`), you may pass an object to this method of any of the parameter's subclasses (e.g., `Student` or `String`). When an object (e.g., a `Student` object or a `String` object) is used in the method, the particular implementation of the method of the object that is invoked (e.g., `toString`) is determined dynamically.

Casting Objects

You have already used the casting operator to convert variables of one primitive type to another. **Casting** can also be used to convert an object of one class type to another within an inheritance hierarchy. In the preceding section, the statement

```
m(new Student());
```

assigns the object `new Student()` to a parameter of the `Object` type. This statement is equivalent to:

```
Object o = new Student(); // Implicit casting  
m(o);
```



The statement `Object o = new Student()`, known as implicit casting, is legal because an instance of `Student` is automatically an instance of `Object`.

Why Casting Is Necessary?

Suppose you want to assign the object reference `o` to a variable of the `Student` type using the following statement:

```
Student b = o;
```

A compile error would occur. Why does the statement `Object o = new Student()` work and the statement `Student b = o` doesn't? This is because a `Student` object is always an instance of `Object`, but an `Object` is not necessarily an instance of `Student`. Even though you can see that `o` is really a `Student` object, the compiler is not so clever to know it. To tell the compiler that `o` is a `Student` object, use an explicit casting. The syntax is similar to the one used for casting among primitive data types. Enclose the target object type in parentheses and place it before the object to be cast, as follows:

```
Student b = (Student)o; // Explicit casting
```


Casting From Superclass to Subclass

Explicit casting must be used when casting an object from a superclass to a subclass. This type of casting may not always succeed.

```
Apple x = (Apple)fruit;
```

```
Orange x = (Orange)fruit;
```

The instanceof Operator

Use the `instanceof` operator to test whether an object is an instance of a class:

```
Object myObject = new Circle();  
... // Some lines of code  
/** Perform casting if myObject is an instance of  
    Circle */  
if (myObject instanceof Circle) {  
    System.out.println("The circle diameter is " +  
        ((Circle)myObject).getDiameter());  
    ...  
}
```

Java 16 instanceof Pattern Matching

When using the `if` statement to test if an object is an instance of a class, you can specify a **binding variable**. If the result of the `instanceof` operator is true, then the object being tested is assigned to the binding variable. This new syntax, known as **`instanceof` pattern matching**, became a standard feature since Java 16. You can simplify the code in lines 15-24 using this new feature as follows:

Java 16 New Features on instanceof

```
if (myObject instanceof Circle circle) {  
    System.out.println("The circle area is " +  
        circle.getArea());  
    System.out.println("The circle diameter is " +  
        circle.getDiameter());  
}  
else if (myObject instanceof Rectangle rectangle) {  
    System.out.println("The rectangle area is " +  
        rectangle.getArea());  
}
```

Tip

To help understand casting, you may also consider the analogy of fruit, apple, and orange with the Fruit class as the superclass for Apple and Orange. An apple is a fruit, so you can always safely assign an instance of Apple to a variable for Fruit. However, a fruit is not necessarily an apple, so you have to use explicit casting to assign an instance of Fruit to a variable of Apple.

Example: Demonstrating Polymorphism and Casting

This example creates two geometric objects: a circle, and a rectangle, invokes the `displayGeometricObject` method to display the objects. The `displayGeometricObject` displays the area and diameter if the object is a circle, and displays area if the object is a rectangle.

[CastingDemo](#)

The equals Method

The `equals()` method compares the contents of two objects. The default implementation of the `equals` method in the `Object` class is as follows:

```
public boolean equals(Object obj) {  
    return this == obj;  
}
```

For example,
the `equals`
method is
overridden in
the `Circle`
class.

```
public boolean equals(Object o) {  
    if (o instanceof Circle) {  
        return radius == ((Circle)o).radius;  
    }  
    else  
        return false;  
}
```

Note (3 of 4)

The `==` comparison operator is used for comparing two primitive data type values or for determining whether two objects have the same references. The `equals` method is intended to test whether two objects have the same contents, provided that the method is modified in the defining class of the objects. The `==` operator is stronger than the `equals` method, in that the `==` operator checks whether the two reference variables refer to the same object.

The ArrayList Class

You can create an array to store objects. But the array's size is fixed once the array is created. Java provides the `ArrayList` class that can be used to store an unlimited number of objects.

java.util.ArrayList<E>	
<code>+ArrayList()</code>	Creates an empty list.
<code>+add(o: E) : void</code>	Appends a new element <code>o</code> at the end of this list.
<code>+add(index: int, o: E) : void</code>	Adds a new element <code>o</code> at the specified index in this list.
<code>+clear(): void</code>	Removes all the elements from this list.
<code>+contains(o: Object): boolean</code>	Returns true if this list contains the element <code>o</code> .
<code>+get(index: int) : E</code>	Returns the element from this list at the specified index.
<code>+indexOf(o: Object) : int</code>	Returns the index of the first matching element in this list.
<code>+isEmpty(): boolean</code>	Returns true if this list contains no elements.
<code>+lastIndexOf(o: Object) : int</code>	Returns the index of the last matching element in this list.
<code>+remove(o: Object): boolean</code>	Removes the element <code>o</code> from this list.
<code>+size(): int</code>	Returns the number of elements in this list.
<code>+remove(index: int) : boolean</code>	Removes the element at the specified index.
<code>+set(index: int, o: E) : E</code>	Sets the element at the specified index.

Generic Type

`ArrayList` is known as a generic class with a generic type `E`. You can specify a concrete type to replace `E` when creating an `ArrayList`. For example, the following statement creates an `ArrayList` and assigns its reference to variable `cities`. This `ArrayList` object can be used to store strings.

```
ArrayList<String> cities = new ArrayList<String>();
```

```
ArrayList<String> cities = new ArrayList<>();
```

[TestArrayList](#)

Differences and Similarities Between Arrays and ArrayList

Operation	Array	ArrayList
Creating an array/ArrayList	<pre>String[] a = new String[10]</pre>	<pre>ArrayList<String> list = new ArrayList<>();</pre>
Accessing an element	<pre>a[index]</pre>	<pre>list.get(index);</pre>
Updating an element	<pre>a[index] = "London";</pre>	<pre>list.set(index, "London");</pre>
Returning size	<pre>a.length</pre>	<pre>list.size();</pre>
Adding a new element	-	<pre>list.add("London");</pre>
Inserting a new element	-	<pre>list.add(index, "London");</pre>
Removing an element	-	<pre>list.remove(index);</pre>
Removing an element	-	<pre>list.remove(Object);</pre>
Removing all elements	-	<pre>list.clear();</pre>

DistinctNumbers

Array Lists From/to Arrays

Creating an ArrayList from an array of objects:

```
String[] array = {"red", "green", "blue"};
```

```
ArrayList<String> list = new  
ArrayList<>(Arrays.asList(array));
```

Creating an array of objects from an ArrayList:

```
String[] array1 = new String[list.size()];  
list.toArray(array1);
```

max and min in an Array List

```
String[] array = {"red", "green", "blue"};
```

```
System.out.println(java.util.Collections.max(  
    new ArrayList<String>(Arrays.asList(array))));
```

```
String[] array = {"red", "green", "blue"};
```

```
System.out.println(java.util.Collections.min(  
    new ArrayList<String>(Arrays.asList(array))));
```

Shuffling an Array List

```
Integer[] array = {3, 5, 95, 4, 15, 34, 3, 6, 5};  
ArrayList<Integer> list = new  
    ArrayList<>(Arrays.asList(array));  
java.util.Collections.shuffle(list);  
System.out.println(list);
```

Stack Animation

<https://liveexample.pearsoncmg.com/dsanimation/StackBook.html>

Stack Animation by Y. Daniel Liang

Enter a value and click the Push button to push the value into the stack. Click the Pop button to remove the top element from the stack.

Top →

5
3
3
4

Enter a value:

The MyStack Classes

A stack to hold objects.

MyStack

MyStack	
-list: ArrayList	A list to store elements.
+isEmpty(): boolean	Returns true if this stack is empty.
+getSize(): int	Returns the number of elements in this stack.
+peek(): Object	Returns the top element in this stack.
+pop(): Object	Returns and removes the top element in this stack.
+push(o: Object): void	Adds a new element to the top of this stack.
+search(o: Object): int	Returns the position of the first element in the stack from the top that matches the specified element.

The `protected` Modifier

- The `protected` modifier can be applied on data and methods in a class. A protected data or a protected method in a public class can be accessed by any class in the same package or its subclasses, even if the subclasses are in a different package.
- `private`, `default`, `protected`, `public`

Visibility increases
—————→
`private, none (if no modifier is used), protected, public`

Accessibility Summary

Modifier on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass	Accessed from a different package
public	✓	✓	✓	✓
protected	✓	✓	✓	-
default	✓	✓	-	-
private	✓	-	-	-

Visibility Modifiers



A Subclass Cannot Weaken the Accessibility

A subclass may override a protected method in its superclass and change its visibility to public. However, a subclass cannot weaken the accessibility of a method defined in the superclass. For example, if a method is defined as public in the superclass, it must be defined as public in the subclass.

Note (4 of 4)

The modifiers are used on classes and class members (data and methods), except that the **final** modifier can also be used on local variables in a method. A final local variable is a constant inside a method.

The `final` Modifier

- The `final` class cannot be extended:

```
final class Math {  
  
    ...  
  
}
```

- The `final` variable is a constant:

```
final static double PI = 3.14159;
```

- The `final` method cannot be overridden by its subclasses.

Lombok: Generating Boilerplate Code Using Annotations

When you write Java code, there are lot of getter and setter methods. It is tedious to write all these boilerplate code.

Project Lombok comes to rescue. Project Lombok is a Java library that provides annotations to tell the Java compiler to automatically generate boilerplate code such as getter and setter methods for data fields. To use Lombok, you need download a jar file named lombok.jar from

<https://projectlombok.org/download> .

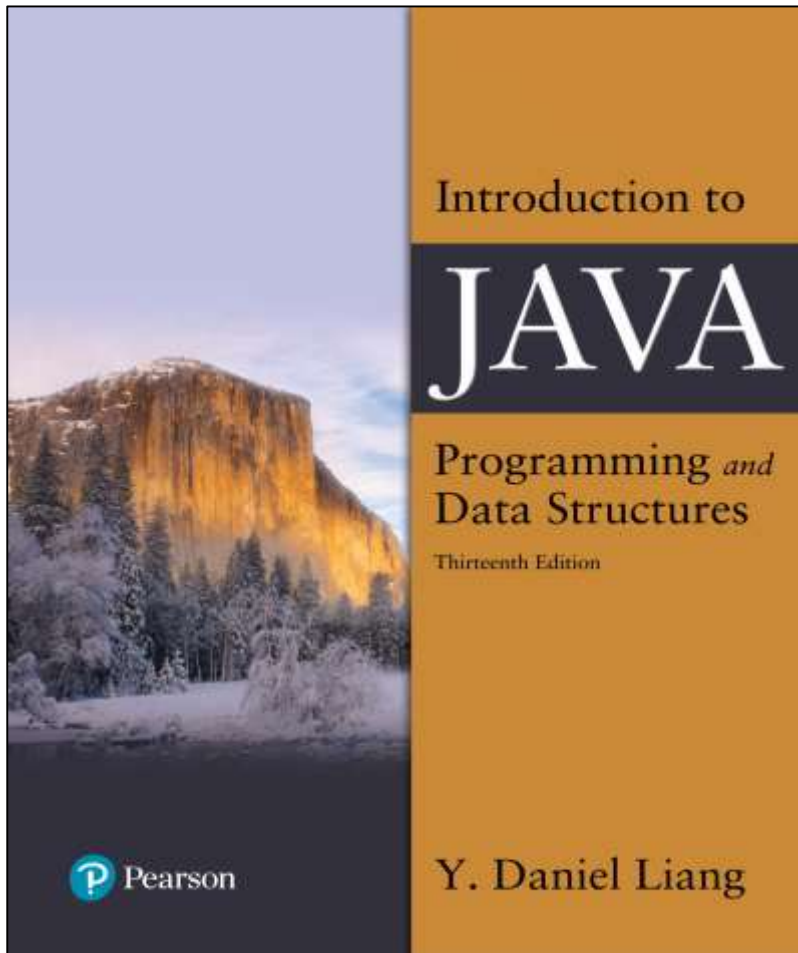
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 12

Exception Handling and
Text IO

Motivations

When a program runs into a runtime error, the program terminates abnormally. How can you handle the runtime error so that the program can continue to run or terminate gracefully? This is the subject we will introduce in this chapter.

Objectives (1 of 4)

12.1 To get an overview of exceptions and exception handling (§12.2).

12.2 To explore the advantages of using exception handling (§12.2).

12.3 To distinguish exception types: **Error** (fatal) vs **Exception** (nonfatal) and checked versus unchecked (§12.3).

12.4 To declare exceptions in a method header (§12.4.1).

12.5 To throw exceptions in a method (§12.4.2).

12.6 To write a **try-catch** block to handle exceptions (§12.4.3).

12.7 To explain how an exception is propagated (§12.4.3).

Objectives (2 of 4)

12.8 To obtain information from an exception object (§12.4.4).

12.9 To develop applications with exception handling (§12.4.5).

12.10 To use the **finally** clause in a **try-catch** block (§12.5).

12.11 To use exceptions only for unexpected errors (§12.6).

12.12 To rethrow exceptions in a **catch** block (§12.7).

12.13 To create chained exceptions (§12.8).

12.14 To define custom exception classes (§12.9).

Objectives (3 of 4)

12.15 To discover file/directory properties, to delete and rename files/directories, and to create directories using the **File** class (§12.10).

12.16 To write data to a file using the **PrintWriter** class (§12.11.1).

12.17 To use try-with-resources to ensure that the resources are closed automatically (§12.11.2).

12.18 To read data from a file using the **Scanner** class (§12.11.3).

12.19 To understand how data is read using a **Scanner** (§12.11.4).

12.20 To develop a program that replaces text in a file (§12.11.5).

12.21 To read data from the Web (§12.12).

Objectives (4 of 4)

- **12.22** To develop a Web crawler (§12.13).

Exception-Handling Overview

Show runtime error

Quotient

Fix it using an `if` statement

QuotientWithIf

With a method

QuotientWithMethod

Exception Advantages

QuotientWithException

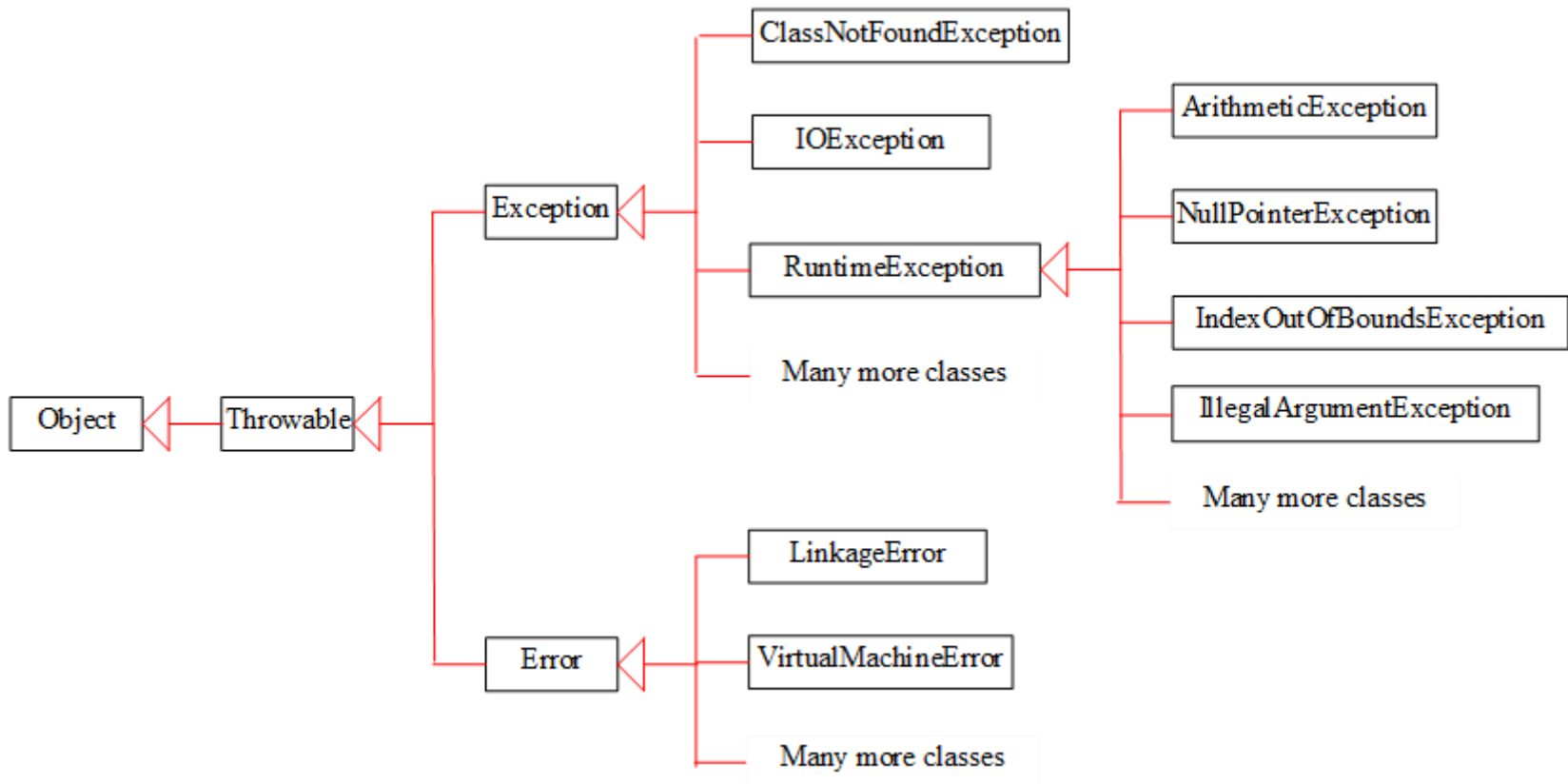
Now you see the **advantages** of using exception handling. It enables a method to throw an exception to its caller. Without this capability, a method must handle the exception or terminate the program.

Handling `InputMismatchException`

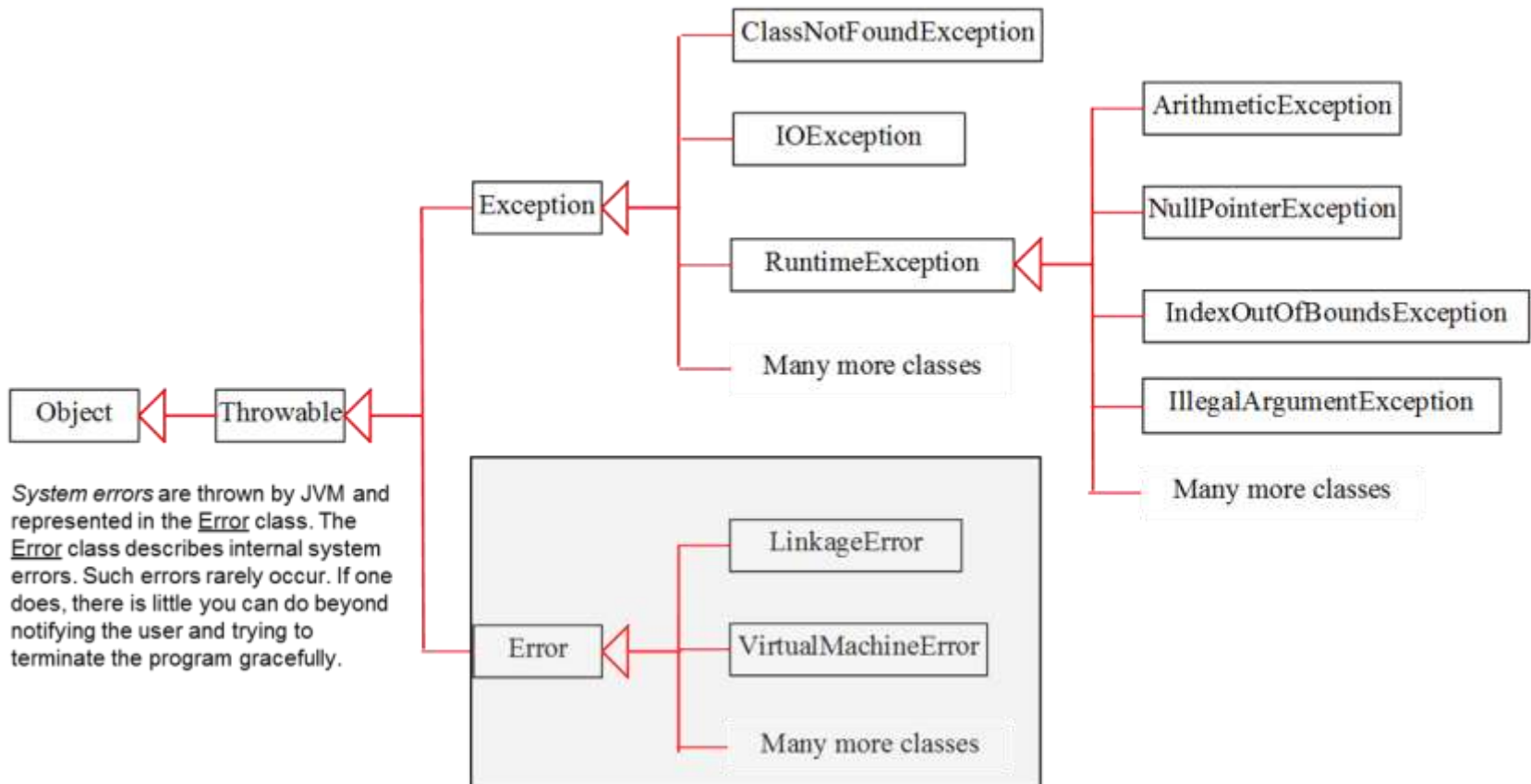
`InputMismatchExceptionDemo`

By handling `InputMismatchException`, your program will continuously read an input until it is correct.

Exception Types

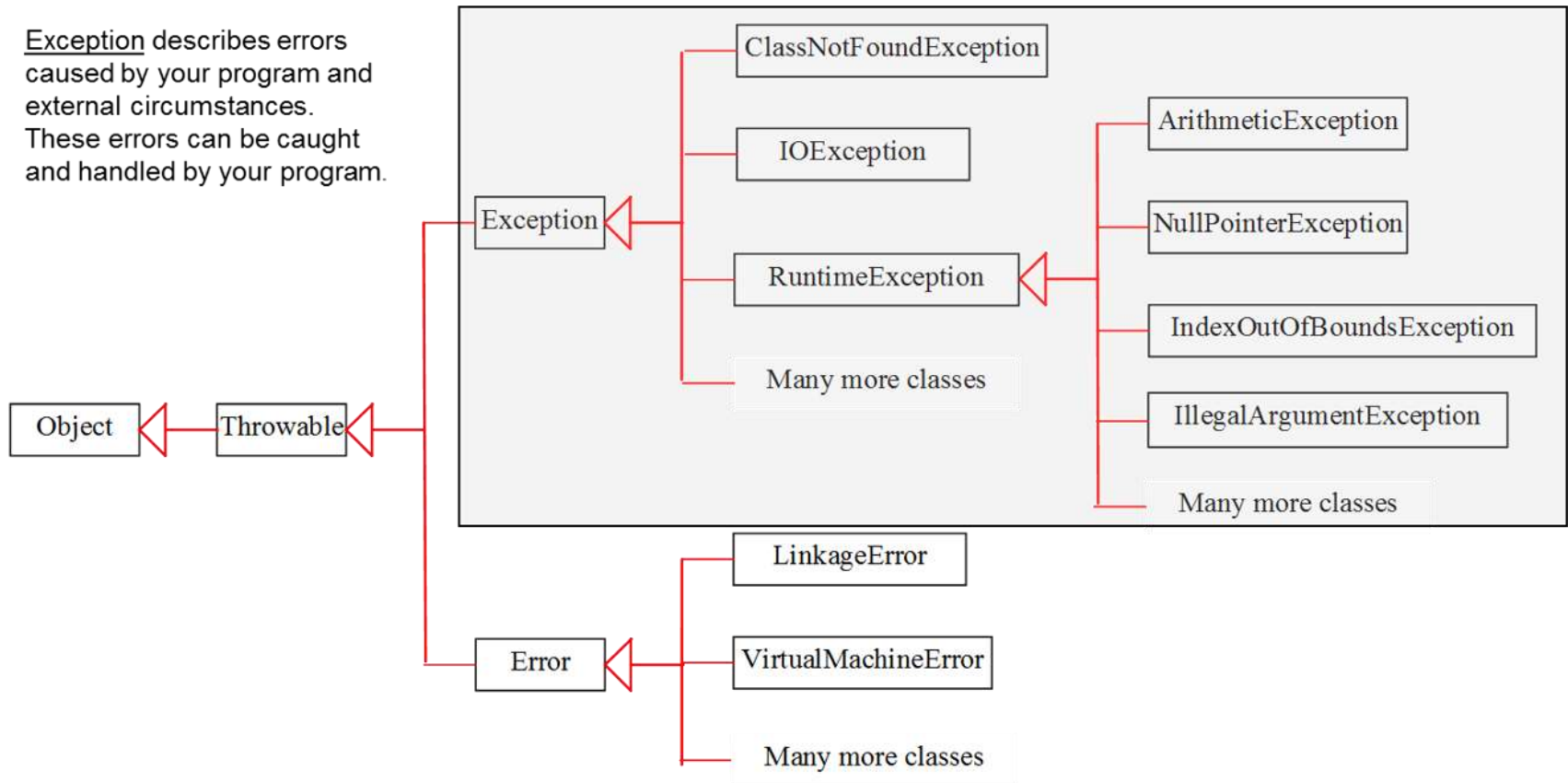


System Errors

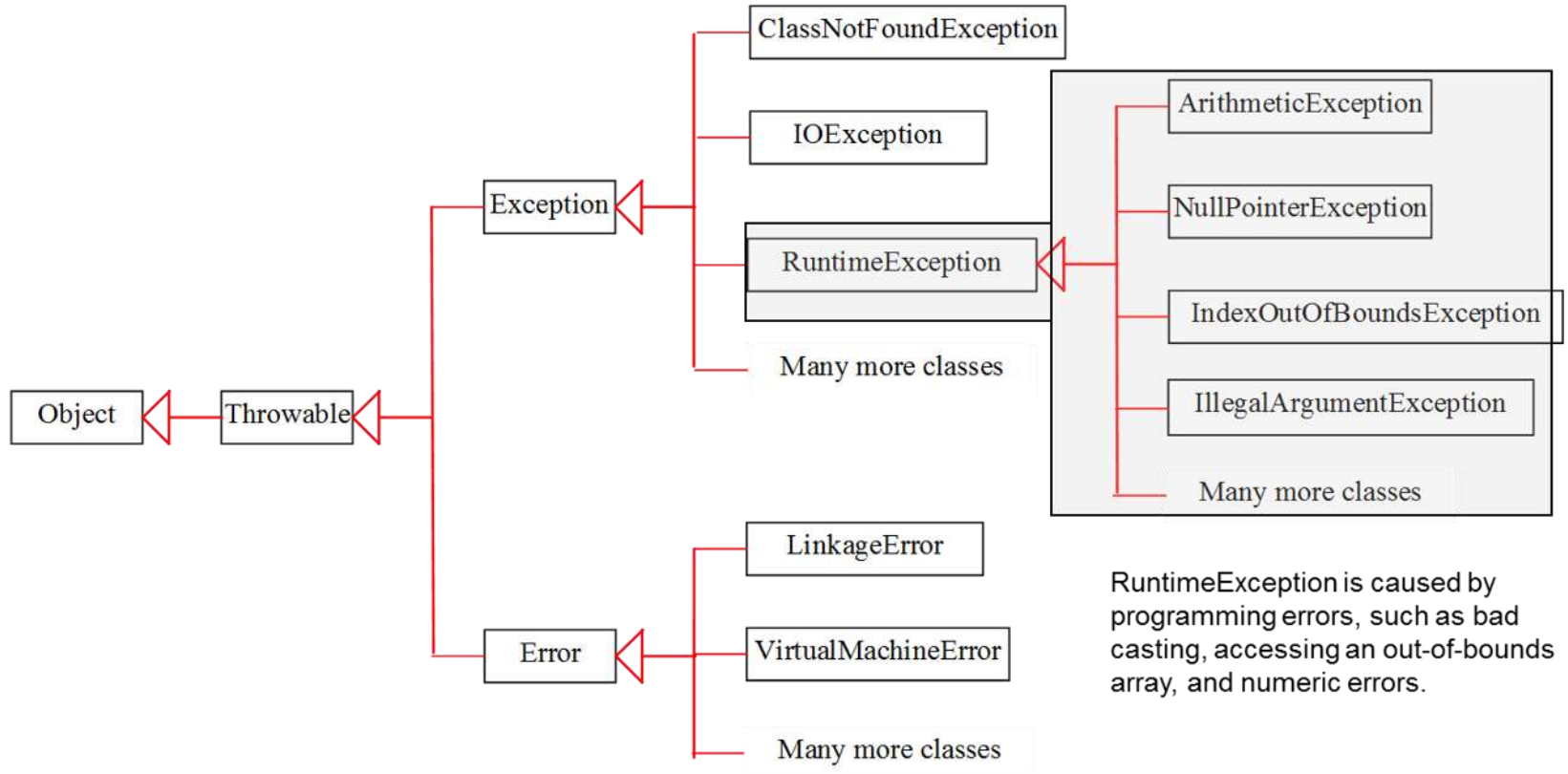


Exceptions

Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



Runtime Exceptions



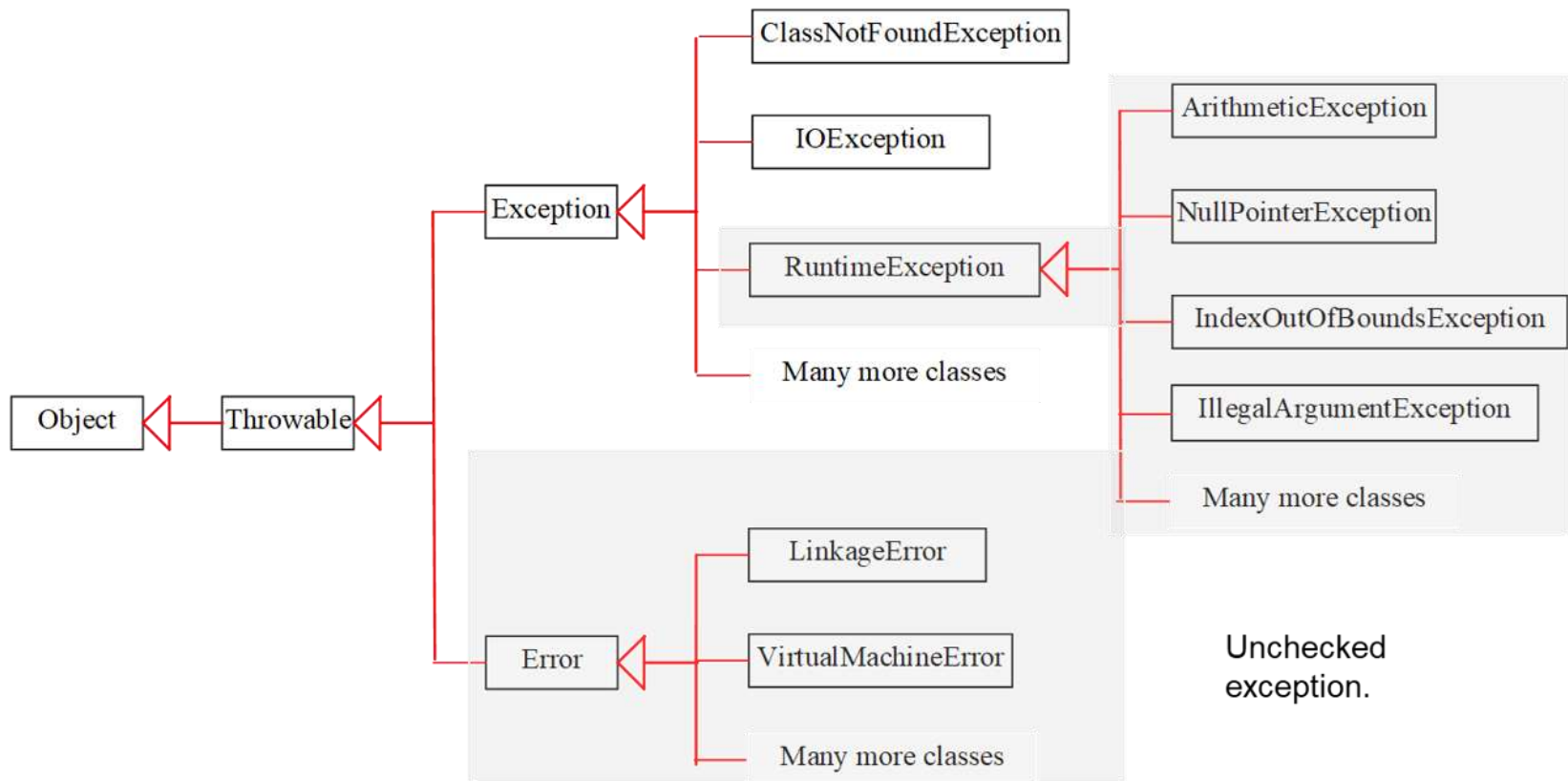
Checked Exceptions v.s Unchecked Exceptions

RuntimeException, **Error** and their subclasses are known as **unchecked exceptions**. All other exceptions are known as **checked exceptions**, meaning that the compiler forces the programmer to check and deal with the exceptions.

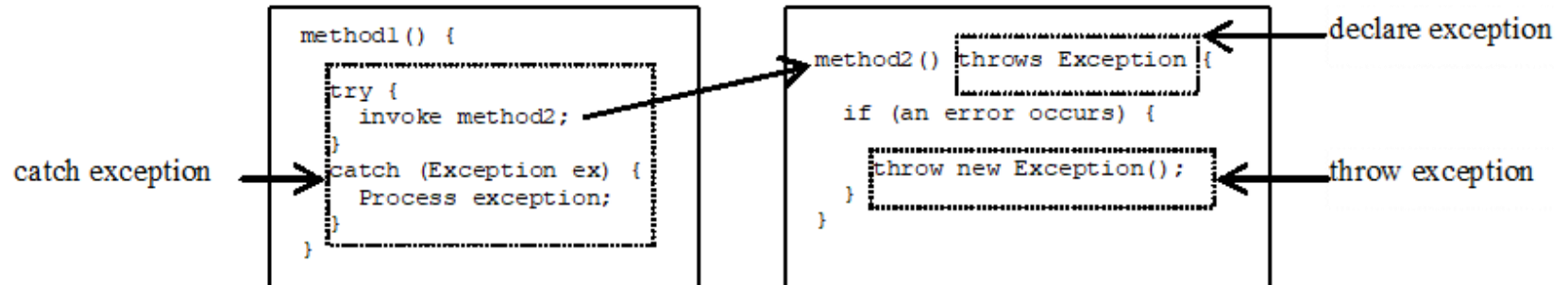
Unchecked Exceptions (1 of 2)

In most cases, unchecked exceptions reflect programming logic errors that are not recoverable. For example, a **`NullPointerException`** is thrown if you access an object through a reference variable before an object is assigned to it; an **`IndexOutOfBoundsException`** is thrown if you access an element in an array outside the bounds of the array. These are the logic errors that should be corrected in the program. Unchecked exceptions can occur anywhere in the program. To avoid cumbersome overuse of try-catch blocks, Java does not mandate you to write code to catch unchecked exceptions.

Unchecked Exceptions (2 of 2)



Declaring, Throwing, and Catching Exceptions



Declaring Exceptions

Every method must state the types of checked exceptions it might throw. This is known as **declaring exceptions**.

```
public void myMethod()  
    throws IOException
```

```
public void myMethod()  
    throws IOException, OtherException
```

Throwing Exceptions

When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as **throwing an exception**. Here is an example,

```
throw new TheException();
```

```
TheException ex = new TheException();  
throw ex;
```

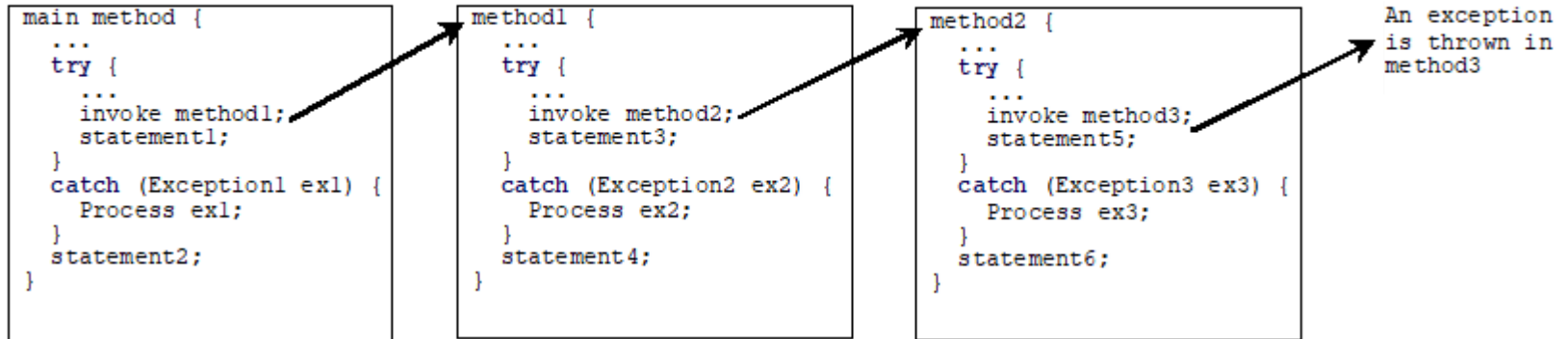
Throwing Exceptions Example

```
/** Set a new radius */  
public void setRadius(double newRadius)  
    throws IllegalArgumentException {  
    if (newRadius >= 0)  
        radius = newRadius;  
    else  
        throw new IllegalArgumentException(  
            "Radius cannot be negative");  
}
```

Catching Exceptions (1 of 2)

```
try {  
    statements; // Statements that may throw exceptions  
}  
catch (Exception1 exVar1) {  
    handler for exception1;  
}  
catch (Exception2 exVar2) {  
    handler for exception2;  
}  
...  
catch (ExceptionN exVar3) {  
    handler for exceptionN;  
}
```

Catching Exceptions (2 of 2)



Call Stack



Catch or Declare Checked Exceptions (1 of 2)

Suppose p2 is defined as follows:

```
void p2() throws IOException {  
    if (a file does not exist) {  
        throw new IOException("File does not exist");  
    }  
  
    ...  
}
```

Catch or Declare Checked Exceptions (2 of 2)

Java forces you to deal with checked exceptions. If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method. For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1() {  
    try {  
        p2();  
    }  
    catch (IOException ex) {  
        ...  
    }  
}
```

(a)

```
void p1() throws IOException {  
    p2();  
}
```

(b)

Example: Declaring, Throwing, and Catching Exceptions

- Objective: This example demonstrates declaring, throwing, and catching exceptions by modifying the **setRadius** method in the Circle class defined in Chapter 9. The new **setRadius** method throws an exception if radius is negative.

[CircleWithException](#)

[TestCircleWithException](#)

Rethrowing Exceptions

```
try {  
    statements;  
}  
catch(TheException ex) {  
    perform operations before exits;  
    throw ex;  
}
```

The finally Clause

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Trace a Program Execution (1 of 11)

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Suppose no exceptions in the statements

Next statement;

Trace a Program Execution (2 of 11)

```
try {  
    statements;  
}  
catch(TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

The final block is
always executed

Next statement;

Trace a Program Execution (3 of 11)

```
try {  
    statements;  
}  
catch (TheException ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Next statement in
the method is
executed

Next statement;

Trace a Program Execution (4 of 11)

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

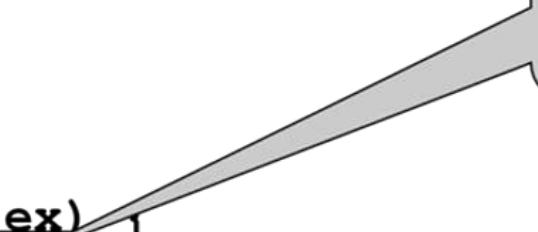
Suppose an exception
of type Exception1 is
thrown in statement2

Next statement;

Trace a Program Execution (5 of 11)

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Next statement;



The exception is handled.

Trace a Program Execution (6 of 11)

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

The final block is
always executed.

Next statement;

Trace a Program Execution (7 of 11)

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch (Exception1 ex) {  
    handling ex;  
}  
finally {  
    finalStatements;  
}
```

Next statement;

The next statement in the method is now executed.

Trace a Program Execution (8 of 11)

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
catch(Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    finalStatements;  
}
```

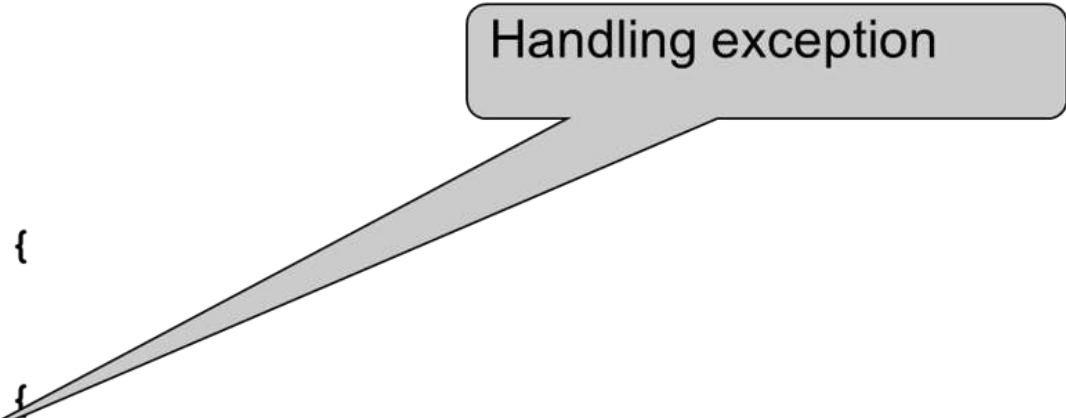
statement2 throws
an exception of type
Exception2.

Next statement;

Trace a Program Execution (9 of 11)

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
catch(Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    finalStatements;  
}
```

Next statement;

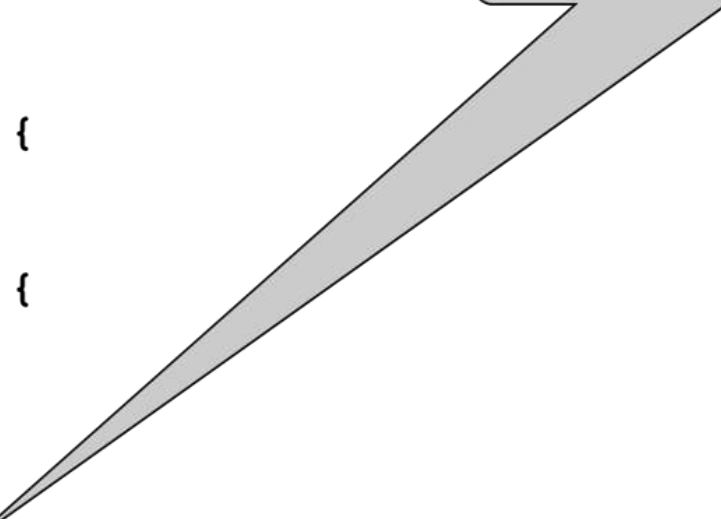


Handling exception

Trace a Program Execution (10 of 11)

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
catch(Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    finalStatements;  
}
```

Execute the final block



Next statement;

Trace a Program Execution (11 of 11)

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
catch(Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    finalStatements;  
}
```

Rethrow the exception
and control is
transferred to the caller

Next statement;

Cautions When Using Exceptions

- Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify. Be aware, however, that exception handling usually requires more time and resources because it requires instantiating a new exception object, rolling back the call stack, and propagating the errors to the calling methods.

When to Throw Exceptions

- An exception occurs in a method. If you want the exception to be processed by its caller, you should create an exception object and throw it. If you can handle the exception in the method where it occurs, there is no need to throw it.

When to Use Exceptions (1 of 2)

When should you use the try-catch block in the code? You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {  
  
    System.out.println(refVar.toString());  
  
}  
  
catch (NullPointerException ex) {  
  
    System.out.println("refVar is null");  
  
}
```

When to Use Exceptions (2 of 2)

is better to be replaced by

```
if (refVar != null)
```

```
    System.out.println(refVar.toString());
```

```
else
```

```
    System.out.println("refVar is null");
```

Defining Custom Exception Classes

- Use the exception classes in the API whenever possible.
- Define custom exception classes if the predefined classes are not sufficient.
- Define custom exception classes by extending Exception or a subclass of Exception.

Custom Exception Class Example

In Listing 13.8, the `setRadius` method throws an exception if the radius is negative. Suppose you wish to pass the radius to the handler, you have to create a custom exception class.

[InvalidRadiusException](#)

[CircleWithRadiusException](#)

[TestCircleWithRadiusException](#)

Assertions

An assertion is a Java statement that enables you to assert an assumption about your program. An assertion contains a Boolean expression that should be true during program execution. Assertions can be used to assure program correctness and avoid logic errors.

Declaring Assertions

An **assertion** is declared using the new Java keyword **assert** in JDK 1.4 as follows:

```
assert assertion; or  
assert assertion : detailMessage;
```

where **assertion** is a Boolean expression and **detailMessage** is a primitive-type or an Object value.

Executing Assertions

When an assertion statement is executed, Java evaluates the assertion. If it is false, an `AssertionError` will be thrown. The `AssertionError` class has a no-arg constructor and seven overloaded single-argument constructors of type `int`, `long`, `float`, `double`, `boolean`, `char`, and `Object`.

For the first assert statement with no detail message, the no-arg constructor of `AssertionError` is used. For the second assert statement with a detail message, an appropriate `AssertionError` constructor is used to match the data type of the message. Since `AssertionError` is a subclass of `Error`, when an assertion becomes false, the program displays a message on the console and exits.

Executing Assertions Example

```
public class AssertionDemo {  
    public static void main(String[] args) {  
        int i; int sum = 0;  
        for (i = 0; i < 10; i++) {  
            sum += i;  
        }  
        assert i == 10;  
        assert sum > 10 && sum < 5 * 10 : "sum is  
        " + sum;  
    }  
}
```


Compiling Programs With Assertions

Since **assert** is a new Java keyword introduced in JDK 1.4, you have to compile the program using a JDK 1.4 compiler. Furthermore, you need to include the switch **–source 1.4** in the compiler command as follows:

```
javac –source 1.4 AssertionDemo.java
```

Note: If you use JDK 1.5, there is no need to use the **–source 1.4** option in the command.

Running Programs With Assertions

By default, the assertions are disabled at runtime. To enable it, use the switch `–enableassertions`, or `–ea` for short, as follows:

```
java –ea AssertionDemo
```

Assertions can be selectively enabled or disabled at class level or package level. The disable switch is `–disableassertions` or `–da` for short. For example, the following command enables assertions in package **package1** and disables assertions in class **Class1**.

```
java –ea:package1 –da:Class1 AssertionDemo
```

Using Exception Handling or Assertions (1 of 4)

Assertion should not be used to replace exception handling. Exception handling deals with unusual circumstances during program execution. Assertions are to assure the correctness of the program. Exception handling addresses robustness and assertion addresses correctness. Like exception handling, assertions are not used for normal tests, but for internal consistency and validity checks. Assertions are checked at runtime and can be turned on or off at startup time.

Using Exception Handling or Assertions (2 of 4)

Do not use assertions for argument checking in public methods. Valid arguments that may be passed to a public method are considered to be part of the method's contract. The contract must always be obeyed whether assertions are enabled or disabled. For example, the following code in the Circle class should be rewritten using exception handling.

```
public void setRadius(double newRadius) {  
    assert newRadius >= 0;  
    radius = newRadius;  
}
```

Using Exception Handling or Assertions (3 of 4)

Use assertions to reaffirm assumptions. This gives you more confidence to assure correctness of the program. A common use of assertions is to replace assumptions with assertions in the code.

Using Exception Handling or Assertions (4 of 4)

Another good use of assertions is place assertions in a switch statement without a default case. For example,

```
switch (month) {  
    case 1: ... ; break;  
    case 2: ... ; break;  
    ...  
    case 12: ... ; break;  
    default: assert false : "Invalid month: "  
    + month  
}
```

The **File** Class

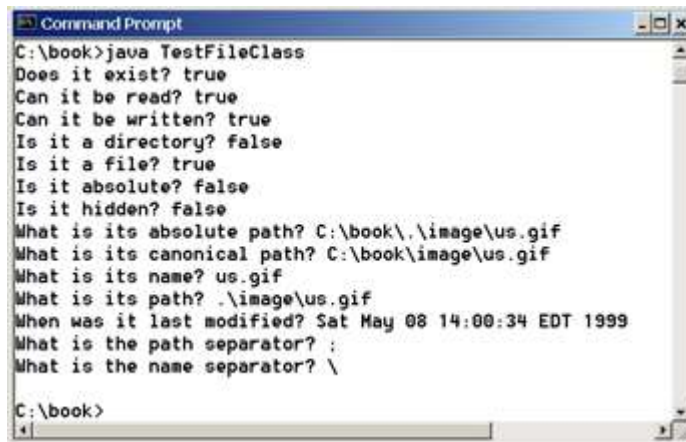
The **File** class is intended to provide an abstraction that deals with most of the machine-dependent complexities of files and path names in a machine-independent fashion. The filename is a string. The **File** class is a wrapper class for the file name and its directory path.

Obtaining File Properties and Manipulating File

java.io.File	
+File(pathname: String)	Creates a File object for the specified path name. The path name may be a directory or a file.
+File(parent: String, child: String)	Creates a File object for the child under the directory parent. The child may be a file name or a subdirectory.
+File(parent: File, child: String)	Creates a File object for the child under the directory parent. The parent is a File object. In the preceding constructor, the parent is a string.
+exists(): boolean	Returns true if the file or the directory represented by the File object exists.
+canRead(): boolean	Returns true if the file represented by the File object exists and can be read.
+canWrite(): boolean	Returns true if the file represented by the File object exists and can be written.
+isDirectory(): boolean	Returns true if the File object represents a directory.
+isFile(): boolean	Returns true if the File object represents a file.
+isAbsolute(): boolean	Returns true if the File object is created using an absolute path name.
+isHidden(): boolean	Returns true if the file represented in the File object is hidden. The exact definition of <i>hidden</i> is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period(.) character.
+getAbsolutePath(): String	Returns the complete absolute file or directory name represented by the File object.
+getCanonicalPath(): String	Returns the same as <code>getAbsolutePath()</code> except that it removes redundant names, such as <code>..</code> and <code>..</code> , from the path name, resolves symbolic links (on Unix), and converts drive letters to standard uppercase (on Windows).
+getName(): String	Returns the last name of the complete directory and file name represented by the File object. For example, new <code>File("c:\\book\\test.dat").getName()</code> returns <code>test.dat</code> .
+getPath(): String	Returns the complete directory and file name represented by the File object. For example, new <code>File("c:\\book\\test.dat").getPath()</code> returns <code>c:\\book\\test.dat</code> .
+getParent(): String	Returns the complete parent directory of the current directory or the file represented by the File object. For example, new <code>File("c:\\book\\test.dat").getParent()</code> returns <code>c:\\book</code> .
+lastModified(): long	Returns the time that the file was last modified.
+length(): long	Returns the size of the file, or 0 if it does not exist or if it is a directory.
+listFile(): File[]	Returns the files under the directory for a directory File object.
+delete(): boolean	Deletes the file or directory represented by this File object. The method returns true if the deletion succeeds.
+renameTo(dest: File): boolean	Renames the file or directory represented by this File object to the specified name represented in dest. The method returns true if the operation succeeds.
+mkdir(): boolean	Creates a directory represented in this File object. Returns true if the the directory is created successfully.
+mkdirs(): boolean	Same as <code>mkdir()</code> except that it creates directory along with its parent directories if the parent directories do not exist.

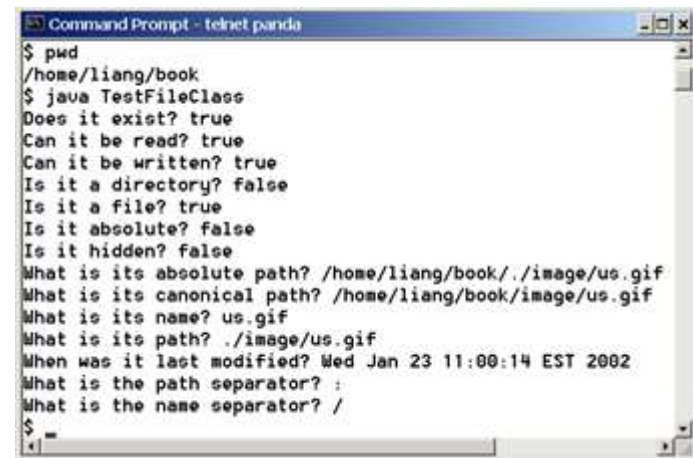
Problem: Explore File Properties

Objective: Write a program that demonstrates how to create files in a platform-independent way and use the methods in the File class to obtain their properties. The following figures show a sample run of the program on Windows and on Unix.



```
Command Prompt
C:\book>java TestFileClass
Does it exist? true
Can it be read? true
Can it be written? true
Is it a directory? false
Is it a file? true
Is it absolute? false
Is it hidden? false
What is its absolute path? C:\book\image\us.gif
What is its canonical path? C:\book\image\us.gif
What is its name? us.gif
What is its path? image\us.gif
When was it last modified? Sat May 08 14:00:34 EDT 1999
What is the path separator? ;
What is the name separator? \

C:\book>
```



```
Command Prompt - telnet panda
$ pwd
/home/liang/book
$ java TestFileClass
Does it exist? true
Can it be read? true
Can it be written? true
Is it a directory? false
Is it a file? true
Is it absolute? false
Is it hidden? false
What is its absolute path? /home/liang/book/image/us.gif
What is its canonical path? /home/liang/book/image/us.gif
What is its name? us.gif
What is its path? image/us.gif
When was it last modified? Wed Jan 23 11:00:14 EST 2002
What is the path separator? :
What is the name separator? /

$
```

TestFileClass

Text I/O

A **File** object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file. In order to perform I/O, you need to create objects using appropriate Java I/O classes. The objects contain the methods for reading/writing data from/to a file. This section introduces how to read/write strings and numeric values from/to a text file using the **Scanner** and **PrintWriter** classes.

Writing Data Using PrintWriter

java.io.PrintWriter	
+PrintWriter(filename: String)	Creates a PrintWriter for the specified file.
+print(s: String): void	Writes a string.
+print(c: char): void	Writes a character.
+print(cArray: char[]): void	Writes an array of character.
+print(i: int): void	Writes an int value.
+print(l: long): void	Writes a long value.
+print(f: float): void	Writes a float value.
+print(d: double): void	Writes a double value.
+print(b: boolean): void	Writes a boolean value.
Also contains the overloaded println methods.	A println method acts like a print method; additionally it prints a line separator. The line separator string is defined by the system. It is \r\n on Windows and \n on Unix. The printf method was introduced in §4.6, “Formatting Console Output and Strings.”
Also contains the overloaded printf methods.	

WriteData

Try-With-Resources

Programmers often forget to close the file. JDK 7 provides the followings new try-with-resources syntax that automatically closes the files.

```
try (declare and create resources) {  
    Use the resource to process the file;  
}
```

[WriteDataWithAutoClose](#)

Reading Data Using Scanner

java.util.Scanner	
+Scanner(source: File)	Creates a Scanner object to read data from the specified file.
+Scanner(source: String)	Creates a Scanner object to read data from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has another token in its input.
+next(): String	Returns next token as a string.
+nextByte(): byte	Returns next token as a byte.
+nextShort(): short	Returns next token as a short.
+nextInt(): int	Returns next token as an int.
+nextLong(): long	Returns next token as a long.
+nextFloat(): float	Returns next token as a float.
+nextDouble(): double	Returns next token as a double.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern.

ReadData

Problem: Replacing Text

Write a class named **ReplaceText** that replaces a string in a text file with a new string. The filename and strings are passed as command-line arguments as follows:

```
java ReplaceText sourceFile targetFile oldString newString
```

For example, invoking

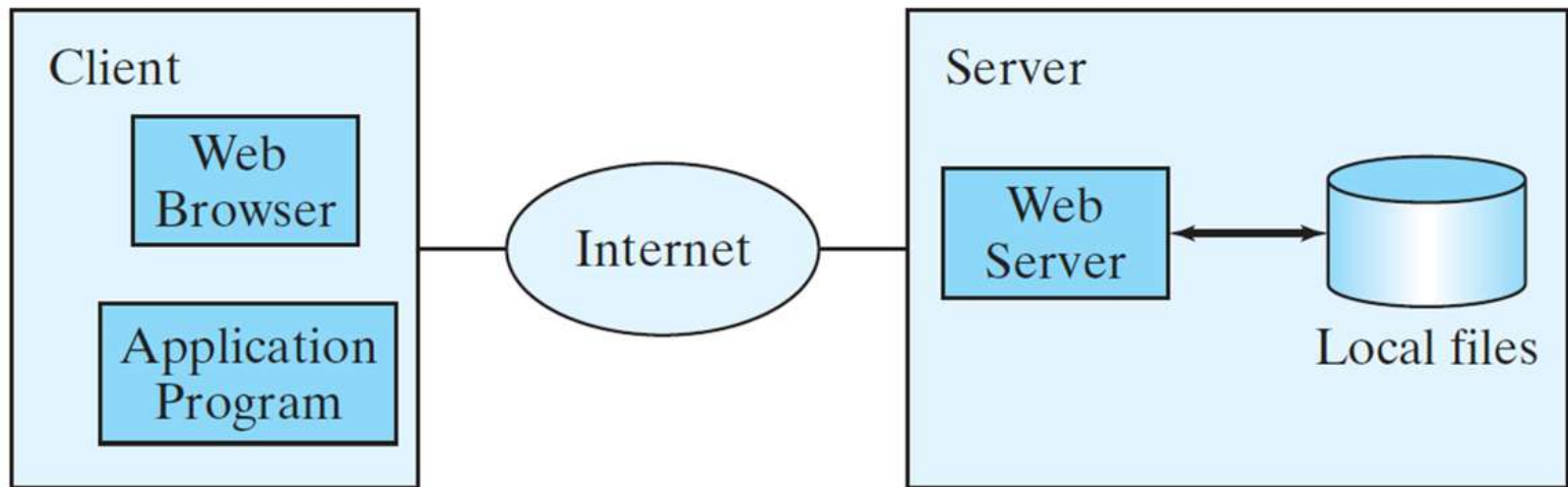
```
java ReplaceText FormatString.java t.txt StringBuilder  
StringBuffer
```

replaces all the occurrences of **StringBuilder** by **StringBuffer** in FormatString.java and saves the new file in t.txt.

[ReplaceText](#)

Reading Data From the Web (1 of 2)

Just like you can read data from a file on your computer, you can read data from a file on the Web.



Reading Data From the Web (2 of 2)

```
URL url = new URL("www.google.com/index.html");
```

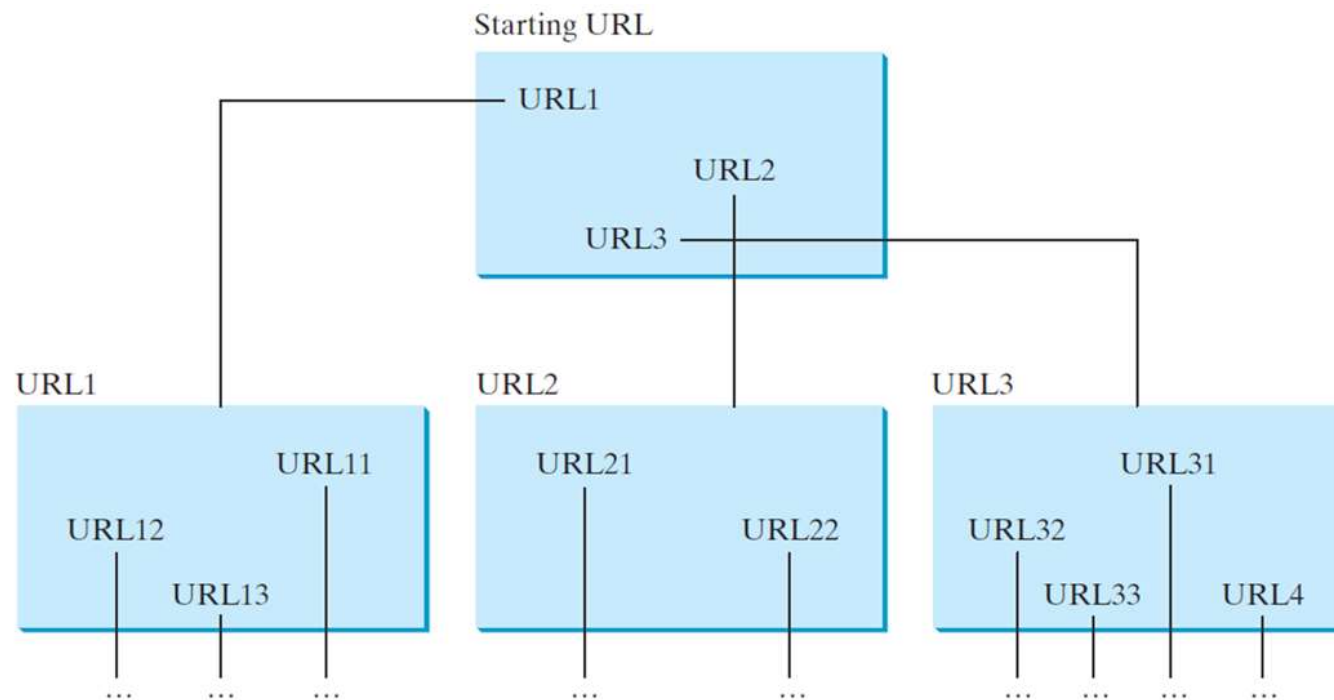
After a **URL** object is created, you can use the **openStream()** method defined in the **URL** class to open an input stream and use this stream to create a **Scanner** object as follows:

```
Scanner input = new Scanner(url.openStream());
```

[ReadFileFromURL](#)

Case Study: Web Crawler (1 of 3)

This case study develops a program that travels the Web by following hyperlinks.



Case Study: Web Crawler (2 of 3)

The program follows the URLs to traverse the Web. To avoid that each URL is traversed only once, the program maintains two lists of URLs. One list stores the URLs pending for traversing and the other stores the URLs that have already been traversed. The algorithm for this program can be described as follows:

Case Study: Web Crawler (3 of 3)

Add the starting URL to a list named `listOfPendingURLs`;

```
while listOfPendingURLs is not empty {
```

```
    Remove a URL from listOfPendingURLs;
```

```
    if this URL is not in listOfTraversedURLs {
```

```
        Add it to listOfTraversedURLs;
```

```
        Display this URL;
```

```
        Exit the while loop when the size of S is equal to 100.
```

```
        Read the page from this URL and for each URL contained in the page {
```

```
            Add it to listOfPendingURLs if it is not in listOfTraversedURLs;
```

```
        }
```

```
    }
```

```
}
```

[WebCrawler](#)

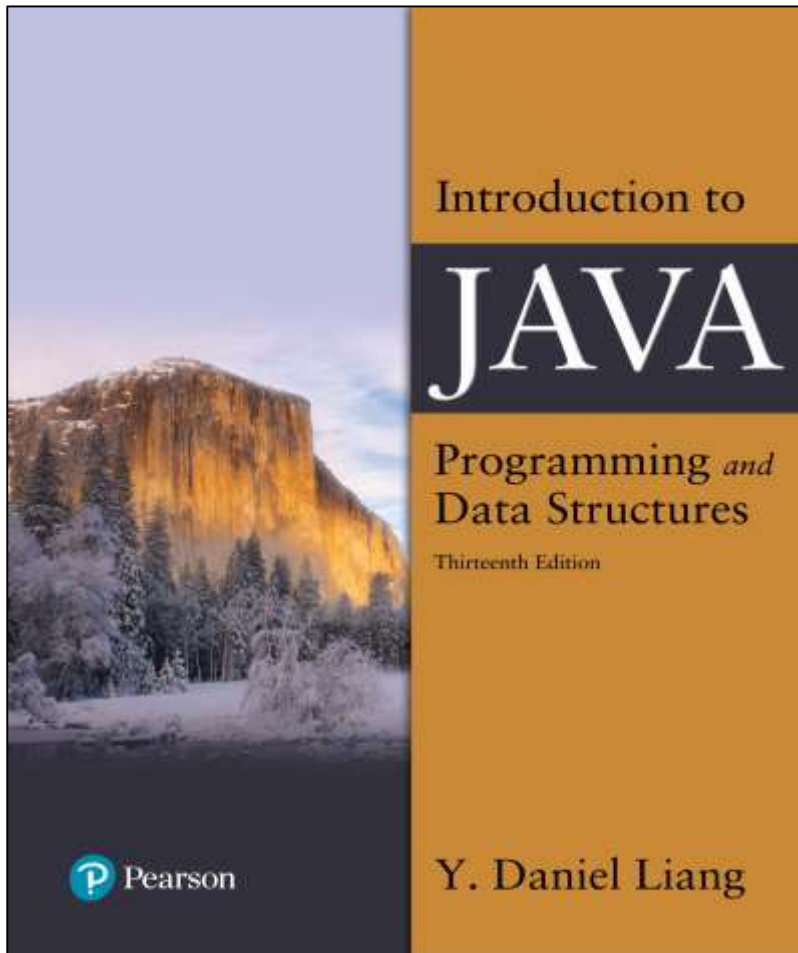
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 13

Abstract Classes and Interfaces

Motivations

- You have learned how to write simple programs to create and display GUI components. Can you write the code to respond to user actions, such as clicking a button to perform an action?
- In order to write such code, you have to know about interfaces. An **interface** is for defining common behavior for classes (including unrelated classes). Before discussing interfaces, we introduce a closely related subject: abstract classes.

Objectives (1 of 2)

13.1 To design and use abstract classes (§13.2).

13.2 To generalize numeric wrapper classes, **BigInteger**, and **BigDecimal** using the abstract **Number** class (§13.3).

13.3 To process a calendar using the **Calendar** and **GregorianCalendar** classes (§13.4).

13.4 To specify common behavior for objects using interfaces (§13.5).

13.5 To define interfaces and define classes that implement interfaces (§13.5).

13.6 To define a natural order using the **Comparable** interface (§13.6).

13.7 To make objects cloneable using the **Cloneable** interface (§13.7).

Objectives (2 of 2)

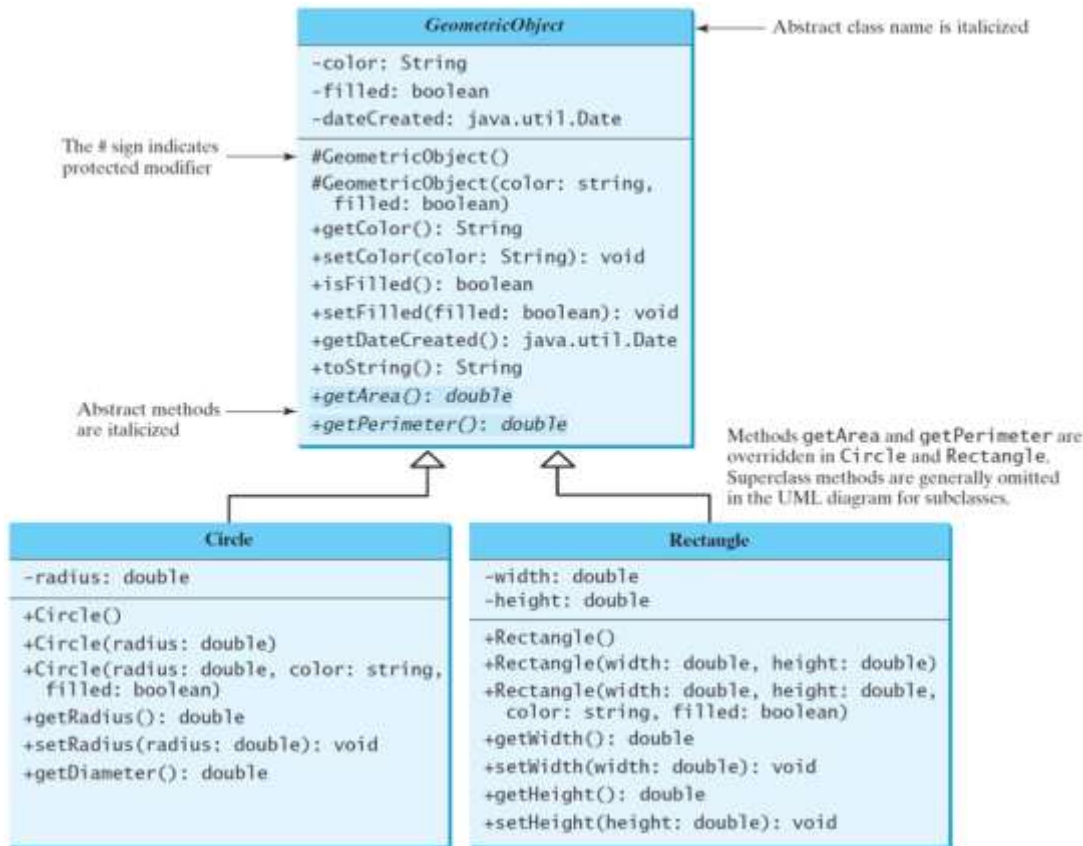
13.8 To explore the similarities and differences among concrete classes, abstract classes, and interfaces (§13.8).

13.9 To design the **Rational** class for processing rational numbers (§13.9).

13.10 To design classes that follow the class-design guidelines (§13.10).

13.11 To use a simple one-line code to define a record (§13.11).

Abstract Classes and Abstract Methods



[GeometricObject](#)

[Circle](#)

[Rectangle](#)

[TestGeometricObject](#)

Abstract Method in Abstract Class

An abstract method cannot be contained in a nonabstract class. If a subclass of an abstract superclass does not implement all the abstract methods, the subclass must be defined abstract. In other words, in a nonabstract subclass extended from an abstract class, all the abstract methods must be implemented, even if they are not used in the subclass.

Object Cannot Be Created From Abstract Class

An abstract class cannot be instantiated using the `new` operator, but you can still define its constructors, which are invoked in the constructors of its subclasses. For instance, the constructors of `GeometricObject` are invoked in the `Circle` class and the `Rectangle` class.

Abstract Class Without Abstract Method

A class that contains abstract methods must be abstract. However, it is possible to define an abstract class that contains no abstract methods. In this case, you cannot create instances of the class using the new operator. This class is used as a base class for defining a new subclass.

Superclass of Abstract Class May Be Concrete

A subclass can be abstract even if its superclass is concrete. For example, the `Object` class is concrete, but its subclasses, such as `GeometricObject`, may be abstract.

Concrete Method Overridden to Be Abstract

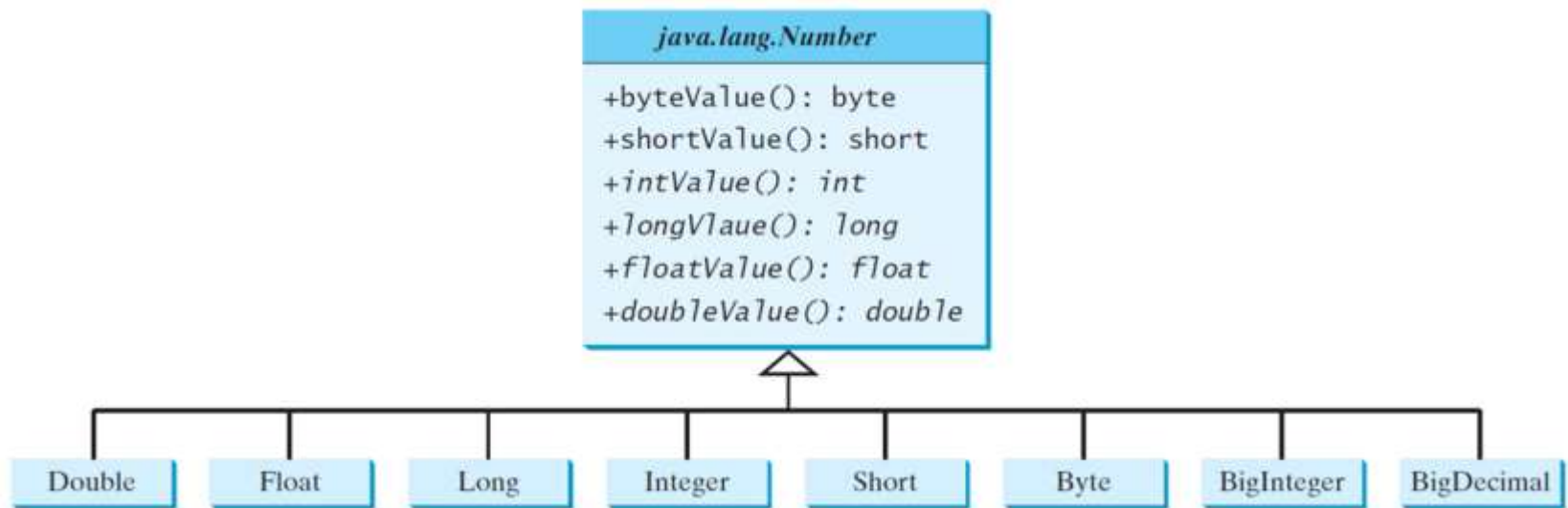
A subclass can override a method from its superclass to define it abstract. This is rare, but useful when the implementation of the method in the superclass becomes invalid in the subclass. In this case, the subclass must be defined abstract.

Abstract Class as Type

You cannot create an instance from an abstract class using the new operator, but an abstract class can be used as a data type. Therefore, the following statement, which creates an array whose elements are of `GeometricObject` type, is correct.

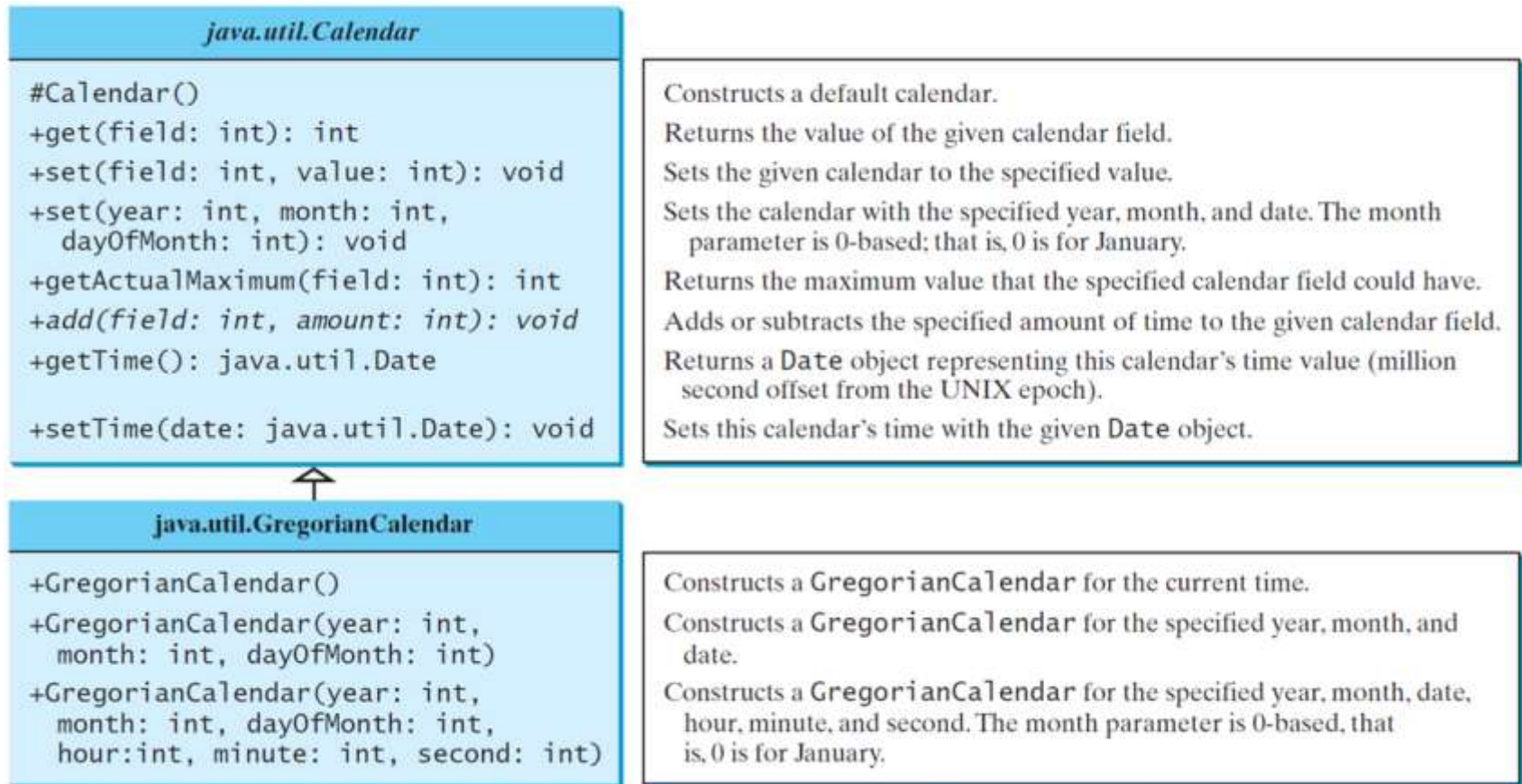
```
GeometricObject[] geo = new GeometricObject[10];
```

Case Study: The Abstract Number Class



[LargestNumbers](#)

The Abstract Calendar Class and Its GregorianCalendar Subclass (1 of 2)



The Abstract Calendar Class and Its GregorianCalendar Subclass (2 of 2)

An instance of `java.util.Date` represents a specific instant in time with millisecond precision.

`java.util.Calendar` is an abstract base class for extracting detailed information such as year, month, date, hour, minute and second from a `Date` object. Subclasses of `Calendar` can implement specific calendar systems such as Gregorian calendar, Lunar Calendar and Jewish calendar. Currently, `java.util.GregorianCalendar` for the Gregorian calendar is supported in the Java API.

The `GregorianCalendar` Class

You can use `new GregorianCalendar()` to construct a default `GregorianCalendar` with the current time and use `new GregorianCalendar(year, month, date)` to construct a `GregorianCalendar` with the specified year, month, and date. The month parameter is 0-based, i.e., 0 is for January.

The Get Method in Calendar Class

The `get(int field)` method defined in the `Calendar` class is useful to extract the date and time information from a `Calendar` object. The fields are defined as constants, as shown in the following.

Constant	Description
Year	The year of the calendar.
Month	The month of the calendar, with 0 for January.
Date	The day of the calendar.
Hour	The hour of the calendar (12-hour notation).
Hour_of_day	The hour of the calendar (24-hour notation).
Minute	The minute of the calendar.
Second	The second of the calendar.
Day_of_week	The day number within the week, with 1 for Sunday.
Day_of_month	Same as Date .
Day_of_year	The day number in the year, with 1 for the first day of the year.
Week_of_month	The week number within the month, with 1 for the first week.
Week_of_year	The week number within the year, with 1 for the first week.
AM_PM	Indicator for AM or PM (0 for AM and 1 for PM).

Getting Date/Time Information From Calendar

[TestCalendar](#)

Interfaces

What is an interface?

Why is an interface useful?

How do you define an interface?

How do you use an interface?

What Is an Interface? Why Is an Interface Useful?

An interface is a classlike construct that contains only constants and abstract methods. In many ways, an interface is similar to an abstract class, but the intent of an interface is to specify common behavior for objects. For example, you can specify that the objects are comparable, edible, cloneable using appropriate interfaces.

Define an Interface

To distinguish an interface from a class, Java uses the following syntax to define an interface:

```
public interface InterfaceName {  
    constant declarations;  
    abstract method signatures;  
}
```

Example:

```
public interface Edible {  
    /** Describe how to eat */  
    public abstract String howToEat();  
}
```

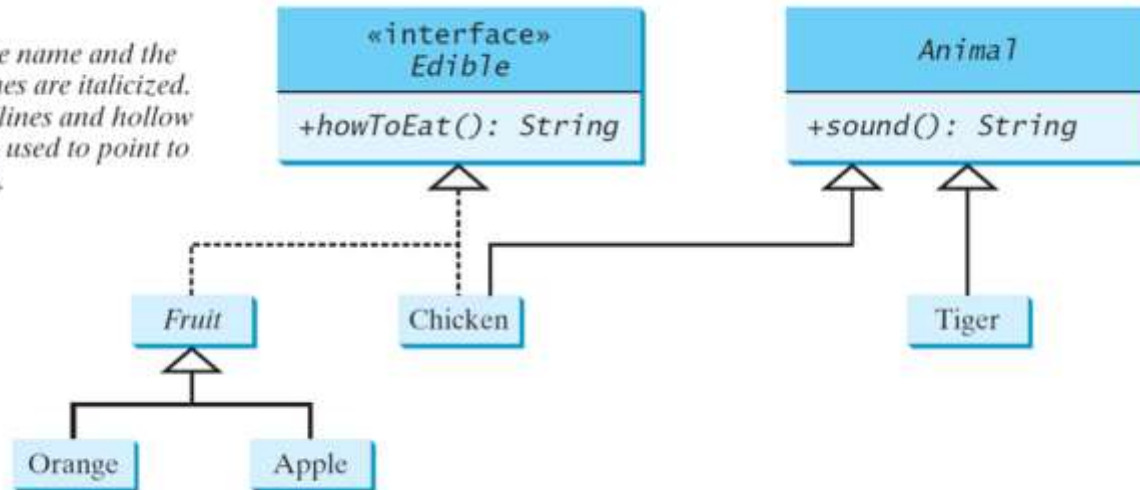

Interface Is a Special Class

An interface is treated like a special class in Java. Each interface is compiled into a separate bytecode file, just like a regular class. Like an abstract class, you cannot create an instance from an interface using the new operator, but in most cases you can use an interface more or less the same way you use an abstract class. For example, you can use an interface as a data type for a variable, as the result of casting, and so on.

Example (1 of 2)

You can now use the Edible interface to specify whether an object is edible. This is accomplished by letting the class for the object implement this interface using the implements keyword. For example, the classes Chicken and Fruit implement the Edible interface (See TestEdible).

*Notation:
The interface name and the
method names are italicized.
The dashed lines and hollow
triangles are used to point to
the interface.*



[Edible](#)

[TestEdible](#)

Omitting Modifiers in Interfaces

All data fields are **public final static** and all methods are **public abstract** in an interface. For this reason, these modifiers can be omitted, as shown below:

```
public interface T1 {  
    public static final int K = 1;  
  
    public abstract void p();  
}
```

Equivalent

```
public interface T1 {  
    int K = 1;  
  
    void p();  
}
```

A constant defined in an interface can be accessed using syntax `InterfaceName.CONSTANT_NAME` (e.g., `T1.K`).

Example: The Comparable Interface

```
// This interface is defined in
```

```
// java.lang package
```

```
package java.lang;
```

```
public interface Comparable<E> {
```

```
    public int compareTo(E o);
```

```
}
```

The `toString`, `equals`, and `hashCode` Methods

Each wrapper class overrides the `toString`, `equals`, and `hashCode` methods defined in the `Object` class. Since all the numeric wrapper classes and the `Character` class implement the `Comparable` interface, the `compareTo` method is implemented in these classes.

Integer and BigInteger Classes

```
public class Integer extends Number
    implements Comparable<Integer> {
    // class body omitted

    @Override
    public int compareTo(Integer o) {
        // Implementation omitted
    }
}
```

```
public class BigInteger extends Number
    implements Comparable<BigInteger> {
    // class body omitted

    @Override
    public int compareTo(BigInteger o) {
        // Implementation omitted
    }
}
```

String and Date Classes

```
public class String extends Object
    implements Comparable<String> {
    // class body omitted

    @Override
    public int compareTo(String o) {
        // Implementation omitted
    }
}
```

```
public class Date extends Object
    implements Comparable<Date> {
    // class body omitted

    @Override
    public int compareTo(Date o) {
        // Implementation omitted
    }
}
```

Example (2 of 2)

1. `System.out.println(new Integer(3).compareTo(new Integer(5)));`
2. `System.out.println("ABC".compareTo("ABE"));`
3. `java.util.Date date1 = new java.util.Date(2013, 1, 1);`
4. `java.util.Date date2 = new java.util.Date(2012, 1, 1);`
5. `System.out.println(date1.compareTo(date2));`

Generic sort Method

Let **n** be an **Integer** object, **s** be a **String** object, and **d** be a **Date** object. All the following expressions are **true**.

```
n instanceof Integer  
n instanceof Object  
n instanceof Comparable
```

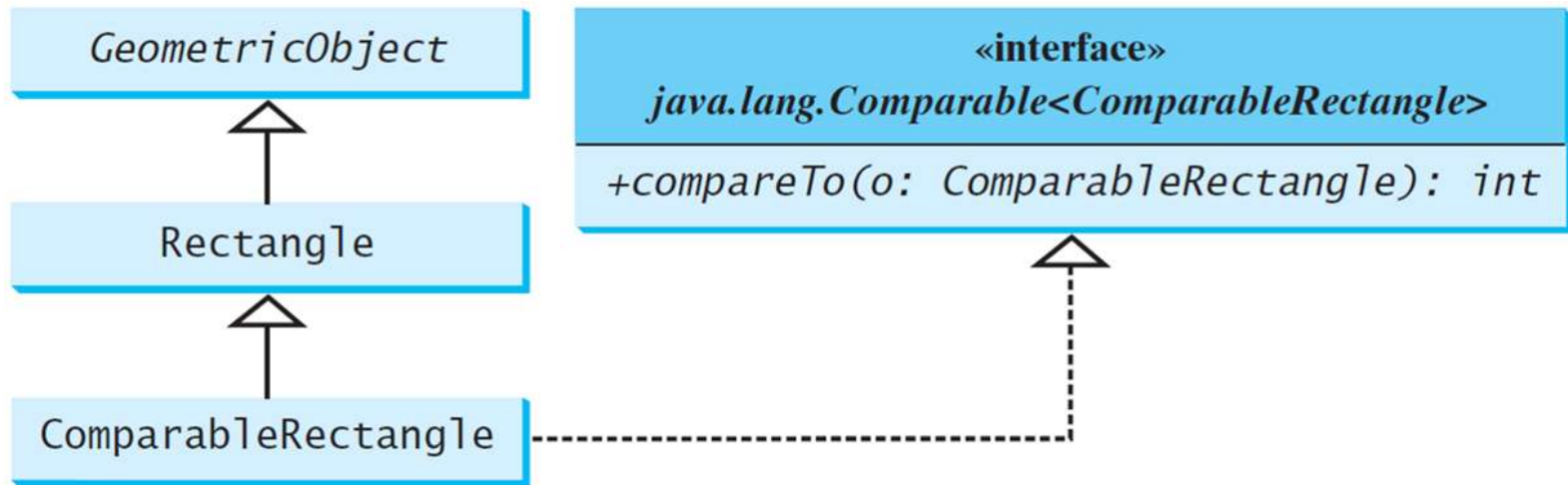
```
s instanceof String  
s instanceof Object  
s instanceof Comparable
```

```
d instanceof java.util.Date  
d instanceof Object  
d instanceof Comparable
```

The `java.util.Arrays.sort(array)` method requires that the elements in an array are instances of `Comparable<E>`.

[SortComparableObjects](#)

Defining Classes to Implement Comparable



ComparableRectangle

SortRectangles

The Cloneable Interfaces

Marker Interface: An empty interface.

A marker interface does not contain constants or methods. It is used to denote that a class possesses certain desirable properties. A class that implements the **Cloneable** interface is marked cloneable, and its objects can be cloned using the **clone()** method defined in the **Object** class.

```
package java.lang;  
  
public interface Cloneable {  
  
}
```

Examples

Many classes (e.g., Date and Calendar) in the Java library implement Cloneable. Thus, the instances of these classes can be cloned. For example, the following code

```
Calendar calendar = new GregorianCalendar(2003, 2, 1);  
Calendar calendarCopy = (Calendar)calendar.clone();  
System.out.println("calendar == calendarCopy is " +  
    (calendar == calendarCopy));  
System.out.println("calendar.equals(calendarCopy) is " +  
    calendar.equals(calendarCopy));
```

displays

calendar == calendarCopy is false

calendar.equals(calendarCopy) is true

Implementing Cloneable Interface

To define a custom class that implements the `Cloneable` interface, the class must override the `clone()` method in the `Object` class. The following code defines a class named `House` that implements `Cloneable` and `Comparable`.

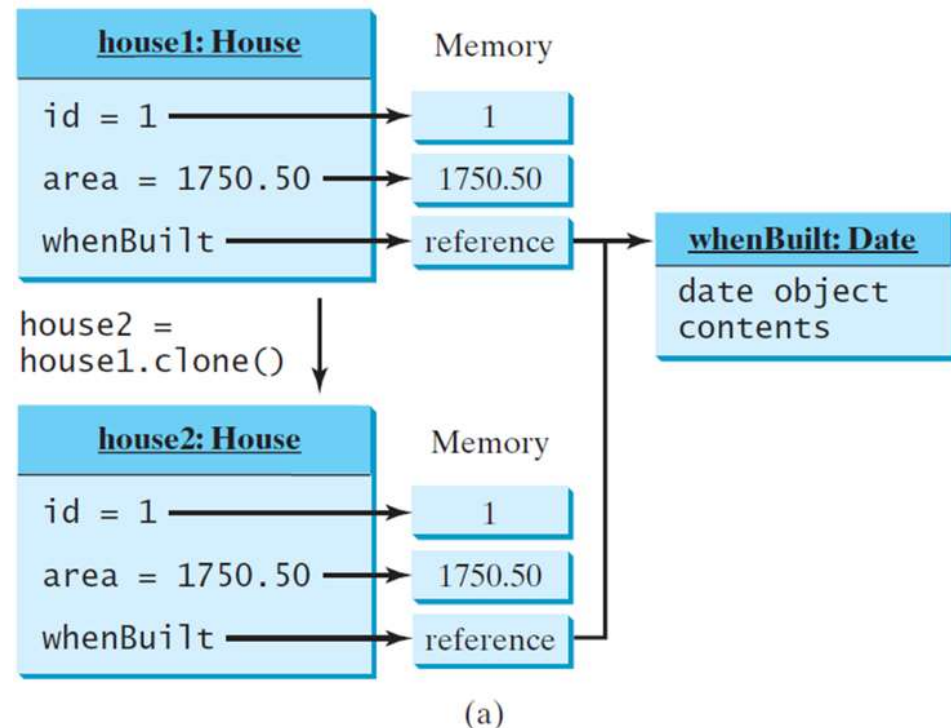
[House](#)

Shallow vs Deep Copy (1 of 2)

```
House house1 = new House(1, 1750.50);
```

```
House house2 = (House)house1.clone();
```

Shallow Copy

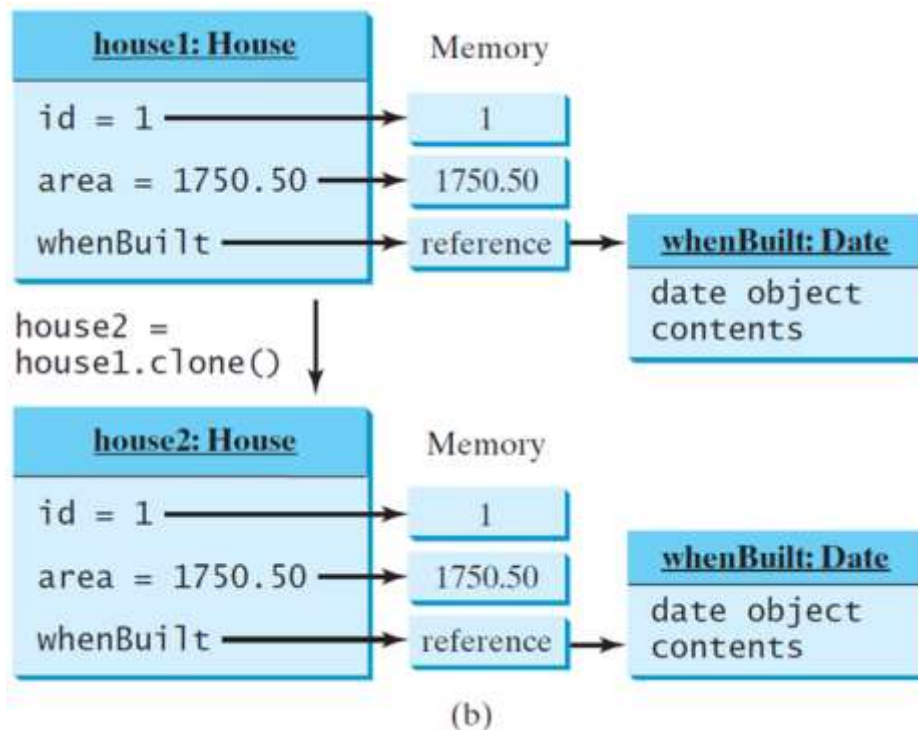


Shallow vs Deep Copy (2 of 2)

```
House house1 = new House(1, 1750.50);
```

```
House house2 = (House)house1.clone();
```

Deep Copy



Interfaces vs Abstract Classes (1 of 2)

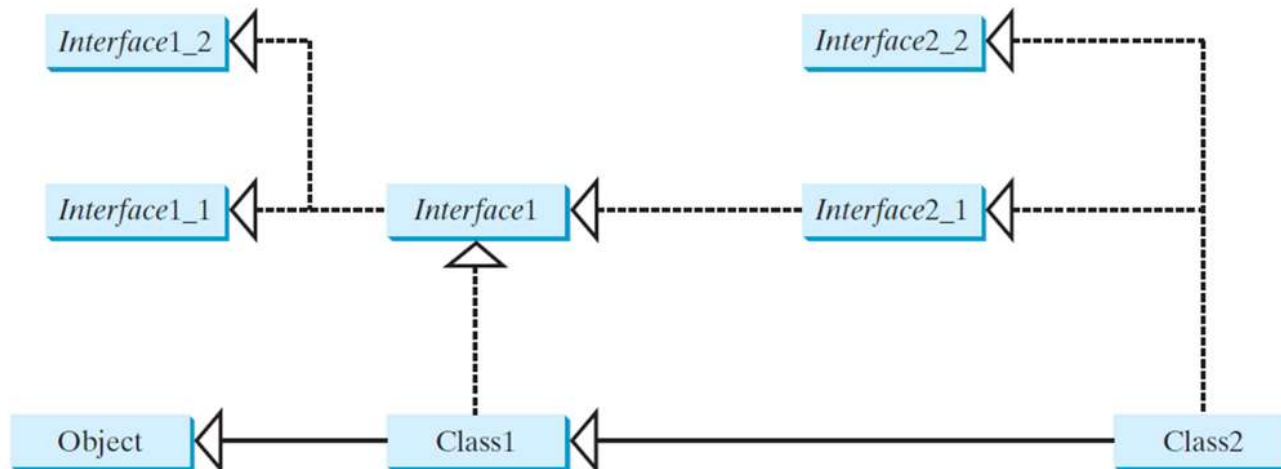
In an interface, the data must be constants; an abstract class can have all types of data.

Each method in an interface has only a signature without implementation; an abstract class can have concrete methods.

	Variables	Constructors	Methods
Abstract class	No restrictions.	Constructors are invoked by subclasses through constructor chaining. An abstract class cannot be instantiated using the new operator.	No restrictions.
Interface	All variables must be public static final	No constructors. An interface cannot be instantiated using the new operator.	All methods must be public abstract instance methods

Interfaces vs Abstract Classes (2 of 2)

All classes share a single root, the Object class, but there is no single root for interfaces. Like a class, an interface also defines a type. A variable of an interface type can reference any instance of the class that implements the interface. If a class extends an interface, this interface plays the same role as a superclass. You can use an interface as a data type and cast a variable of an interface type to its subclass, and vice versa.



Suppose that *c* is an instance of *Class2*. *c* is also an instance of *Object*, *Class1*, *Interface1*, *Interface1_1*, *Interface1_2*, *Interface2_1*, and *Interface2_2*.

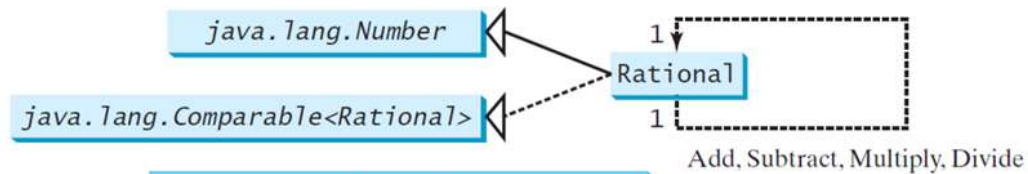
Caution: Conflict Interfaces

In rare occasions, a class may implement two interfaces with conflict information (e.g., two same constants with different values or two methods with same signature but different return type). This type of errors will be detected by the compiler.

Whether to Use an Interface or a Class?

Abstract classes and interfaces can both be used to model common features. How do you decide whether to use an interface or a class? In general, a strong is-a relationship that clearly describes a parent-child relationship should be modeled using classes. For example, a staff member is a person. A weak is-a relationship, also known as an is-kind-of relationship, indicates that an object possesses a certain property. A weak is-a relationship can be modeled using interfaces. For example, all strings are comparable, so the String class implements the Comparable interface. You can also use interfaces to circumvent single inheritance restriction if multiple inheritance is desired. In the case of multiple inheritance, you have to design one as a superclass, and others as interface.

The Rational Class



Rational
-numerator: long -denominator: long
+Rational() +Rational(numerator: long, denominator: long) +getNumerator(): long +getDenominator(): long +add(secondRational: Rational): Rational +subtract(secondRational: Rational): Rational +multiply(secondRational: Rational): Rational +divide(secondRational: Rational): Rational +toString(): String +gcd(n: long, d: long): long

The numerator of this rational number.
The denominator of this rational number.
Creates a rational number with numerator 0 and denominator 1.
Creates a rational number with a specified numerator and denominator.
Returns the numerator of this rational number.
Returns the denominator of this rational number.
Returns the addition of this rational number with another.
Returns the subtraction of this rational number with another.
Returns the multiplication of this rational number with another.
Returns the division of this rational number with another.
Returns a string in the form "numerator/denominator." Returns the numerator if denominator is 1.
Returns the greatest common divisor of n and d.

[Rational](#)

[TestRationalClass](#)

Designing a Class (1 of 5)

(Coherence) A class should describe a single entity, and all the class operations should logically fit together to support a coherent purpose. You can use a class for students, for example, but you should not combine students and staff in the same class, because students and staff have different entities.

Designing a Class (2 of 5)

(Separating responsibilities) A single entity with too many responsibilities can be broken into several classes to separate responsibilities. The classes `String`, `StringBuilder`, and `StringBuffer` all deal with strings, for example, but have different responsibilities. The `String` class deals with immutable strings, the `StringBuilder` class is for creating mutable strings, and the `StringBuffer` class is similar to `StringBuilder` except that `StringBuffer` contains synchronized methods for updating strings.

Designing a Class (3 of 5)

Classes are designed for reuse. Users can incorporate classes in many different combinations, orders, and environments. Therefore, you should design a class that imposes no restrictions on what or when the user can do with it, design the properties to ensure that the user can set properties in any order, with any combination of values, and design methods to function independently of their order of occurrence.

Designing a Class (4 of 5)

Provide a public no-arg constructor and override the **equals** method and the **toString** method defined in the **Object** class whenever possible.

Designing a Class (5 of 5)

Follow standard Java programming style and naming conventions. Choose informative names for classes, data fields, and methods. Always place the data declaration before the constructor, and place constructors before methods. Always provide a constructor and initialize variables to avoid programming errors.

Using Visibility Modifiers (1 of 2)

Each class can present two contracts – one for the users of the class and one for the extenders of the class. Make the fields private and accessor methods public if they are intended for the users of the class. Make the fields or method protected if they are intended for extenders of the class. The contract for the extenders encompasses the contract for the users. The extended class may increase the visibility of an instance method from protected to public, or change its implementation, but you should never change the implementation in a way that violates that contract.

Using Visibility Modifiers (2 of 2)

A class should use the private modifier to hide its data from direct access by clients. You can use get methods and set methods to provide users with access to the private data, but only to private data you want the user to see or to modify. A class should also hide methods not intended for client use. The gcd method in the Rational class is private, for example, because it is only for internal use within the class.

Using the Static Modifier

A property that is shared by all the instances of the class should be declared as a static property.

Records

Often you need to create a simple object to hold data. Before Java 16, you have to define a class and write a lot of boilerplate code. This is tedious, however. Since Java 14, you can define a record to simplify coding. For example, the following code defines a record for Teachers.

```
public record Teacher(String name, java.util.Date date) {}
```

This simple one-line code is roughly equivalent to the following class definition in LiveExample 13.14.

[Teacher](#)

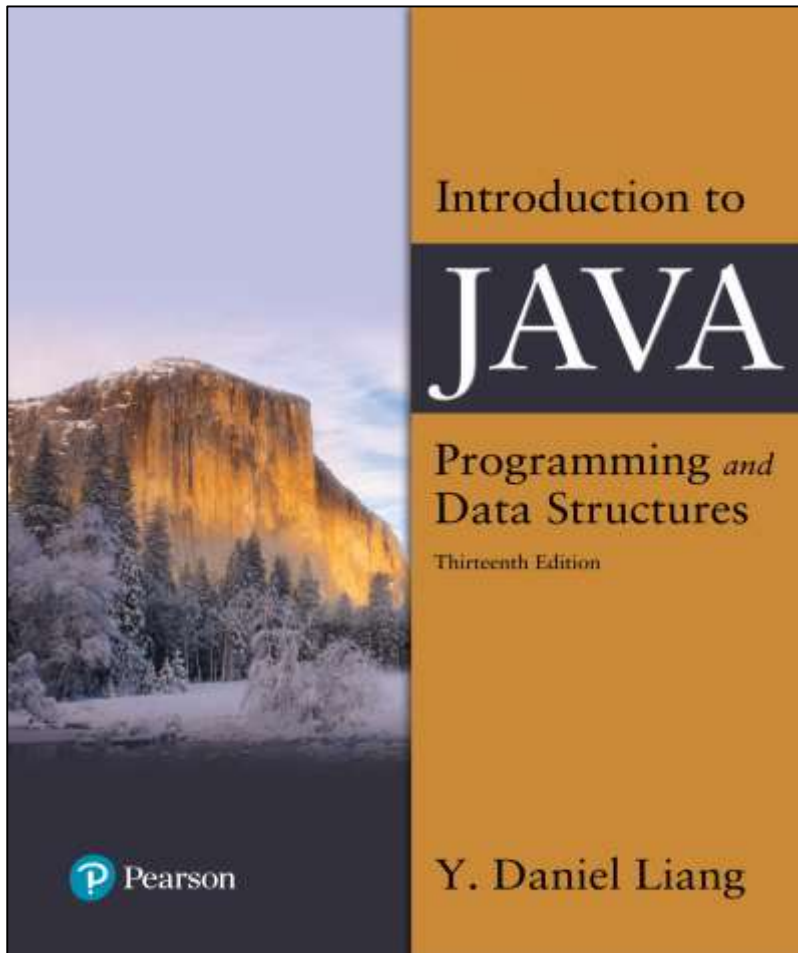
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 14

JavaFX Basics

Motivations

JavaFX is a new framework for developing Java GUI programs. The JavaFX API is an excellent example of how the object-oriented principle is applied. This chapter serves two purposes. First, it presents the basics of JavaFX programming. Second, it uses JavaFX to demonstrate OOP. Specifically, this chapter introduces the framework of JavaFX and discusses JavaFX GUI components and their relationships.

Objectives (1 of 2)

14.1 To distinguish between JavaFX, Swing, and AWT (§14.2).

14.2 To write a simple JavaFX program and understand the relationship among stages, scenes, and nodes (§14.3).

14.3 To create user interfaces using panes, UI controls, and shapes (§14.4).

14.4 To use binding properties to synchronize property values (§14.5).

14.5 To use the common properties **style** and **rotate** for nodes (§14.6).

14.6 To create colors using the **Color** class (§14.7).

14.7 To create fonts using the **Font** class (§14.8).

Objectives (2 of 2)

14.8 To create images using the **Image** class and to create image views using the **ImageView** class (§14.9).

14.9 To layout nodes using **Pane**, **StackPane**, **FlowPane**, **GridPane**, **BorderPane**, **HBox**, and **VBox** (§14.10).

14.10 To display text using the **Text** class and create shapes using **Line**, **Circle**, **Rectangle**, **Ellipse**, **Arc**, **Polygon**, and **Polyline** (§14.11).

14.11 To develop the reusable GUI components **ClockPane** for displaying an analog clock (§14.12).

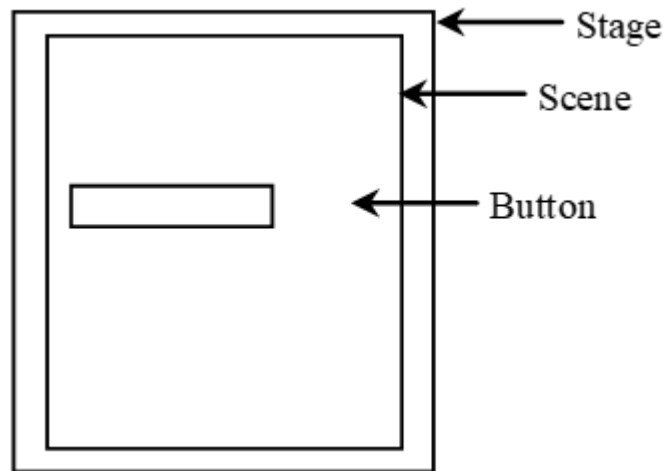
JavaFX vs Swing and AWT

Swing and AWT are replaced by the JavaFX platform for developing rich Internet applications.

When Java was introduced, the GUI classes were bundled in a library known as the **Abstract Windows Toolkit (AWT)**. AWT is fine for developing simple graphical user interfaces, but not for developing comprehensive GUI projects. In addition, AWT is prone to platform-specific bugs. The AWT user-interface components were replaced by a more robust, versatile, and flexible library known as **Swing components**. Swing components are painted directly on canvases using Java code. Swing components depend less on the target platform and use less of the native GUI resource. With the release of Java 8, Swing is replaced by a completely new GUI platform known as **JavaFX**.

Basic Structure of JavaFX

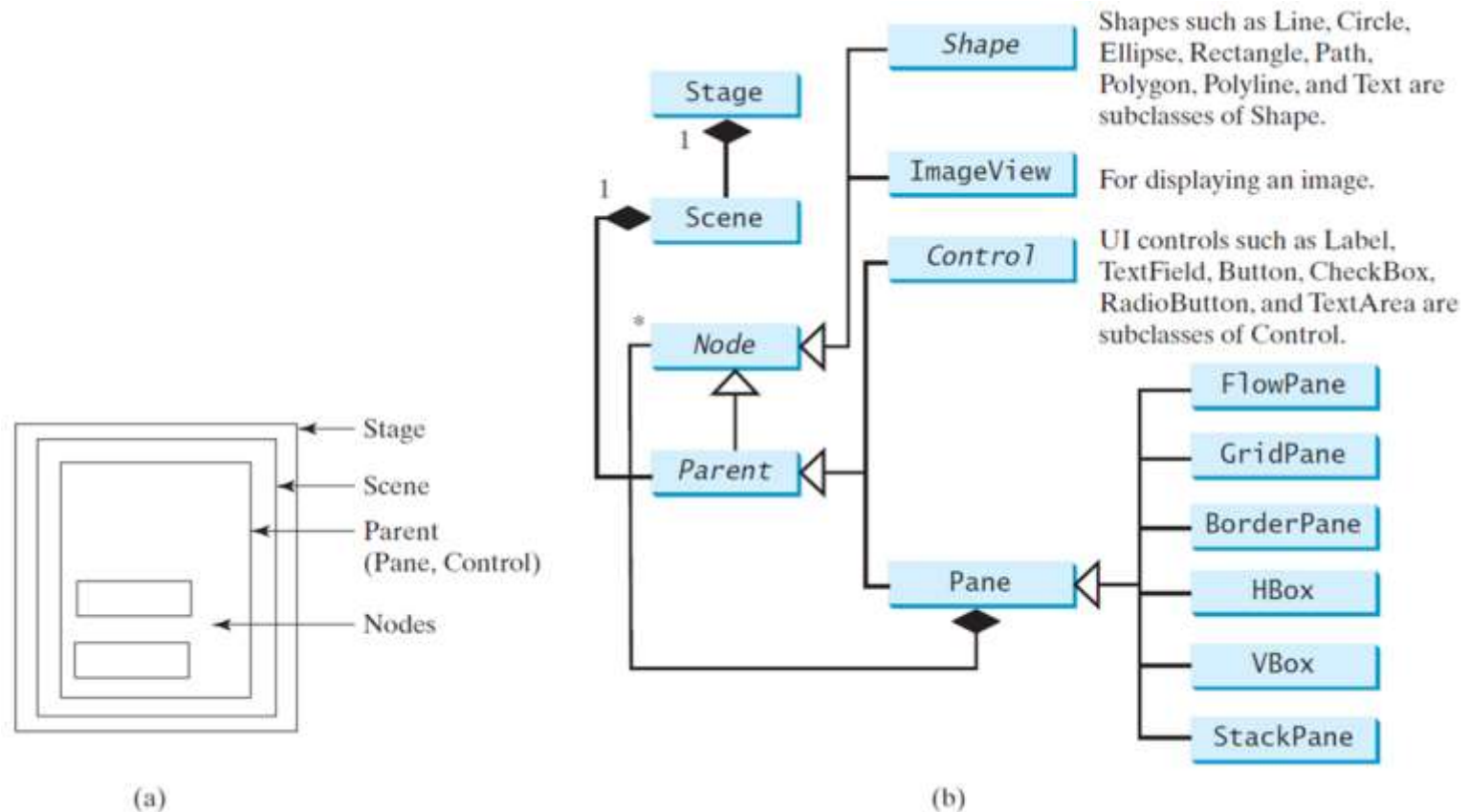
- Application
- Override the start(Stage) method
- Stage, Scene, and Nodes



[MyJavaFX](#)

[MultipleStageDemo](#)

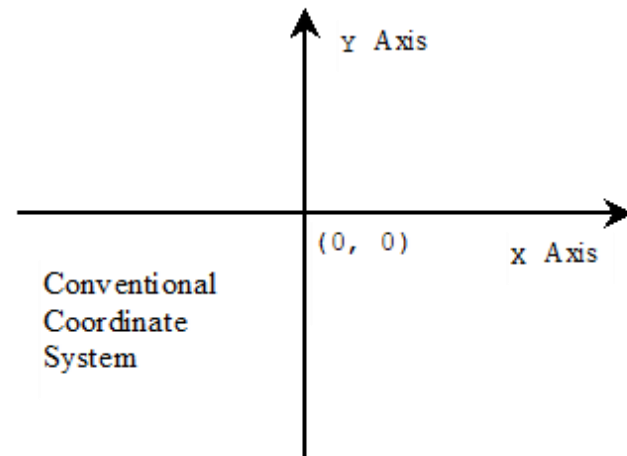
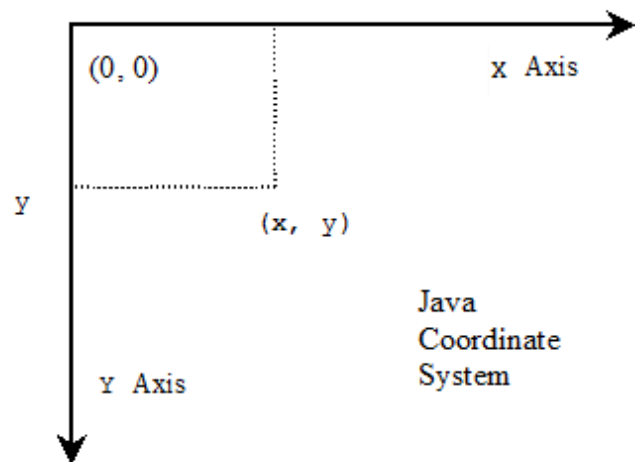
Panes, UI Controls, and Shapes



ButtonInPane

Display a Shape

This example displays a circle in the center of the pane.



[ShowCircle](#)

Binding Properties

JavaFX introduces a new concept called **binding property** that enables a **target object** to be bound to a **source object**. If the value in the source object changes, the target property is also changed automatically. The target object is simply called a **binding object** or a **binding property**.

ShowCircleCentered

Binding Property: Getter, Setter, and Property Getter

```
public class SomeClassName {  
  
    private PropertyType x;  
  
    /** Value getter method */  
    public propertyValueType getX() { ... }  
  
    /** Value setter method */  
    public void setX(propertyValueType value) { ... }  
  
    /** Property getter method */  
    public PropertyType  
        xProperty() { ... }  
}
```

(a) x is a binding property

```
public class Circle {  
  
    private DoubleProperty centerX;  
  
    /** Value getter method */  
    public double getCenterX() { ... }  
  
    /** Value setter method */  
    public void setCenterX(double value) { ... }  
  
    /** Property getter method */  
    public DoubleProperty centerXProperty() { ... }  
}
```

(b) centerX is binding property

Uni/Bidirectional Binding

[BindingDemo](#)

[BidirectionalBindingDemo](#)

Common Properties and Methods for Nodes

- style: set a JavaFX CSS style
- rotate: Rotate a node

[NodeStyleRotateDemo](#)

The Color Class

javafx.scene.paint.Color

-red: double
-green: double
-blue: double
-opacity: double

+Color(r: double, g: double, b: double, opacity: double)
+brighter(): Color
+darker(): Color
+color(r: double, g: double, b: double): Color
+color(r: double, g: double, b: double, opacity: double): Color
+rgb(r: int, g: int, b: int): Color
+rgb(r: int, g: int, b: int, opacity: double): Color

The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

The red value of this Color (between 0.0 and 1.0).

The green value of this Color (between 0.0 and 1.0).

The blue value of this Color (between 0.0 and 1.0).

The opacity of this Color (between 0.0 and 1.0).

Creates a Color with the specified red, green, blue, and opacity values.

Creates a Color that is a brighter version of this Color.

Creates a Color that is a darker version of this Color.

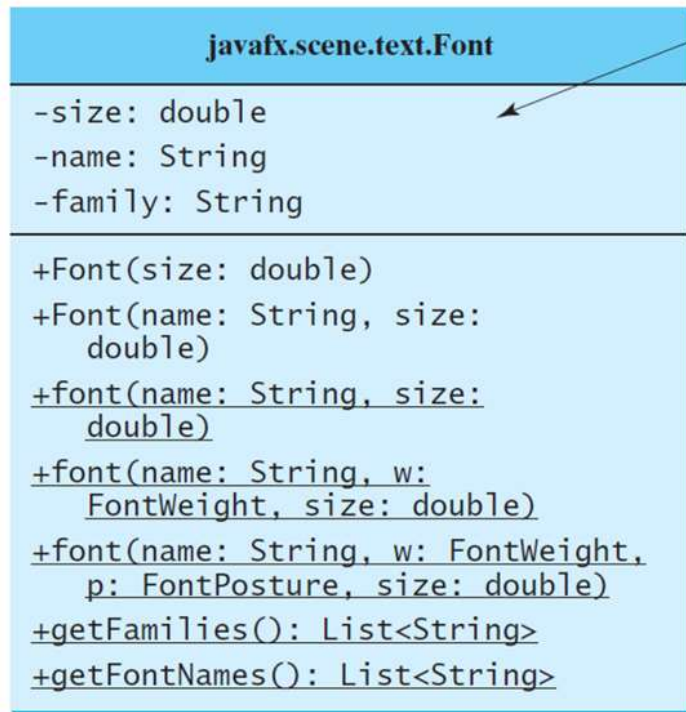
Creates an opaque Color with the specified red, green, and blue values.

Creates a Color with the specified red, green, blue, and opacity values.

Creates a Color with the specified red, green, and blue values in the range from 0 to 255.

Creates a Color with the specified red, green, and blue values in the range from 0 to 255 and a given opacity.

The Font Class



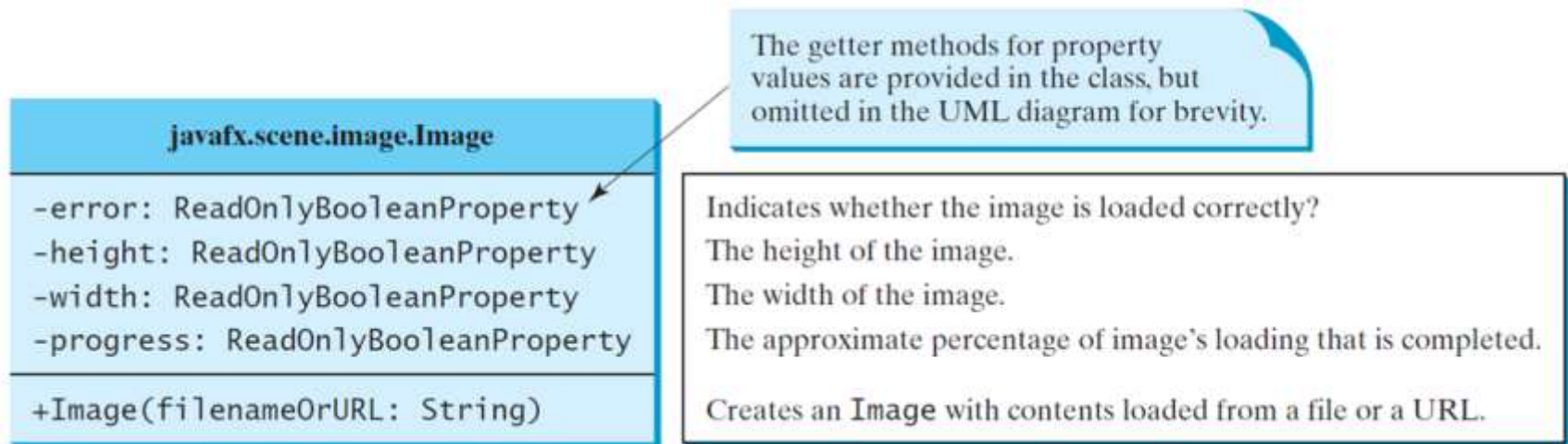
The getter methods for property values are provided in the class, but omitted in the UML diagram for brevity.

The size of this font.
The name of this font.
The family of this font.

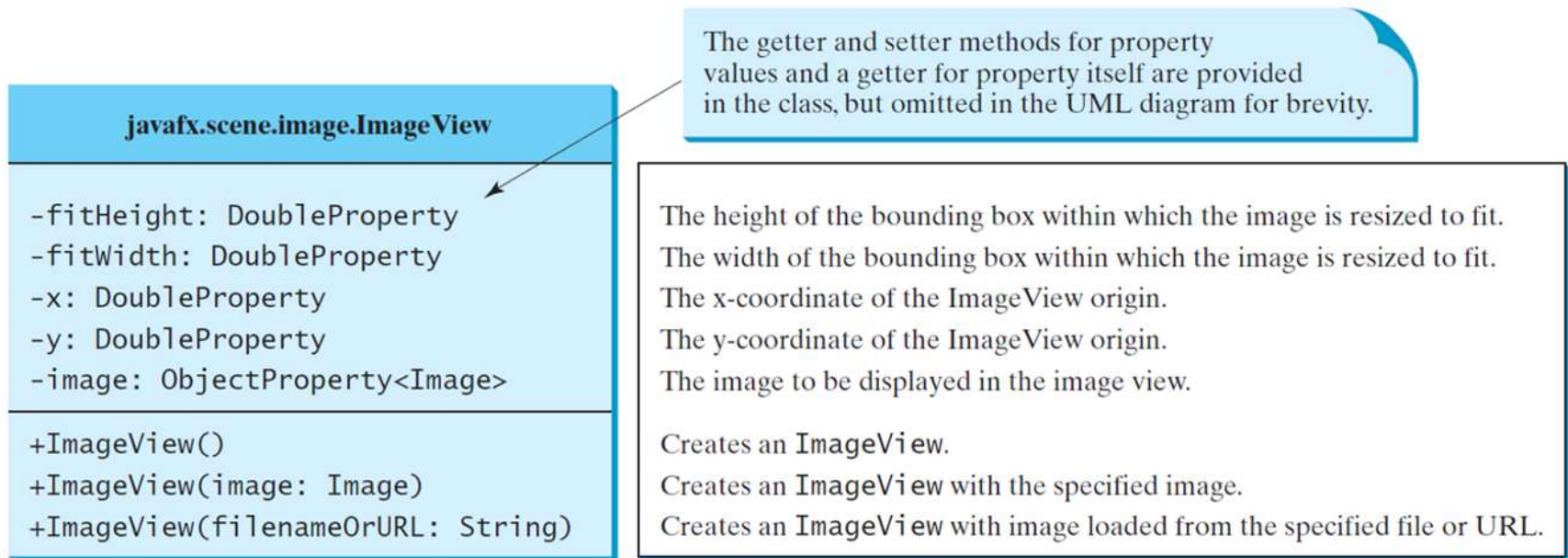
Creates a `Font` with the specified size.
Creates a `Font` with the specified full font name and size.
Creates a `Font` with the specified name and size.
Creates a `Font` with the specified name, weight, and size.
Creates a `Font` with the specified name, weight, posture, and size.
Returns a list of font family names.
Returns a list of full font names including family and weight.

FontDemo

The Image Class



The ImageView Class



ShowImage

Layout Panes

JavaFX provides many types of panes for organizing nodes in a container.

Class	Description
Pane	Base class for layout panes. It contains the <code>getChildren()</code> method for returning a list of nodes in the pane.
StackPane	Places the nodes on top of each other in the center of the pane.
FlowPane	Places the nodes row-by-row horizontally or column-by-column vertically.
GridPane	Places the nodes in the cells in a two-dimensional grid.
BorderPane	Places the nodes in the top, right, bottom, left, and center regions.
HBox	Places the nodes in a single row.
VBox	Places the nodes in a single column.

FlowPane

javafx.scene.layout.FlowPane

-alignment: `ObjectProperty<Pos>`
-orientation:
 `ObjectProperty<Orientation>`
-hgap: `DoubleProperty`
-vgap: `DoubleProperty`

+FlowPane()
+FlowPane(hgap: double, vgap:
 double)
+FlowPane(orientation:
 `ObjectProperty<Orientation>`)
+FlowPane(orientation:
 `ObjectProperty<Orientation>`,
 hgap: double, vgap: double)

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The overall alignment of the content in this pane (default: `Pos.LEFT`).
The orientation in this pane (default: `Orientation.HORIZONTAL`).

The horizontal gap between the nodes (default: 0).
The vertical gap between the nodes (default: 0).

Creates a default `FlowPane`.

Creates a `FlowPane` with a specified horizontal and vertical gap.

Creates a `FlowPane` with a specified orientation.

Creates a `FlowPane` with a specified orientation, horizontal gap and vertical gap.

MultipleStageDemo

GridPane

javafx.scene.layout.GridPane

-alignment: ObjectProperty<Pos>
-gridLinesVisible: BooleanProperty
-hgap: DoubleProperty
-vgap: DoubleProperty

+GridPane()
+add(child: Node, columnIndex: int, rowIndex: int): void
+addColumn(columnIndex: int, children: Node...): void
+addRow(rowIndex: int, children: Node...): void
+getColumnIndex(child: Node): int
+setColumnIndex(child: Node, columnIndex: int): void
+getRowIndex(child: Node): int
+setRowIndex(child: Node, rowIndex: int): void
+setHalignment(child: Node, value: HPos): void
+setValignment(child: Node, value: VPos): void

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

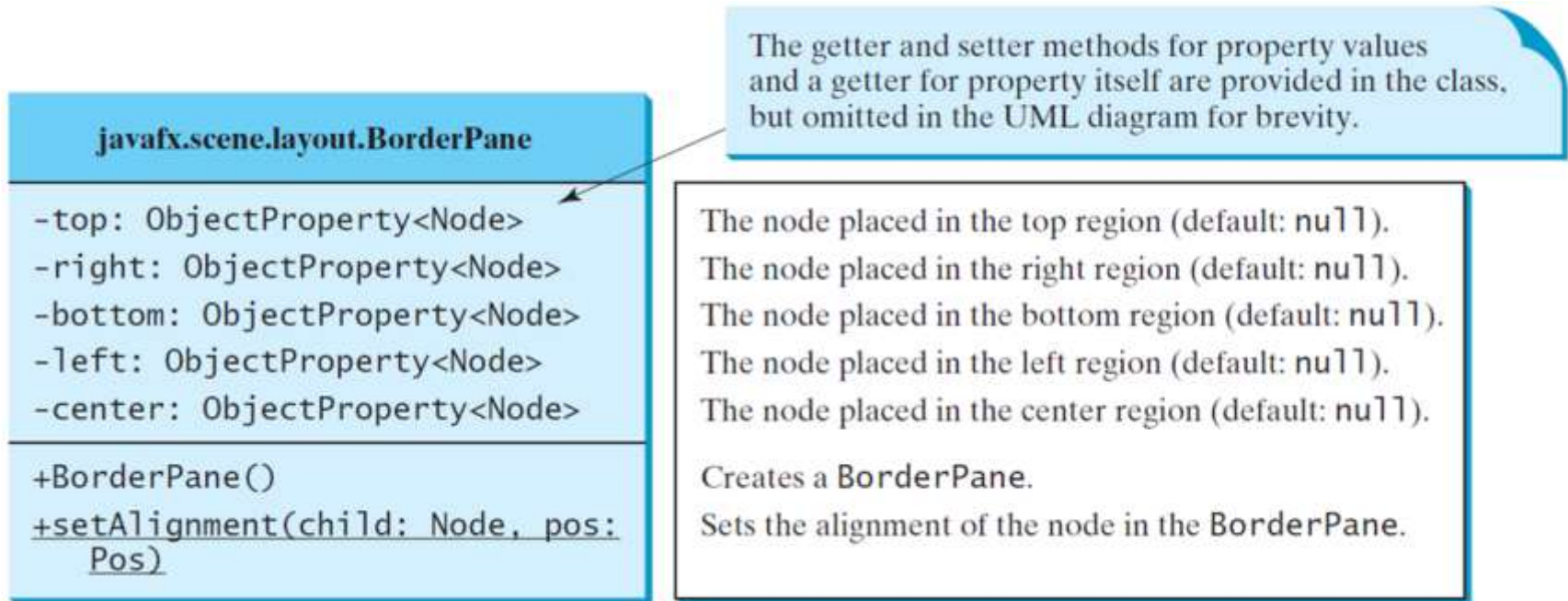
The overall alignment of the content in this pane (default: Pos.LEFT).
Is the grid line visible? (default: false)

The horizontal gap between the nodes (default: 0).
The vertical gap between the nodes (default: 0).

Creates a GridPane.
Adds a node to the specified column and row.
Adds multiple nodes to the specified column.
Adds multiple nodes to the specified row.
Returns the column index for the specified node.
Sets a node to a new column. This method repositions the node.
Returns the row index for the specified node.
Sets a node to a new row. This method repositions the node.
Sets the horizontal alignment for the child in the cell.
Sets the vertical alignment for the child in the cell.

[ShowGridPane](#)

BorderPane



ShowBorderPane

HBox

javafx.scene.layout.HBox

-alignment: ObjectProperty<Pos>
-fillHeight: BooleanProperty
-spacing: DoubleProperty

+HBox()
+HBox(spacing: double)
+setMargin(node: Node, value: Insets): void

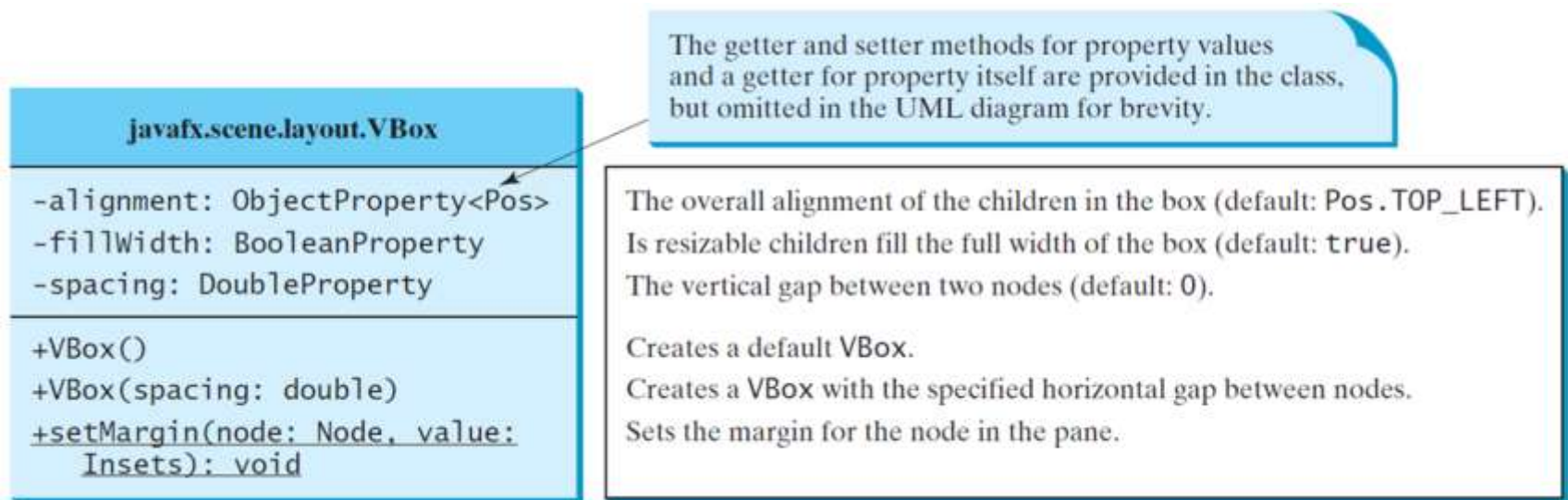
The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The overall alignment of the children in the box (default: `Pos.TOP_LEFT`).
Is resizable children fill the full height of the box (default: `true`).
The horizontal gap between two nodes (default: 0).

Creates a default HBox.

Creates an HBox with the specified horizontal gap between nodes.
Sets the margin for the node in the pane.

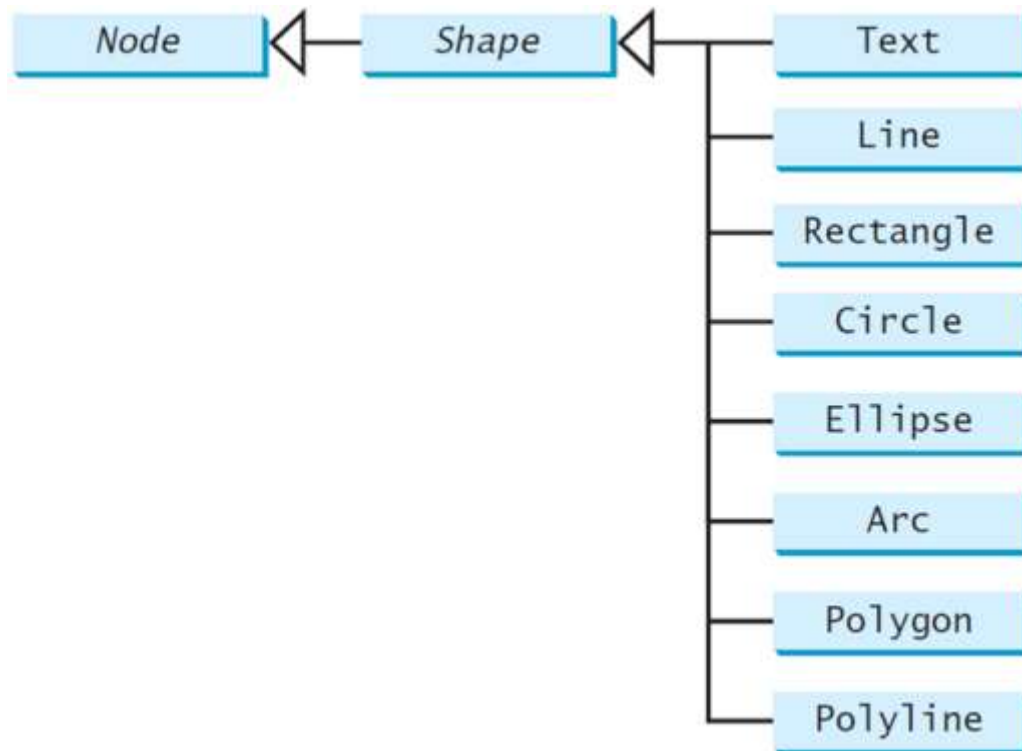
VBox



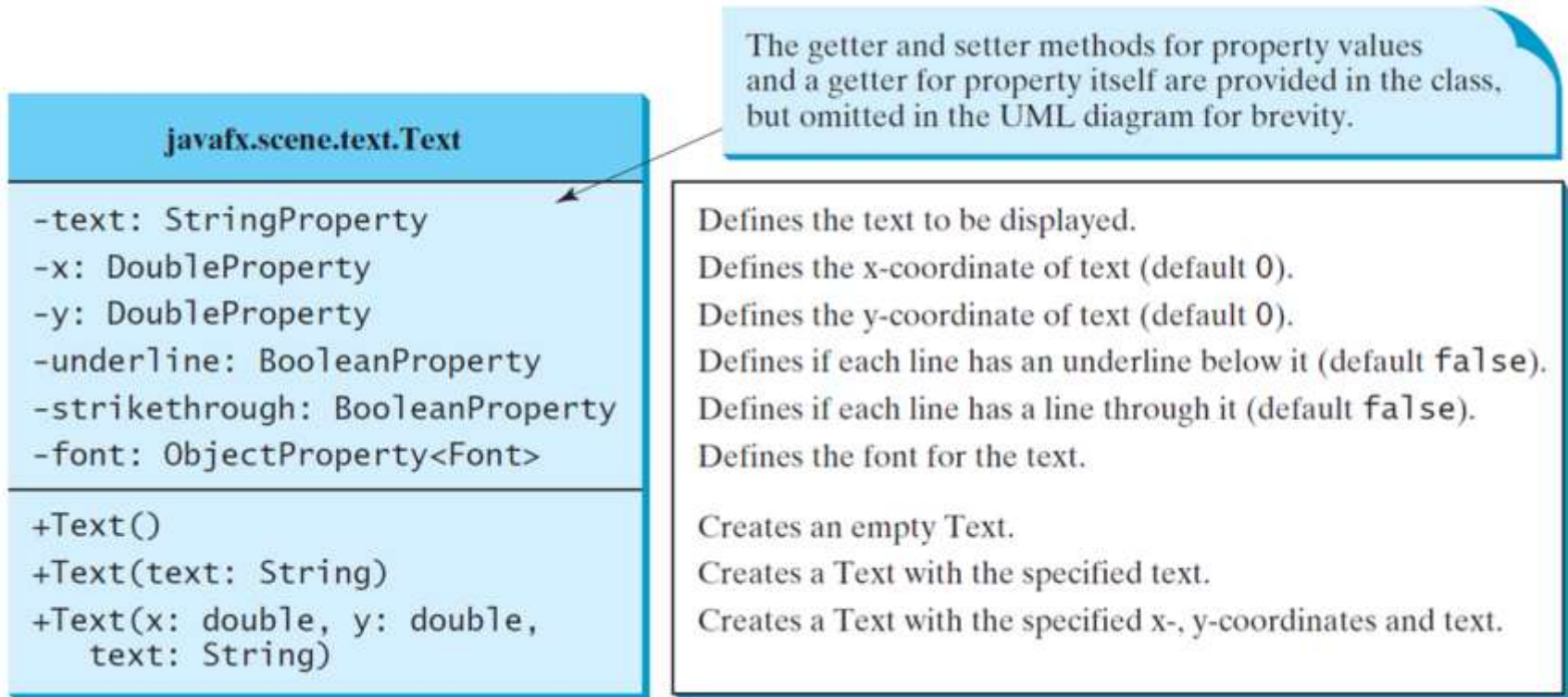
ShowHBoxVBox

Shapes

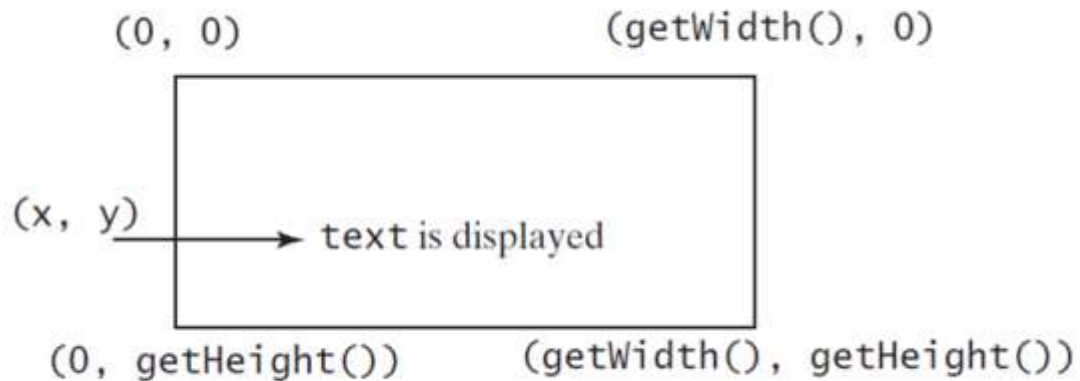
JavaFX provides many shape classes for drawing texts, lines, circles, rectangles, ellipses, arcs, polygons, and polylines.



Text



Text Example



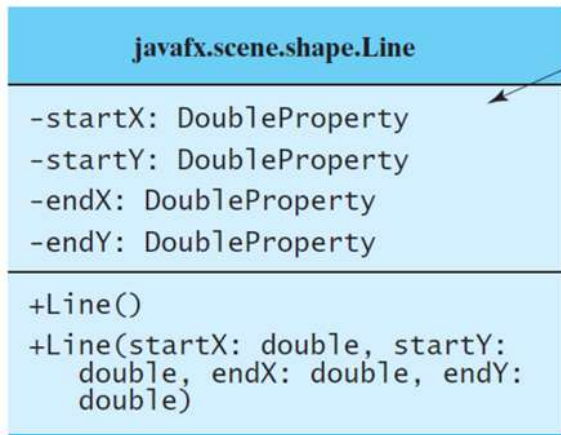
(a) `Text(x, y, text)`



(b) *Three Text objects are displayed*

ShowText

Line



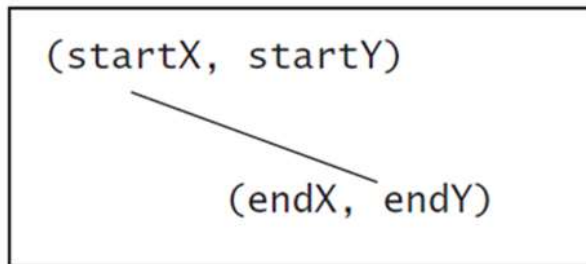
The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The x-coordinate of the start point.
The y-coordinate of the start point.
The x-coordinate of the end point.
The y-coordinate of the end point.

Creates an empty `Line`.
Creates a `Line` with the specified starting and ending points.

`(0, 0)`

`(getWidth(), 0)`

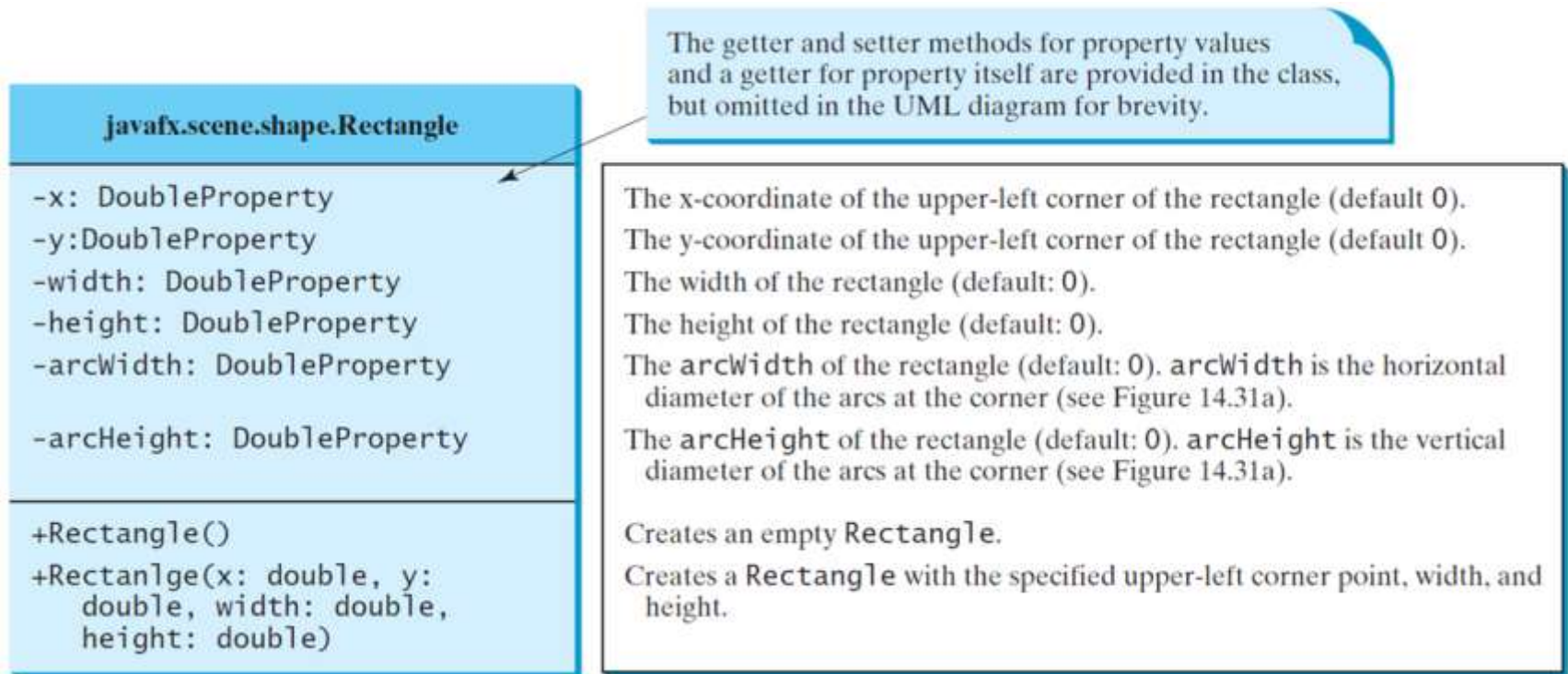


`(0, getHeight())`

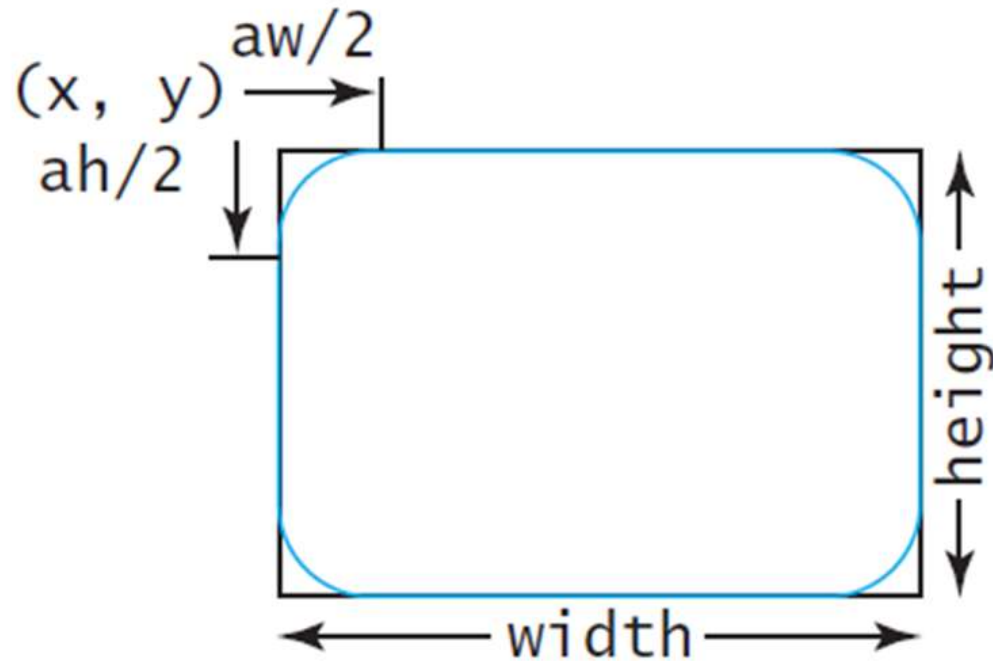
`(getWidth(), getHeight())`

[ShowLine](#)

Rectangle



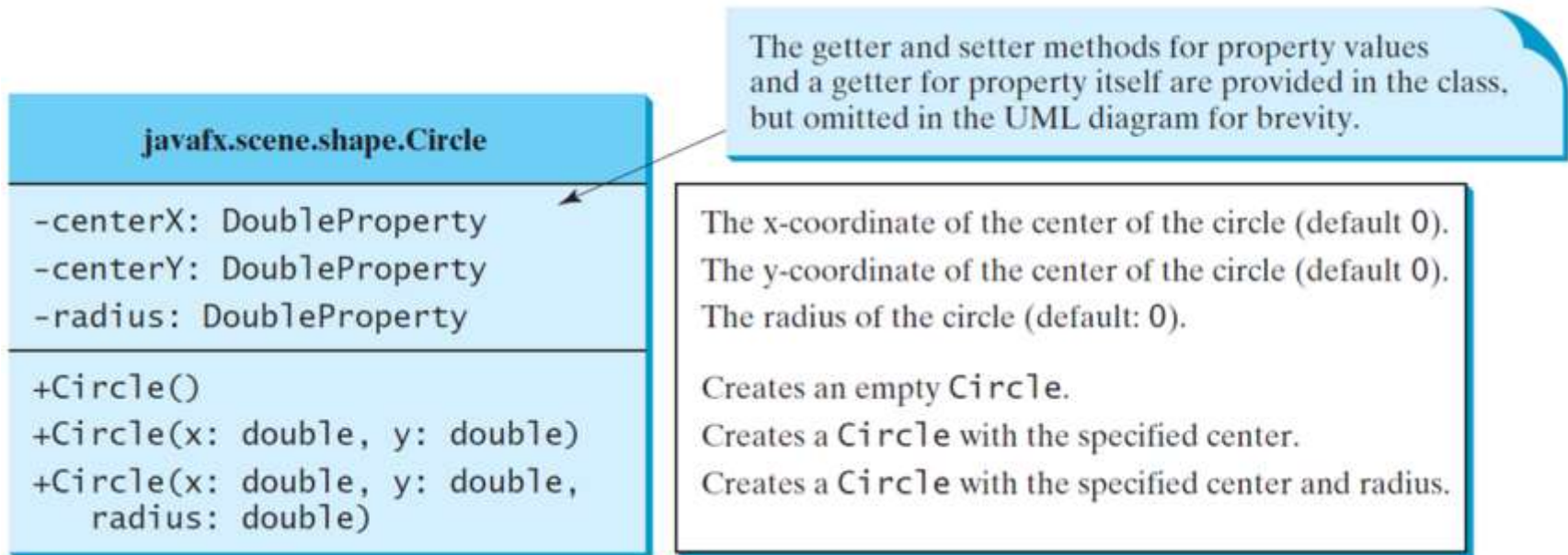
Rectangle Example



(a) `Rectangle(x, y, w, h)`

[ShowRectangle](#)

Circle



Ellipse

<code>javafx.scene.shape.Ellipse</code>
<code>-centerX: DoubleProperty</code> <code>-centerY: DoubleProperty</code> <code>-radiusX: DoubleProperty</code> <code>-radiusY: DoubleProperty</code>
<code>+Ellipse()</code> <code>+Ellipse(x: double, y: double)</code> <code>+Ellipse(x: double, y: double, radiusX: double, radiusY: double)</code>

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The x-coordinate of the center of the ellipse (default 0).

The y-coordinate of the center of the ellipse (default 0).

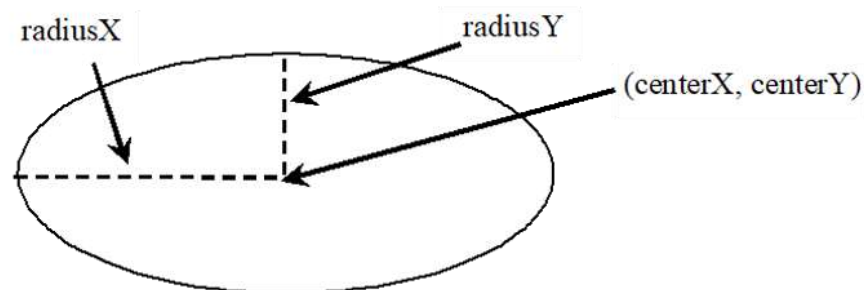
The horizontal radius of the ellipse (default: 0).

The vertical radius of the ellipse (default: 0).

Creates an empty `Ellipse`.

Creates an `Ellipse` with the specified center.

Creates an `Ellipse` with the specified center and radiuses.



[ShowEllipse](#)

Arc

javafx.scene.shape.Arc

-centerX: DoubleProperty
-centerY: DoubleProperty
-radiusX: DoubleProperty
-radiusY: DoubleProperty
-startAngle: DoubleProperty
-length: DoubleProperty
-type: ObjectProperty<ArcType>

+Arc()
+Arc(x: double, y: double,
radiusX: double, radiusY:
double, startAngle: double,
length: double)

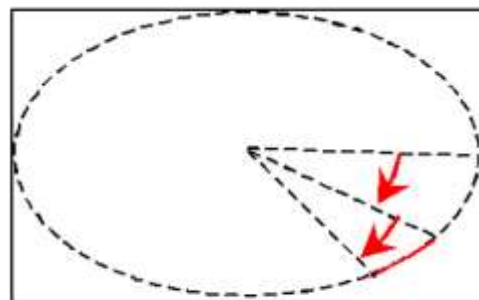
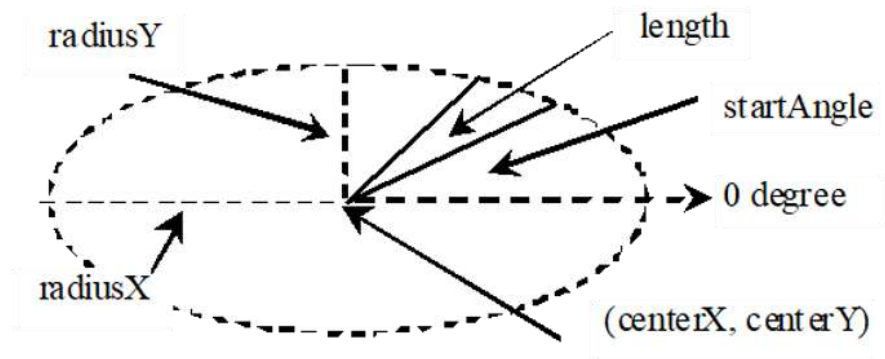
The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The x-coordinate of the center of the ellipse (default 0).
The y-coordinate of the center of the ellipse (default 0).
The horizontal radius of the ellipse (default: 0).
The vertical radius of the ellipse (default: 0).
The start angle of the arc in degrees.
The angular extent of the arc in degrees.
The closure type of the arc (ArcType.OPEN, ArcType.CHORD, ArcType.ROUND).

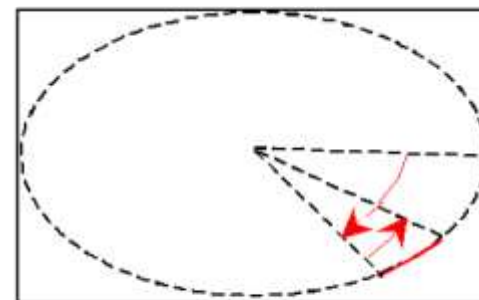
Creates an empty Arc.

Creates an Arc with the specified arguments.

Arc Examples



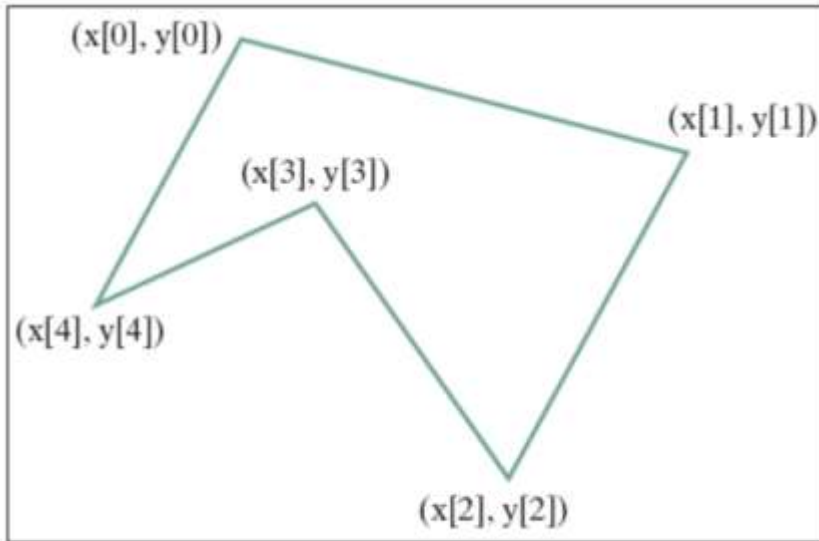
(a) Negative starting angle -30° and negative spanning angle -20°



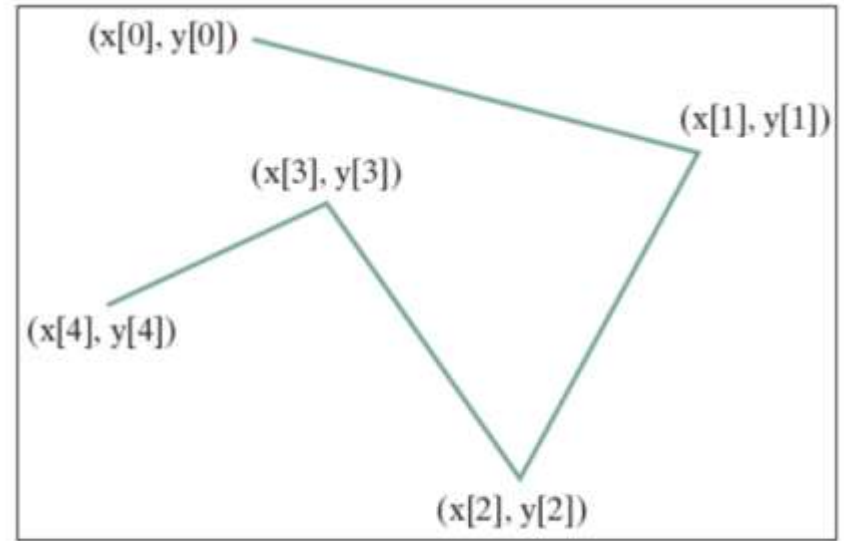
(b) Negative starting angle -50° and positive spanning angle 20°

[ShowArc](#)

Polygon and Polyline

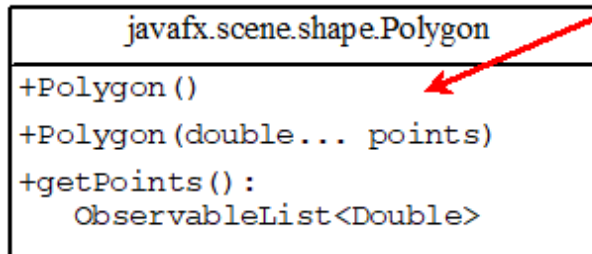


(a) Polygon



(b) Polyline

Polygon



The `getter` and `setter` methods for property values and a `getter` for property itself are provided in the class, but omitted in the UML diagram for brevity.

Creates an empty polygon.

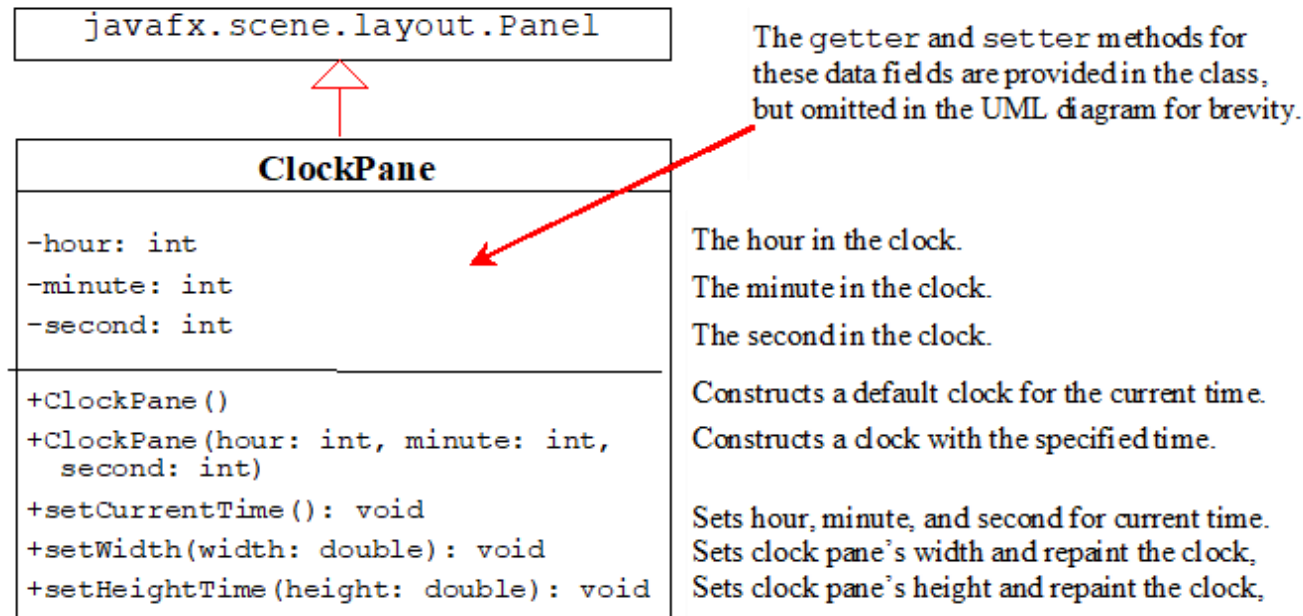
Creates a polygon with the given points.

Returns a list of double values as x- and y-coordinates of the points.

[ShowPolygon](#)

Case Study: The ClockPane Class

This case study develops a class that displays a clock on a pane.



[ClockPane](#)

Use the ClockPane Class

[DisplayClock](#)

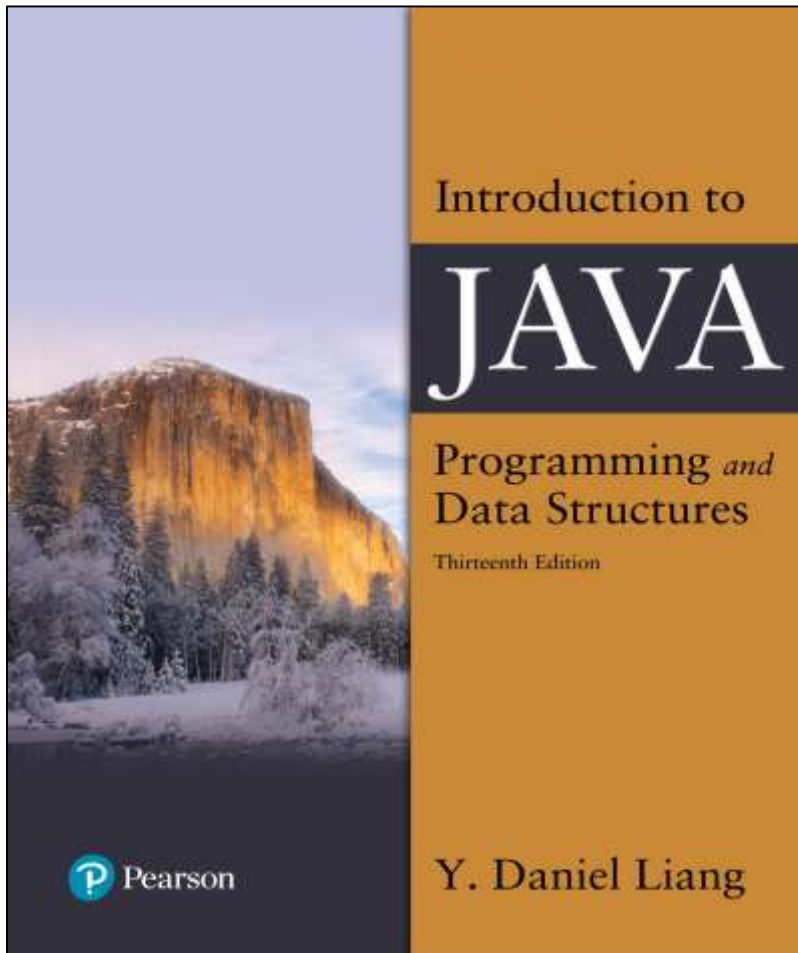
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 15

Event-Driven
Programming and
Animations

Motivations

Suppose you want to write a GUI program that lets the user enter a loan amount, annual interest rate, and number of years and click the **Compute Payment** button to obtain the monthly payment and total payment. How do you accomplish the task? You have to use **event-driven programming** to write the code to respond to the button-clicking event.



The screenshot shows a window titled "LoanCalculator" with a standard Windows title bar (minimize, maximize, close buttons). Inside the window, there are five input fields with corresponding labels to their left: "Annual Interest Rate:" with the value "4.5", "Number of Years:" with the value "4", "Loan Amount:" with the value "5000", "Monthly Payment:" with the value "\$114.02", and "Total Payment:" with the value "\$5472.84". At the bottom right of the window is a button labeled "Calculator".

Label	Value
Annual Interest Rate:	4.5
Number of Years:	4
Loan Amount:	5000
Monthly Payment:	\$114.02
Total Payment:	\$5472.84

[LoanCalculator](#)

Objectives (1 of 2)

15.1 To get a taste of event-driven programming (§15.1).

15.2 To describe events, event sources, and event classes (§15.2).

15.3 To define handler classes, register handler objects with the source object, and write the code to handle events (§15.3).

15.4 To define handler classes using inner classes (§15.4).

15.5 To define handler classes using anonymous inner classes (§15.5).

15.6 To simplify event handling using lambda expressions (§15.6).

Objectives (2 of 2)

15.7 To develop a GUI application for a loan calculator (§15.7).

15.8 To write programs to deal with **MouseEvent**s (§15.8).

15.9 To write programs to deal with **KeyEvent**s (§15.9).

15.10 To create listeners for processing a value change in an observable object (§15.10).

15.11 To use the **Animation**, **PathTransition**, **FadeTransition**, and **Timeline** classes to develop animations (§15.11).

15.12 To develop an animation for simulating a bouncing ball (§15.12).

15.13 To draw, color, and resize a US map (§15.13).

Procedural vs Event-Driven Programming

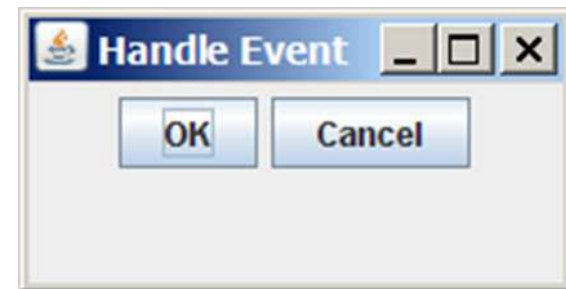
- **Procedural programming** is executed in procedural order.
- In event-driven programming, code is executed upon activation of events.

Taste of Event-Driven Programming

The example displays a button in the frame. A message is displayed on the console when a button is clicked.



```
Command Prompt - java HandleEvent
C:\book>java HandleEvent
OK button clicked
Cancel button clicked
OK button clicked
```

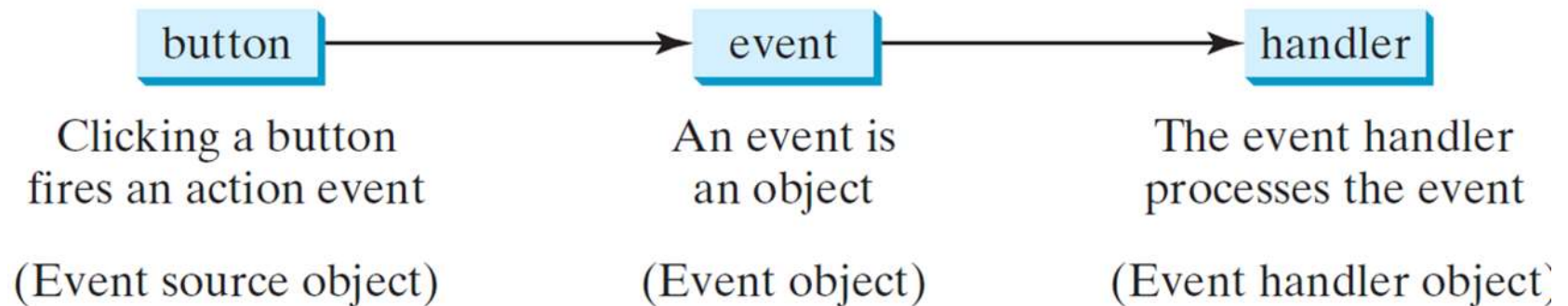


[HandleEvent](#)

Handling GUI Events

Source object (e.g., button)

Listener object contains a method for processing the event.



Trace Execution (1 of 3)

```
public class HandleEvent extends Application {  
    public void start(Stage primaryStage) {  
        ...  
        OKHandlerClass handler1 = new OKHandlerClass();  
        btOK.setOnAction(handler1);  
        CancelHandlerClass handler2 = new CancelHandlerClass(),  
        btCancel.setOnAction(handler2);  
        ...  
        primaryStage.show(); // Display the stage  
    }  
}
```

1. Start from the main method to create a window and display it



```
class OKHandlerClass implements EventHandler<ActionEvent> {  
    @Override  
    public void handle(ActionEvent e) {  
        System.out.println("OK button clicked");  
    }  
}
```

Trace Execution (2 of 3)

```
public class HandleEvent extends Application {  
    public void start(Stage primaryStage) {  
        ...  
        OKHandlerClass handler1 = new OKHandlerClass();  
        btOK.setOnAction(handler1);  
        CancelHandlerClass handler2 = new CancelHandlerClass();  
        btCancel.setOnAction(handler2);  
        ...  
        primaryStage.show(); // Display the stage  
    }  
}
```

```
class OKHandlerClass implements EventHandler<ActionEvent> {  
    @Override  
    public void handle(ActionEvent e) {  
        System.out.println("OK button clicked");  
    }  
}
```

2. Click OK



Trace Execution (3 of 3)

```
public class HandleEvent extends Application {  
    public void start(Stage primaryStage) {  
        ...  
        OKHandlerClass handler1 = new OKHandlerClass();  
        btOK.setOnAction(handler1);  
        CancelHandlerClass handler2 = new CancelHandlerClass();  
        btCancel.setOnAction(handler2);  
        ...  
        primaryStage.show(); // Display the stage  
    }  
}
```

3. The JVM
invokes the
listener's handle
method



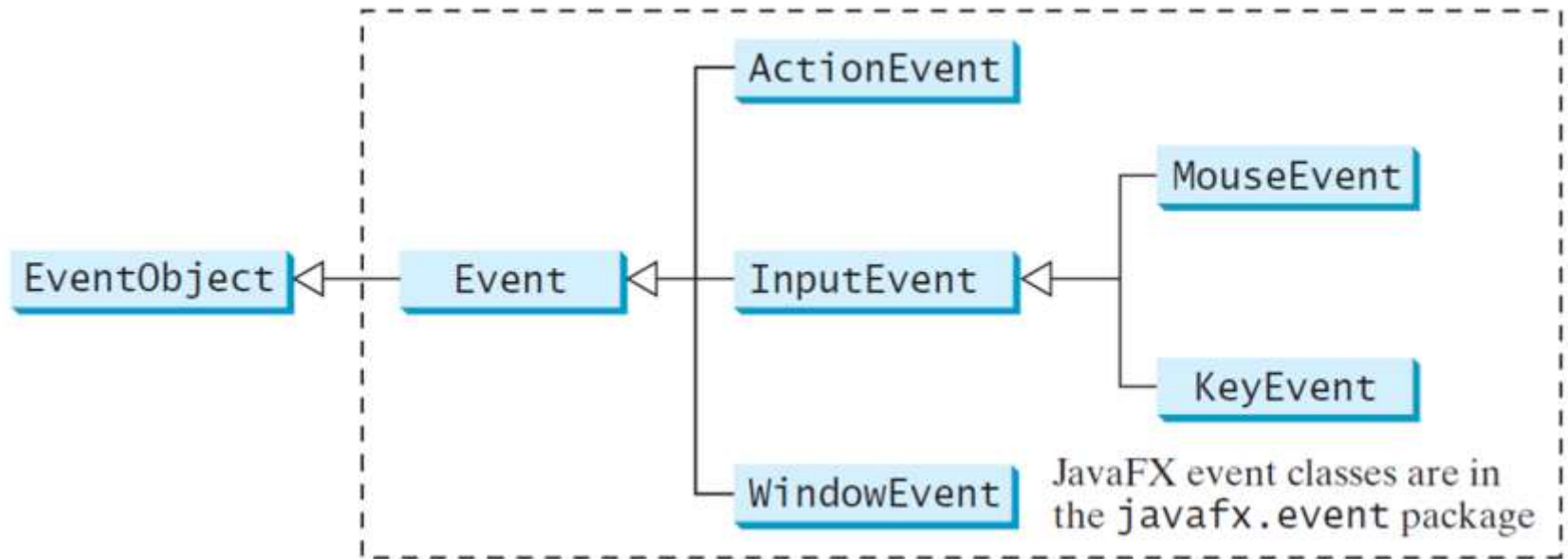
```
class OKHandlerClass implements EventHandler<ActionEvent> {  
    @Override  
    public void handle(ActionEvent e) {  
        System.out.println("OK button clicked");  
    }  
}
```



Events

- An **event** can be defined as a type of signal to the program that something has happened.
- The event is generated by external user actions such as mouse movements, mouse clicks, or keystrokes.

Event Classes



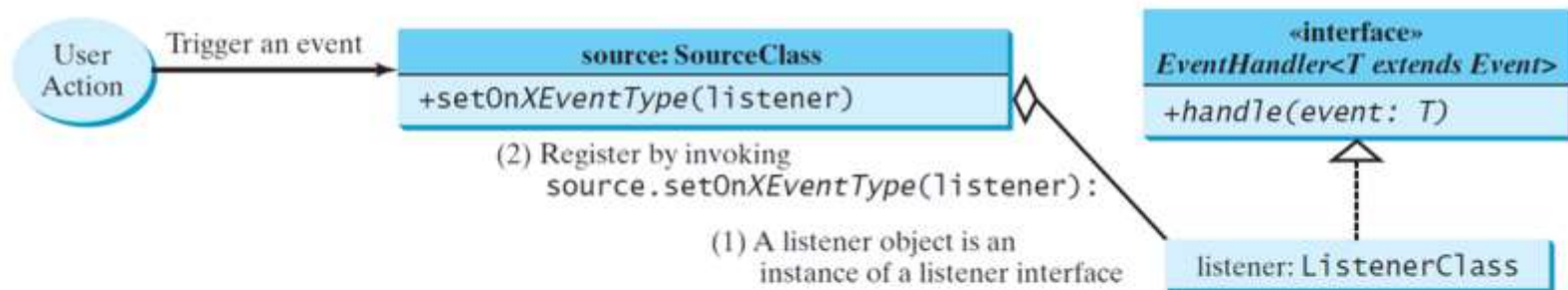
Event Information

An event object contains whatever properties are pertinent to the event. You can identify the source object of the event using the `getSource()` instance method in the `EventObject` class. The subclasses of `EventObject` deal with special types of events, such as button actions, window events, mouse movements, and keystrokes. Table 15.1 lists external user actions, source objects, and event types generated.

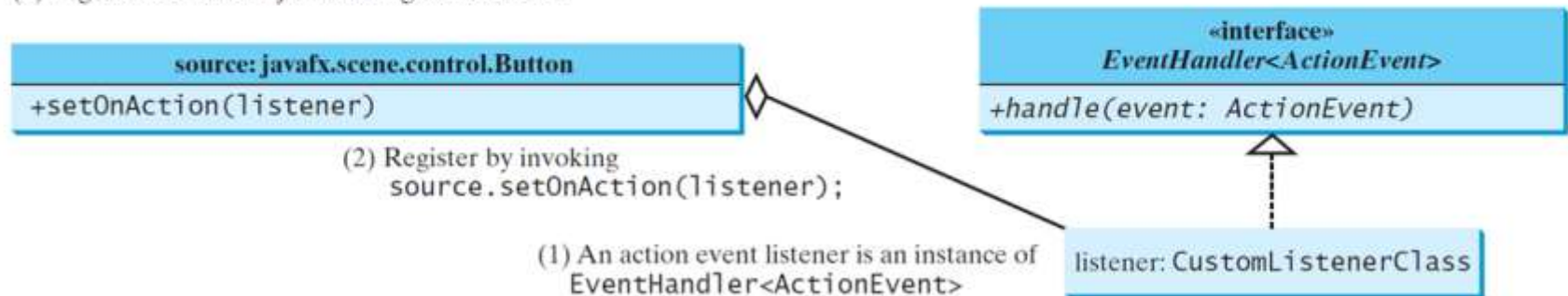
Selected User Actions and Handlers

User Action	Source Object	Event Type Fired	Event Registration Method
Click a button	Button	ActionEvent	setOnAction (EventHandler<ActionEvent>)
Press Enter in a text field	TextField	ActionEvent	setOnAction (EventHandler<ActionEvent>)
Check or uncheck	RadioButton	ActionEvent	setOnAction (EventHandler<ActionEvent>)
Check or uncheck	CheckBox	ActionEvent	setOnAction (EventHandler<ActionEvent>)
Select a new; item	ComboBox	ActionEvent	setOnAction (EventHandler<ActionEvent>)
Mouse pressed	Node, Scene	MouseEvent	setOnMousePressed (EventHandler<MouseEvent>)
Mouse released	-	-	setOnMouseReleased (EventHandler<MouseEvent>)
Mouse clicked	-	-	setOnMouseClicked (EventHandler<MouseEvent>)
Mouse entered	-	-	setOnMouseEntered (EventHandler<MouseEvent>)
Mouse exited	-	-	setOnMouseExited (EventHandler<MouseEvent>)
Mouse moved	-	-	setOnMouseMoved (EventHandler<MouseEvent>)
Mouse dragged	-	-	setOnMouseDragged (EventHandler<MouseEvent>)
Key pressed:	Node, Scene	KeyEvent	setOnKeyPressed (EventHandler<KeyEvent>)
Key released	-	-	setOnKeyReleased (EventHandler<KeyEvent>)
Key typed	-	-	setOnKeyTyped (EventHandler<KeyEvent>)

The Delegation Model



(a) A generic source object with a generic event T



(b) A Button source object with an ActionEvent

The Delegation Model: Example

```
Button btOK = new Button("OK");
```

```
OKHandlerClass handler = new  
OKHandlerClass();
```

```
btOK.setAction(handler);
```

Example: First Version for Controlcircle (No Listeners)

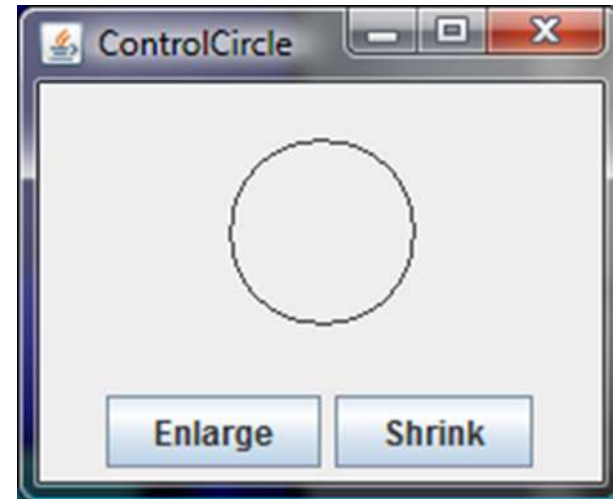
Now let us consider to write a program that uses two buttons to control the size of a circle.



[ControlCircleWithoutEventHandling](#)

Example: Second Version for ControlCircle (With Listener for Enlarge)

Now let us consider to write a program that uses two buttons to control the size of a circle.



[ControlCircle](#)

Inner Class Listeners

A listener class is designed specifically to create a listener object for a GUI component (e.g., a button). It will not be shared by other applications. So, it is appropriate to define the listener class inside the frame class as an inner class.

Inner Classes (1 of 4)

Inner class: A class is a member of another class.

Advantages: In some applications, you can use an inner class to make programs simple.

An inner class can reference the data and methods defined in the outer class in which it nests, so you do not need to pass the reference of the outer class to the constructor of the inner class.

[ShowInnerClass](#)

Inner Classes (2 of 4)

```
public class Test {  
    ...  
}  
  
public class A {  
    ...  
}
```

(a)

```
public class Test {  
    ...  
  
    // Inner class  
    public class A {  
        ...  
    }  
}
```

(b)

```
// OuterClass.java: inner class demo  
public class OuterClass {  
    private int data;  
  
    /** A method in the outer class */  
    public void m() {  
        // Do something  
    }  
  
    // An inner class  
    class InnerClass {  
        /** A method in the inner class */  
        public void mi() {  
            // Directly reference data and method  
            // defined in its outer class  
            data++;  
            m();  
        }  
    }  
}
```

(c)

Inner Classes (3 of 4)

Inner classes can make programs simple and concise.

An inner class supports the work of its containing outer class and is compiled into a class named **`OuterClassName$InnerClassName.class`**. For example, the inner class `InnerClass` in `OuterClass` is compiled into **`OuterClass$InnerClass.class`**.

Inner Classes (4 of 4)

- An inner class can be declared public, protected, or private subject to the same visibility rules applied to a member of the class.
- An inner class can be declared static. A static inner class can be accessed using the outer class name. A static inner class cannot access nonstatic members of the outer class

Anonymous Inner Classes (1 of 3)

- An anonymous inner class must always extend a superclass or implement an interface, but it cannot have an explicit `extends` or `implements` clause.
- An anonymous inner class must implement all the abstract methods in the superclass or in the interface.
- An anonymous inner class always uses the no-arg constructor from its superclass to create an instance. If an anonymous inner class implements an interface, the constructor is `Object()`.
- An anonymous inner class is compiled into a class named `OuterClassName$n.class`. For example, if the outer class `Test` has two anonymous inner classes, these two classes are compiled into `Test$1.class` and `Test$2.class`.

Anonymous Inner Classes (2 of 3)

Inner class listeners can be shortened using anonymous inner classes. An **anonymous inner class** is an inner class without a name. It combines declaring an inner class and creating an instance of the class in one step. An anonymous inner class is declared as follows:

```
new SuperClassName/InterfaceName() {  
    // Implement or override methods in superclass or  
    interface  
    // Other methods if necessary  
}
```

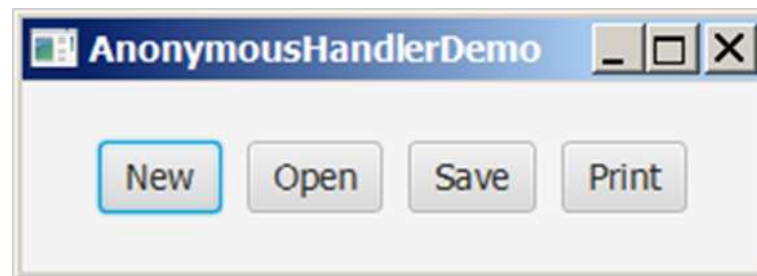
Anonymous Inner Classes (3 of 3)

```
public void start(Stage primaryStage) {  
    // Omitted  
  
    btEnlarge.setOnAction(  
        new EnlargeHandler());  
}  
  
class EnlargeHandler  
    implements EventHandler<ActionEvent> {  
    public void handle(ActionEvent e) {  
        circlePane.enlarge();  
    }  
}
```

(a) Inner class EnlargeListener

```
public void start(Stage primaryStage) {  
    // Omitted  
  
    btEnlarge.setOnAction(  
        new class EnlargeHandler  
            implements EventHandler<ActionEvent>() {  
                public void handle(ActionEvent e) {  
                    circlePane.enlarge();  
                }  
            }  
    );  
}
```

(b) Anonymous inner class



[AnonymousHandlerDemo](#)

Simplifying Event Handling Using Lambda Expressions

Lambda expression is a new feature in Java 8. Lambda expressions can be viewed as an anonymous method with a concise syntax. For example, the following code in (a) can be greatly simplified using a lambda expression in (b) in three lines.

```
btEnlarge.setOnAction(  
    new EventHandler<ActionEvent>() {  
        @Override  
        public void handle(ActionEvent e) {  
            // Code for processing event e  
        }  
    }  
);
```

(a) Anonymous inner class event handler

```
btEnlarge.setOnAction(e -> {  
    // Code for processing event e  
});
```

(b) Lambda expression event handler

Basic Syntax for a Lambda Expression

The basic syntax for a lambda expression is either

```
(type1 param1, type2 param2, ...) -> expression
```

or

```
(type1 param1, type2 param2, ...) -> { statements; }
```

The data type for a parameter may be explicitly declared or implicitly inferred by the compiler. The parentheses can be omitted if there is only one parameter without an explicit data type.

Single Abstract Method Interface (SAM)

The statements in the lambda expression is all for that method. If it contains multiple methods, the compiler will not be able to compile the lambda expression. So, for the compiler to understand lambda expressions, the interface must contain exactly one abstract method. Such an interface is known as a **functional interface**, or a **Single Abstract Method (SAM)** interface.

[AnonymousHandlerDemo](#)

Problem: Loan Calculator

LoanCalculator

The MouseEvent Class

`javafx.scene.input.MouseEvent`

```
+getButton(): MouseButton  
+getClickCount(): int  
+getX(): double  
+getY(): double  
+getSceneX(): double  
+getSceneY(): double  
+getScreenX(): double  
+getScreenY(): double  
+isAltDown(): boolean  
+isControlDown(): boolean  
+isMetaDown(): boolean  
+isShiftDown(): boolean
```

Indicates which mouse button has been clicked.

Returns the number of mouse clicks associated with this event.

Returns the *x*-coordinate of the mouse point in the event source node.

Returns the *y*-coordinate of the mouse point in the event source node.

Returns the *x*-coordinate of the mouse point in the scene.

Returns the *y*-coordinate of the mouse point in the scene.

Returns the *x*-coordinate of the mouse point in the screen.

Returns the *y*-coordinate of the mouse point in the screen.

Returns true if the `Alt` key is pressed on this event.

Returns true if the `Control` key is pressed on this event.

Returns true if the mouse `Meta` button is pressed on this event.

Returns true if the `Shift` key is pressed on this event.

[MouseEventDemo](#)

The KeyEvent Class

javafx.scene.input.KeyEvent

+getCharacter(): String
+getCode(): KeyCode
+getText(): String
+isAltDown(): boolean
+isControlDown(): boolean
+isMetaDown(): boolean
+isShiftDown(): boolean

Returns the character associated with the key in this event.

Returns the key code associated with the key in this event.

Returns a string describing the key code.

Returns true if the **Alt** key is pressed on this event.

Returns true if the **Control** key is pressed on this event.

Returns true if the mouse **Meta** button is pressed on this event.

Returns true if the **Shift** key is pressed on this event.

[KeyEventDemo](#)

The KeyCode Constants

Constant	Description
Home	The Home key
End	The End key
Page_up	The Page Up key
Pace_down	The Page Down key
Up	The up-arrow key
Down	The down-arrow key
Left	The left-arrow key
Right	The right-arrow key
Escape	The Esc key
Tab	The Tab key

Constant	Description
Control	The Control key
Shift	The Shift key
Back_space	The Backspace key
Caps	The Caps Lock key
Num_lock	The Num Lock key
Enter	The Enter key
Undefined	The keyCode unknown
F1 to F12	The function keys from F1 to F12
0 to 9	The number keys from 0 to 9
A to Z	The letter keys from A to Z

Example: Control Circle With Mouse and Key

ControlCircleWithMouseAndKey

Listeners for Observable Objects

You can add a listener to process a value change in an observable object.

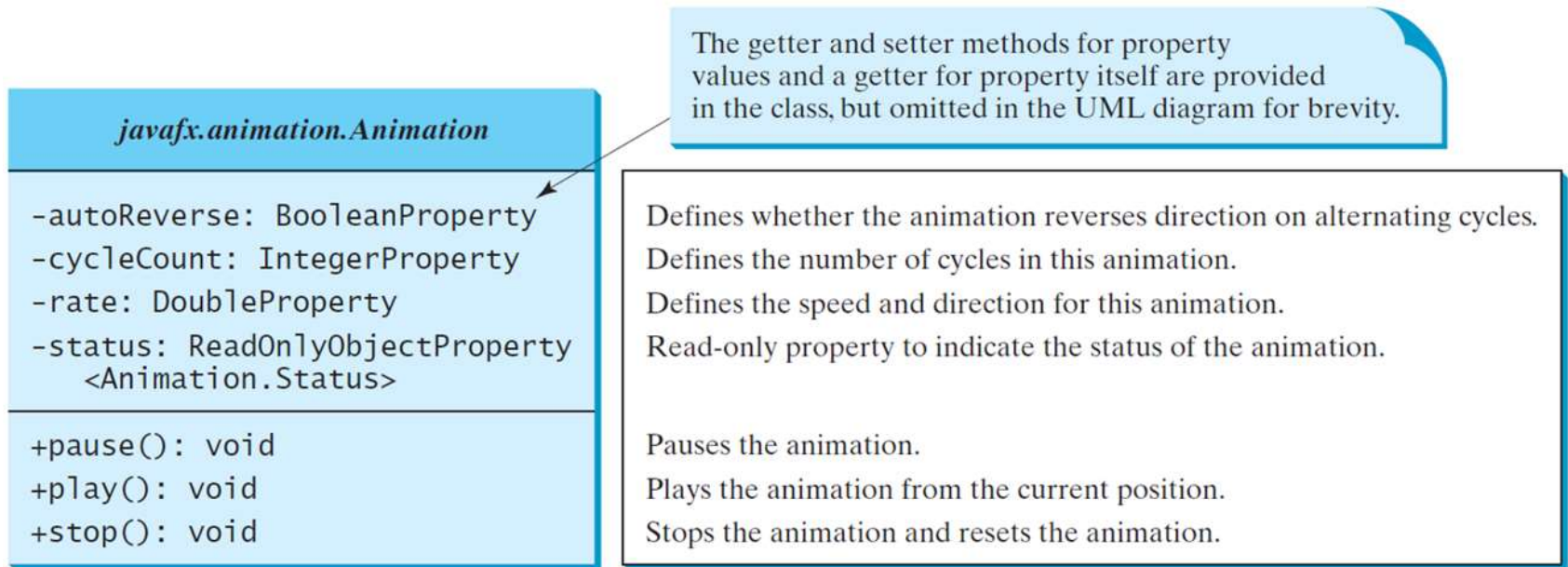
An instance of **Observable** is known as an **observable object**, which contains the **`addListener(InvalidationListener listener)`** method for adding a listener. Once the value is changed in the property, a listener is notified. The listener class should implement the **`InvalidationListener`** interface, which uses the **`invalidated(Observable o)`** method to handle the property value change. Every binding property is an instance of **Observable**.

[ObservablePropertyDemo](#)

[DisplayResizableClock](#)

Animation

JavaFX provides the **Animation** class with the core functionality for all animations.



PathTransition

javafx.animation.PathTransition

-duration: ObjectProperty<Duration>
-node: ObjectProperty<Node>
-orientation: ObjectProperty
 <PathTransition.OrientationType>
-path: ObjectProperty<Shape>

+PathTransition()
+PathTransition(duration: Duration,
 path: Shape)
+PathTransition(duration: Duration,
 path: Shape, node: Node)

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The duration of this transition.

The target node of this transition.

The orientation of the node along the path.

The shape whose outline is used as a path to animate the node move.

Creates an empty PathTransition.

Creates a PathTransition with the specified duration and path.

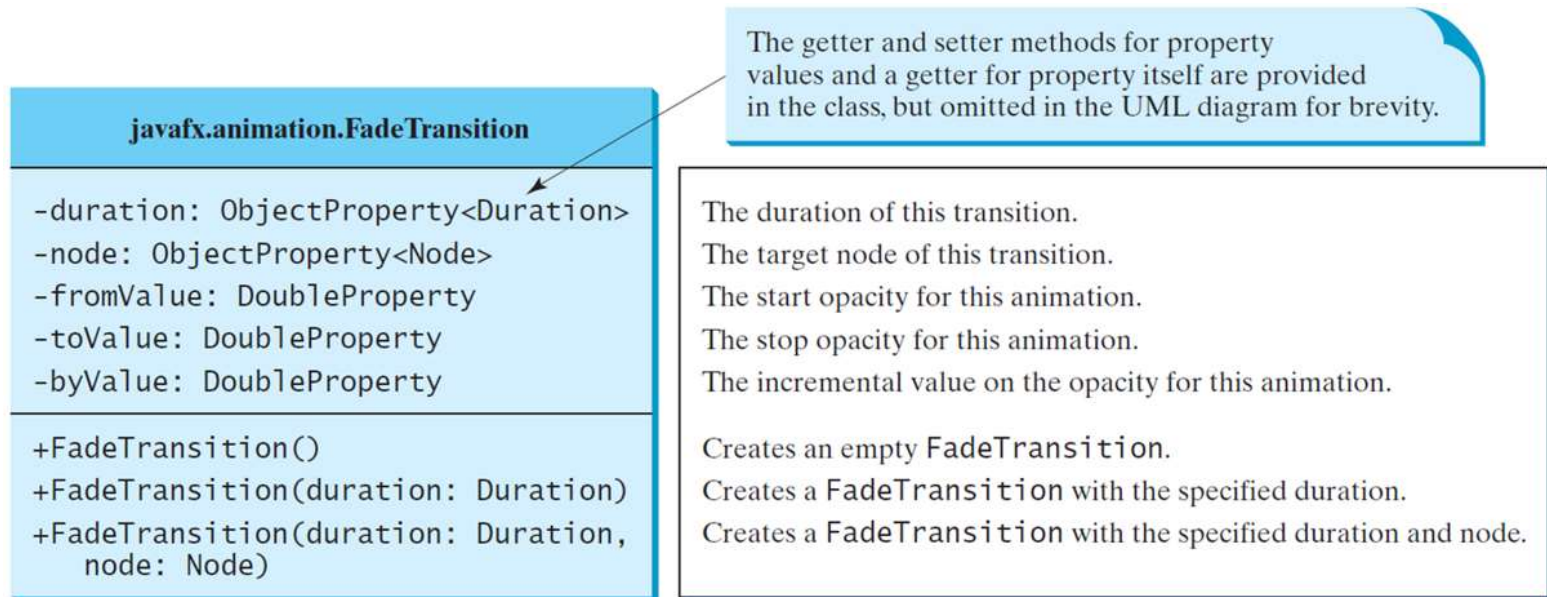
Creates a PathTransition with the specified duration, path, and node.

[PathTransitionDemo](#)

[FlagRisingAnimation](#)

FadeTransition

The **FadeTransition** class animates the change of the opacity in a node over a given time.



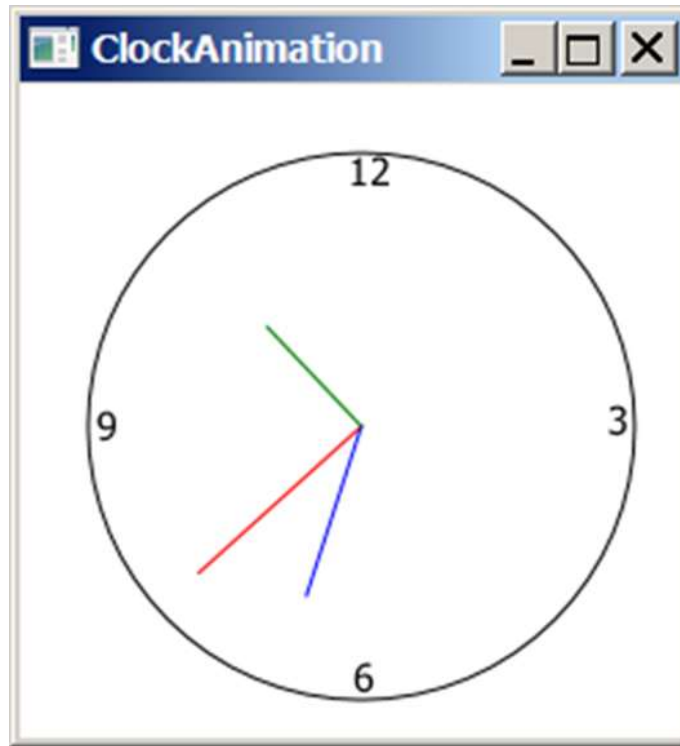
FadeTransitionDemo

Timeline

PathTransition and **FadeTransition** define specialized animations. The **Timeline** class can be used to program any animation using one or more **KeyFrames**. Each **KeyFrame** is executed sequentially at a specified time interval. **Timeline** inherits from **Animation**.

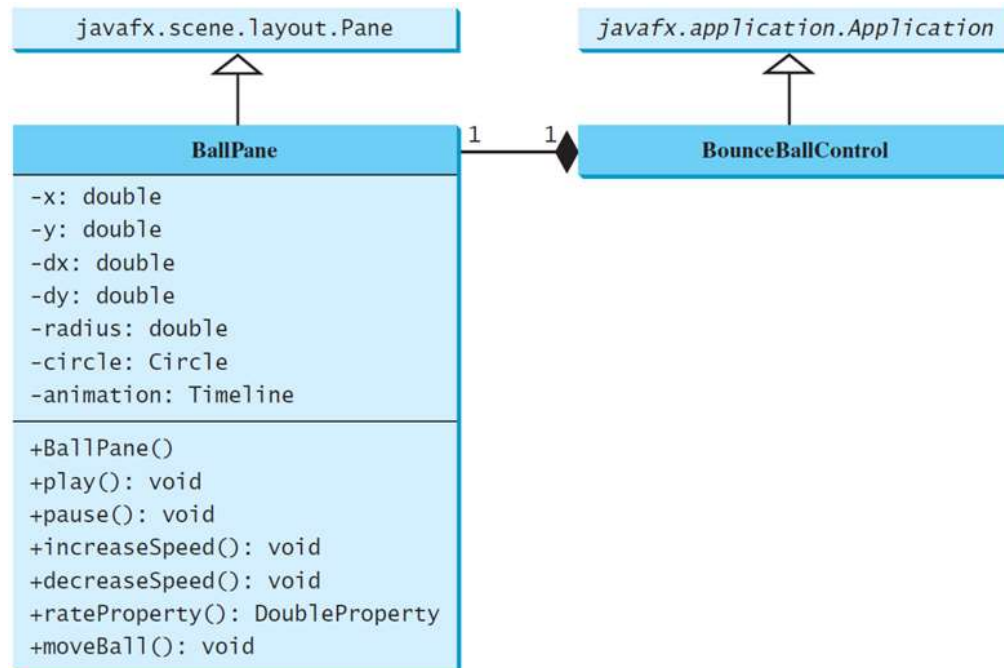
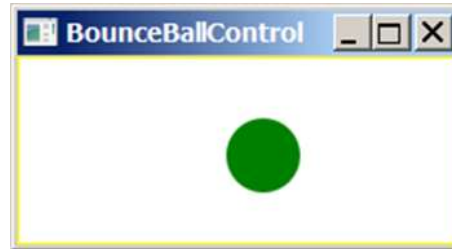
[TimelineDemo](#) Run

Clock Animation



[ClockAnimation](#)

Case Study: Bouncing Ball



BallPane

BounceBallControl

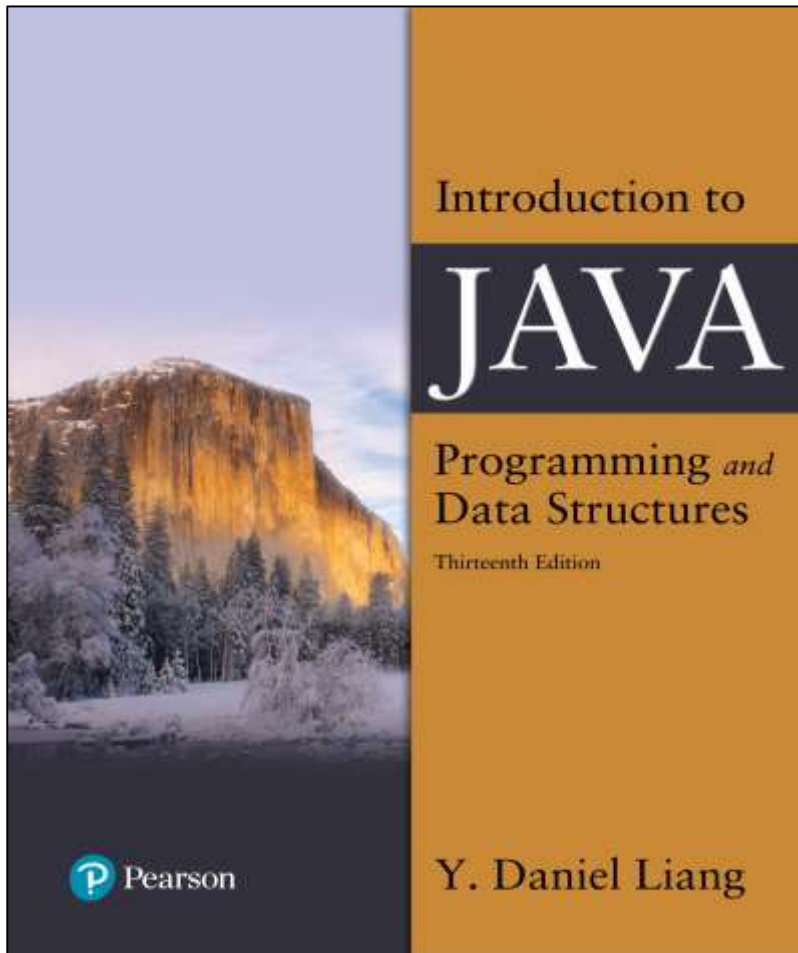
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 16

JavaFX UI Controls and Multimedia

Motivations

A graphical user interface (GUI) makes a system user-friendly and easy to use. Creating a GUI requires creativity and knowledge of how GUI components work. Since the GUI components in Java are very flexible and versatile, you can create a wide assortment of useful user interfaces.

Previous chapters briefly introduced several GUI components. This chapter introduces the frequently used GUI components in detail.

Objectives (1 of 2)

16.1 To create graphical user interfaces with various user-interface controls (§§16.2–16.11).

16.2 To create a label with text and graphic using the **Label** class and explore properties in the abstract **Labeled** class (§16.2).

16.3 To create a button with text and graphic using the **Button** class and set a handler using the **setOnAction** method in the abstract **ButtonBase** class (§16.3).

16.4 To create a check box using the **CheckBox** class (§16.4).

16.5 To create a radio button using the **RadioButton** class and group radio buttons using a **ToggleGroup** (§16.5).

16.6 To enter data using the **TextField** class and password using the **PasswordField** class (§16.6).

16.7 To enter data in multiple lines using the **TextArea** class (§16.7).

Objectives (2 of 2)

16.8 To select a single item using **ComboBox** (§16.8).

16.9 To select a single or multiple items using **ListView** (§16.9).

16.10 To select a range of values using **ScrollBar** (§16.10).

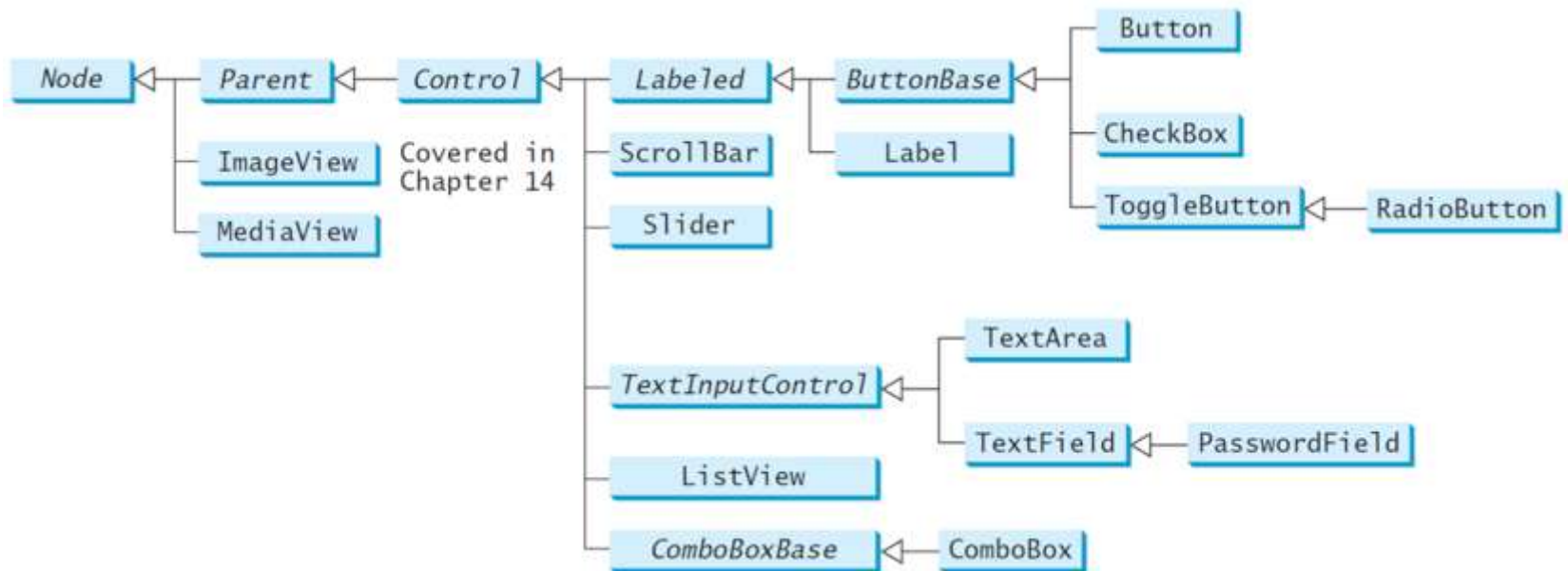
16.11 To select a range of values using **Slider** and explore differences between **ScrollBar** and **Slider** (§16.11).

16.12 To develop a tic-tac-toe game (§16.12).

16.13 To view and play video and audio using the **Media**, **MediaPlayer**, and **MediaView** (§16.13).

16.14 To develop a case study for showing the national flag and play anthem (§16.14).

Frequently Used UI Controls



Throughout this book, the prefixes **lbl**, **bt**, **chk**, **rb**, **tf**, **pf**, **ta**, **cbo**, **lv**, **scb**, **sld**, and **mp** are used to name reference variables for **Label**, **Button**, **CheckBox**, **RadioButton**, **TextField**, **PasswordField**, **TextArea**, **ComboBox**, **ListView**, **ScrollBar**, **Slider**, and **MediaPlayer**.

Labeled

A **label** is a display area for a short text, a node, or both. It is often used to label other controls (usually text fields). Labels and buttons share many common properties. These common properties are defined in the **Labeled** class.

javafx.scene.control.Labeled

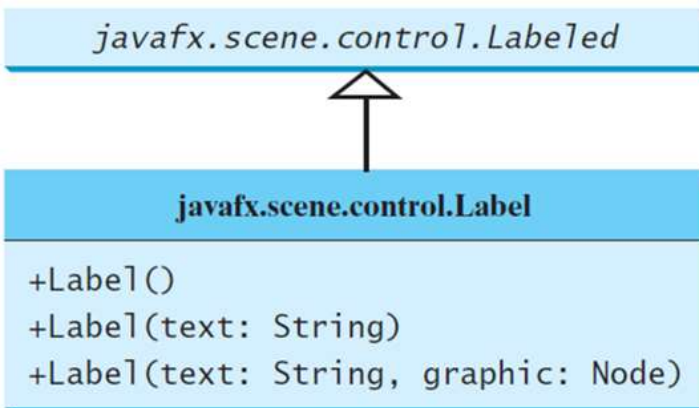
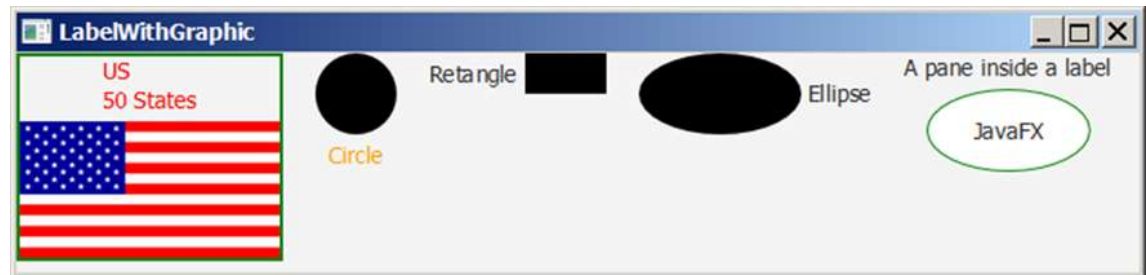
-alignment: ObjectProperty<Pos>
-contentDisplay:
 ObjectProperty<ContentDisplay>
-graphic: ObjectProperty<Node>
-graphicTextGap: DoubleProperty
-textFill: ObjectProperty<Paint>
-text: StringProperty
-underline: BooleanProperty
-wrapText: BooleanProperty

The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

Specifies the alignment of the text and node in the labeled.
Specifies the position of the node relative to the text using the constants TOP, BOTTOM, LEFT, and RIGHT defined in ContentDisplay.
A graphic for the labeled.
The gap between the graphic and the text.
The paint used to fill the text.
A text for the labeled.
Whether text should be underlined.
Whether text should be wrapped if the text exceeds the width.

Label

The Label class defines labels.

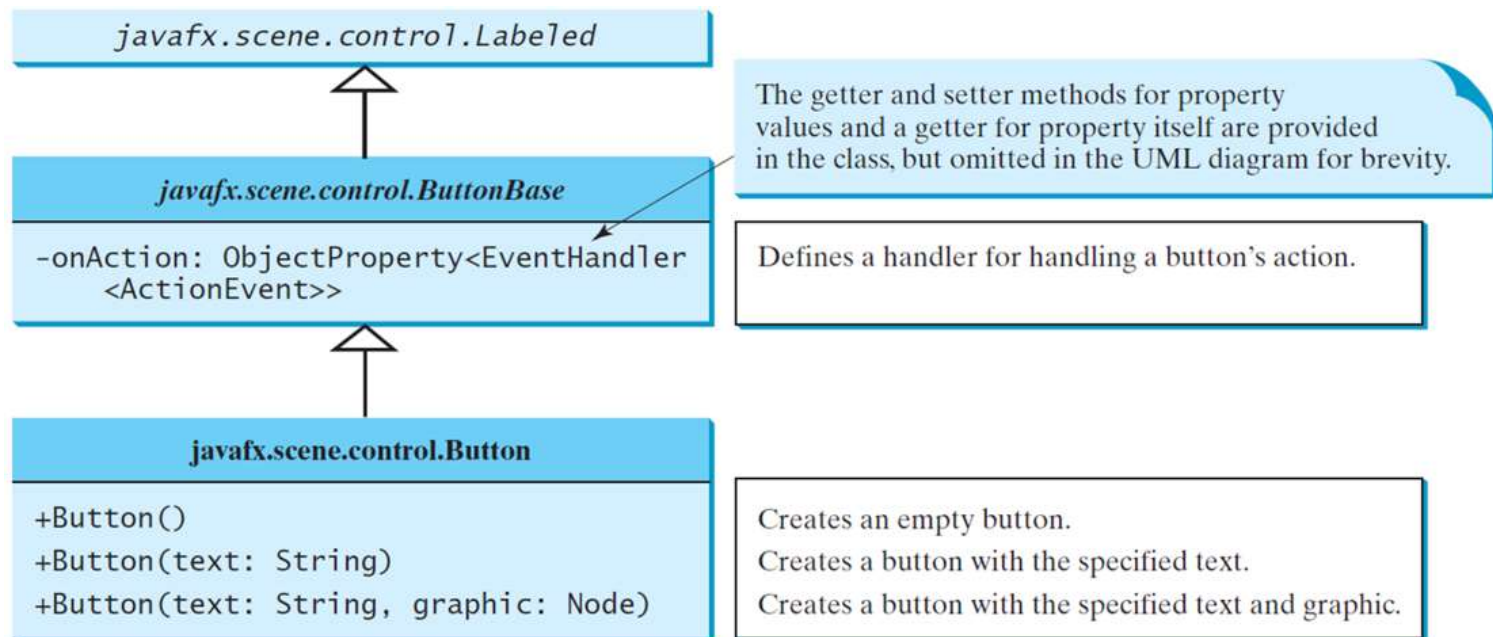


Creates an empty label.
Creates a label with the specified text.
Creates a label with the specified text and graphic.

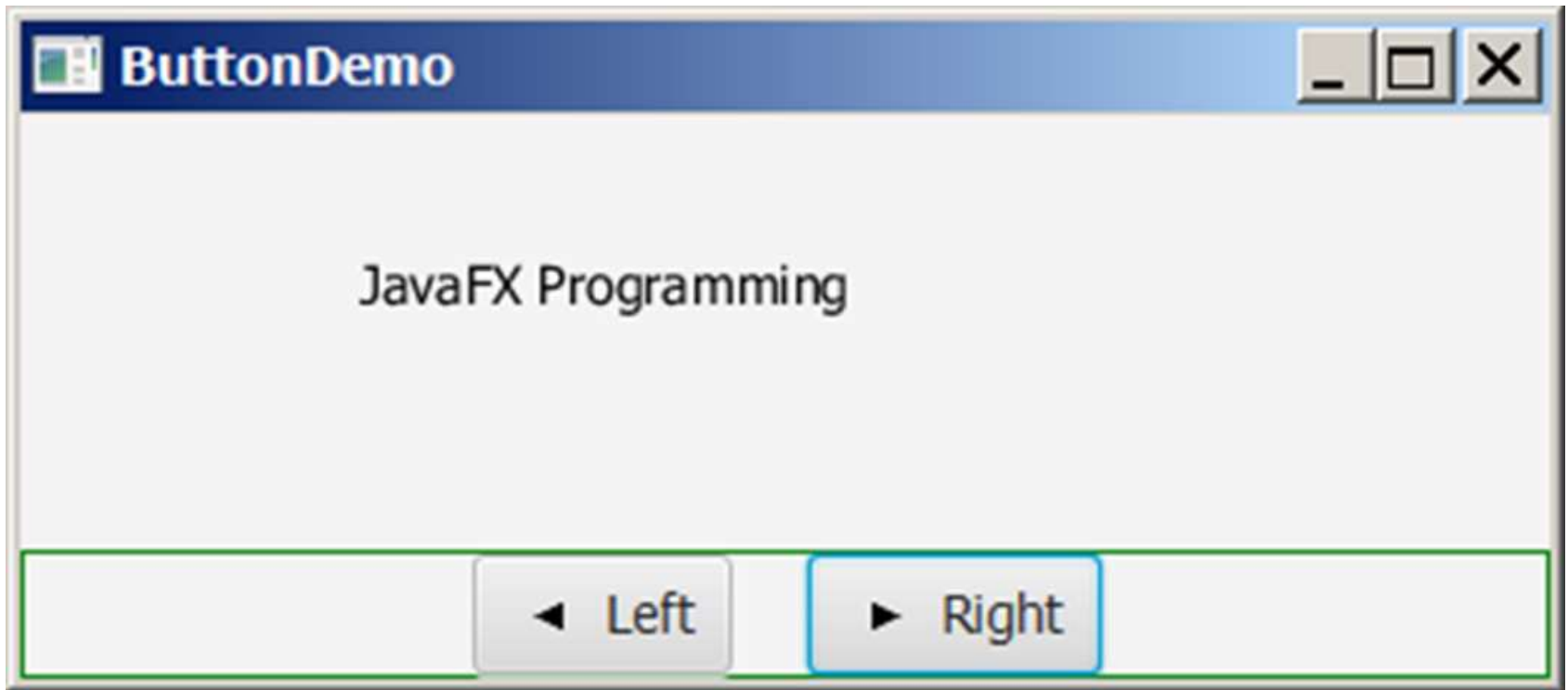
LabelWithGraphic

Buttonbase and Button

A **button** is a control that triggers an action event when clicked. JavaFX provides regular buttons, toggle buttons, check box buttons, and radio buttons. The common features of these buttons are defined in **ButtonBase** and **Labeled** classes.



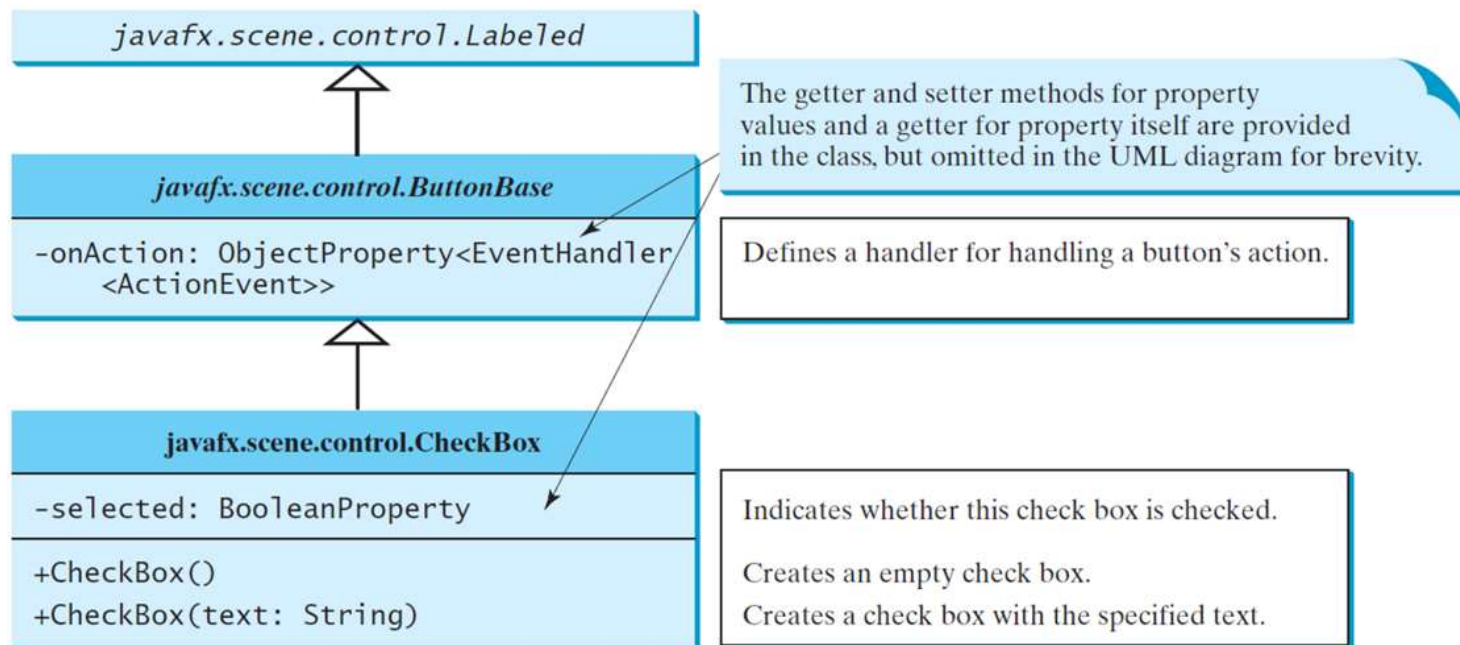
Button Example



[ButtonDemo](#)

CheckBox

A **CheckBox** is used for the user to make a selection. Like **Button**, **CheckBox** inherits all the properties such as **onAction**, **text**, **graphic**, **alignment**, **graphicTextGap**, **textFill**, **contentDisplay** from **ButtonBase** and **Labeled**.



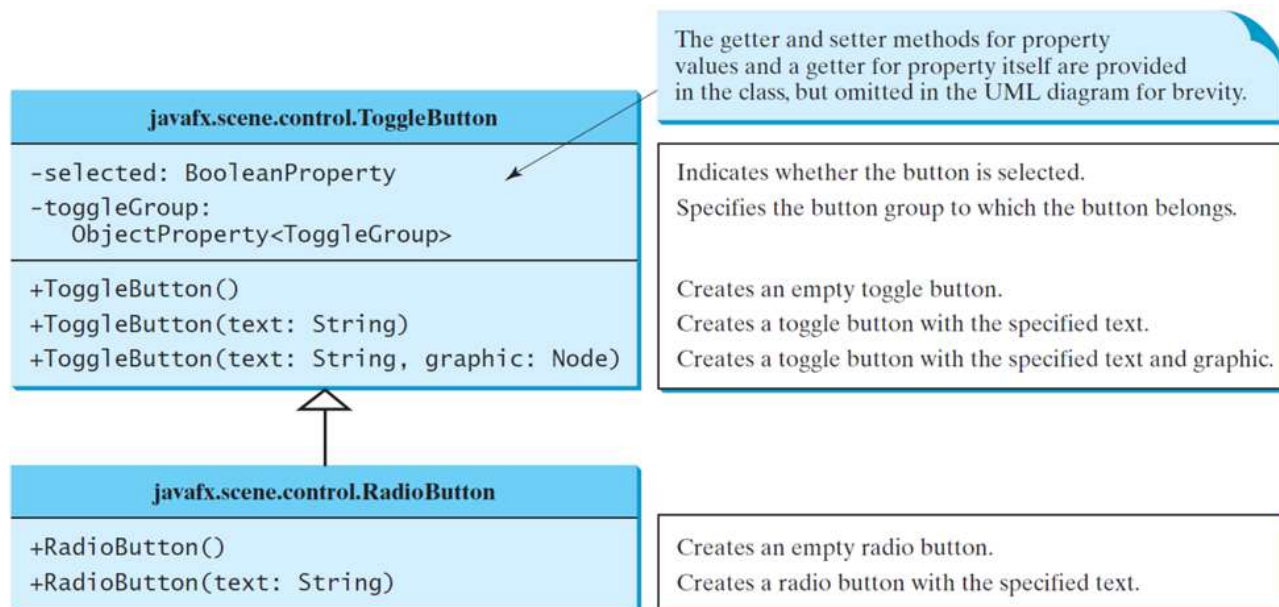
CheckBox Example



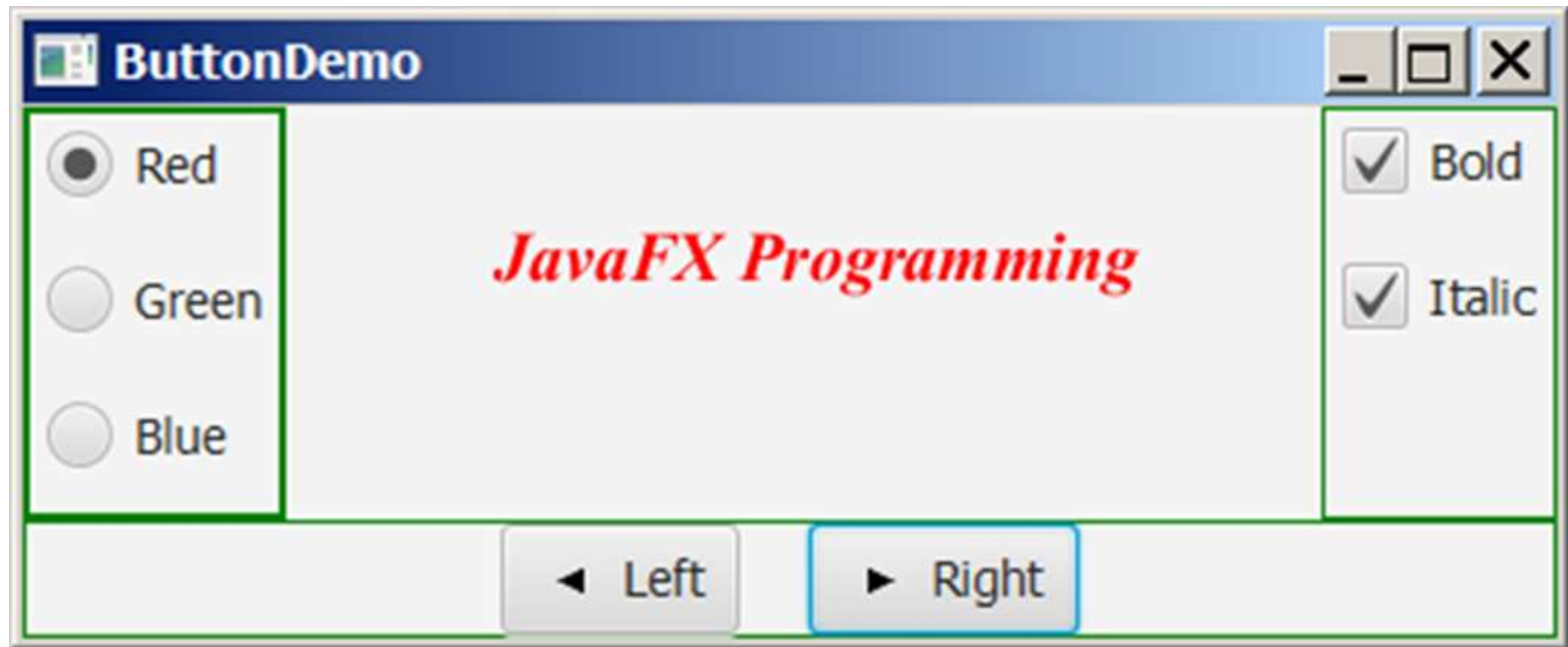
[CheckBoxDemo](#)

RadioButton

Radio buttons, also known as **option buttons**, enable you to choose a single item from a group of choices. In appearance radio buttons resemble check boxes, but check boxes display a square that is either checked or blank, whereas radio buttons display a circle that is either filled (if selected) or blank (if not selected).



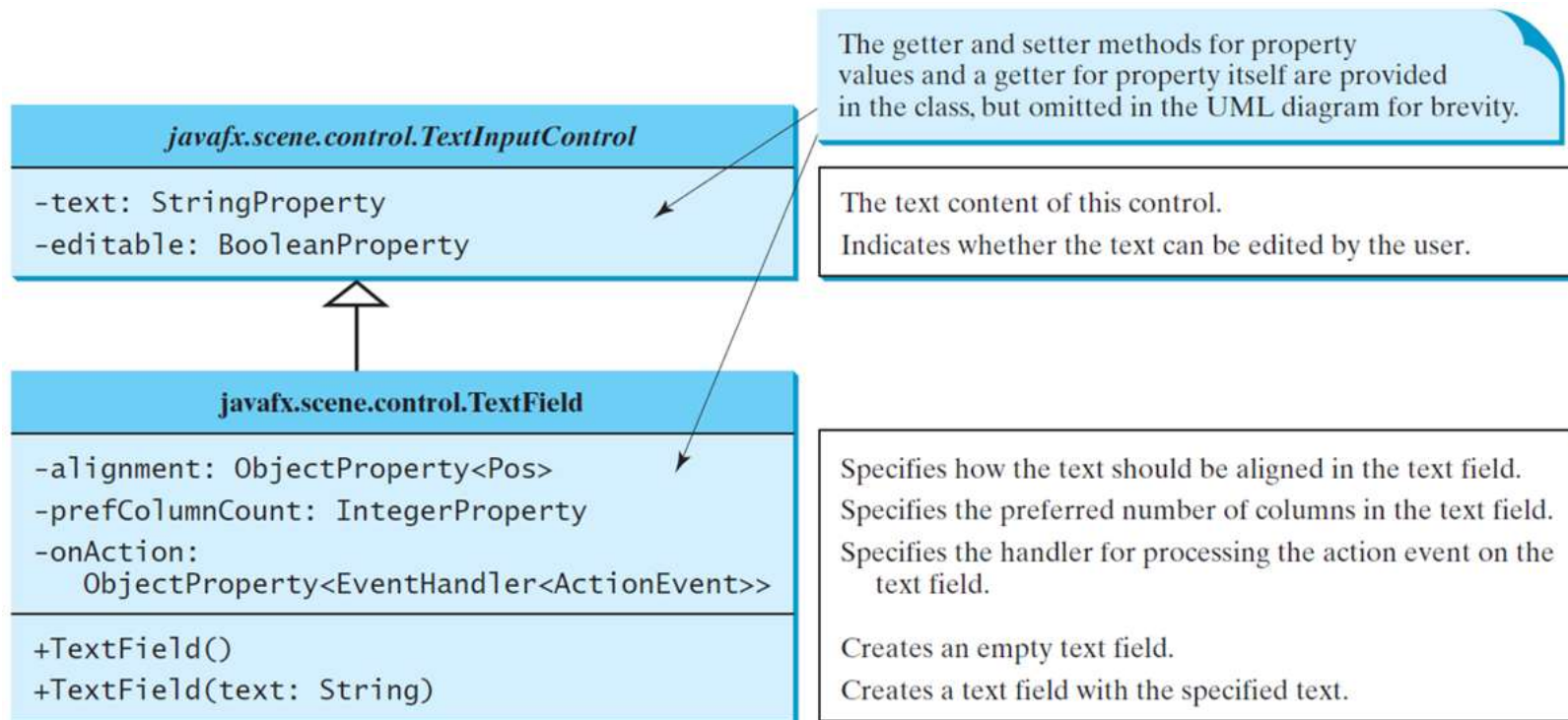
RadioButton Example



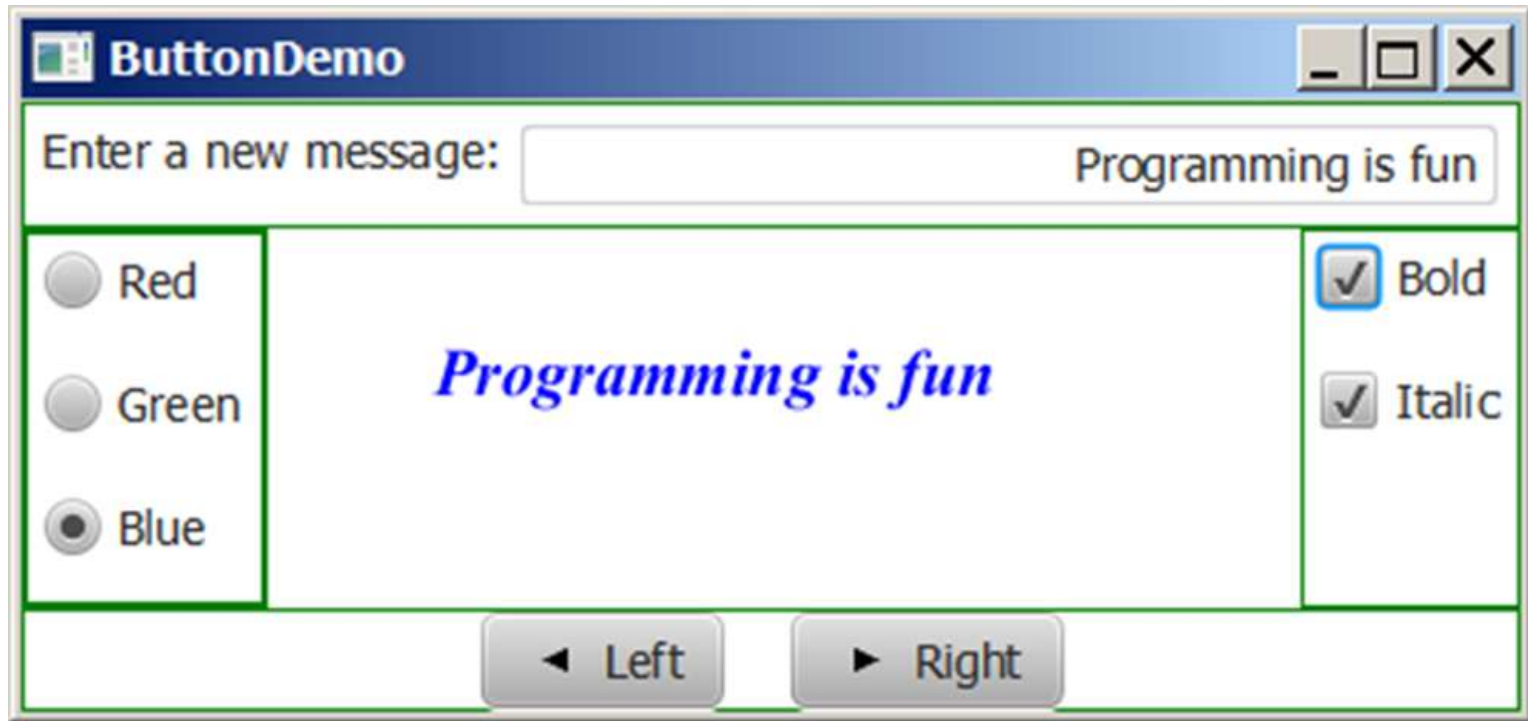
[RadioButtonDemo](#)

TextField

A text field can be used to enter or display a string.
TextField is a subclass of **TextInputControl**.



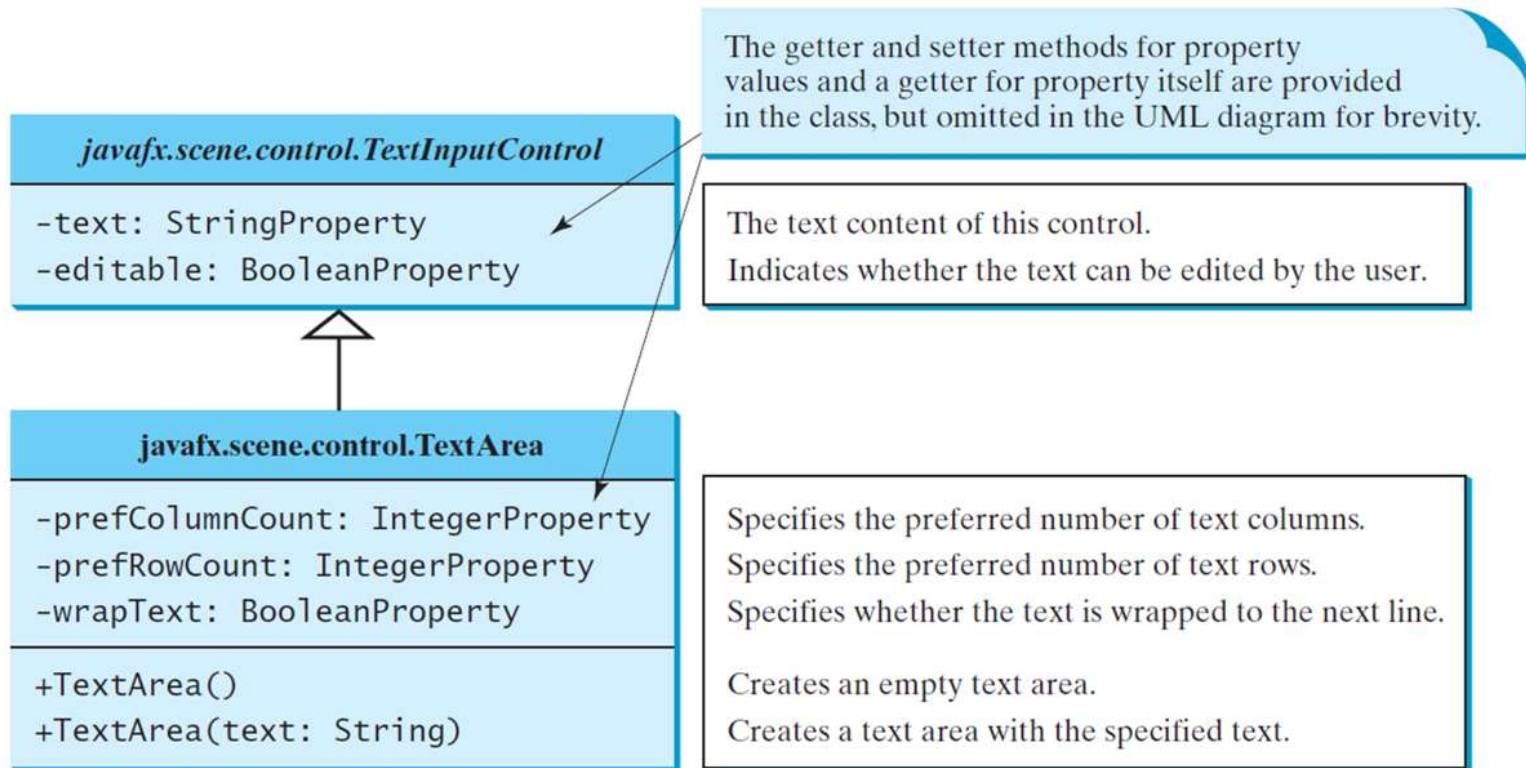
TextField Example



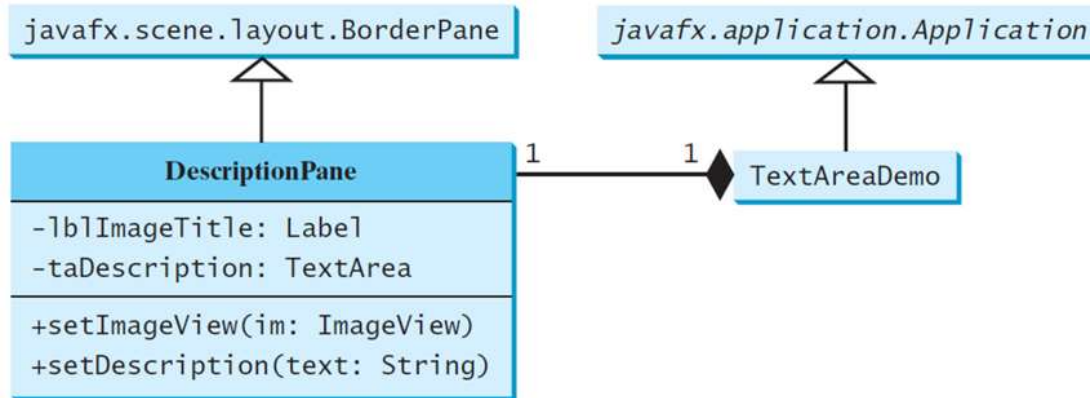
[TextFieldDemo](#)

TextArea

A **TextArea** enables the user to enter multiple lines of text.



TextArea Example

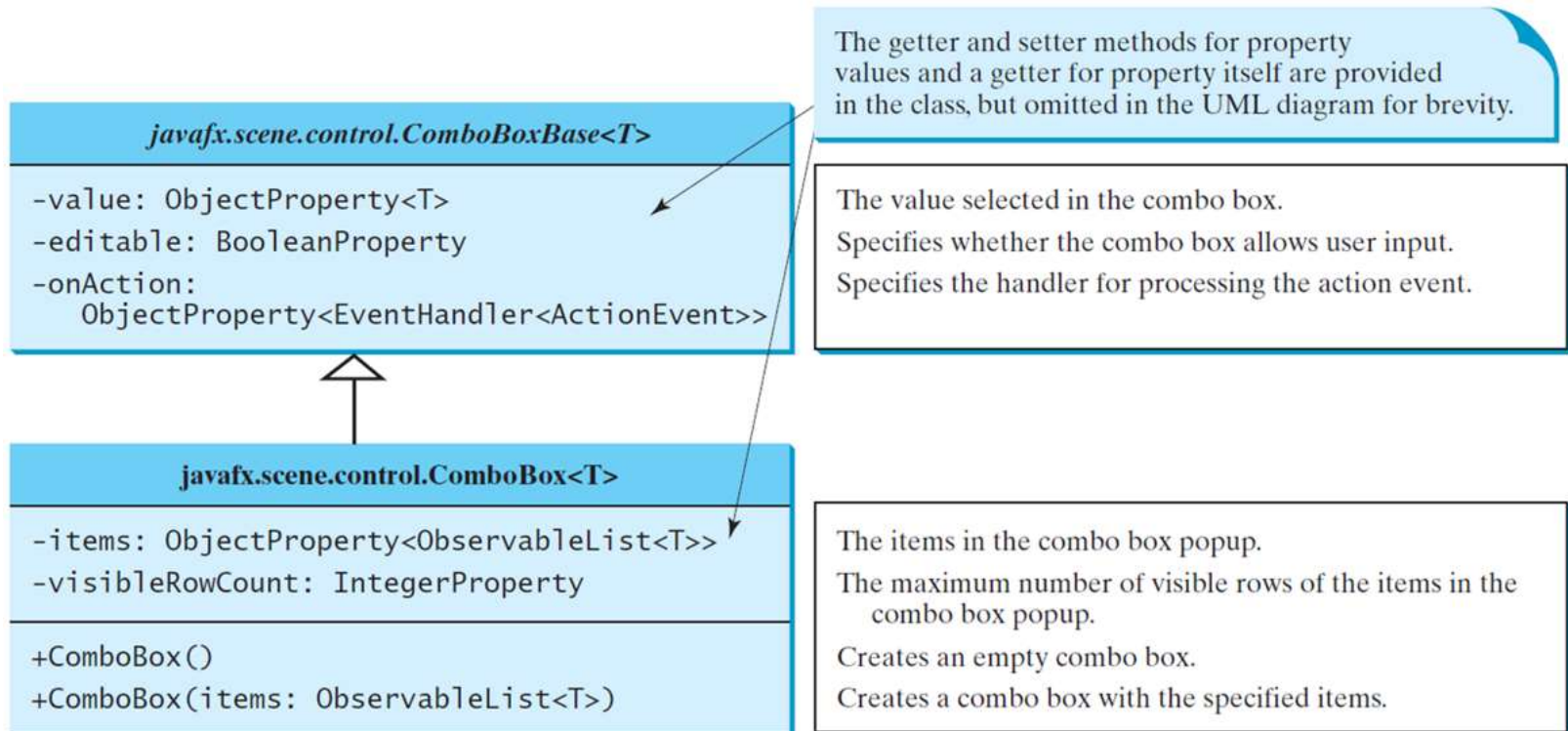


[DescriptionPane](#)

[TextAreaDemo](#)

ComboBox

A combo box, also known as a choice list or drop-down list, contains a list of items from which the user can choose.



ComboBox Example

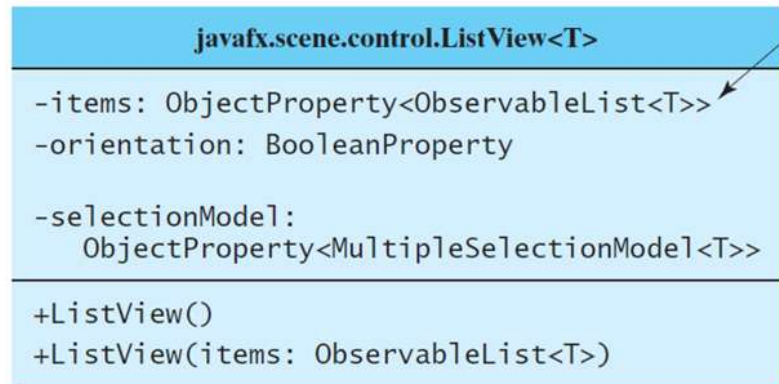
This example lets users view an image and a description of a country's flag by selecting the country from a `comboBox`.



[ComboBoxDemo](#)

ListView

A **list view** is a component that performs basically the same function as a combo box, but it enables the user to choose a single value or multiple values.



The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The items in the list view.

Indicates whether the items are displayed horizontally or vertically in the list view.

Specifies how items are selected. The `SelectionMode` is also used to obtain the selected items.

Creates an empty list view.

Creates a list view with the specified items.

Example: Using Listview

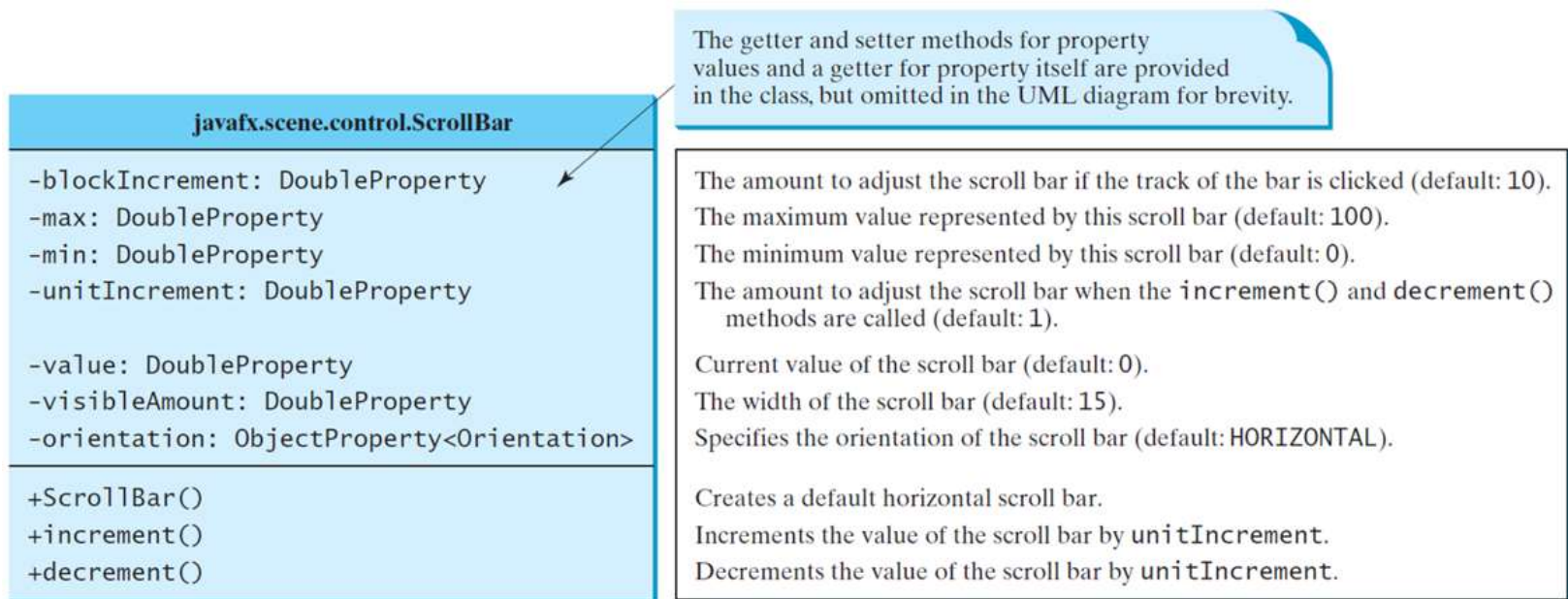
This example gives a program that lets users select countries in a list and display the flags of the selected countries in the labels.



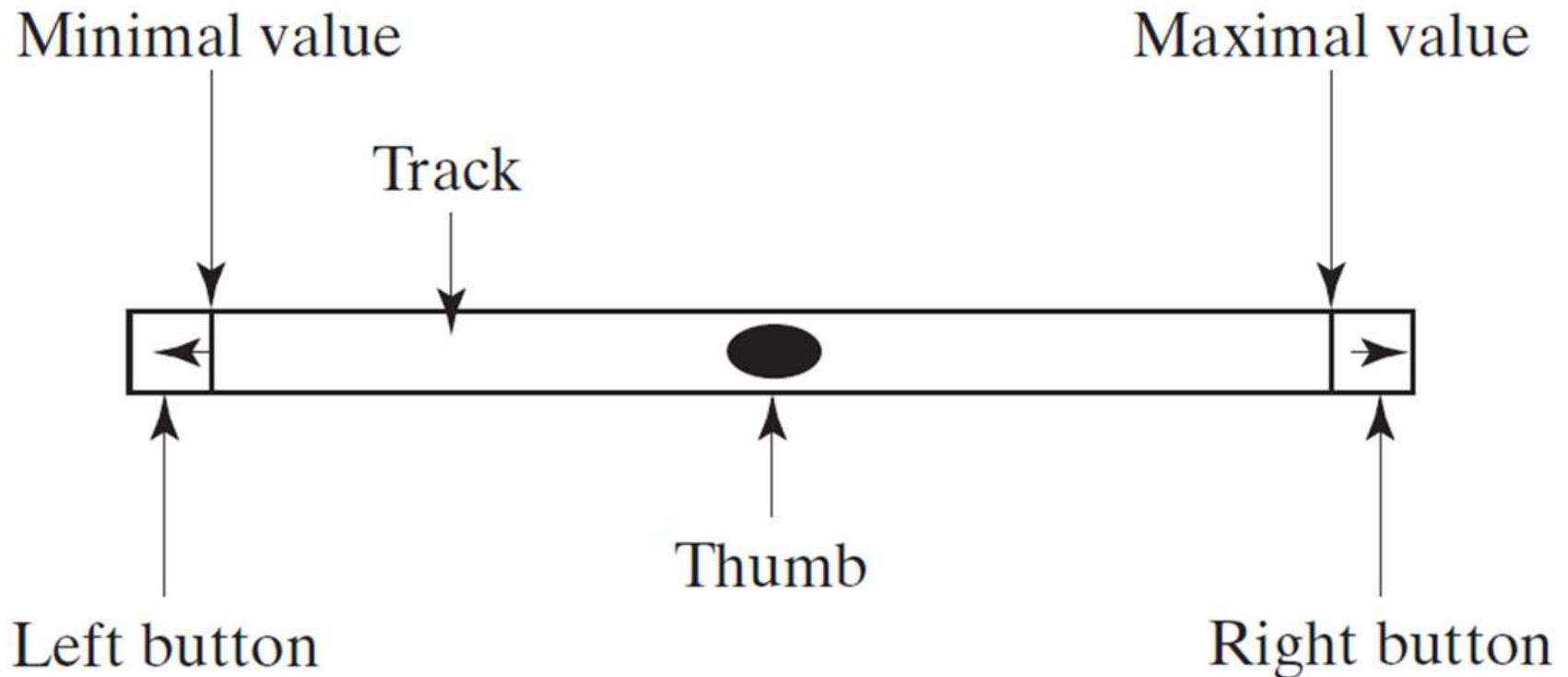
[ListViewDemo](#)

ScrollBar

A **scroll bar** is a control that enables the user to select from a range of values. The `scrollbar` appears in two styles: **horizontal** and **vertical**.

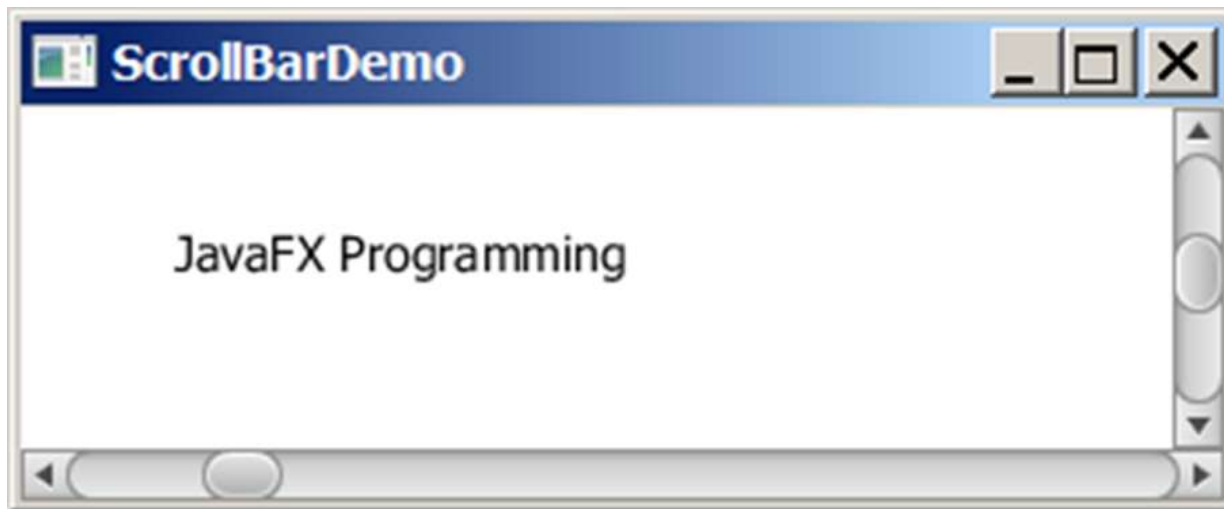


Scroll Bar Properties



Example: Using Scrollbars

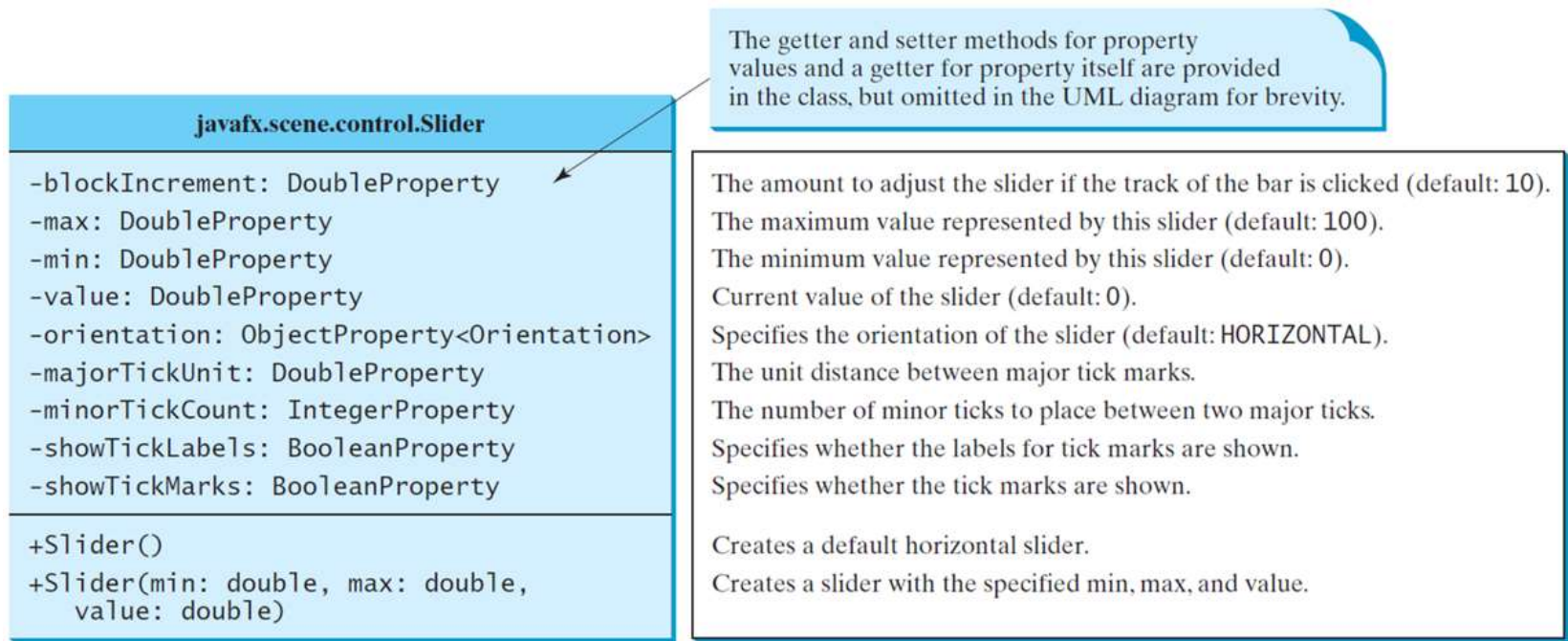
This example uses horizontal and vertical scrollbars to control a message displayed on a panel. The horizontal scrollbar is used to move the message to the left or the right, and the vertical scrollbar to move it up and down.



[ScrollBarDemo](#)

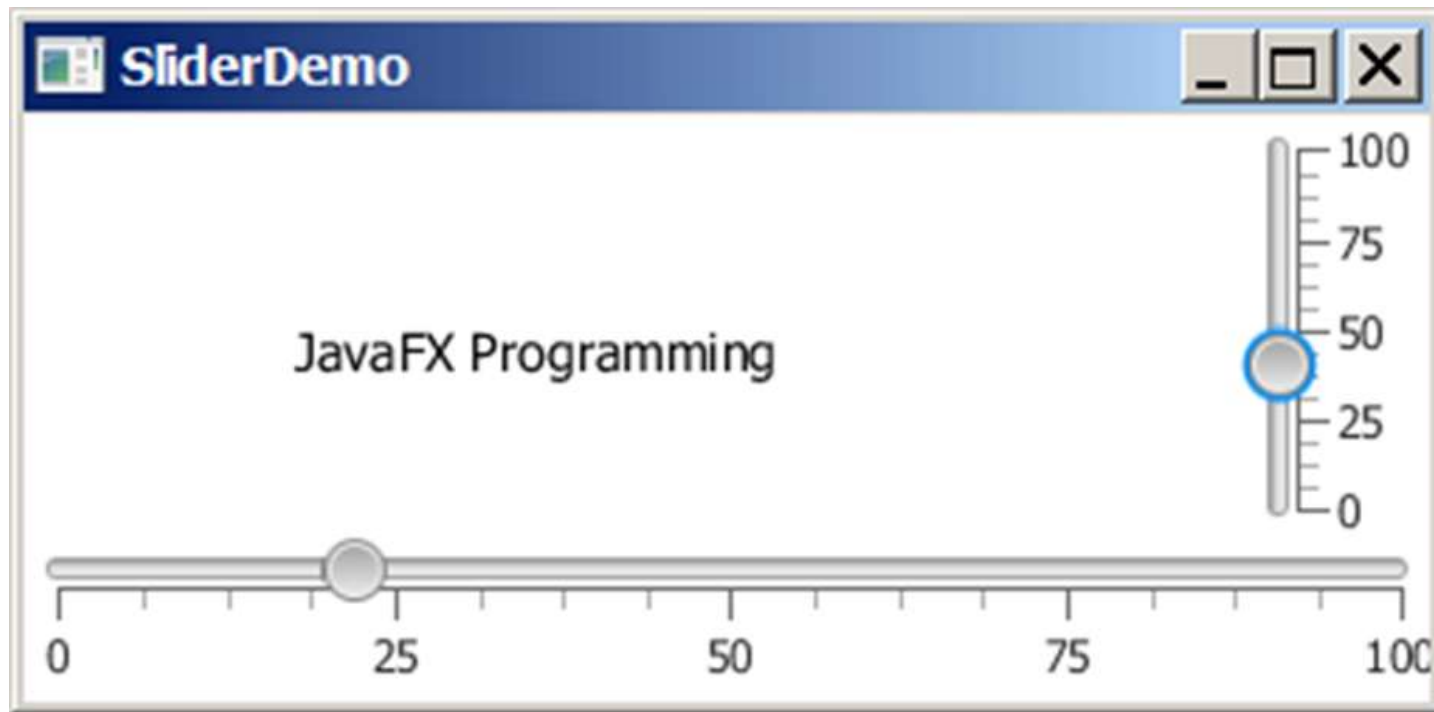
Slider

Slider is similar to ScrollBar, but Slider has more properties and can appear in many forms.



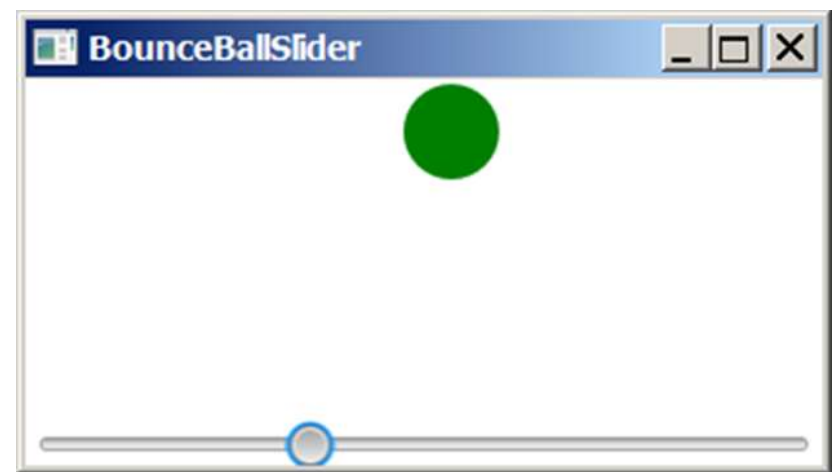
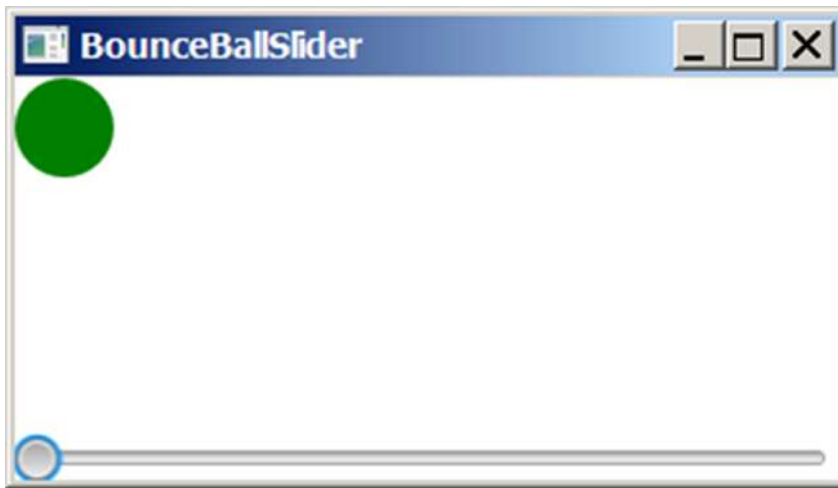
Example: Using Sliders

Rewrite the preceding program using the sliders to control a message displayed on a panel instead of using scroll bars.



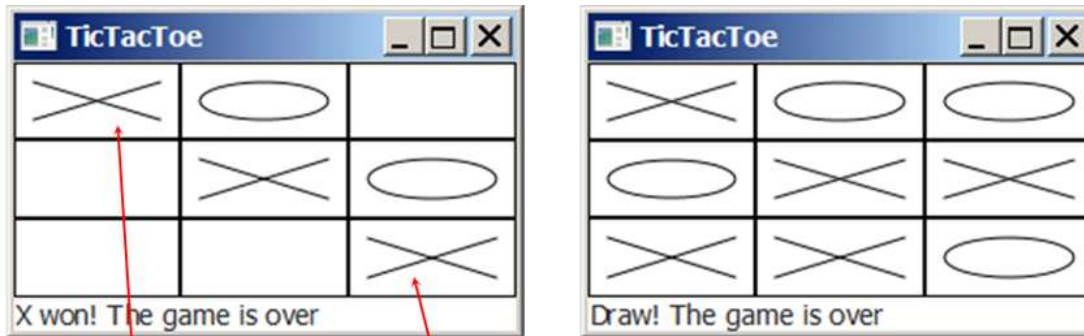
Case Study: Bounce Ball

Listing 15.17 gives a program that displays a bouncing ball. You can add a slider to control the speed of the ball movement.



[SliderDemo](#)

Case Study: TicTacToe (1 of 2)



`javafx.scene.layout.Pane`



`Cell`

`-token: char`

`+getToken(): char`

`+setToken(token: char): void`

`-handleMouseClicked(): void`

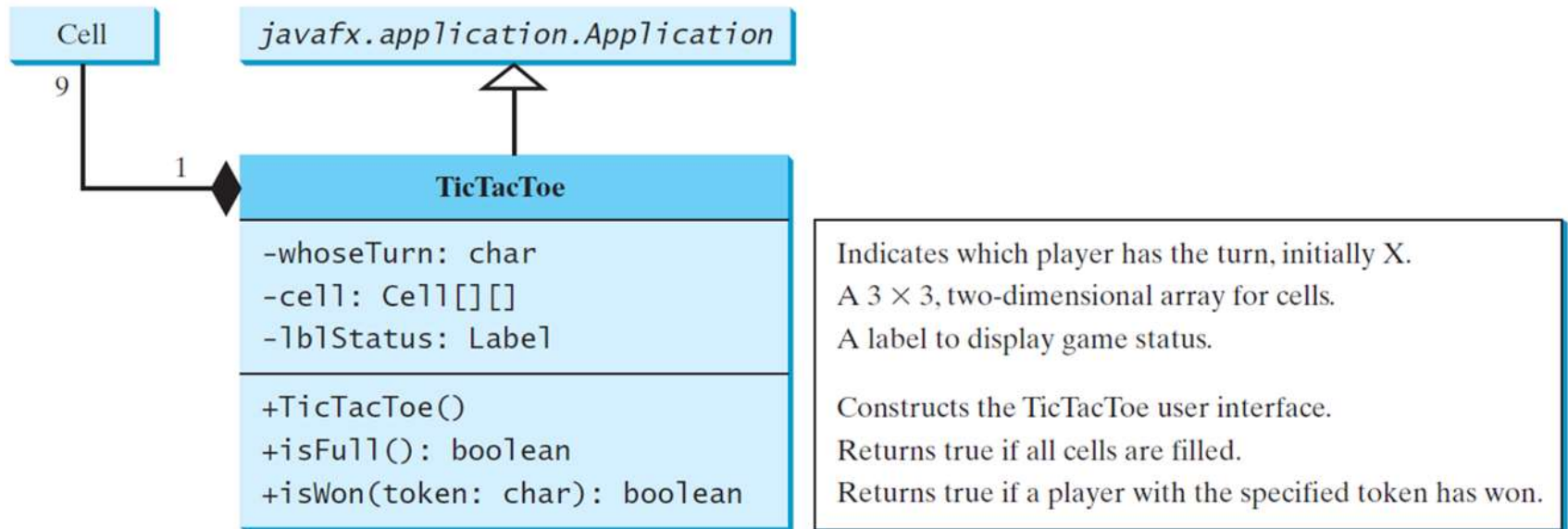
Token used in the cell (default: ' ').

Returns the token in the cell.

Sets a new token in the cell.

Handles a mouse click event.

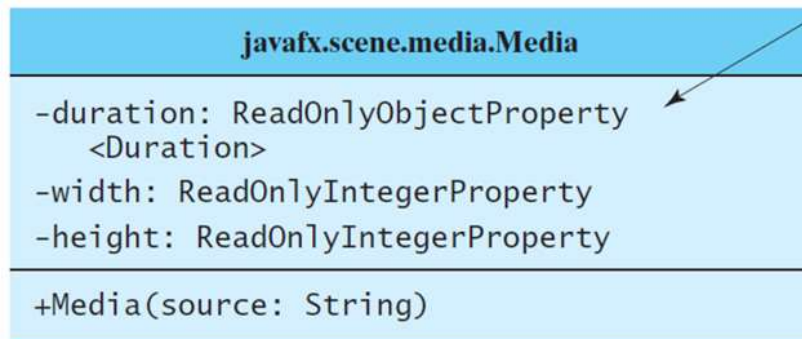
Case Study: TicTacToe (2 of 2)



TicTacToe

Media

You can use the **Media** class to obtain the source of the media, the **MediaPlayer** class to play and control the media, and the **MediaView** class to display the video.



The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

The durations in seconds of the source media.

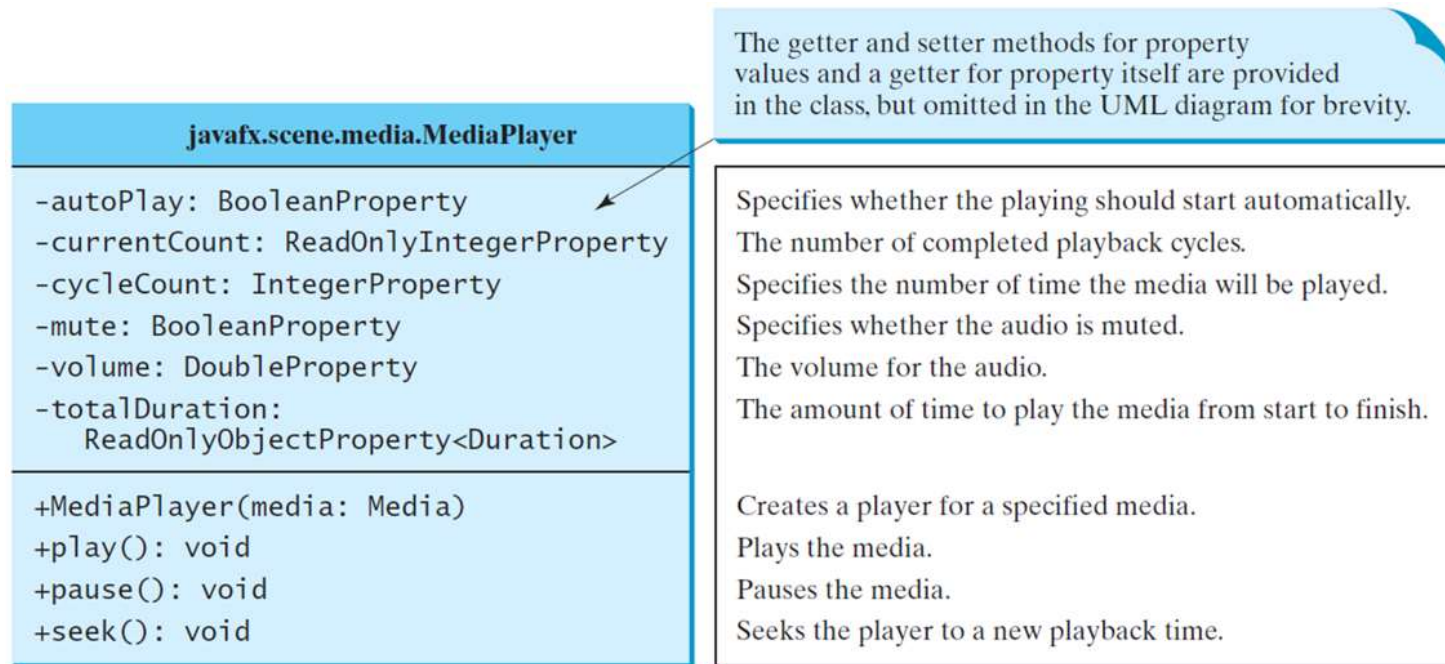
The width in pixels of the source video.

The height in pixels of the source video.

Creates a `Media` from a URL source.

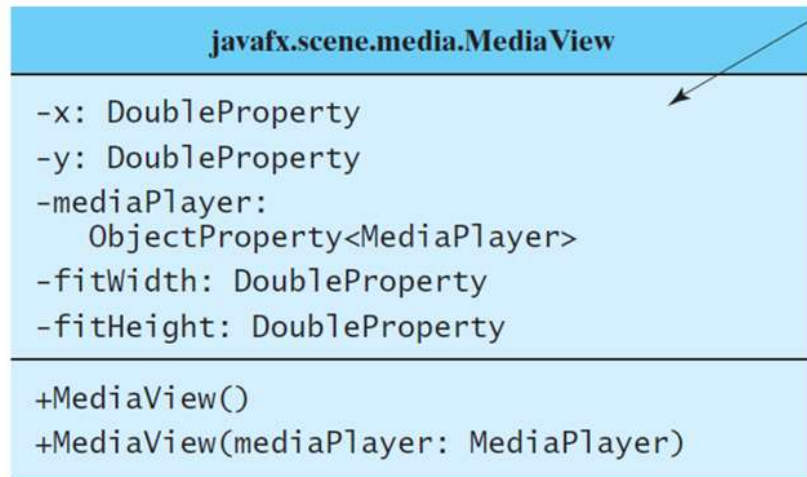
MediaPlayer

The **MediaPlayer** class plays and controls the `media` with properties such as **autoPlay**, **currentCount**, **cycleCount**, **mute**, **volume**, and **totalDuration**.



MediaView

The **MediaView** class is a subclass of **Node** that provides a view of the **Media** being played by a **MediaPlayer**. The **MediaView** class provides the properties for viewing the `media`.



The getter and setter methods for property values and a getter for property itself are provided in the class, but omitted in the UML diagram for brevity.

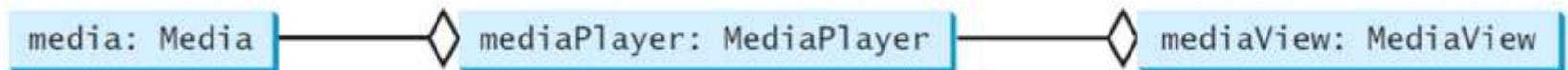
Specifies the current x-coordinate of the media view.
Specifies the current y-coordinate of the media view.
Specifies a media player for the media view.

Specifies the width of the view for the media to fit.
Specifies the height of the view for the media to fit.

Creates an empty media view.
Creates a media view with the specified media player.

Example: Using Media

This example displays a video in a view. You can use the play/pause button to play or pause the video and use the rewind button to restart the video, and use the slider to control the volume of the audio.



[MediaDemo](#)

Case Study: National Flags and Anthems

This case study presents a program that displays a nation's flag and plays its anthem.



[FlagAnthem](#)

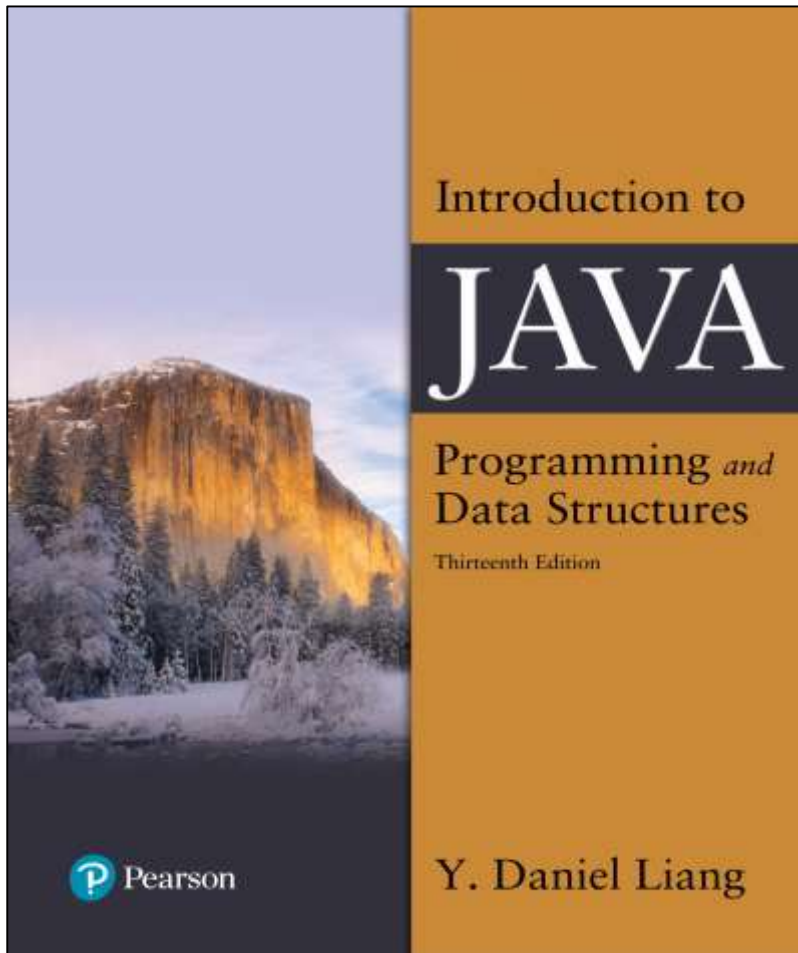
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 19

Generics

Objectives (1 of 2)

19.1 To know the benefits of generics (§19.1).

19.2 To use generic classes and interfaces (§19.2).

19.3 To define generic classes and interfaces (§19.3).

19.4 To explain why generic types can improve reliability and readability (§19.3).

19.5 To define and use generic methods and bounded generic types (§19.4).

19.6 To develop a generic sort method to sort an array of Comparable objects (§19.5).

19.7 To use raw types for backward compatibility (§19.6).

Objectives (2 of 2)

19.8 To explain why wildcard generic types are necessary (§19.7).

19.9 To describe generic-type erasure and list certain restrictions and limitations on generic types caused by type erasure (§19.8).

19.10 To design and implement generic matrix classes (§19.9).

Why Do You Get a Warning?

```
public class ShowUncheckedWarning {  
    public static void main(String[] args) {  
        java.util.ArrayList list =  
            new java.util.ArrayList();  
        list.add("Java Programming");  
    }  
}
```

To understand the compile warning on this line, you need to learn JDK 1.6 generics.

Fix the Warning

```
public class ShowUncheckedWarning {  
    public static void main(String[] args) {  
        java.util.ArrayList<String> list =  
            new java.util.ArrayList<String>();  
        list.add("Java Programming");  
    }  
}
```

No compile warning on this line.

What Is Generics?

Generics is the capability to parameterize types. With this capability, you can define a class or a method with generic types that can be substituted using concrete types by the compiler. For example, you may define a generic stack class that stores the elements of a generic type. From this generic class, you may create a stack object for holding strings and a stack object for holding numbers. Here, strings and numbers are concrete types that replace the generic type.

Why Generics?

The key benefit of generics is to enable errors to be detected at compile time rather than at runtime. A generic class or method permits you to specify allowable types of objects that the class or method may work with. If you attempt to use the class or method with an incompatible object, a compile error occurs.

Generic Type

```
package java.lang;

public interface Comparable {
    public int compareTo(Object o)
}
```

(a) Prior to JDK 1.5

```
package java.lang;

public interface Comparable<T> {
    public int compareTo(T o)
}
```

(b) JDK 1.5

Generic Instantiation

Runtime error

```
Comparable c = new Date();
System.out.println(c.compareTo("red"));
```

(a) Prior to JDK 1.5

```
Comparable<Date> c = new Date();
System.out.println(c.compareTo("red"));
```

(b) JDK 1.5

Compile error

Improves reliability

Generic ArrayList in JDK 1.5

java.util.ArrayList

```
+ArrayList()  
+add(o: Object): void  
+add(index: int, o: Object): void  
+clear(): void  
+contains(o: Object): boolean  
+get(index: int): Object  
+indexOf(o: Object): int  
+isEmpty(): boolean  
+lastIndexOf(o: Object): int  
+remove(o: Object): boolean  
+size(): int  
+remove(index: int): boolean  
+set(index: int, o: Object): Object
```

(a) ArrayList before JDK 1.5

java.util.ArrayList<E>

```
+ArrayList()  
+add(o: E): void  
+add(index: int, o: E): void  
+clear(): void  
+contains(o: Object): boolean  
+get(index: int): E  
+indexOf(o: Object): int  
+isEmpty(): boolean  
+lastIndexOf(o: Object): int  
+remove(o: Object): boolean  
+size(): int  
+remove(index: int): boolean  
+set(index: int, o: E): E
```

(b) ArrayList since JDK 1.5

No Casting Needed

```
ArrayList<Double> list = new ArrayList<>();
```

```
list.add(5.5); // 5.5 is automatically converted to new  
Double(5.5)
```

```
list.add(3.0); // 3.0 is automatically converted to new  
Double(3.0)
```

```
Double doubleObject = list.get(0); // No casting is needed
```

```
double d = list.get(1); // Automatically converted to double
```

Declaring Generic Classes and Interfaces

GenericStack<E>	
-list: java.util.ArrayList<E>	An array list to store elements.
+GenericStack() +getSize(): int +peek(): E +pop(): E +push(o: E): void +isEmpty(): boolean	Creates an empty stack. Returns the number of elements in this stack. Returns the top element in this stack. Returns and removes the top element in this stack. Adds a new element to the top of this stack. Returns true if the stack is empty.

[GenericStack](#)

Generic Static Methods

```
public static <E> void print(E[] list) {  
    for (int i = 0; i < list.length; i++)  
        System.out.print(list[i] + " ");  
    System.out.println();  
}  
  
public static void print(Object[] list) {  
    for (int i = 0; i < list.length; i++)  
        System.out.print(list[i] + " ");  
    System.out.println();  
}
```

Bounded Generic Type

```
public static void main(String[] args ) {  
    Rectangle rectangle = new Rectangle(2, 2);  
    Circle circle = new Circle (2);  
    System.out.println("Same area? " +  
        equalArea(rectangle, circle));  
}
```

```
public static <E extends GeometricObject>  
boolean  
    equalArea(E object1, E object2) {  
    return object1.getArea() ==  
        object2.getArea();  
}
```

Raw Type and Backward Compatibility

```
// raw type
```

```
ArrayList list = new ArrayList();
```

This is **roughly** equivalent to

```
ArrayList<Object> list = new ArrayList<Object>();
```

Raw Type Is Unsafe

```
// Max.java: Find a maximum object
public class Max {
    /** Return the maximum between two objects */
    public static Comparable max(Comparable o1, Comparable o2) {
        if (o1.compareTo(o2) > 0)
            return o1;
        else
            return o2;
    }
}
```

Runtime Error:

```
Max.max("Welcome", 23); // No compile error
```

Avoiding Unsafe Raw Types

Use

```
new ArrayList<ConcreteType>()
```

Instead of

```
new ArrayList();
```

[TestArrayListNew](#)

Make It Safe

```
// Max1.java: Find a maximum object
public class Max1 {
    /** Return the maximum between two objects */
    public static <E extends Comparable<E>> E max(E o1,
        E o2) {
        if (o1.compareTo(o2) > 0)
            return o1;
        else
            return o2;
    }
}
```

```
Max.max("Welcome", 23);
```


Wildcards

Why wildcards are necessary? See this example.

[WildCardNeedDemo](#)

? unbounded wildcard

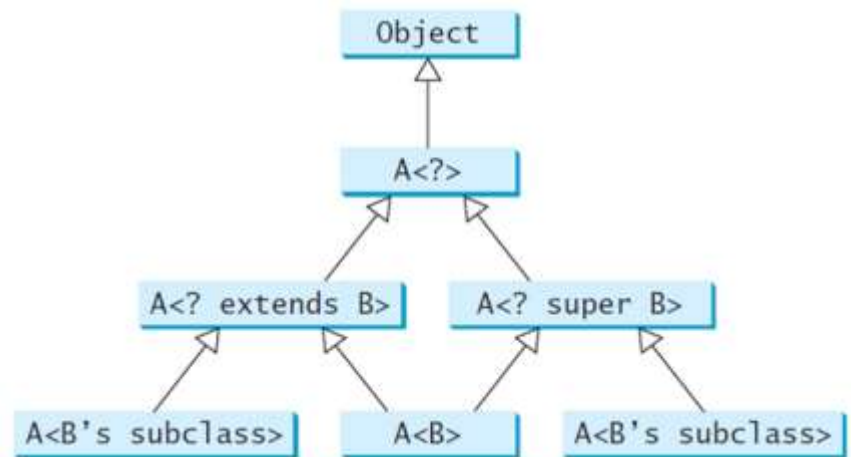
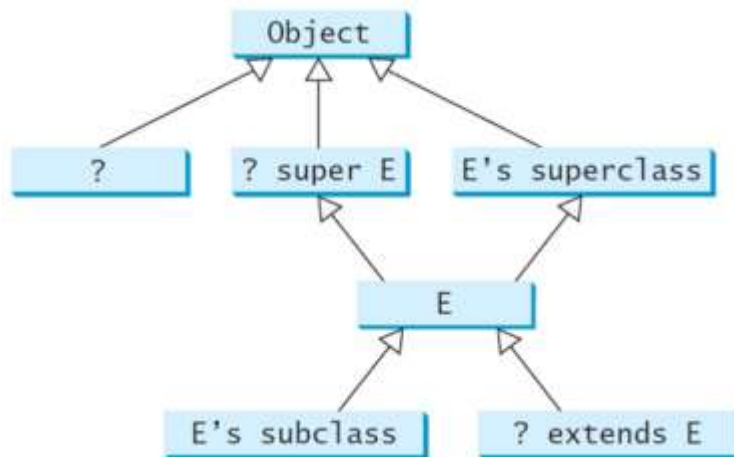
? extends T bounded wildcard

? super T lower bound wildcard

[AnyWildCardDemo](#)

[SuperWildCardDemo](#)

Generic Types and Wildcard Types



Erasure and Restrictions on Generics

Generics are implemented using an approach called **type erasure**. The compiler uses the generic type information to compile the code, but erases it afterwards. So the generic information is not available at runtime. This approach enables the generic code to be backward-compatible with the legacy code that uses raw types.

Compile Time Checking

For example, the compiler checks whether generics is used correctly for the following code in (a) and translates it into the equivalent code in (b) for runtime use. The code in (b) uses the raw type.

```
ArrayList<String> list = new ArrayList<>();  
list.add("Oklahoma");  
String state = list.get(0);
```

(a)

```
ArrayList list = new ArrayList();  
list.add("Oklahoma");  
String state = (String) list.get(0);
```

(b)

Important Facts

It is important to note that a generic class is shared by all its instances regardless of its actual generic type.

```
GenericStack<String> stack1 = new GenericStack<>();
```

```
GenericStack<Integer> stack2 = new GenericStack<>();
```

Although `GenericStack<String>` and `GenericStack<Integer>` are two types, but there is only one class `GenericStack` loaded into the JVM.

Restrictions on Generics

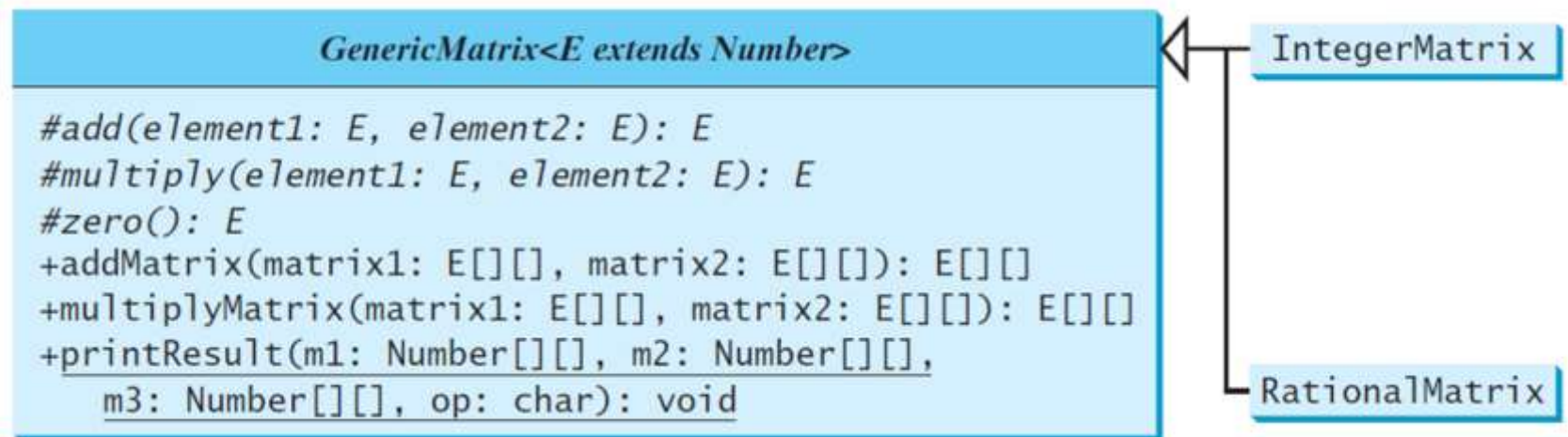
- Restriction 1: Cannot Create an Instance of a Generic Type. (i.e., `new E()`).
- Restriction 2: Generic Array Creation is Not Allowed. (i.e., `new E[100]`).
- Restriction 3: A Generic Type Parameter of a Class Is Not Allowed in a Static Context.
- Restriction 4: Exception Classes Cannot be Generic.

Designing Generic Matrix Classes

Objective: This example gives a generic class for matrix arithmetic. This class implements matrix addition and multiplication common for all types of matrices.

[GenericMatrix](#)

UML Diagram



Source Code

Objective: This example gives two programs that utilize the `GenericMatrix` class for integer matrix arithmetic and rational matrix arithmetic.

[IntegerMatrix](#)

[TestIntegerMatrix](#)

[RationalMatrix](#)

[TestRationalMatrix](#)

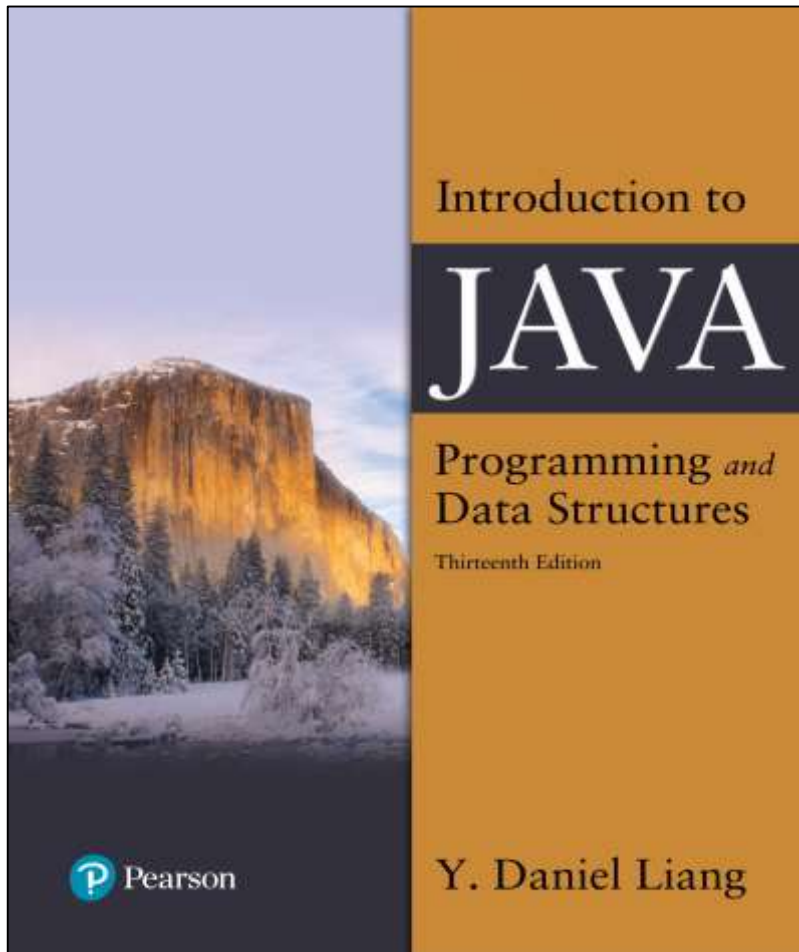
Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 24

Implementing Lists, Stacks,
Queues, and Priority
Queues

Objectives (1 of 2)

24.1 To design common operations of lists in an interface and make the interface a subtype of `Collection` (§24.2).

24.2 To design and implement an array list using an array (§24.3).

24.3 To create a linked list using a linked structure (§24.4).

24.4 To design `MyLinkedList` class (§24.5).

24.5 To implement `MyLinkedList` class (§24.6).

24.6 To compare the performance between `MyArrayList` and `MyLinkedList` (§24.7).

Objectives (2 of 2)

24.7 To explore variations of linked lists (§24.8).

24.8 To design and implement a stack class using an array list and a queue class using a linked list (§24.9).

24.9 To design and implement a priority queue using a heap (§24.10).

Lists

A list is a popular data structure to store data in sequential order. For example, a list of students, a list of available rooms, a list of cities, and a list of books, etc. can be stored using lists. The common operations on a list are usually the following:

- Retrieve an element from this list.
- Insert a new element to this list.
- Delete an element from this list.
- Find how many elements are in this list.
- Find if an element is in this list.
- Find if this list is empty.

Two Ways to Implement Lists

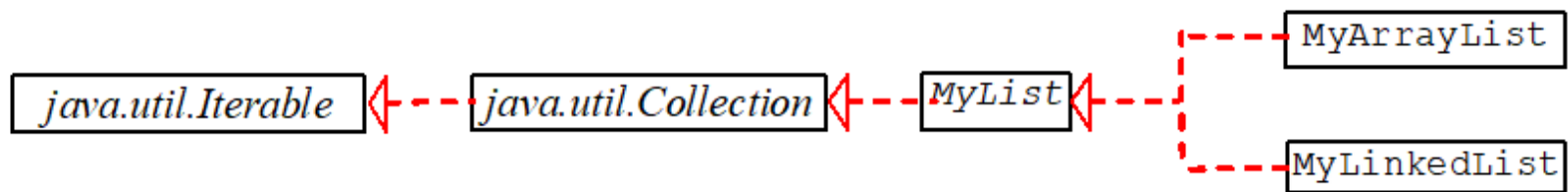
There are two ways to implement a list.

Using arrays. One is to use an array to store the elements. The array is dynamically created. If the capacity of the array is exceeded, create a new larger array and copy all the elements from the current array to the new array.

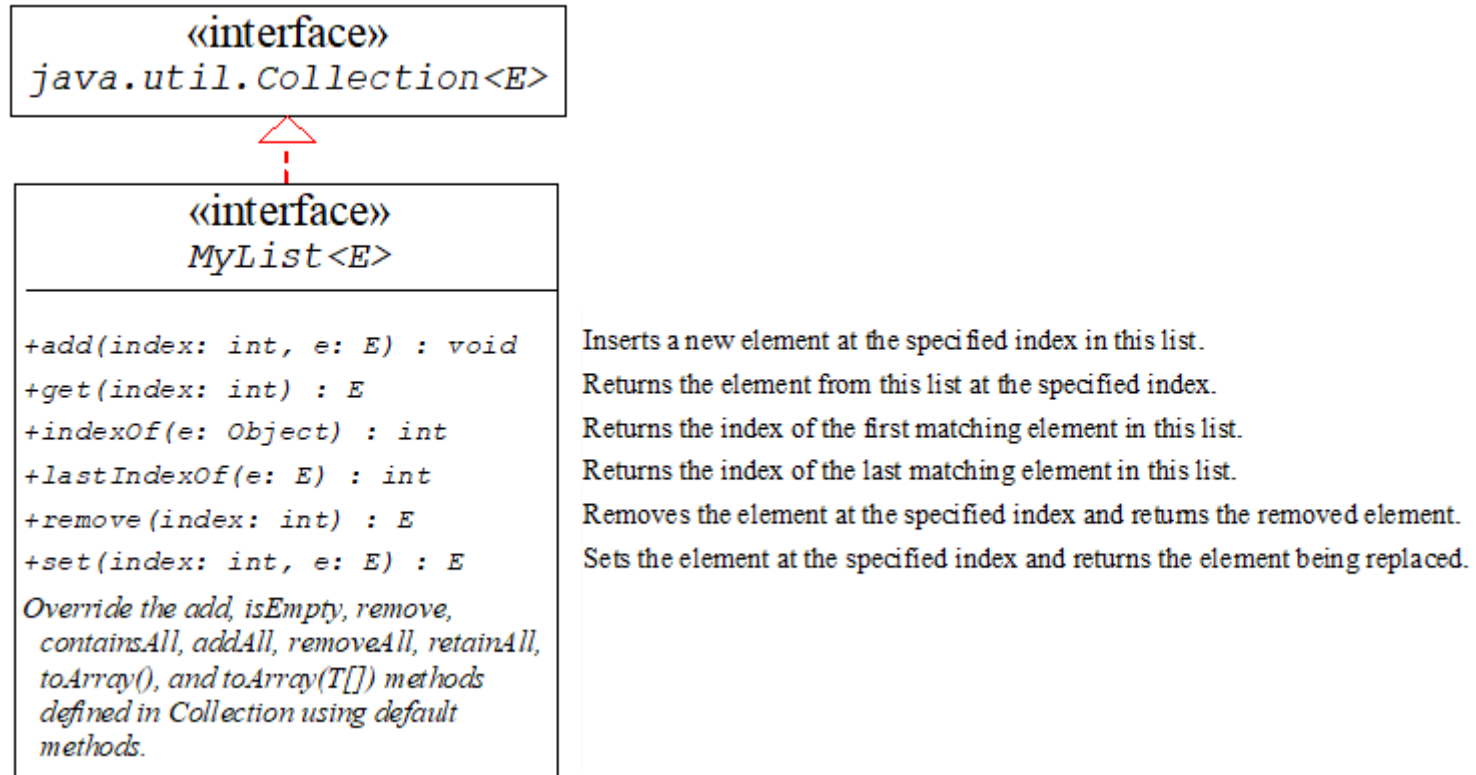
Using linked list. The other approach is to use a linked structure. A linked structure consists of nodes. Each node is dynamically created to hold an element. All the nodes are linked together to form a list.

Design of ArrayList and LinkedList

For convenience, let's name these two classes: `MyArrayList` and `MyLinkedList`. These two classes have common operations, but different data fields. The common operations can be generalized in an interface or an abstract class. Prior to Java 8, a popular design strategy is to define common operations in an interface and provide an abstract class for partially implementing the interface. So, the concrete class can simply extend the abstract class without implementing the full interface. Java 8 enables you to define default methods. You can provide default implementation for some of the methods in the interface rather than in an abstract class.



MyList Interface



[MyList](#)

Array Lists

Array is a fixed-size data structure. Once an array is created, its size cannot be changed. Nevertheless, you can still use array to implement dynamic data structures. The trick is to create a new larger array to replace the current array if the current array cannot hold new elements in the list.

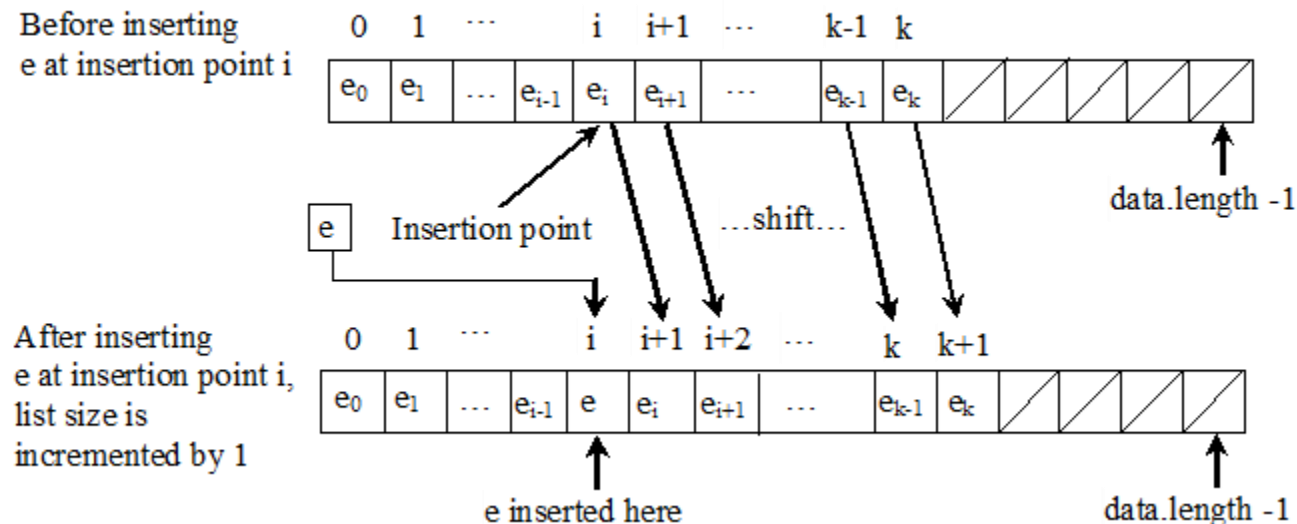
Initially, an array, say data of `Object[]` type, is created with a default size. When inserting a new element into the array, first ensure there is enough room in the array. If not, create a new array with the size as twice as the current one. Copy the elements from the current array to the new array. The new array now becomes the current array.

Array List Animation

<https://liveexample.pearsoncmg.com/dsanimation/ArrayList eBook.html>

Insertion

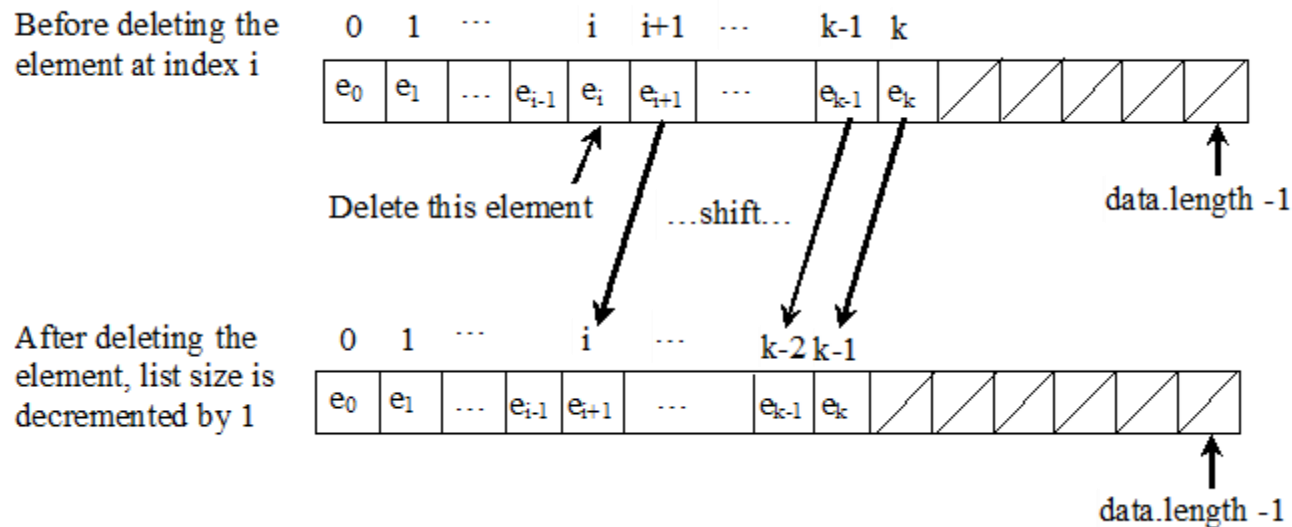
Before inserting a new element at a specified index, shift all the elements after the index to the right and increase the list size by 1.



[Add Method Animation](#)

Deletion

To remove an element at a specified index, shift all the elements after the index to the left by one position and decrease the list size by 1.



[Remove Method Animation](#)

Implementing MyArrayList

«interface»
MyList<E>



MyArrayList<E>

-data: E[]
-size: int

+MyArrayList()
+MyArrayList(objects: E[])
+trimToSize(): void

-ensureCapacity(): void
-checkIndex(index: int): void

Array for storing elements in this array list.

The number of elements in the array list.

Creates a default array list.

Creates an array list from an array of objects.

Trims the capacity of this array list to the list's current size.

Doubles the current array size if needed.

Throws an exception if the index is out of bounds in the list.

[MyArrayList](#)

[TestMyArrayList](#)

Linked Lists

Since `MyArrayList` is implemented using an array, the methods `get(int index)` and `set(int index, Object o)` for accessing and modifying an element through an index and the `add(Object o)` for adding an element at the end of the list are efficient. However, the methods `add(int index, Object o)` and `remove(int index)` are inefficient because it requires shifting potentially a large number of elements. You can use a linked structure to implement a list to improve efficiency for adding and removing an element anywhere in a list.

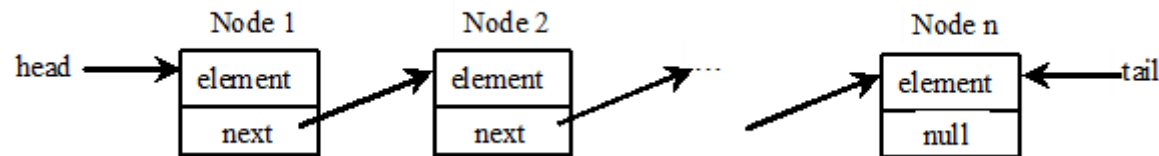
Linked List Animation

<https://liveexample.pearsoncmg.com/dsanimation/LinkedListSteBook.html>

The screenshot shows a web browser window titled "LinkedList Animation by Y. Daniel Liang". The address bar displays the URL "www.cs.armstrong.edu/liang/animation/web/LinkedList.html". The page content includes instructions: "Enter a value and click the Search, Insert, or Delete button to search, insert, or delete the value from the list. Enter a value and an index and then click the Insert button to insert the value in the specified index. Enter an index and then click the Delete button to delete the value in the specified index." Below the text is a diagram of a linked list with four nodes. The first node contains the value 5 and is pointed to by an arrow labeled "head". The second node contains 65, the third contains 7, and the fourth contains 17 and is pointed to by an arrow labeled "tail". Each node is represented as a box divided into two parts: the left part for the value and the right part for the next pointer. Arrows connect the next pointer of one node to the next node. At the bottom of the interface, there are input fields for "Enter a value:" (containing 17) and "Enter an index:" (empty), followed by three buttons: "Search", "Insert" (highlighted in blue), and "Delete".

Nodes in Linked Lists

A linked list consists of nodes. Each node contains an element, and each node is linked to its next neighbor. Thus a node can be defined as a class, as follows:



```
class Node<E> {  
    E element;  
    Node<E> next;  
    public Node(E o) {  
        element = o;  
    }  
}
```

Adding Three Nodes (1 of 4)

The variable **head** refers to the first node in the list, and the variable **tail** refers to the last node in the list. If the list is empty, both are **null**. For example, you can create three nodes to store three strings in a list, as follows:

Step 1: Declare **head** and **tail**:

```
Node<String> head = null;  
Node<String> tail = null;
```

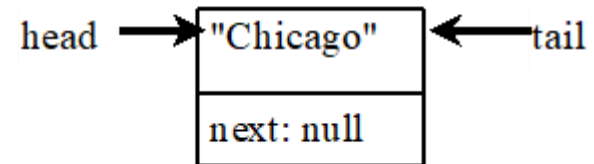
The list is empty now

Adding Three Nodes (2 of 4)

Step 2: Create the first node and insert it to the list:

```
head = new Node<>("Chicago");  
tail = head;
```

After the first node is inserted

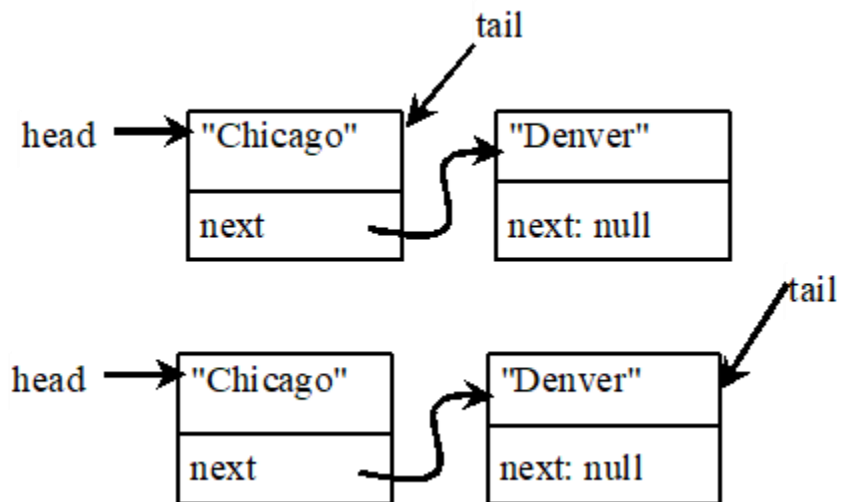


Adding Three Nodes (3 of 4)

Step 3: Create the second node and insert it to the list:

```
tail.next = new Node<>("Denver");
```

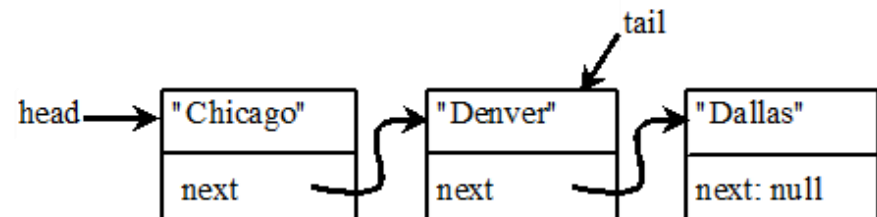
```
tail = tail.next;
```



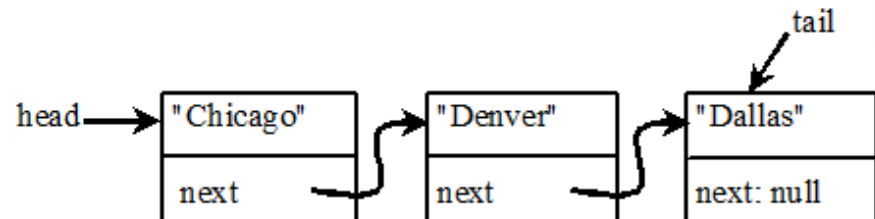
Adding Three Nodes (4 of 4)

Step 4: Create the third node and insert it to the list:

```
tail.next =  
    new Node<>("Dallas");
```



```
tail = tail.next;
```

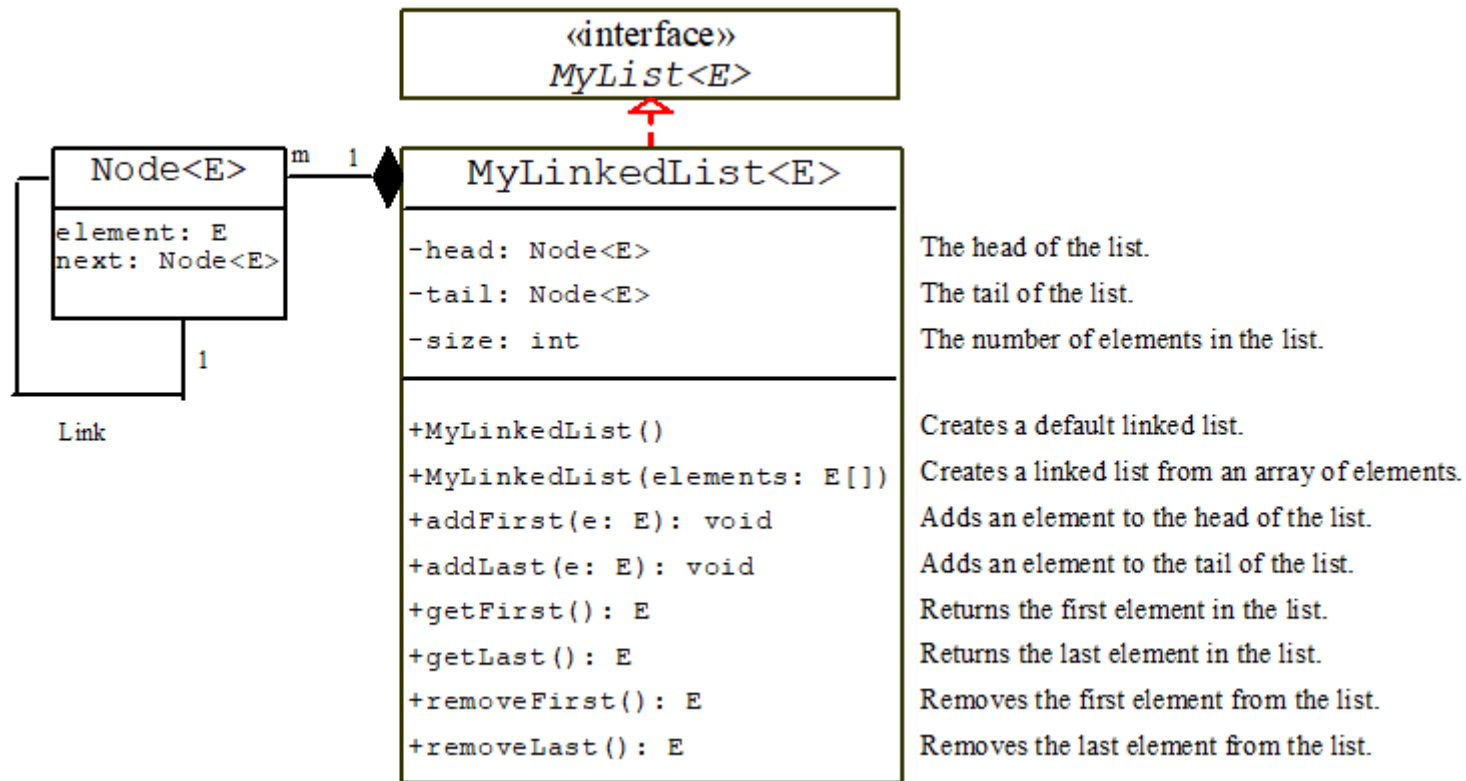


Traversing All Elements in the List

Each node contains the element and a data field named **next** that points to the next node. If the node is the last in the list, its pointer data field **next** contains the value **null**. You can use this property to detect the last node. For example, you may write the following loop to traverse all the nodes in the list.

```
Node<E> current = head;
while (current != null) {
    System.out.println(current.element);
    current = current.next;
}
```

MyLinkedList



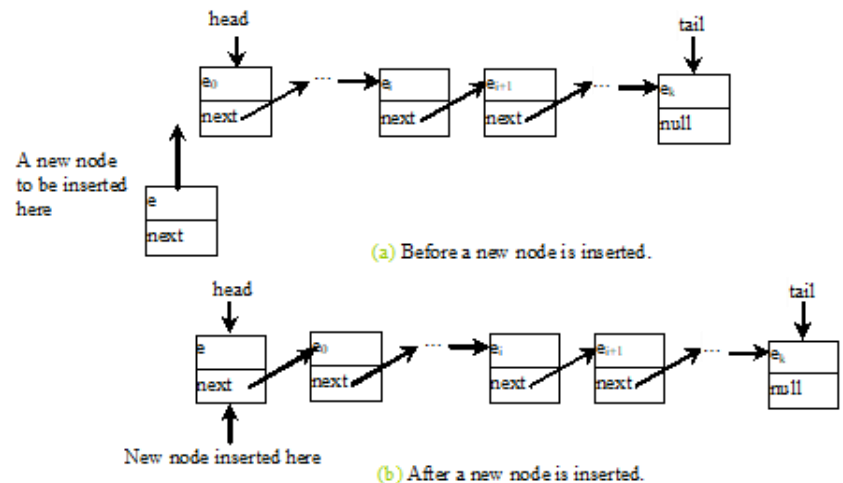
[MyLinkedList](#)

[TestMyLinkedList](#)

Implementing addFirst(E e)

```
public void addFirst(E e) {  
    Node<E> newNode = new  
    Node<>(e);  
    newNode.next = head;  
    head = newNode;  
    size++;  
    if (tail == null)  
        tail = head;  
}
```

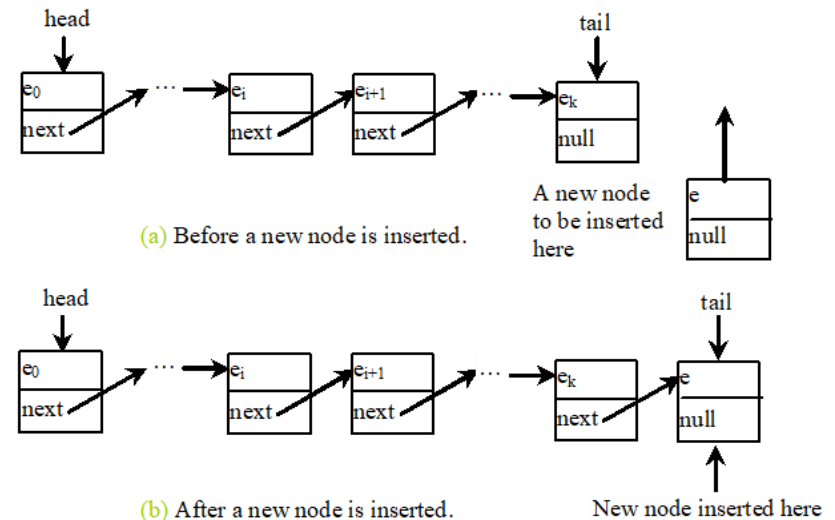
[addFirst Animation](#)



Implementing addLast(E e)

```
public void addLast(E e) {  
    if (tail == null) {  
        head = tail = new Node<>(e);  
    }  
    else {  
        tail.next = new Node<>(e);  
        tail = tail.next;  
    }  
    size++;  
}
```

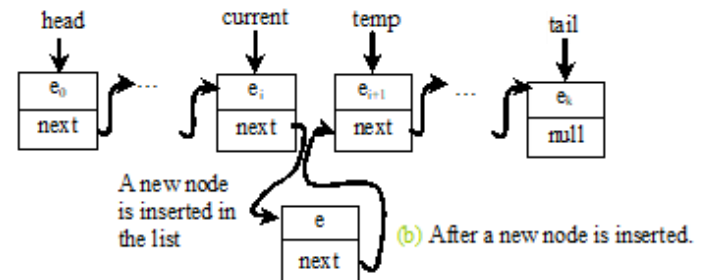
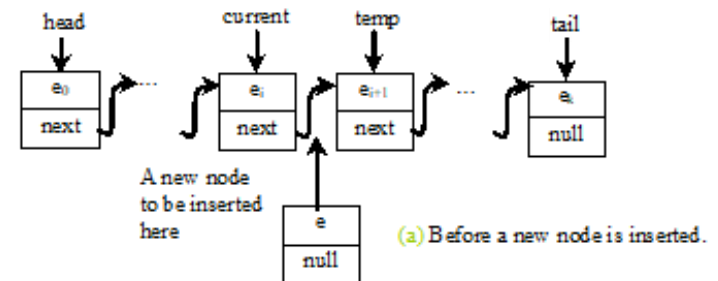
[addLast Animation](#)



Implementing add(int index, E e)

```
public void add(int index, E e) {  
    if (index == 0) addFirst(e);  
    else if (index >= size) addLast(e);  
    else {  
        Node<E> current = head;  
        for (int i = 1; i < index; i++)  
            current = current.next;  
        Node<E> temp = current.next;  
        current.next = new Node<>(e);  
        (current.next).next = temp;  
        size++;  
    }  
}
```

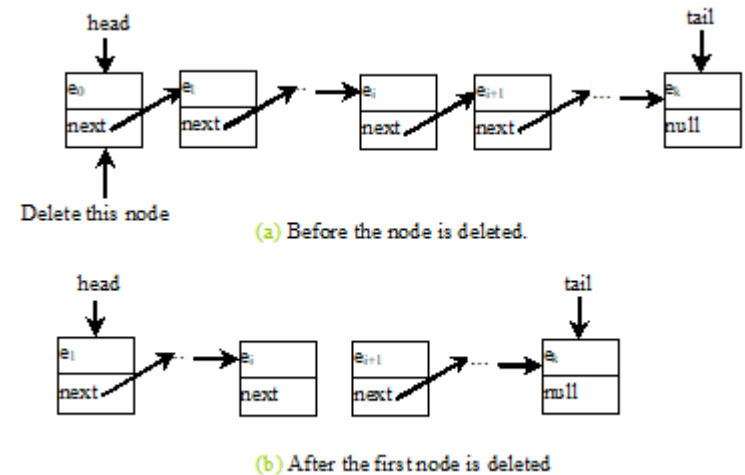
Add(index, e) Animation



Implementing removeFirst()

```
public E removeFirst() {  
    if (size == 0) return null;  
    else {  
        Node<E> temp = head;  
        head = head.next;  
        size--;  
        if (head == null) tail = null;  
        return temp.element;  
    }  
}
```

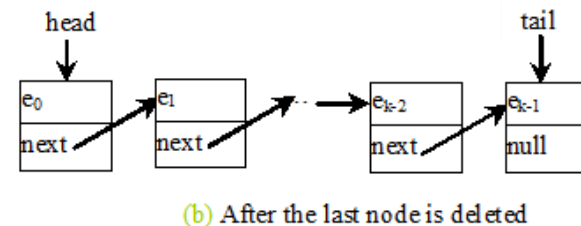
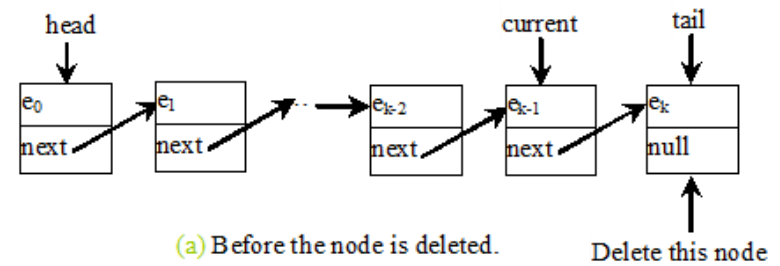
[removeFirst Animation](#)



Implementing `removeLast()` (1 of 2)

```
public E removeLast() {  
    if (size == 0) return null;  
    else if (size == 1)  
    {  
        Node<E> temp = head;  
        head = tail = null;  
        size = 0;  
        return temp.element;  
    }  
}
```

[addLast Animation](#)



Implementing `removeLast()` (2 of 2)

else

```
{  
    Node<E> current = head;  
    for (int i = 0; i < size - 2; i++)  
        current = current.next;  
    Node temp = tail;  
    tail = current;  
    tail.next = null;  
    size--;  
    return temp.element;  
}
```

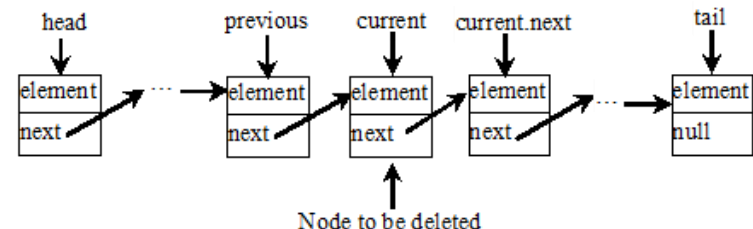
Implementing `remove(int index)` (1 of 2)

```
public E remove(int index) {  
    if (index < 0 || index >= size) return null;  
    else if (index == 0) return removeFirst();  
    else if (index == size - 1) return  
        removeLast();  
    else {  
        Node<E> previous = head;  
        for (int i = 1; i < index; i++) {  
            previous = previous.next;  
        }  
    }  
}
```

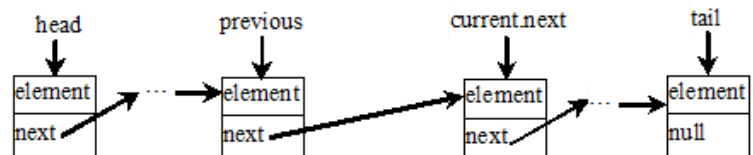
Implementing `remove(int index)` (2 of 2)

```
Node<E> current = previous.next;  
previous.next = current.next;  
size--;  
return current.element;  
}  
}
```

[remove Animation](#)



(a) Before the node is deleted.



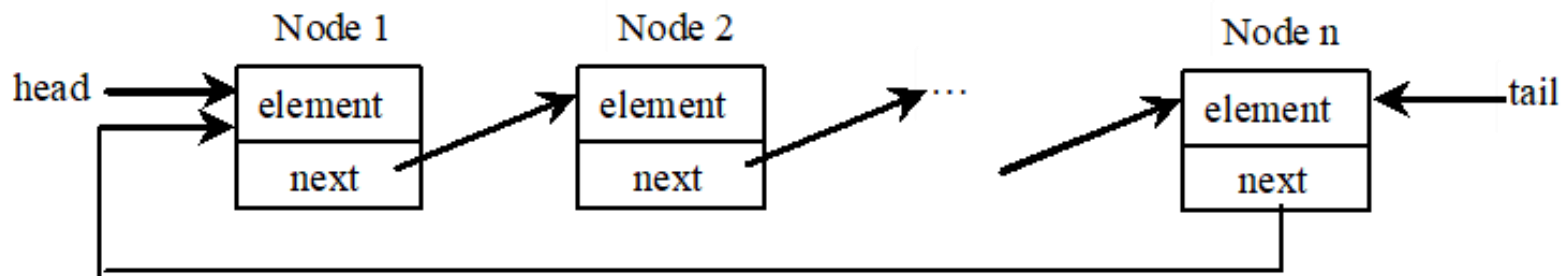
(b) After the node is deleted.

Time Complexity for ArrayList and LinkedList

Methods	MyArrayList/ArrayList	MyLinkedList/LinkedList
<code>add(e: E)</code>	$O(1)$	$O(1)$
<code>add(index: int, e: E)</code>	$O(n)$	$O(n)$
<code>clear()</code>	$O(1)$	$O(1)$
<code>contains(e: E)</code>	$O(n)$	$O(n)$
<code>get(index: int)</code>	$O(1)$	$O(n)$
<code>indexOf(e: E)</code>	$O(n)$	$O(n)$
<code>isEmpty()</code>	$O(1)$	$O(1)$
<code>lastIndexOf(e: E)</code>	$O(n)$	$O(n)$
<code>remove(e: E)</code>	$O(n)$	$O(n)$
<code>size()</code>	$O(1)$	$O(1)$
<code>remove(index: int)</code>	$O(n)$	$O(n)$
<code>set(index: int, e: E)</code>	$O(n)$	$O(n)$
<code>addFirst(e: E)</code>	$O(n)$	$O(1)$
<code>removeFirst()</code>	$O(n)$	$O(1)$

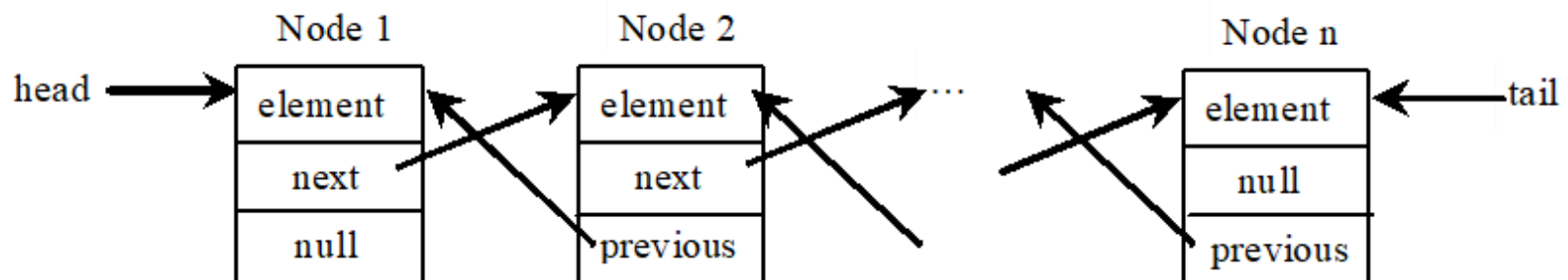
Circular Linked Lists

A **circular, singly linked list** is like a singly linked list, except that the pointer of the last node points back to the first node.



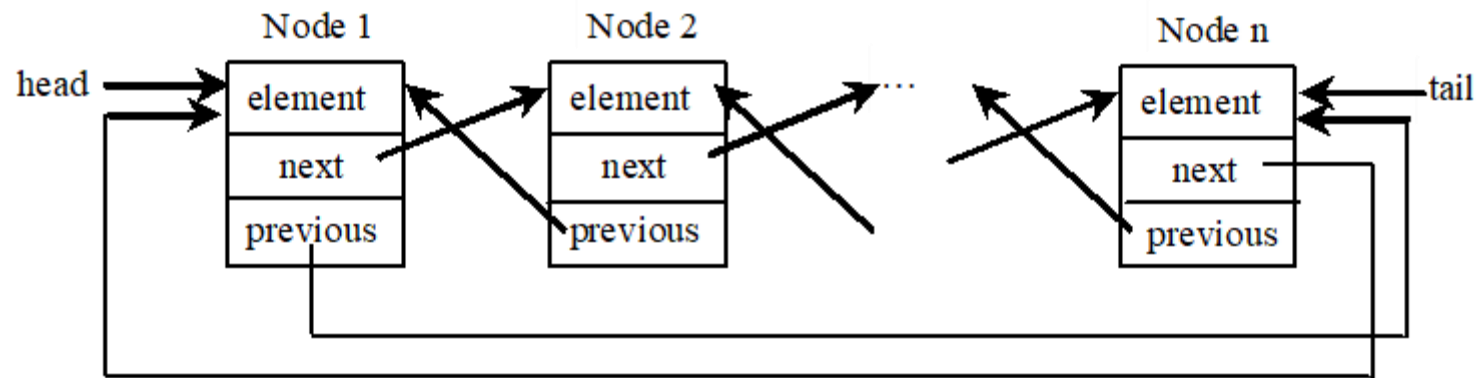
Doubly Linked Lists

A **doubly linked list** contains the nodes with two pointers. One points to the next node and the other points to the previous node. These two pointers are conveniently called a **forward pointer** and a **backward pointer**. So, a doubly linked list can be traversed forward and backward.



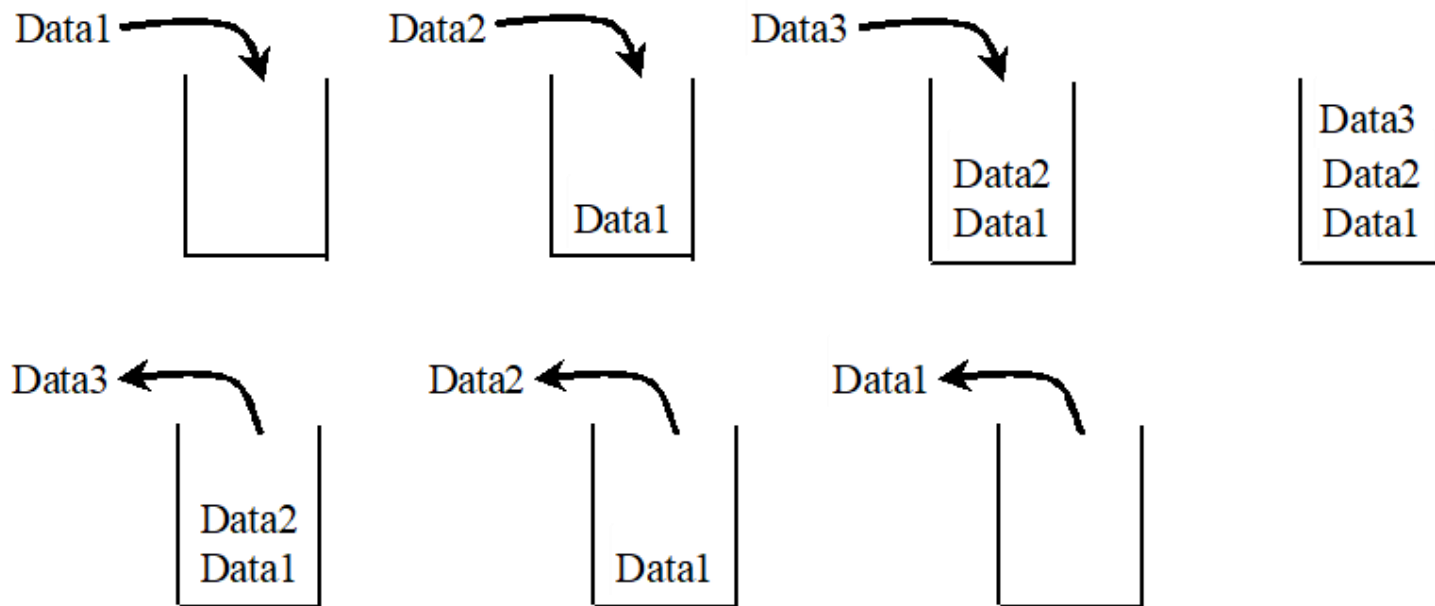
Circular Doubly Linked Lists

A **circular, doubly linked** list is doubly linked list, except that the forward pointer of the last node points to the first node and the backward pointer of the first pointer points to the last node.



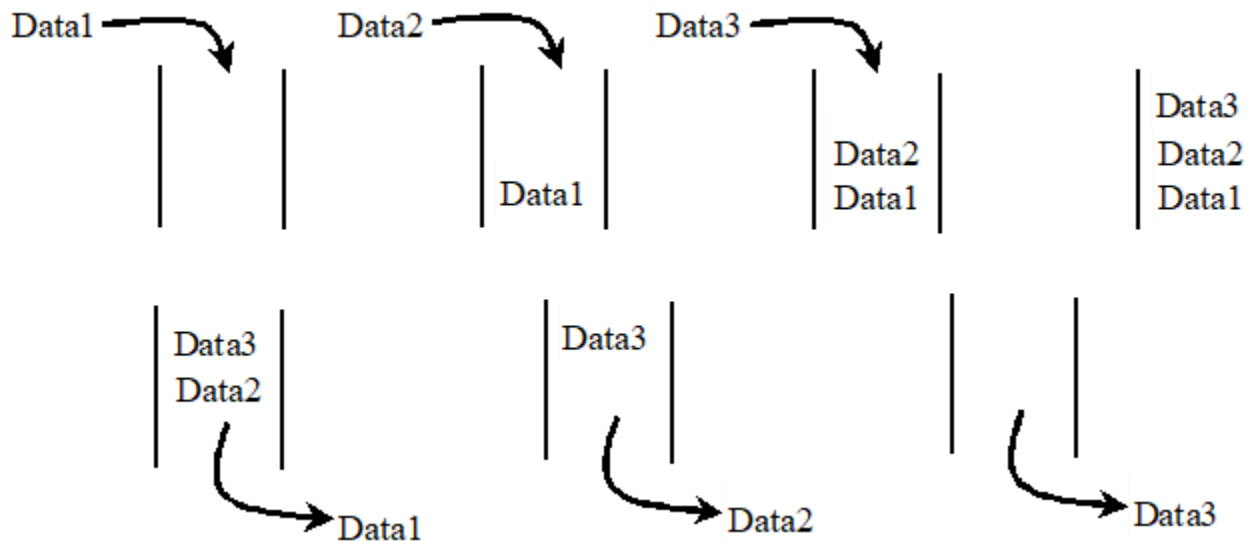
Stacks

A stack can be viewed as a special type of list, where the elements are accessed, inserted, and deleted only from the end, called the top, of the stack.



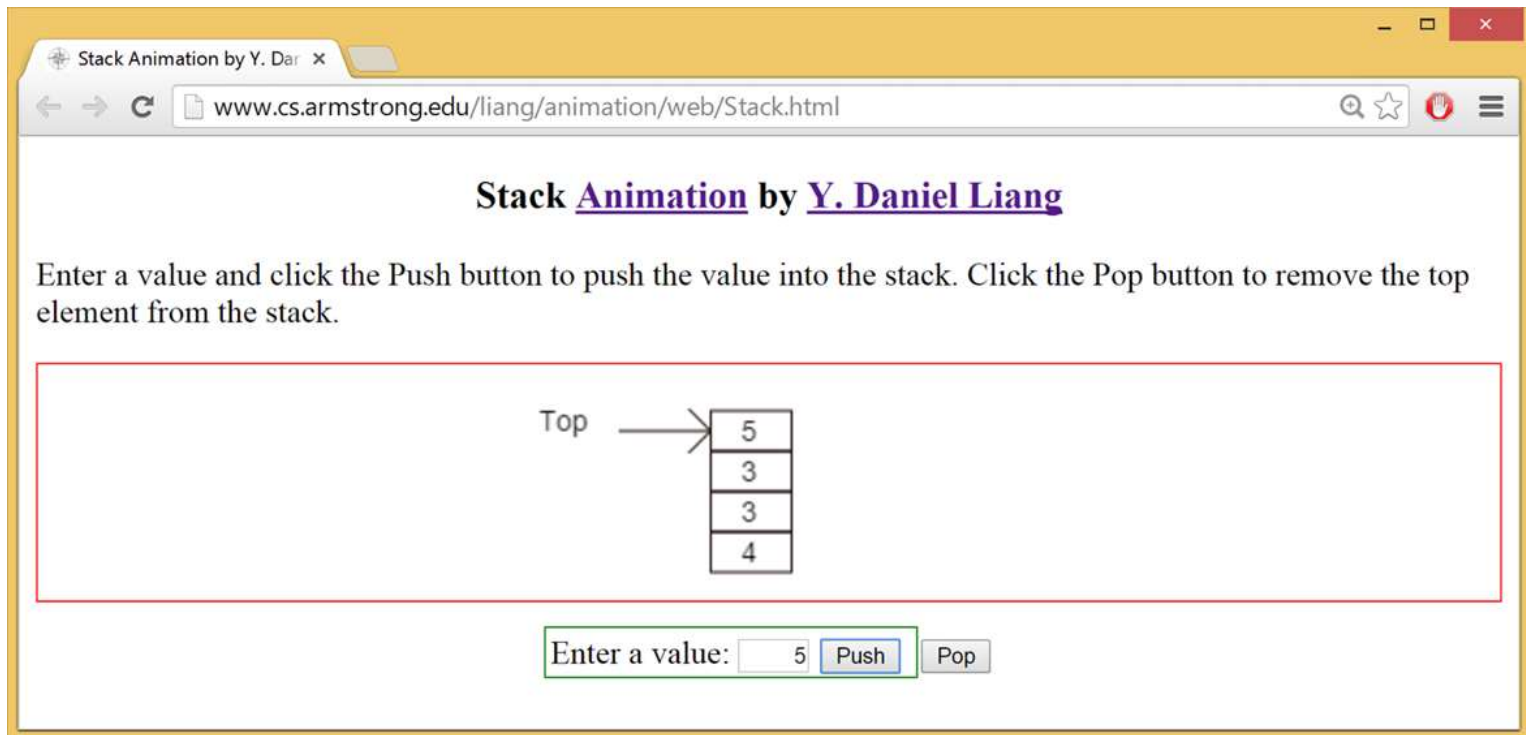
Queues

A queue represents a waiting list. A queue can be viewed as a special type of list, where the elements are inserted into the end (*tail*) of the queue, and are accessed and deleted from the beginning (*head*) of the queue.



Stack Animation

<https://liveexample.pearsoncmg.com/dsanimation/StackBook.html>



The screenshot shows a web browser window titled "Stack Animation by Y. Daniel Liang". The address bar displays the URL "www.cs.armstrong.edu/liang/animation/web/Stack.html". The main content area has the title "Stack Animation by Y. Daniel Liang" and a text instruction: "Enter a value and click the Push button to push the value into the stack. Click the Pop button to remove the top element from the stack." Below the text is a diagram of a stack. A horizontal arrow labeled "Top" points to the top of a vertical container. The container has four cells, with the top cell containing the number 5, followed by 3, 3, and 4 from top to bottom. At the bottom of the interface, there is a text input field with the value "5", a "Push" button, and a "Pop" button.

Queue Animation

<https://liveexample.pearsoncmg.com/dsanimation/QueueBook.html>

Stack Animation by Y. Dar x

www.cs.armstrong.edu/liang/animation/web/Queue.html

Queue Animation by Y. Daniel Liang

Enter a value and click the Enqueue button to append the value into the tail of the queue. Click the Dequeue button to remove the element from the head of the queue.

head

tail

5	45	2	4	21
---	----	---	---	----

Enter a value: 21 Enqueue Dequeue

Implementing Stacks and Queues

Using an array list to implement Stack

Use a linked list to implement Queue

Since the insertion and deletion operations on a stack are made only at the end of the stack, using an array list to implement a stack is more efficient than a linked list. Since deletions are made at the beginning of the list, it is more efficient to implement a queue using a linked list than an array list. This section implements a stack class using an array list and a queue using a linked list.

Design of the Stack and Queue Classes

- There are two ways to design the stack and queue classes:
 - Using inheritance: You can define the stack class by extending the array list class, and the queue class by extending the linked list class.



(a) Using inheritance

- Using composition: You can define an array list as a data field in the stack class, and a linked list as a data field in the queue class.



(b) Using composition

Composition is Better

Both designs are fine, but using composition is better because it enables you to define a complete new stack class and queue class without inheriting the unnecessary and inappropriate methods from the array list and linked list.

MyStack and MyQueue

GenericStack

GenericStack<E>	
-list: java.util.ArrayList<E>	An arraylist to store elements.
+GenericStack()	Creates an empty stack.
+getSize(): int	Returns the number of elements in this stack.
+peek(): E	Returns the top element in this stack.
+pop(): E	Returns and removes the top element in this stack.
+push(o: E): void	Adds a new element to the top of this stack.
+isEmpty(): boolean	Returns true if the stack is empty.

GenericQueue

GenericQueue<E>	
-list: LinkedList<E>	
+enqueue(e: E): void	Adds an element to this queue.
+dequeue(): E	Removes an element from this queue.
+getSize(): int	Returns the number of elements from this queue.

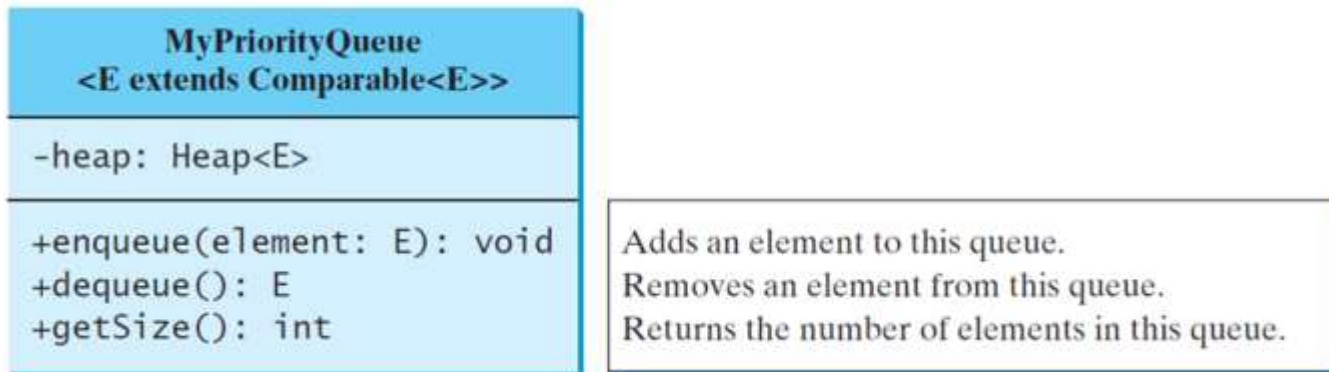
Example: Using Stacks and Queues

Write a program that creates a stack using **MyStack** and a queue using **MyQueue**. It then uses the **push** (**enqueue**) method to add strings to the stack (**queue**) and the **pop** (**dequeue**) method to remove strings from the stack (**queue**).

[TestStackQueue](#)

Priority Queue

A regular queue is a first-in and first-out data structure. Elements are appended to the end of the queue and are removed from the beginning of the queue. In a priority queue, elements are assigned with priorities. When accessing elements, the element with the highest priority is removed first. A priority queue has a largest-in, first-out behavior. For example, the emergency room in a hospital assigns patients with priority numbers; the patient with the highest priority is treated first.



[MyPriorityQueue](#)

[TestPriorityQueue](#)

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.